How to Scale Your Model

Austin et al., Google DeepMind

Contents

How to Scale Your Model: A Systems View of LLMs on TPUs **2**
Table of Contents . . . 2

Introduction **3**
High-Level Outline . . . 4
Links to Sections . . . 7

All About Rooflines **9**
Where Does the Time Go? . . . 9
A Few Problems to Work . . . 13

How to Think About TPUs **16**
What Is a TPU? . . . 16
TPU Networking . . . 20
Key Takeaways . . . 23
Worked Problems . . . 25
Appendix . . . 28

Sharded Matrices and How to Multiply Them **33**
Partitioning Notation and Collective Operations . . . 33
Computation With Sharded Arrays . . . 40
A Deeper Dive into TPU Communication Primitives . . . 47
What Have We Learned? . . . 50
Some Problems to Work . . . 51

All the Transformer Math You Need to Know **56**
Counting Dots . . . 56
Transformer Accounting . . . 57
Global FLOPs and Params Calculation . . . 59
Miscellaneous Math . . . 61
What Should You Take Away from this Section? . . . 62
A Few Problems to Work . . . 62
Appendix . . . 64

How to Parallelize a Transformer for Training **67**
What Do We Mean By Scaling? . . . 67
Takeaways from LLM Training on TPUs . . . 82
Some Problems to Work . . . 83
Appendix . . . 84

Training LLaMA 3 on TPUs **85**
Worked Problems . . . 88

All About Transformer Inference **89**
The Basics of Transformer Inference . . . 89
Tricks for Improving Generation Throughput and Latency . . . 96
Distributing Inference Over Multiple Accelerators . . . 97
Designing an Effective Inference Engine . . . 100
Worked Problems . . . 103
Appendix . . . 106

Serving LLaMA 3 on TPUs **113**
What's the LLaMA Serving Story? . . . 113
Visualizing the Latency Throughput Tradeoff . . . 117
Worked Problems . . . 119

How to Profile TPU Code **121**
A Thousand-Foot View of the TPU Software Stack . . . 121
The JAX Profiler: A Multi-Purpose TPU Profiler . . . 122
Worked Problems . . . 129

Programming TPUs in JAX **131**
How Does Parallelism Work in JAX? . . . 131
Worked Problems . . . 138

Conclusions and Further Reading **142**
Acknowledgments . . . 142
Further Reading . . . 142
Feedback . . . 143

How to Think About GPUs **144**
What Is a GPU? . . . 144
Networking . . . 151
Beyond the node level . . . 153
How Do Collectives Work on GPUs? . . . 156
Rooflines for LLM Scaling on GPUs . . . 163
Acknowledgements and Further Reading . . . 169
Appendix A: How does this change with GB200? . . . 169
Appendix B: More networking details . . . 170

How to Scale Your Model: A Systems View of LLMs on TPUs

A comprehensive guide to training and serving large language models

Table of Contents

- Introduction
- Chapter 1: All About Rooflines
- Chapter 2: How to Think About TPUs
- Chapter 3: Sharded Matrices and How to Multiply Them
- Chapter 4: All the Transformer Math You Need to Know
- Chapter 5: How to Parallelize a Transformer for Training
- Chapter 6: Training LLaMA 3 on TPUs
- Chapter 7: All About Transformer Inference
- Chapter 8: Serving LLaMA 3 on TPUs
- Chapter 9: How to Profile TPU Code
- Chapter 10: Programming TPUs in JAX
- Chapter 11: Conclusions and Further Reading
- Chapter 12: How to Think About GPUs

Introduction



Much of deep learning still boils down to a kind of black magic, but optimizing the performance of your models doesn't have to — even at huge scale! Relatively simple principles apply everywhere — from dealing with a single accelerator to tens of thousands — and understanding them lets you do many useful things:

- Ballpark how close parts of your model are to their theoretical optimum.
- Make informed choices about different parallelism schemes at different scales (how you split the computation across multiple devices).
- Estimate the cost and time required to train and run large Transformer models.
- Design algorithms that take advantage of specific hardware affordances.
- Design hardware driven by an explicit understanding of what limits current algorithm performance.

Expected background: We're going to assume you have a basic understanding of LLMs and the Transformer architecture but not necessarily how they operate at scale. You should know the basics of LLM training and ideally have some basic familiarity with JAX. Some useful background reading might include this blog post on the Transformer architecture and the original Transformer paper. Also check out this list for more useful concurrent and future reading.

Goals & Feedback: By the end, you should feel comfortable estimating the best parallelism scheme for a Transformer model on a given hardware platform, and roughly how long training and inference should take. If you don't, email us or leave a comment! We'd love to know how we could make this clearer.

You might also enjoy reading the new Section 12 on NVIDIA GPUs!

Why should you care?

Three or four years ago, I don't think most ML researchers would have needed to understand any of the content in this book. But today even "small" models run so close to hardware limits that doing novel research requires you to think about efficiency at scale.[1] **A 20% win on benchmarks is irrelevant if it comes at a 20% cost**

[1]Historically, ML research has followed something of a tick-tock cycle between systems innovations and software improvements. Alex Krizhevsky had to write unholy CUDA code to make CNNs fast but within a couple of years, libraries like Theano and TensorFlow meant you didn't have to. Maybe that will happen here too and everything in this book will be abstracted away in a few years. But

to roofline efficiency. Promising model architectures routinely fail either because they *can't* run efficiently at scale or because no one puts in the work to make them do so.

The goal of "model scaling" is to be able to increase the number of chips used for training or inference while achieving a proportional, linear increase in throughput. This is known as "*strong scaling*". Although adding additional chips ("parallelism") usually decreases the computation time, it also comes at the cost of added communication between chips. When communication takes longer than computation we become "communication bound" and cannot scale strongly.[2] If we understand our hardware well enough to anticipate where these bottlenecks will arise, we can design or reconfigure our models to avoid them.[3]

Our goal in this book is to explain how TPU (and GPU) hardware works and how the Transformer architecture has evolved to perform well on current hardware. We hope this will be useful both for researchers designing new architectures and for engineers working to make the current generation of LLMs run fast.

High-Level Outline

The overall structure of this book is as follows:

Section 1 explains roofline analysis and what factors can limit our ability to scale (communication, computation, and memory). Section 2 and Section 3 talk in detail about how TPUs work, both as individual chips and — of critical importance — as an interconnected system with inter-chip links of limited bandwidth and latency. We'll answer questions like:

- How long should a matrix multiply of a certain size take? At what point is it bound by compute or by memory or communication bandwidth?
- How are TPUs wired together to form training clusters? How much bandwidth does each part of the system have?
- How long does it take to gather, scatter, or re-distribute arrays across multiple TPUs?
- How do we efficiently multiply matrices that are distributed differently across devices?

Five years ago ML had a colorful landscape of architectures — ConvNets, LSTMs, MLPs, Transformers — but now we mostly just have the Transformer. We strongly believe it's worth understanding every piece of the Transformer architecture: the exact sizes of every matrix, where normalization occurs, how many parameters and FLOPs[4] are in each part. Section 4 goes through this "Transformer math" carefully, showing how to count the parameters and FLOPs for both training and inference. This tells us how much memory our model will use, how much time we'll spend on compute or comms, and when attention will become important relative to the feed-forward blocks.

Section 5: Training and Section 7: Inference are the core of this book, where we discuss the fundamental question: given a model of some size and some number of chips, how do I parallelize my model to stay in the "strong scaling" regime? This is a simple question with a surprisingly complicated answer. At a high level, there are four primary parallelism techniques used to split models over multiple chips (**data**, **tensor**, **pipeline**, and **expert**), and a number of other techniques to reduce the memory requirements (**rematerialization**, **optimizer/model sharding (aka ZeRO)**, **host offload**, **gradient accumulation**). We discuss many of these here.

scaling laws have pushed our models perpetually to the very frontier of our hardware, and it seems likely that, for the foreseeable future, doing cutting-edge research will be inextricably tied to an understanding of how to efficiently scale models to large hardware topologies.

[2] As your computation time decreases, you also typically face bottlenecks at the level of a single chip. Your shiny new TPU or GPU may be rated to perform 500 trillion operations per second, but if you aren't careful it can just as easily do a tenth of that if it's bogged down moving parameters around in memory. The interplay of per-chip computation, memory bandwidth, and total memory is critical to the scaling story.

[3] Hardware designers face the inverse problem: building hardware that provides just enough compute, bandwidth, and memory for our algorithms while minimizing cost. You can imagine how stressful this "co-design" problem is: you have to bet on what algorithms will look like when the first chips actually become available, often 2 to 3 years down the road. The story of the TPU is a resounding success in this game. Matrix multiplication is a unique algorithm in the sense that it uses far more FLOPs per byte of memory than almost any other (N FLOPs per byte), and early TPUs and their systolic array architecture achieved far better perf / $ than GPUs did at the time they were built. TPUs were designed for ML workloads, and GPUs with their Tensor Cores are rapidly changing to fill this niche as well. But you can imagine how costly it would have been if neural networks had not taken off, or had changed in some fundamental way that TPUs (which are inherently less flexible than GPUs) could not handle.

[4] FLoating point OPs, basically the total number of adds and multiplies required. While many sources take FLOPs to mean "operations per second", we use FLOPs/s to indicate that explicitly.

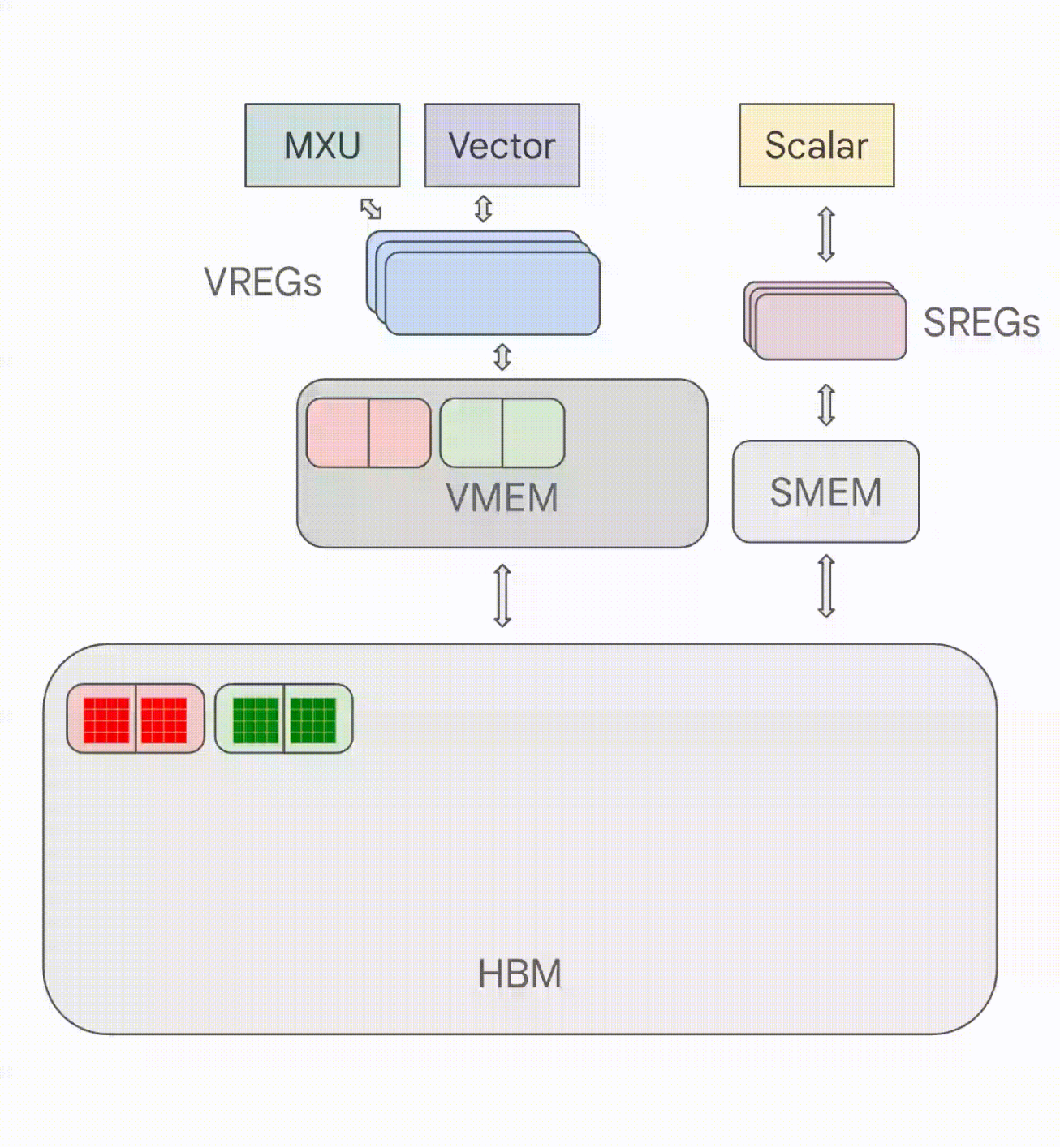
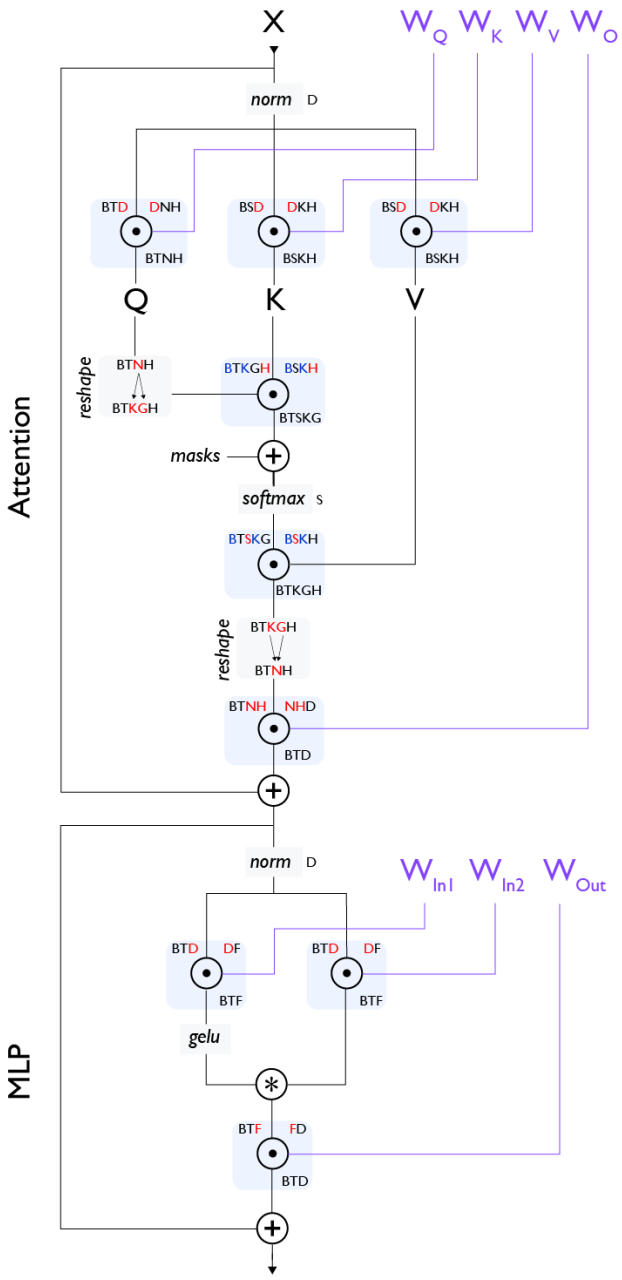


Figure 1: Figure: a diagram from Section 2 showing how a TPU performs an elementwise product. Depending on the size of our arrays and the bandwidth of various links, we can find ourselves compute-bound (using the full hardware compute capacity) or memory-bound (bottlenecked by memory loading).



symbol	dimension
B	batch
L	number of layers
T	sequence length (query)
S	sequence length (key value)
V	vocab
D	d_model, embedding dimension
F	MLP hidden dimension
H	attention head dimension
N	number of query heads
K	number of key/value heads
G	q heads per kv head = N // K

Figure 2: Figure: a standard Transformer layer with each matrix multiplication (matmul) shown as a dot inside a circle. All parameters (excluding norms) are shown in purple. Section 4 walks through this diagram in more detail.

We hope by the end of these sections you should be able to choose among them yourself for new architectures or settings. Section 6 and Section 8 are practical tutorials that apply these concepts to LLaMA 3, a popular open-source model.

Finally, Section 9 and Section 10 look at how to implement some of these ideas in JAX and how to profile and debug your code when things go wrong. Section 12 is a new section that dives into GPUs as well.

Throughout we try to give you problems to work for yourself. Please feel no pressure to read all the sections or read them in order. And please leave feedback. For the time being, this is a draft and will continue to be revised. Thank you!

We'd like to acknowledge James Bradbury and Blake Hechtman who derived many of the ideas in this book.

Without further ado, here is Section 1 about TPU rooflines.

Links to Sections

This series is probably longer than it needs to be, but we hope that won't deter you. The first three chapters are preliminaries and can be skipped if you're already familiar with the material, although they introduce notation used later. The final three parts might be the most practically useful, since they explain how to work with real models.

Part 1: Preliminaries

- **Chapter 1: A Brief Intro to Roofline Analysis**. Algorithms are bounded by three things: compute, communication, and memory. We can use these to approximate how fast our algorithms will run.
- **Chapter 2: How to Think About TPUs**. How do TPUs work? How does that affect what models we can train and serve?
- **Chapter 3: Sharded Matrices and How to Multiply Them**. Here we explain model sharding and multi-TPU parallelism by way of our favorite operation: (sharded) matrix multiplications.

Part 2: Transformers

- **Chapter 4: All the Transformer Math You Need to Know**. How many FLOPs does a Transformer use in its forward and backward pass? Can you calculate the number of parameters? The size of its KV caches? We work through this math here.
- **Chapter 5: How to Parallelize a Transformer for Training**. FSDP. Megatron sharding. Pipeline parallelism. Given some number of chips, how do I train a model of a given size with a given batch size as efficiently as possible?
- **Chapter 6: Training LLaMA 3 on TPUs**. How would we train LLaMA 3 on TPUs? How long would it take? How much would it cost?
- **Chapter 7: All About Transformer Inference**. Once we've trained a model, we have to serve it. Inference adds a new consideration — latency — and changes up the memory landscape. We'll talk about how disaggregated serving works and how to think about KV caches.
- **Chapter 8: Serving LLaMA 3 on TPUs**. How much would it cost to serve LLaMA 3 on TPU v5e? What are the latency/throughput tradeoffs?

Part 3: Practical Tutorials

- **Chapter 9: How to Profile TPU Code**. Real LLMs are never as simple as the theory above. Here we explain the JAX + XLA stack and how to use the JAX/TensorBoard profiler to debug and fix real issues.
- **Chapter 10: Programming TPUs in JAX**. JAX provides a bunch of magical APIs for parallelizing computation, but you need to know how to use them. Fun examples and worked problems.

Part 4: Conclusions and Bonus Content

- **Chapter 11: Conclusions and Further Reading**. Closing thoughts and further reading on TPUs and LLMs.

- **Chapter 12: How to Think About GPUs**. A bonus section about GPUs, how they work, how they're networked, and how their rooflines differ from TPUs.

Chapter 1

All About Rooflines

Where Does the Time Go?

Let's start with an extremely simple question: *why does an algorithm take 50ms instead of 50s or 5ms*? What is actually happening within the model that takes substantial time and how long should we expect it to take?

Computation: A deep learning model is effectively a bunch of matrix multiplications, each composed of floating-point multiplication and addition 'operations' (FLOPs). Our accelerator speed determines how long these take to compute:

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}} \tag{1}$$

For instance, an NVIDIA H100 can perform about 9.89e14 bfloat16[5] FLOPs/s while a TPU v6e can perform 9.1e14 FLOPs/s.[6] That means doing 1e12 FLOPs on an H100 will take (roughly) `1e12 / 9.89e14 = 1.01ms` and `1e12 / 9.1e14 = 1.1ms` on a TPU v6e.[7]

Communication within a chip: *Within an accelerator*, tensors need to be transferred between accelerator memory (HBM) and the compute cores. You'll see the bandwidth of this link referred to as "HBM bandwidth".[8] On an H100, this is about 3.35TB/s and on TPU v6e this is about 1.6TB/s.

Communication between chips: When we distribute a model *across* multiple accelerators, tensors frequently need to be transferred between them. There are often a few options for this on our hardware (ICI, DCN, and PCIe), each with different bandwidths.

Whether the communication is within a chip or between chips, we measure this in bytes/s and estimate the total communication time with:

$$T_{\text{comms}} = \frac{\text{Communication Bytes}}{\text{Network/Memory Bandwidth Bytes/s}} \tag{2}$$

Typically (but not always), computation within a single chip can be overlapped with communication within a chip and between chips. This means **we can lower-bound training and inference time by using the maximum of computation and communication time**. We can also **upper-bound with their sum**. In practice, we optimize against the maximum as the algebra is simpler and we can usually come close to this bound by overlapping our communication and computation. If we optimize with the maximum in mind then the lower and upper bounds differ by at most a factor of 2 since $T_{\text{math}} + T_{\text{comms}} \leq 2 * \max(T_{\text{math}}, T_{\text{comms}})$. We then increase accuracy beyond this by modeling 'overlap regions' and overheads, which can be informed by profiling your specific model and target system.

$$T_{\text{lower}} = \max(T_{\text{math}}, T_{\text{comms}}) \tag{3}$$

[5] bf16 is short for bfloat16, a 16-bit floating point format often used in ML.

[6] H100s and B200s can usually only achieve around 80-85% of the claimed peak FLOPs, while TPUs can get closer to 95% in normal use.

[7] Note that these chips are priced differently, and this comparison does not normalize to cost.

[8] NVIDIA also calls this "memory bandwidth."

$$T_{\text{upper}} = T_{\text{math}} + T_{\text{comms}} \tag{4}$$

If we assume we can perfectly overlap communication and computation, when $T_{\text{math}} > T_{\text{comms}}$, we see full utilization from our hardware. We call this being "compute-bound". When $T_{\text{comms}} > T_{\text{math}}$, we tend to be "communication-bound"[9] and at least some fraction of our accelerator FLOPs/s is wasted waiting for data to be passed around. One way to tell if an operation will be compute or communication-bound is to look at its "**arithmetic intensity**" or "**operational intensity**".

Definition: the arithmetic intensity of an algorithm is given by the ratio of the total FLOPs it performs to the number of bytes it needs to communicate — either within a chip or between chips.

$$\text{Arithmetic Intensity} = \frac{\text{Computation FLOPs}}{\text{Communication Bytes}} \tag{5}$$

Arithmetic intensity measures the "FLOPs per byte" of a given operation. To a first order, when our arithmetic intensity is high, T_{math} is large compared to T_{comms} and we typically use most of the available FLOPs. When the opposite is true, we spend more time on comms and waste FLOPs. The point where this crossover happens is the "peak arithmetic intensity" of our hardware, the ratio of peak accelerator FLOPs/s to accelerator bandwidth.

$$\begin{aligned} T_{\text{math}} > T_{\text{comms}} &\Leftrightarrow \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}} > \frac{\text{Communication Bytes}}{\text{Bandwidth Bytes/s}} \\ &\Leftrightarrow \frac{\text{Computation FLOPs}}{\text{Communication Bytes}} > \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} \\ &\Leftrightarrow \text{Intensity(Computation)} > \text{Intensity(Accelerator)} \end{aligned}$$

The quantity Intensity(Accelerator) is the arithmetic intensity at which our accelerator achieves its peak FLOPs/s. **For the TPU v5e MXU, this is about 240 FLOPs/byte**, since the TPU can perform `1.97e14` FLOPs/s and load `8.2e11` bytes/s from HBM.[10] That means if an algorithm has a lower arithmetic intensity than 240 FLOPs/byte, it will be bound by byte loading and thus we won't make good use of our hardware.[11] Let's look at one such example:

Example (dot product): to compute the dot product of two vectors in bfloat16 precision, `x • y: bf16[N], bf16[N] → bf16[1]`, we need to load x and y from memory, each of which has $2 * N = 2N$ bytes, perform N multiplications and $N - 1$ additions, and write 2 bytes back into HBM

$$\text{Intensity(dot product)} = \frac{\text{Total FLOPs}}{\text{Total Bytes}} = \frac{N + N - 1}{2N + 2N + 2} = \frac{2N - 1}{4N + 2} \to \frac{1}{2} \tag{6}$$

as $N \to \infty$. So the dot product has an arithmetic intensity of $\frac{1}{2}$ or, put another way, the dot product does 0.5 floating point operations per byte loaded. This means our arithmetic intensity is lower than that of our hardware and we will be communication-bound.[12]

[9] We'll use "communication-bound", "comms-bound", "memory-bound", and "bandwidth-bound" interchangeably throughout this book.

[10] The MXU is the matrix multiply unit on the TPU. We specify this here because the TPU has other accelerators like the VPU that are responsible for elementwise operations that have a different peak FLOPs/s.

[11] This is only true if the algorithm loads its weights from HBM and runs in the MXU. As we'll discuss in the next section, we can sometimes store parameters in VMEM which has a much higher bandwidth. Many algorithms also run in the VPU, which has different performance characteristics.

[12] The 240 number above is not the correct comparison here since, as you will see in the next section, a dot-product is performed on the VPU and not the MXU. The TPU v5p VPU can do roughly 7e12 FLOPs / second per core, so its critical intensity is around 3, which means we are still somewhat comms-bound here. Either way, the fact that our intensity is low and constant means it is difficult to be compute-bound on most hardware.

Visualizing rooflines

We can visualize the tradeoff between memory and compute using a **roofline plot**, which plots the peak achievable FLOPs/s (throughput) of an algorithm on our hardware (the y-axis) against the arithmetic intensity of that algorithm (the x-axis). Here's an example log-log plot:

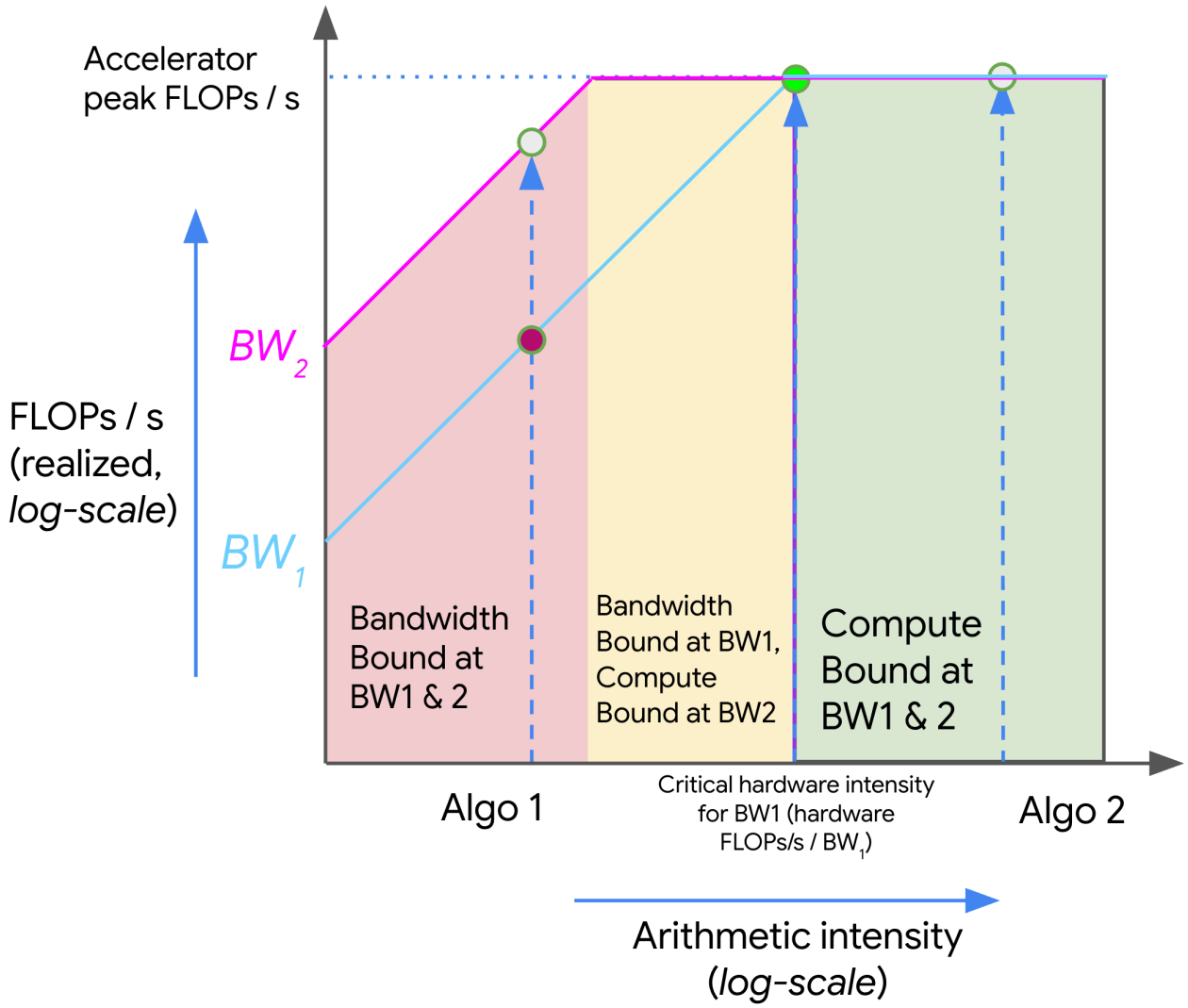


Figure 3: Figure: an example roofline plot showing two algorithms with different arithmetic intensities (Algo 1 and Algo 2) and their corresponding theoretical peak throughput under different bandwidths (BW1 and BW2). In the red area, an algorithm is bandwidth bound at both bandwidths and is wasting some fraction of the hardware's peak FLOPs/s. The yellow area is bandwidth-bound only at the lower bandwidth (BW1). The green area is compute-bound at all bandwidths. Here, we are using the peak FLOPs/s of the accelerator and increasing bandwidth or improving intensity yield no benefit.

Above, as the intensity increases (moving left to right), we initially see a linear increase in the performance of our algorithm (in FLOPs/s) until we hit the critical arithmetic intensity of the hardware, 240 in the case of the TPU v5e. Any algorithm with a lower intensity will be bandwidth (BW) bound and limited by the peak memory bandwidth (shown in red). Any algorithm to the right will fully utilize our FLOPs (shown in green). Here, Algo 1 is comms-bound and uses only a fraction of the total hardware FLOPs/s. Algo 2 is compute-bound. We can generally improve the performance of an algorithm either by increasing its arithmetic intensity or by increasing the memory bandwidth available (moving from BW1 to BW2).

Matrix multiplication

Let's look at our soon-to-be favorite algorithm: matrix multiplication (aka matmul). Given two matrices X and Y, we can write $X * Y \to Z$ where X has shape bf16$[B, D]$, Y has shape bf16$[D, F]$, and Z has shape bf16$[B, F]$. To do the matmul we need to load $2DF + 2BD$ bytes, perform $2BDF$ FLOPs, and write $2BF$ bytes back.[13] [14] Thus:

$$\text{Intensity(matmul)} = \frac{2BDF}{2BD + 2DF + 2BF} = \frac{BDF}{BD + DF + BF} \tag{7}$$

We can get a nice simplification if we assume our "batch size" B is small relative to D and F. Then we get

$$\frac{BDF}{BD + DF + BF} \approx \frac{BDF}{DF} = B \tag{8}$$

$$\text{Intensity(matmul)} > \text{Intensity(TPU)} \implies B > \frac{1.97e14}{8.20e11} = 240 \tag{9}$$

This is a reasonable assumption for Transformer matmuls since we typically have a local (per-replica) token batch size $B < 1024$ (*note, not sequences*) but D and $F > 8000$. Thus we generally become compute-bound when our per-replica[15] batch size is greater than 240 tokens, a very simple rule!

Takeaway: for a bfloat16 matmul to be compute-bound on most TPUs, we need our batch size to be greater than 240 (tokens).[16]

This comes with a few notable caveats we'll explore in the problems below, particularly with respect to quantization (e.g., if we quantize our activations but still do full-precision FLOPs), but it's a good rule to remember. For GPUs, this number is slightly higher (closer to 300), but the same conclusion generally holds. When we decompose a big matmul into smaller matmuls, the tile sizes also matter.[17] We'll discuss the lower-level GPU and TPU details in the next section.

Network communication rooflines

All the rooflines we've discussed so far have been memory-bandwidth rooflines, *all within a single chip*. This shouldn't be taken as a rule. In fact, most of the rooflines we'll care about in this book involve communication between chips: usually matrix multiplications that involve matrices sharded across multiple TPUs.

To pick a somewhat contrived example, say we want to multiply two big matrices $X \sim$ bf16$[B, D]$ and $Y \sim$ bf16$[D, F]$ which are split evenly across 2 TPUs/GPUs (along the D dimension). To do this multiplication (as we'll see in Section 3), we can multiply half of each matrix on each TPU (`Z0 = X[:, :D // 2] @ Y[:D // 2, :]` on TPU 0 and `Z1 = X[:, D // 2:] @ Y[D // 2:, :]` on TPU 1) and then copy the resulting "partial sums" to the other TPU and add them together. Say we can copy `4.5e10` bytes/s in each direction and perform `1.97e14` FLOPs/s on each chip. What are T_{math} and T_{comms}?

T_{math} is clearly half of what it was before, since each TPU is doing half the work, i.e.[18]

[13] Technically we perform $BF \times (2D - 1)$ FLOPs but this is close enough. This comes from BDF multiplications and $BF * (D - 1)$ additions. Section 4 has more details.

[14] Although the output of a matmul is technically float32 we usually cast down to bfloat16 before copying back to HBM.

[15] We say per-replica because, if we do some kind of model sharding to increase the number of chips used in the matmul, we scale both our available compute and memory bandwidth by the same amount. Thus the critical batch size is true per independent copy of the model weights.

[16] Note that this is *not* the batch size in the usual sense, where it means the batch size in sequences. It turns out most rooflines depend purely on the number of tokens, whether they belong to the same or different sequences. For instance if you have a batch size of 512 sequences of 4096 tokens on 2048 GPUs, you have a total batch size of `512 * 4096 = 2M` tokens, and a local batch size of 1k tokens.

[17] When we do a large matrix multiplication, we need to break it down into smaller tiles which fit into VMEM/SMEM/TMEM, the higher-bandwidth on-chip memory. This causes us to load chunks multiple times, so it's no longer quite true that we only load $O(N^2)$ bytes. Consider an $(m, k) \cdot (k, n)$ matmul with tile sizes bm, bk, bn. Let $tm = m/bm$, etc. Then the total FLOPs is $2 \cdot tm \cdot tn \cdot tk \cdot bm \cdot bn \cdot bk$ and the total bytes are $2 \cdot tm \cdot tn \cdot (tk \cdot (bm \cdot bk + bk \cdot bn) + bm \cdot bn)$. Ignoring the last term, we have an intensity of $bm \cdot bn/(bm + bn)$, which is similar to the above.

[18] We're ignoring the FLOPs required to add the two partial sums together (another BF addition), but this is basically negligible.

$T_{\text{math}} = \frac{2BDF}{2 \cdot \text{Accelerator FLOPs/s}} = \frac{BDF}{1.97e14}$

Now what about T_{comms}? This now refers to the communication time between chips! This is just the total bytes sent divided by the network bandwidth, i.e.

$T_{\text{comms}} = \frac{2BF}{\text{Network Bandwidth}} = \frac{2BF}{4.5e10}$

Therefore we become compute-bound (now with respect to the inter-chip network) when Intensity(matmul (2-chips)) > Intensity(TPU w.r.t. inter-chip network) or equivalently when $\frac{BDF}{2BF} = \frac{D}{2} > \frac{1.97e14}{4.5e10} = 4377$ or $D > 8755$. Note that, unlike before, the critical threshold now depends on D and not B! Try to think why that is. This is just one such example, but we highlight that this kind of roofline is critical to knowing when we can parallelize an operation across multiple TPUs.

A Few Problems to Work

Question 1 [int8 matmul]: Say we want to do the matmul $X[B, D] \cdot_D Y[D, F] \to Z[B, F]$[19] in int8 precision (1 byte per parameter) instead of bfloat16 (2 bytes per parameter) since TPUs/GPUs can do matmuls faster in lower precision.

1. How many bytes need to be loaded from memory? How many need to be written back to memory?
2. How many total OPs are performed?
3. What is the arithmetic intensity?
4. What is a roofline estimate for T_{math} and T_{comms}? What are reasonable upper and lower bounds for the runtime of the whole operation?

Assume our HBM bandwidth is `8.2e11` bytes/s and our int8 peak OPs/s is `3.94e14` (about 2x bfloat16).

Click here for the answer.

1. Because we're storing our parameters in int8, we have 1 byte per parameter, so we have $BD + DF$ bytes loaded from HBM and BF written back.
2. This is the same as in bfloat16, but in theory int8 OPs/s should be faster. So this is still $2BDF$ OPs.
3. Arithmetic intensity is $2BDF/(BD + DF + BF)$. If we make the same assumption as above about $B \ll D$ and $B \ll F$, we get an arithmetic intensity of $2B$, meaning our rule becomes $B >$ HBM int8 arithmetic intensity/2. Using the numbers given, this int8 intensity is `3.94e14 / 8.2e11 = 480`, so the rule is $B > 480/2 = 240$. Note that this is basically unchanged!
4. $T_{\text{math}} = 2BDF/3.94e14$ and $T_{\text{comms}} = (BD + DF + BF)/8.2e11$, so a reasonable lower bound is $\max(T_{\text{math}}, T_{\text{comms}})$ and an upper bound is $T_{\text{math}} + T_{\text{comms}}$.

Question 2 [int8 + bf16 matmul]: In practice we often do different weight vs. activation quantization, so we might store our weights in very low precision but keep activations (and compute) in a higher precision. Say we want to quantize our weights in int8 but keep activations (and compute) in bfloat16. At what batch size do we become compute bound? Assume `1.97e14` bfloat16 FLOPs/s.

Hint: this means specifically `bf16[B, D] * int8[D, F] -> bf16[B, F]` *where B is the "batch size".*

Click here for the answer.

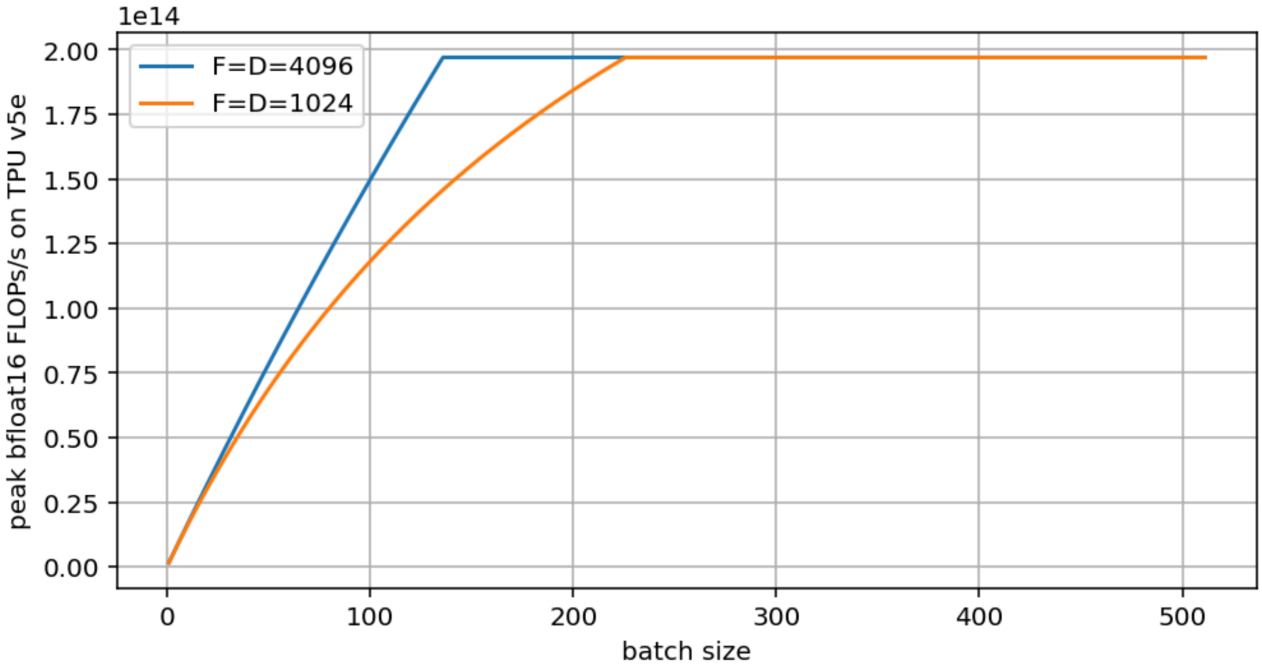
Again assuming B is small, we have 2BDF bfloat16 FLOPs but only DF weights (instead of 2DF in bfloat16). This means we become compute-bound when $2B > 240$ or $B > 120$. This is a lot lower, meaning if we can do int8 weight quantization (which is fairly easy to do) but still do bfloat16 FLOPs, we get a meaningful win in efficiency (although int8 OPs would be better).

Question 3: Taking the setup from Question 2, make a roofline plot of peak FLOPs/s vs. B for $F = D = 4096$ and $F = D = 1024$. *Use the exact number of bytes loaded, not an approximation.*

Click here for the answer.

Here is the plot in question:

[19] Here and throughout we'll use the notation $A \cdot_D B$ to indicate that the multiplication is performing a contraction over the D dimension. This is an abuse of einsum notation.



Note that both models eventually achieve the peak hardware FLOPs/s, but the larger D/F achieve it sooner. D=F=1024 almost doubles the critical batch size. The code to generate this figure is here:

```
import matplotlib.pyplot as plt
import numpy as np

bs = np.arange(1, 512)

def roofline(B, D, F):
  total_flops = 2*B*D*F
  flops_time = total_flops / 1.97e14
  comms_time = (2*B*D + D*F + 2*B*F) / 8.2e11
  total_time = np.maximum(flops_time, comms_time)
  return total_flops / total_time

roofline_big = roofline(bs, 4096, 4096)
roofline_small = roofline(bs, 1024, 1024)

plt.figure(figsize=(8, 4))
plt.plot(bs, roofline_big, label='F=D=4096')
plt.plot(bs, roofline_small, label='F=D=1024')
plt.legend()
plt.xlabel('batch size')
plt.ylabel('peak bfloat16 FLOPs/s on TPU v5e')
plt.grid()
```

Question 4: What if we wanted to perform $\text{int8}[B, D] \cdot_D \text{int8}[B, D, F] \to \text{int8}[B, F]$ where we imagine having a different matrix for each batch element. What is the arithmetic intensity of this operation?

Click here for the answer.

Let's start by looking at the total FLOPs and comms.

1. Total FLOPs: the FLOPs is basically the same, since we're doing B independent $[D] \times [D, F]$ products, the same total work as a single $[B, D] \times [D, F]$ matmul (this is discussed more in Section 4). So this is just $2BDF$.

2. Total comms: we have a lot more comms here: $BD + BDF + BF$.
3. Therefore, our arithmetic intensity is now actually $2BDF/(BD + BDF + BF)$. Since BDF dominates the denominator, this is roughly 2. So instead of it depending on the batch size, this is essentially constant. This is bad because it means we'll basically always be comms bound no matter what.

Question 5 [Memory Rooflines for GPUs]: Using the spec sheet provided by NVIDIA for the H100 SXM, calculate the batch size at which a bfloat16 matrix multiplication will become compute-bound. *Note that the Tensor Core FLOPs numbers are twice the true value since they're only achievable with structured sparsity.*

Click here for the answer.

From the spec sheet, we see that the reported bfloat16 FLOPs value is `1.979e15` FLOPs/s with an asterisk noting "with sparsity". The true value is half this without sparsity, i.e. `9.89e14` FLOPs/s. The memory bandwidth is 3.35TB/s, or `3.35e12` bytes / second. Thus B_{crit} is `9.89e14 / 3.35e12 = 295`, rather similar to the TPU.

That's it for Part 1! For Part 2, looking at how real TPUs handle FLOPs and communication, click here.

Chapter 2

How to Think About TPUs

You might also enjoy reading the new Section 12 on NVIDIA GPUs!

What Is a TPU?

A TPU is basically a compute core that specializes in matrix multiplication (called a TensorCore) attached to a stack of fast memory (called high-bandwidth memory or HBM). Here's a diagram:

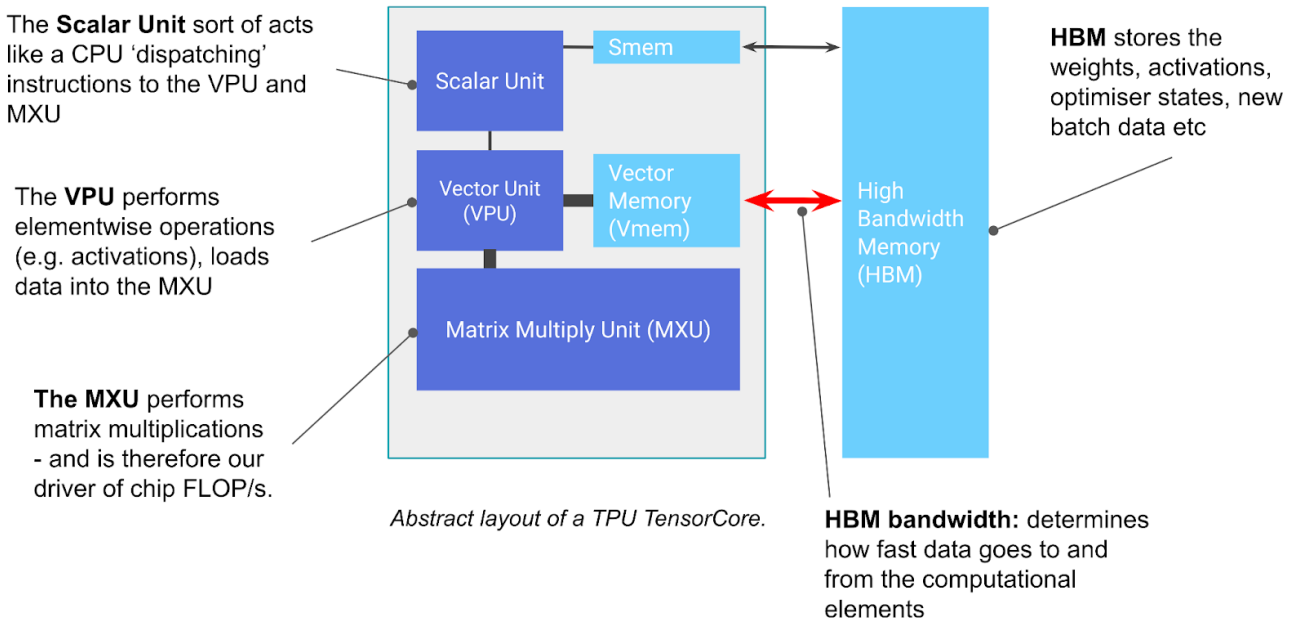


Figure 4: Figure: the basic components of a TPU chip. The TensorCore is the gray left-hand box, containing the matrix-multiply unit (MXU), vector unit (VPU), and vector memory (VMEM).

You can think of the TensorCore as basically just being a really good matrix multiplication machine, but it has a few other functions worth noting. The TensorCore has three key units:

- The **MXU** (Matrix Multiply Unit) is the core of the TensorCore. For most TPU generations, it performs one `bf16[8,128] @ bf16[128,128] -> f32[8,128]` matrix multiply[20] every 8 cycles using a systolic array (see Appendix B for details).
  - This is about `5e13` bf16 FLOPs/s per MXU at 1.5GHz on TPU v5e. Most TensorCores have 2 or 4 MXUs, so e.g. the total bf16 FLOPs/s for TPU v5e is `2e14`.
  - TPUs also support lower precision matmuls with higher throughput (e.g. each TPU v5e chip can do `4e14` int8 OPs/s).

[20]TPU v6e (Trillium) has a 256x256 MXU, while all previous generations use 128x128.

- The **VPU** (Vector Processing Unit) performs general mathematical operations like ReLU activations or pointwise addition or multiplication between vectors. Reductions (sums) are also performed here. Appendix A provides more details.
- **VMEM** (Vector Memory) is an on-chip scratchpad located in the TensorCore, close to the compute units. It is much smaller than HBM (for example, 128 MiB on TPU v5e) but has a much higher bandwidth to the MXU. VMEM operates somewhat like an L1/L2 cache on CPUs but is much larger and programmer-controlled. Data in HBM needs to be copied into VMEM before the TensorCore can do any computation with it.

TPUs are very, very fast at matrix multiplication. It's mainly what they do and they do it well. TPU v5p, one of the most powerful TPUs to date, can do `2.5e14` bf16 FLOPs / second / core or `5e14` bf16 FLOPs / sec / chip. A single pod of 8960 chips can do 4 bf16 exaFLOPs/s. That's *a lot*. That's one of the most powerful supercomputers in the world. And Google has a lot of them.[21]

The diagram above also includes a few other components like SMEM and the scalar unit, which are used for control flow handling and are discussed briefly in Appendix A, but aren't crucial to understand. On the other hand, HBM is important and fairly simple:

- **HBM** (High Bandwidth Memory) is a big chunk of fast memory that stores tensors for use by the TensorCore. HBM usually has capacity on the order of tens of gigabytes (for example, TPU v5e has 16GiB of HBM).
  - When needed for a computation, tensors are streamed out of HBM through VMEM (see below) into the MXU and the result is written from VMEM back to HBM.
  - The bandwidth between HBM and the TensorCore (through VMEM) is known as "HBM bandwidth" (usually around 1-2TB/sec) and limits how fast computation can be done in memory-bound workloads.

Generally, all TPU operations are pipelined and overlapped. To perform a matmul $X \cdot A \to Y$, a TPU would first need to copy chunks of matrices A and X from HBM into VMEM, then load them into the MXU which multiplies chunks of 8x128 (for X) and 128x128 (for A), then copy the result chunk by chunk back to HBM. To do this efficiently, the matmul is pipelined so the copies to/from VMEM are overlapped with the MXU work. This allows the MXU to continue working instead of waiting on memory transfers, keeping matmuls compute-bound, not memory-bound.

Here's an example of how you might perform an elementwise product from HBM:

A matmul would look nearly identical except it would load into the MXU instead of the VPU/Vector unit, and the loads and stores would occur in a different order, since the same weight chunk is used for multiple chunks of activations. You can see chunks of data streaming into VMEM, then into the VREGs (vector registers), then into the Vector Unit, then back into VMEM and HBM. As we're about to see, if the load from HBM to VMEM is slower than the FLOPs in the Vector Unit (or MXU), we become "bandwidth bound" since we're starving the VPU or MXU of work.

Key takeaway: TPUs are very simple. They load weights from HBM into VMEM, then from VMEM into a systolic array which can perform around 200 trillion multiply-adds per second. The HBM $\leftrightarrow$ VMEM and VMEM $\leftrightarrow$ systolic array bandwidths set fundamental limits on what computations TPUs can do efficiently.

VMEM and arithmetic intensity: VMEM is much smaller than HBM but it has a much higher bandwidth to the MXU. As we saw in Section 1, this means if an algorithm can fit all its inputs/outputs in VMEM, it's much less likely to hit communication bottlenecks. This is particularly helpful when a computation has poor arithmetic intensity: VMEM bandwidth is around 22x higher than HBM bandwidth which means an MXU operation reading from/writing to VMEM requires an arithmetic intensity of only 10-20 to achieve peak FLOPs utilization. That means if we can fit our weights into VMEM instead of HBM, our matrix multiplications can be FLOPs bound at much smaller batch sizes. And it means algorithms that fundamentally have a lower arithmetic intensity can still be efficient. VMEM is just so small this is often a challenge.[22]

[21] TPUs, and their systolic arrays in particular, are such powerful hardware accelerators because matrix multiplication is one of the few algorithms that uses $O(n^3)$ compute for $O(n^2)$ bytes. That makes it very easy for an ordinary ALU to be bottlenecked by compute and not by memory bandwidth.

[22] We sometimes talk about VMEM prefetching, which refers to loading weights ahead of time in VMEM so we can mask the cost of loading for our matmuls. For instance, in a normal Transformer we can sometimes load our big feed-forward weights into VMEM during attention, which can hide the cost of the weight load if we're memory bandwidth bound. This requires our weights to be small

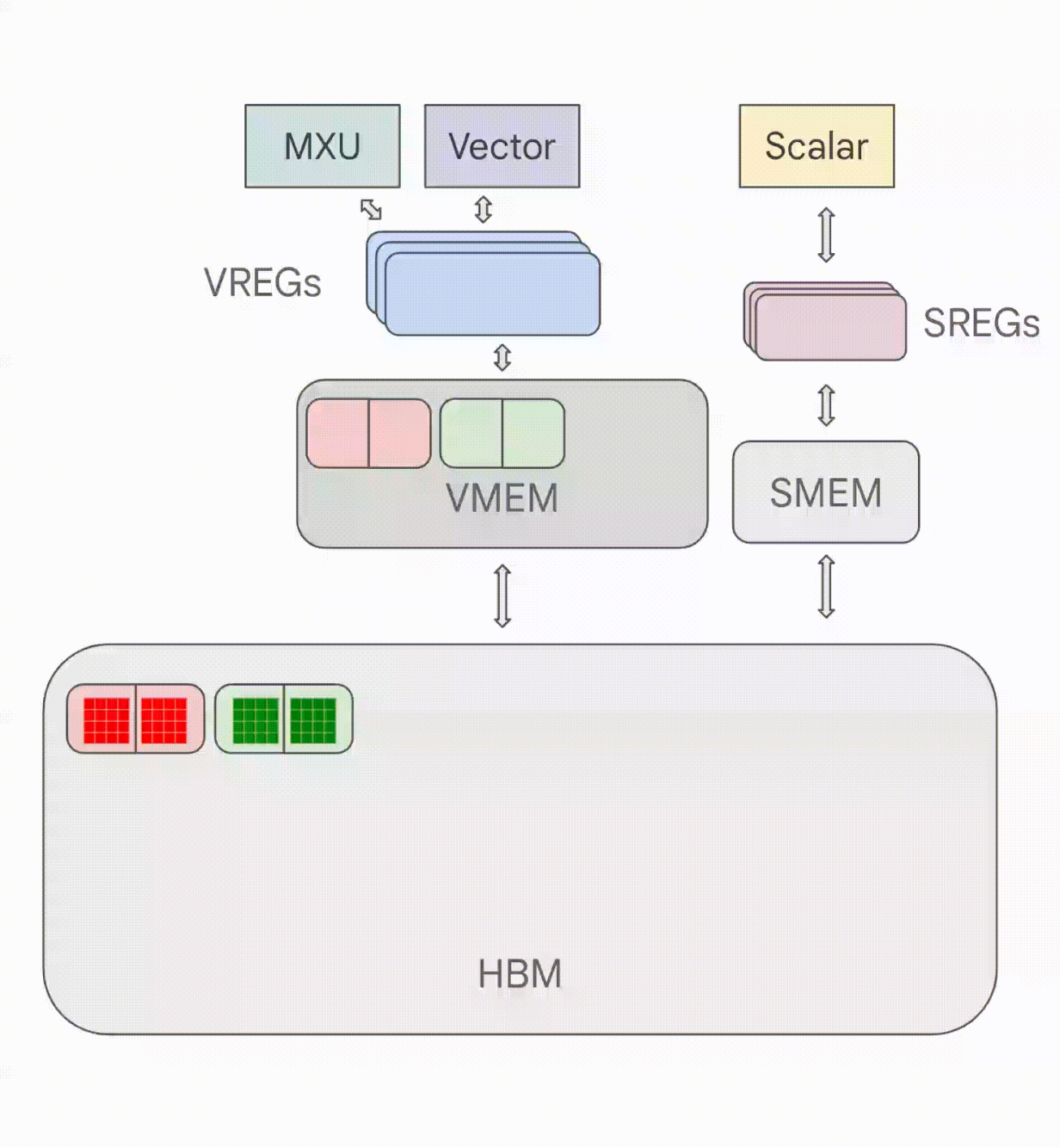
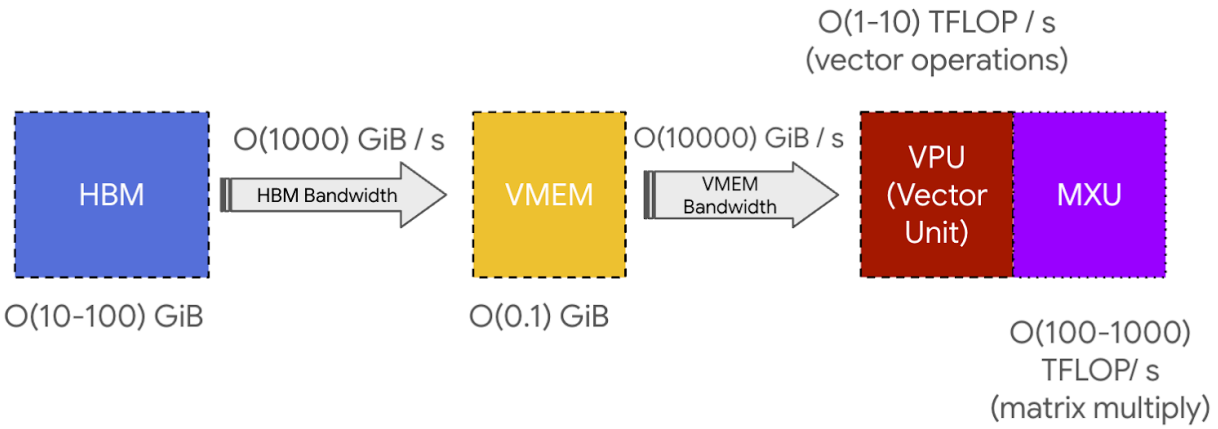
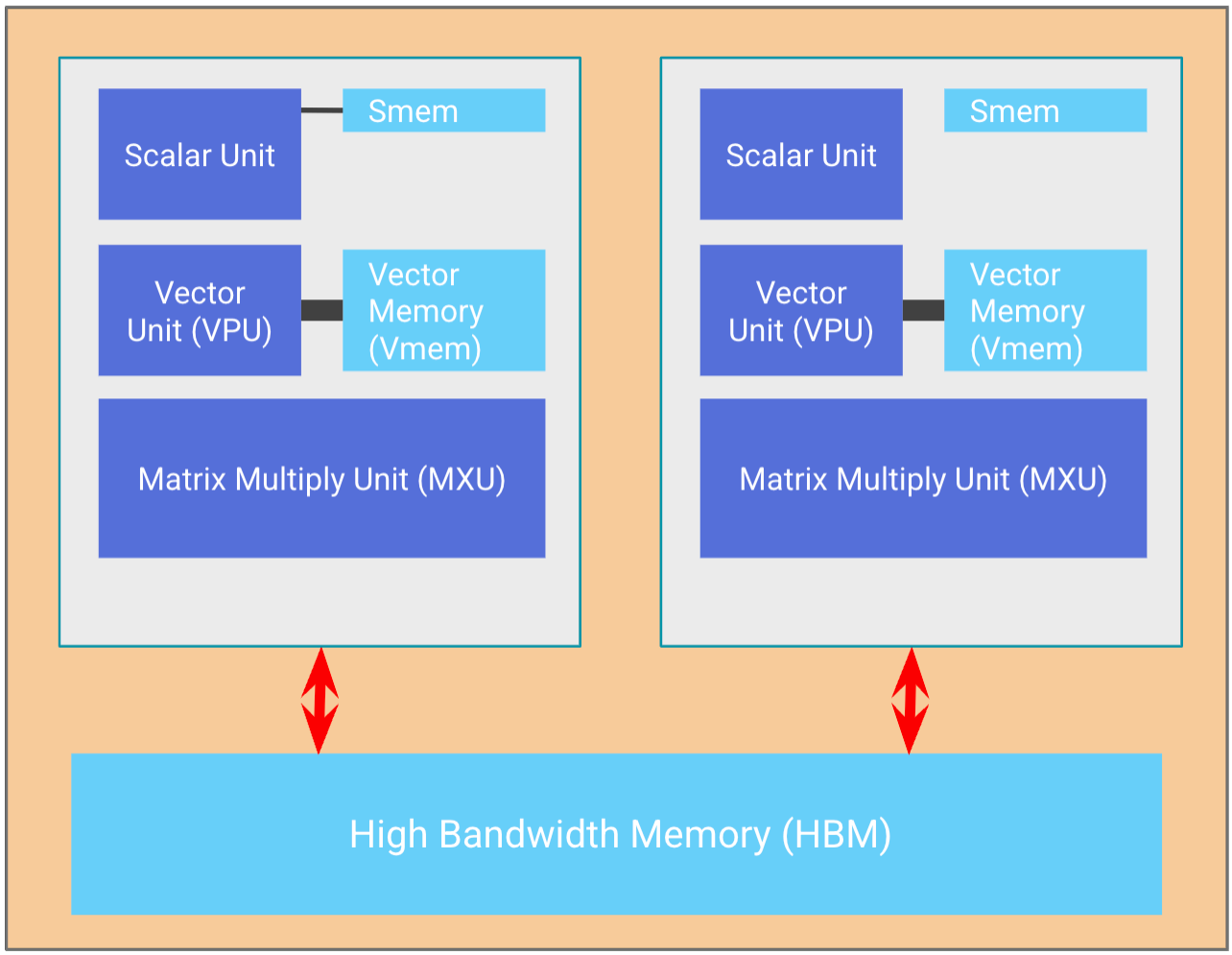


Figure 5: Figure: an animation showing a pointwise product performed on TPU, with bytes loaded from HBM. Note how bytes are streamed out of memory in chunks and partial results are pipelined back without waiting for the full array to be materialized.

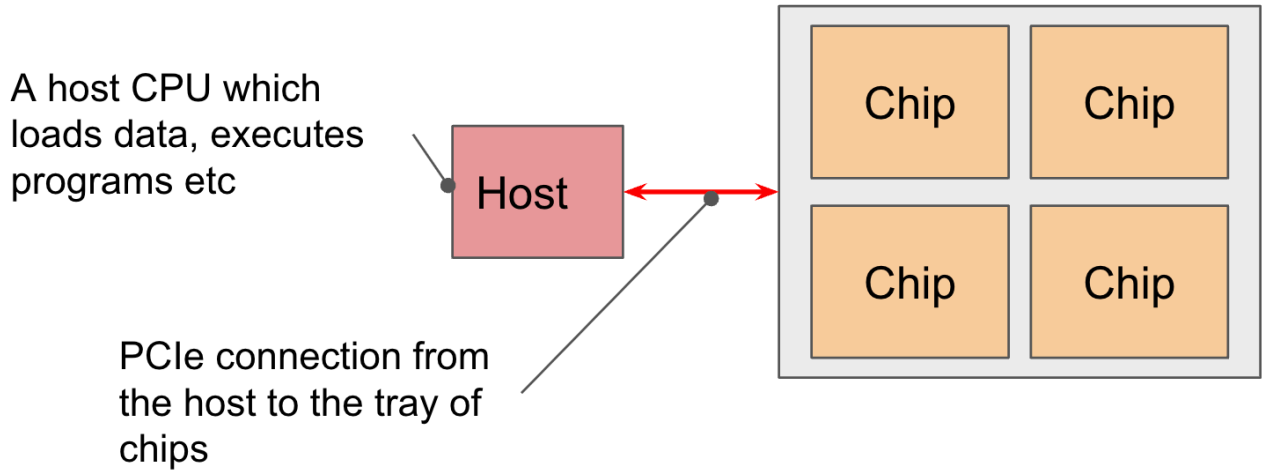


A TPU chip typically (but not always) consists of two TPU cores which share memory and can be thought of as one large accelerator with twice the FLOPs (known as a "megacore" configuration). This is true for v4, v5, and v6 TPUs (TPU v7 removes megacore and instead has a high-bandwidth link between the two cores). Older TPU chips have separate memory and are regarded as two separate accelerators (TPU v3 and older). Inference-optimized chips like the TPU v5e only have one TPU core per chip.



enough or sharded enough to fit a single layer into VMEM with space to spare.

Chips are arranged in **sets of 4 on a 'tray'** connected to a **CPU host via PCIe network.** This is the format most readers will be familiar with, 4 chips (8 cores, though usually treated as 4 logical megacores) exposed through Colab or a single TPU-VM. For inference chips like the TPU v5e, we have 2 trays per host, instead of 1, but also only 1 core per chip, giving us 8 chips = 8 cores.[23]

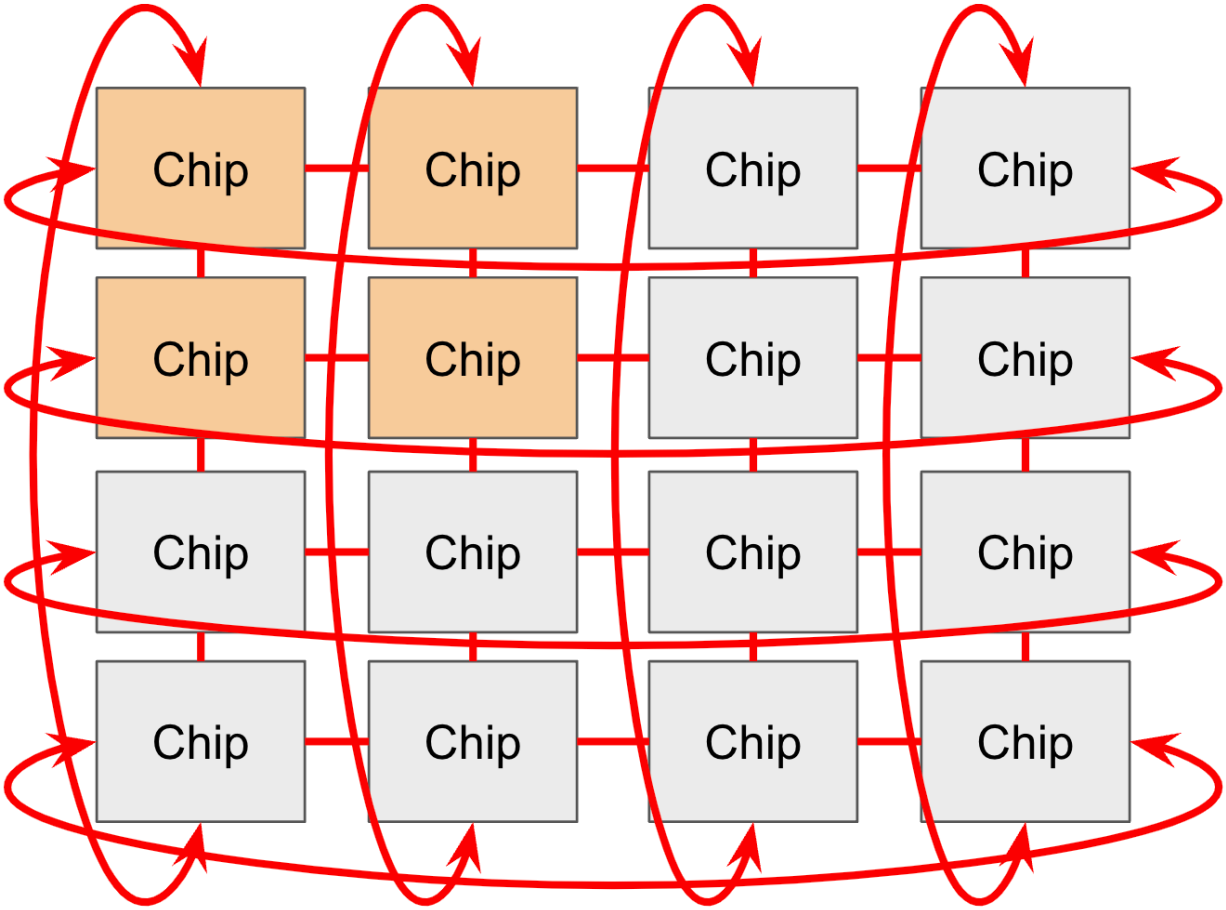


PCIe bandwidth is limited: Like the HBM ↔ VMEM link, the CPU ↔ HBM PCIe connection has a specific bandwidth that limits how quickly you can load from host memory to HBM or vice-versa. PCIe bandwidth for TPU v4 is 16GB / second each way, for example, so close to 100x slower than HBM. We *can* load/offload data into the host (CPU) RAM, but not very quickly.

TPU Networking

Chips are connected to each other through the ICI network in a Pod. In older generations (TPU v2 and TPU v3), inference chips (e.g., TPU v5e), and Trillium (TPU v6e), ICI ("inter-chip interconnects") connects the 4 nearest neighbors (with edge links to form a 2D torus). TPU v4 and TPU v5p are connected to the nearest 6 neighbors (forming a 3D torus). Note these connections do **not** go through their hosts, they are direct links between chips.

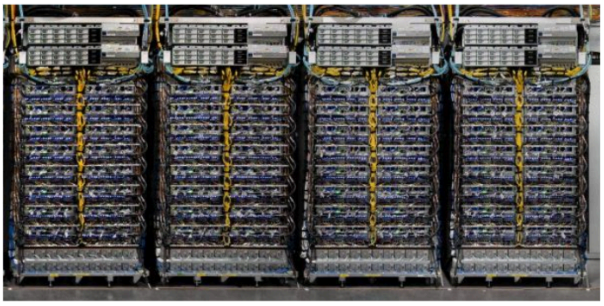
[23] On Cloud TPU VMs, each tray is exposed as part of a separate VM, so there are once again 4 cores visible.



The toroidal structure reduces the maximum distance between any two nodes from N to $N/2$, making communication much faster. TPUs also have a "twisted torus" configuration that wraps the torus in a Mobius-strip like topology to further reduce the average distance between nodes.

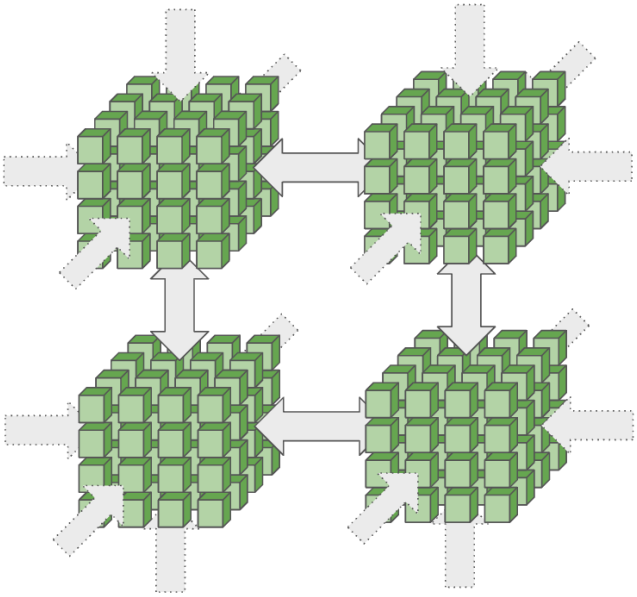
TPU pods (connected by ICI) can get really big: the maximum pod size (called a **superpod**) is `16x16x16` for TPU v4 and `16x20x28` for TPU v5p. These large pods are composed of reconfigurable cubes of `4x4x4` chips connected by optical wraparound links[24] that we can reconfigure to connect very large topologies.

[24]The optical switch is simply a reconfigurable connection with the same ICI bandwidth. It just lets us connect cubes while retaining a wraparound link.

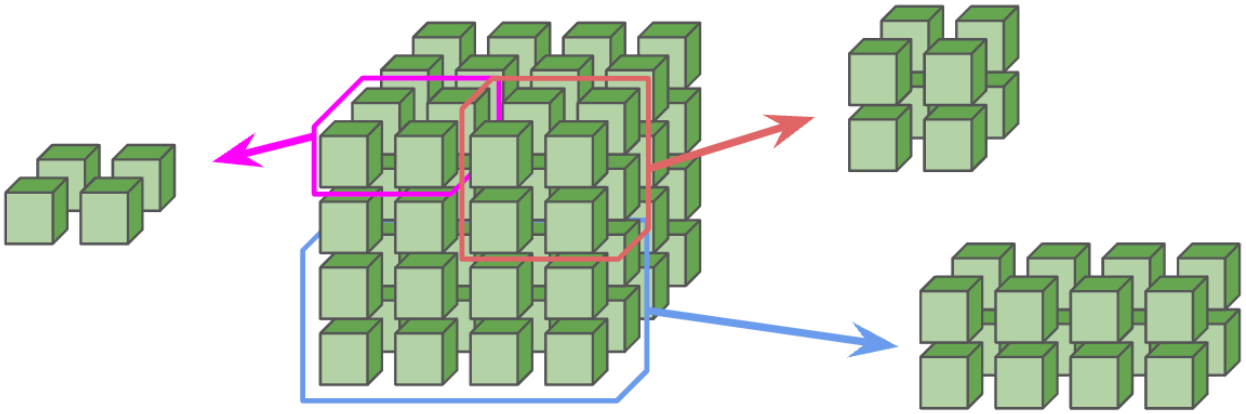


Each rack corresponds to 4x4x4 TPUv4s

These can be configured into arbitrary
4ix4jx4k toruses via optical interconnects



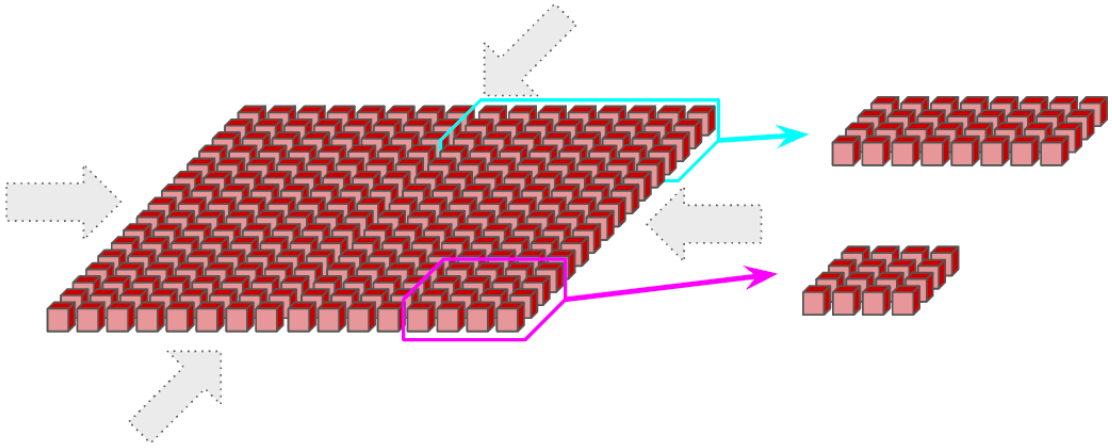
Smaller topologies (e.g. `2x2x1`, `2x2x2`) can also be requested, albeit with no wraparounds. This is an important caveat, since it typically doubles the time of most communication. Any multiple of a full cube (e.g. `4x4x4` or `4x4x8`) will have wraparounds provided by the optical switches.[25]



Smaller slices can also be requested.

TPU v5e and Trillium pods consist of a single `16x16` 2D torus with wraparounds along any axis of size 16 (meaning an `8x16` has a wraparound on the long axis). TPUs v5e and v6e (Trillium) cannot expand beyond a 16x16 torus but pods can still communicate with each other over standard data-center networking (DCN), which connects TPU hosts to each other. Again, smaller topologies can be requested without wraps on dims < 16.

[25] Note that a `2x2x4` won't have any wraparounds since they are provided by the optical switches which are only available on a full cube. A TPU v5e 8x16 *will* have a wraparound on the longer axis, however, since it doesn't use reconfigurable optical networking.



This nearest-neighbor connectivity is a key difference between TPUs and GPUs. GPUs are connected with a hierarchy of switches that approximate a point-to-point connection between every GPU, rather than using local connections like a TPU. Typically, GPUs within a node (8 GPUs for H100 or as many as 72 for B200 NVL72) are directly connected, while larger topologies require O(log(N)) hops between each GPU. On the one hand, that means GPUs can send arbitrary data within a small number of hops. On the other hand, TPUs are dramatically cheaper (since NVLink switches are expensive), simpler to wire together, and can scale to much larger topologies because the number of links per device and the bandwidth per device is constant. Read more here.

ICI is very fast relative to DCN, but is still slower than HBM bandwidth. For instance, a TPU v5p has:

- `2.8e12` bytes/s (2.8 TB/s) of HBM bandwidth per chip.
- `9e10` bytes/s (90 GB/s) of ICI bandwidth per axis, with 3 axes per chip.[26]
- `6.25e9` bytes/s (6.25 GB/s) of DCN (egress) bandwidth per TPU (via 1-2 NICs on each host).[27]

This means that when we split models across multiple chips, we need to be careful to avoid bottlenecking the MXU with slower cross-device communication.

Multi-slice training: A set of ICI-connected TPUs is called a **slice**. Different slices can be connected between each other using DCN, for instance to link slices on different pods. Since DCN is a much slower connection than ICI, we should try to limit how much our computation has to wait for data from DCN. DCN is host-to-host, so to transfer buffers from TPU to TPU over DCN, we first need to transfer over PCIe to the host, then egress over the network, then ingress over the target host network, then over PCIe into HBM.

Key Takeaways

- TPUs are simple and can in most cases be thought of as a matrix multiply unit connected to memory (super fast), other chips over ICI (rather fast), and the rest of the datacenter over DCN (somewhat fast).
- Communication is limited by our various network bandwidths in order of speed:
  - HBM bandwidth: Between a TensorCore and its associated HBM.
  - ICI bandwidth: Between a TPU chip and its nearest 4 or 6 neighbors.
  - PCIe bandwidth: Between a CPU host and its associated tray(s) of chips.
  - DCN bandwidth: Between multiple CPU hosts, typically hosts not connected by ICI.

[26] The page above lists 100 GB/s of bandwidth, which is slightly different from what's listed here. TPU ICI links have slightly different bandwidths depending on the operation being performed. You can generally use the numbers in this doc without worry.

[27] TPU v6e and TPU7x have 12.5e9 bytes/s and v5e has 3.125e9 bytes/s.

- **Within a slice, TPUs are only connected to their nearest neighbors via ICI.** This means communication over ICI between distant chips in a slice needs to hop over the intervening chips first.
- **Weight matrices need to be padded to at least size 128** (256 on TPU v6e) in both dimensions to fill up the MXU (in fact, smaller axes are padded to 128).
- **Lower precision matrix multiplication tends to be faster.** TPUs can do int8 or int4 OPs roughly 2x/4x faster than bfloat16 FLOPs for generations that support it. VPU operations are still performed in fp32.
- To avoid bottlenecking the TPU compute unit, we need to **make sure the amount of communication across each channel is proportional to its speed**.

TPU specs

Here are some specific numbers for our chips:

Model	Pod size	Host size	HBM capacity/chip	HBM BW/chip (bytes/s)	FLOPs/s/chip (bf16)	FLOPs/s/chip (int8)
TPU v3	32x32	4x2	32GB	9.0e11	1.4e14	1.4e14
TPU v4p	16x16x16	2x2x1	32GB	1.2e12	2.75e14	2.75e14
TPU v5p	16x20x28	2x2x1	96GB	2.8e12	4.59e14	9.18e14
TPU v5e	16x16	4x2	16GB	8.2e11	1.97e14	3.94e14
TPU v6e	16x16	4x2	32GB	1.6e12	9.20e14	1.84e15
TPU7x	4x4x576	2x2x1	192GB	7.4e12	2.30e15	4.61e15

Host size refers to the topology of TPUs connected to a single host (e.g. TPU v5e has a single CPU host connected to 8 TPUs in a 4x2 topology). See the TPU7x documentation for more details on the latest generation. Here are interconnect figures:

Model	ICI BW/link (one-way, bytes/s)	ICI BW/link (bidi, bytes/s)
TPU v3	1.0e11	2.0e11
TPU v4p	4.5e10	9.0e10
TPU v5p	9.0e10	1.8e11
TPU v5e	4.5e10	9.0e10
TPU v6e	9.0e10	1.8e11
TPU7x	9.0e10	1.8e11

We include both one-way (unidirectional) bandwidth and bidi (bidirectional) bandwidth since unidirectional bandwidth is more true to the hardware but bidirectional bandwidth occurs more often in equations involving a full ring.[28]

PCIe bandwidth is typically around `1.6e10` bytes / second per TPU (`3.2e10` for TPU v6e), while DCN bandwidth is typically around `6.25e9` bytes / second per TPU (`12.5e9` for TPU v6e and TPU7x, and `3.125e9` for TPU v5e).

[28] By bidi (bidirectional) bandwidth we mean the total bytes that can be sent along a single link in both directions, or equally, the total number of outgoing bytes from a single TPU along a particular axis, assuming we can use both links efficiently. This is true when we have a functioning ring, AKA when we have a wraparound connection on the particular axis. This occurs on inference chips when we have a full 16 axis, or on training chips (v*p) when we have an axis which is a multiple of 4. We prefer to use the bidirectional bandwidth because it appears frequently in calculations involving bidirectional comms.

Worked Problems

These numbers are a little dry, but they let you make basic roofline estimates for model performance. Let's work a few problems to explain why this is useful. You'll see more examples in Part 3.

Question 1 [bounding LLM latency]: Say you want to sample from a 200B parameter model in bf16 that's split across 32 TPU v4p. How long would it take to load all the parameters from HBM into the systolic array? *Hint: use the numbers above.*

Click here for the answer.

Answer: We're loading `sizeof(bf16) * 200e9 = 400e9` bytes on 32 chips, meaning 12.5e9 bytes / chip, each with an HBM bandwidth of 1.23e12. So the load takes around 10ms.

That's pretty cool, because *that's a reasonable lower bound on the latency of sampling* from the model. Each sampling step needs to load all parameters from HBM, so it cannot take less than 10 ms. In practice, at small batch sizes, this is close to being achievable.

Question 2 [TPU details]: Consider a full TPU v5e pod. How many total CPU hosts are there? How many TPU TensorCores? What is the total FLOPs/s for the whole pod? What is the total HBM? Do the same exercise for TPU v5p pod.

Click here for the answer.

Answer: For TPU v5e, each pod is `16x16` and each host is a 4x2 slice, so we have `16*16 / 8 = 32` hosts. For TPU v5e, each TPU has only one core, so we have 256 TensorCores. The total FLOPs/s is `16*16*2e14 = 5.1e16` in bfloat16. Each chip has 16GB of HBM, so that's `256 * 16 = 4TB` of memory.

For a full TPU v5p pod, we have `16x20x28` chips and each host is 2x2x1, so we have `(16*20*28) / (2*2) = 2,240` hosts. For TPU v5p, each TPU has two TensorCores, so we have `8960 * 2 = 17,920` cores. The total FLOPs/s is `8960 * 4.59e14 = 4.1e18` in bfloat16. Each chip has 96GB of HBM, so that's `8960 * 96 = 860TB` of memory.

Question 3 [PCIe operational intensity]: Imagine we're forced to store a big weight matrix A of type bf16$[D, F]$, and a batch of activations x of type bf16$[B, D]$ in host DRAM and want to do a matrix multiplication on them. This is running on a single host, and we're using a single TPU v6e chip attached to it. You can assume $B \ll D$, and $F = 4D$ (we'll see in future chapters why these are reasonable assumptions). What is the smallest batch size B we need to remain FLOPs bound over PCIe? Assume PCIe bandwidth of 1.6e10 bytes / second.

Click here for the answer.

Answer: We have to perform $2BDF$ floating point operations, and each chip can perform `9.2e14` floating point operations per second. This then requires $2BDF/9.2e14$ seconds to perform. We have to load $2DF + 2BD$ bytes from DRAM, and write $2BF$ bytes back to it. We are bottlenecked by PCIe transfer speeds, so we need $2 \cdot (BD + DF + BF)/1.6e10$ seconds to transfer data to and from the TPU. Since we want computation to take longer than weight loading, assuming we can overlap all weight loading with computation, we want $2BDF/9.2e14 > 2 \cdot (BD + DF + BF)/1.6e10$. We can simplify this using our assumptions that $B \ll D$, and $F = 4D$, to get

$\frac{8BD^2}{9.2\times10^{14}} > \frac{8D^2}{1.6\times10^{10}}$

or

$B > \frac{9.2\times10^{14}}{1.6\times10^{10}} \simeq 57{,}500$

Question 4 [general matmul latency]: Let's say we want to multiply a weight matrix int8[16384, 4096] by an activation matrix of size int8[B, 4096] where B is some unknown batch size. Let's say we're on 1 TPU v5e to start.

1. How long will this multiplication take as a function of B? *Hint: it may help to calculate how long it will take to load the arrays from HBM and how long the multiplication will actually take. Which is bottlenecking you?*
2. What if we wanted to run this operation out of VMEM? How long would it take as a function of B?

Click here for the answer.

Answer: (1) The number of operations we need to perform is $2 \cdot 4096 \cdot 16384 \cdot B = 1.3 \times 10^8 \cdot B$. So $T_{\text{math}} = (1.3 \times 10^8 \cdot B)/3.94 \times 10^{14}$ seconds. We need to load $16384 \cdot 4096 + 4096 \cdot B$ bytes from HBM to VMEM, and write back $16384 \cdot B$ bytes from VMEM to HBM. This means $T_{\text{comms}} = (6.7 \times 10^7 + 2 \times 10^4 \cdot B)/8.2 \times 10^{11}$ seconds. Assuming as much overlap of communication and computation as possible, the whole multiplication will take approximately

$$\max\{T_{\text{math}}, T_{\text{comms}}\} = \max\left\{\frac{1.3\times10^8\cdot B}{3.94\times10^{14}}, \frac{6.7\times10^7+2\times10^4\cdot B}{8.2\times10^{11}}\right\}$$

We'll be FLOPs-bound when $\frac{6.7\times10^7+2\times10^4\cdot B}{8.2\times10^{11}} < \frac{1.3\times10^8\cdot B}{3.94\times10^{14}}$, or equivalently, $B > 267$. This is slightly larger than the 240 number we derive in Section 1 because we factor in the full impact of D and F.

(2) If instead we are loading from VMEM, let's consider VMEM bandwidth to the MXU as 22 times the HBM $\leftrightarrow$ VMEM bandwidth. This turns our data loading denominator from 8.2e11 to 1.80e13, and we get $B > 11$. Note that in practice, we cannot dedicate all of our VMEM bandwidth to loading the weight matrix, so in practice it will be closer to 20.

Question 5 [ICI bandwidth]: Let's say we have a TPU v5e `4x4` slice. Let's say we want to send an array of type `bf16[8, 128, 8192]` from `TPU{0,0}` to `TPU{3, 3}`. Let's say the per-hop latency for TPU v5e is $1\mu s$.

1. How soon will the first byte arrive at its destination?
2. How long will the total transfer take?

Click here for the answer.

Answer: In a TPU v5e we have 2D connectivity. Because we have only a `4x4` slice (with no axes of size 16), we have no wraparound connections. Thus there are two ports from which our target chip can receive data, and likewise two ports from which our source chip can send data. The amount of data we have to transfer is `2 * 8 * 128 * 8192 = 1.7e7` bytes. We can transfer from both ports simultaneously (i.e. send half the array right and half down), so we get `2 * 4.5e10 = 9e10` bytes transferred per second, which means it'll take about `1.7e7 / 9e10 = 188us` to transfer the whole array through (assuming we're bandwidth bound). In a `4x4` slice, we have six hops between chips $(0, 0)$ and $(3, 3)$, since there are no wraparound links for axes with fewer than 16 chips. Since the latency of each hop is about $1\mu s$, the first byte will arrive in about `6us` and the total transfer will take about `188 + 6 = 194us`, since the last byte must likewise traverse the six hops after it leaves the source (in general, the latency and bandwidth terms are additive, though here the latency is a small correction).

Question 6 [pulling it all together, hard]: Imagine you have a big matrix **A**: `int8[128 * 1024, 128 * 1024]` sharded evenly across a TPU v5e 4x4 slice but offloaded to host DRAM on each chip. Let's say you want to copy the entire array to TPU{0, 0} and multiply it by a vector `bf16[8, 128 * 1024]`. How long will this take? *Hint: use the numbers above.*

Click here for the answer.

Answer: Let's start by outlining the operations we have to perform. Our array is about 16GB. From the table above, a TPU v5e host has a 4x2 topology, so a 4x4 has 2 hosts. Thus, since our array is evenly sharded, each host effectively contains a chunk of 1/2 of the array, or 8GB. We need to copy these chunks all to TPU{0,0}, which gives us two options:

1. We can copy over DCN and then load the entire unsharded array over PCIe into HBM.
2. We can load our sharded arrays onto their corresponding TPUs, then perform a gather over ICI, then perform the matmul on TPU{0,0}.

It should be clear that option (2) is better. DCN is slow compared to ICI and we'd much prefer to load a big array over many PCIe links rather than just a few (the 8 on host 0). Here's a diagram of part of the system. As described above, note that TPUs are connected to their neighbors by ICI (even across hosts), all TPUs are connected to their host CPU (via PCIe), and hosts are connected by DCN.

Now let's work through how long each piece will take:

1. **PCIe load**: we're loading chunks of 16GB over 16 PCIe links, each of which has `1.6e10` bytes/second bandwidth. Thus this will take about 63ms.

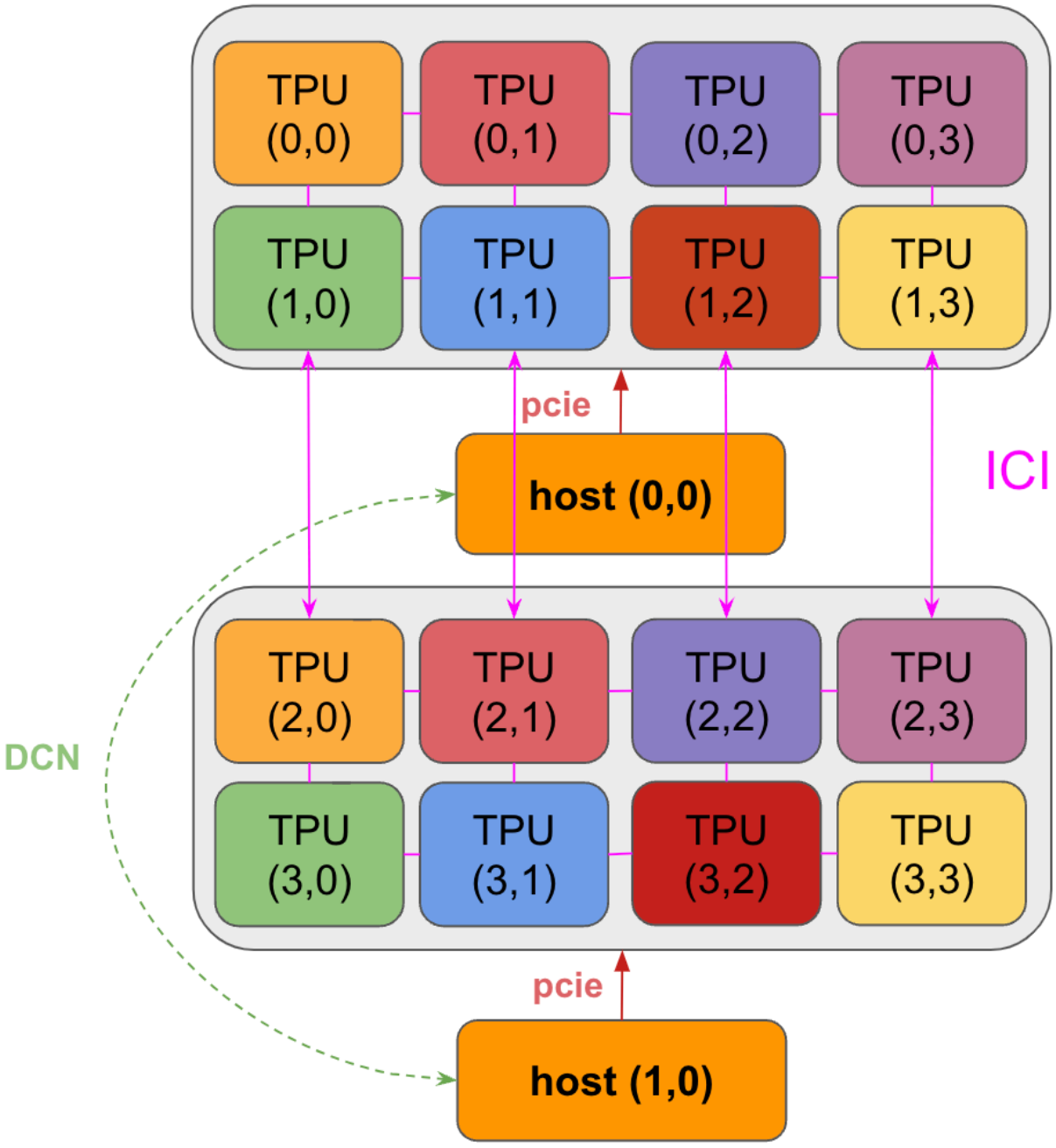


Figure 6: Each chip actually has its own PCIe link to its host, though for clarity only one is shown here.

2. **ICI copy:** each TPU now has 16GB / 16 = 1GB of our array. Our ICI bandwidth is 9e10 bytes/second per link *bidirectional*, and you'll notice from the above diagram that only 2 of the 4 ICI links on the TPU v5e are in use in this topology for TPU{0,0}. Since TPU{0,0} needs to receive a total of 15GB along 2 axes at `4.5e10` bytes/s/link, we can lower bound the time by `15e9 / (4.5e10 * 2) = 167ms`. In practice this probably isn't achievable because the load is very uneven, but it's probably within a factor of 2. As you'll see in Section 3, performing a full AllGather would also take roughly `16e9 / (4.5e10 * 2)`, so this is close to optimal.
3. **HBM → MXU load:** to perform our final matmul, we need to load these 16e9 bytes plus the bf16[8, 128 * 1024] array (another 2MB, so negligible) over HBM bandwidth into the MXU, which will take `16e9 / 8.2e11 = 20ms`.
4. **FLOPs:** we're performing a total of $2 \cdot 8 \cdot 128 \cdot 1024 \cdot 128 \cdot 1024 = 2.7 \times 10^{11}$ FLOPs, and since we can perform `1.97e14` bf16 FLOPs/s, we get 1.4ms.

An upper bound for the total time is the sum of all of these times, but since the TPU can typically overlap these operations, we can think of this as a pipelining problem that's bottlenecked by the slowest piece. Assuming that's true, then the answer is at least 167ms, likely closer to 200ms with imperfect overlapping.

That's it for Part 2! For Part 3, covering partitioning and cross-TPU communication, click here.

Appendix

Appendix A: More on TPU internals

Here we'll dive more deeply into the internal operations of a TPU. Unless otherwise noted, we'll provide specs for a TPU v5p.

VPU

The VPU is the TPU's vector arithmetic core. The VPU consists of a two dimensional SIMD vector machine (the **VPU**) that performs elementwise arithmetic operations like vadd (vector addition) or vmax (elementwise max) and a set of vector registers called **VREGs** that hold data for the VPU and MXU.

VREGs: Each TPU v5p core has 64 32-bit VREGs (32 in TPU v4), giving us a total of about `64 * 8 * 128 * 4 = 256kB` of VREG memory per core (or 2x this for the whole chip since we have two cores). A TPU v5p can load 3 registers from VMEM each cycle, and write 1 register to VMEM each cycle.

VPU: The VPU is a 2D vector arithmetic unit of shape `(8, 128)` where the 128 dimension is referred to as lane axis and the dimension of 8 is referred to as the sublane axis. Each (lane, sublane) pair on v5 contains 4 standard floating-point ALUs which are independent of each other. The VPU executes most arithmetic instructions in one cycle in each of its ALUs (like vadd or vector add) with a latency of 2 cycles, so e.g. in v5 you can add 4 pairs of f32 values together from VREGs in each cycle. A typical VPU instruction might look like `{v2 = vadd.8x128.f32 v0, v1}` where v0 and v1 are input VREGs and v2 is an output VREG.

All lanes and sublanes execute the same program every cycle in a pure SIMD manner, but each ALU can perform a different operation. So we can e.g. process 1 vadd and 1 vsub in a single cycle, each of which operates on two full VREGs and writes the output to a third.

Pop Quiz [Calculating VPU throughput]: Using the above information, calculate how many vector FLOPs/s a TPU v5p can perform. A TPU v5p has a clock speed of about 1.75GHz.

Click here for the answer.

Answer: Each cycle, each core can execute 4 vector instructions on `8 * 128` ALUs. This gives us `8 * 128 * 4` FLOPs/cycle per core, or `8 * 128 * 4 * 1.75e9 = 7e12 FLOPs/s`. Note how much smaller this is than the MXU FLOPs/s of about `2e14` per core (roughly 30x).

Reductions: Generally, communication or reduction across the sublane dimension is easier than across the lane dimension. For instance, the VPU supports an intra-lane shuffle operation that can roll along the axis of size 8

in about a cycle. This can be used to perform efficient reductions along the sublane dimension (just shuffle by 4, 2, and 1 and do 3 pairs of elementwise sums).

Cross-lane reductions are much harder and involve a separate hardware unit called the XLU or "cross lane unit", which is slow and fairly expensive.

Comparison to GPUs: For those familiar with NVIDIA GPUs, each ALU in the VPU is analogous to a CUDA core, and a single VPU lane is analogous to a "Warp Scheduler", i.e. the set of usually 32 CUDA Cores that perform SIMD arithmetic. Reductions within the lane are pretty easy, but if we need to cross lanes, we need to transit at least VMEM/XLU/SMEM which is much slower. See the GPU section for more details.

Scalar Core

The scalar core is the control unit of the TPU. It fetches and dispatches all instructions and executes transfers from HBM into VMEM, and can be programmed to do scalar metadata work. Because the scalar core is single-threaded, one side-effect of this is that each core of the TPU is only capable of creating one DMA request per cycle.

To put this in context, a single scalar core controls a VPU (consisting of 4096 ALUs), 4 MXUs, 2 XLUs, and multiple DMA engines. The highly skewed nature of control per unit compute is a source of hardware efficiency, but also limits the ability to do data dependent vectorization in any interesting way.

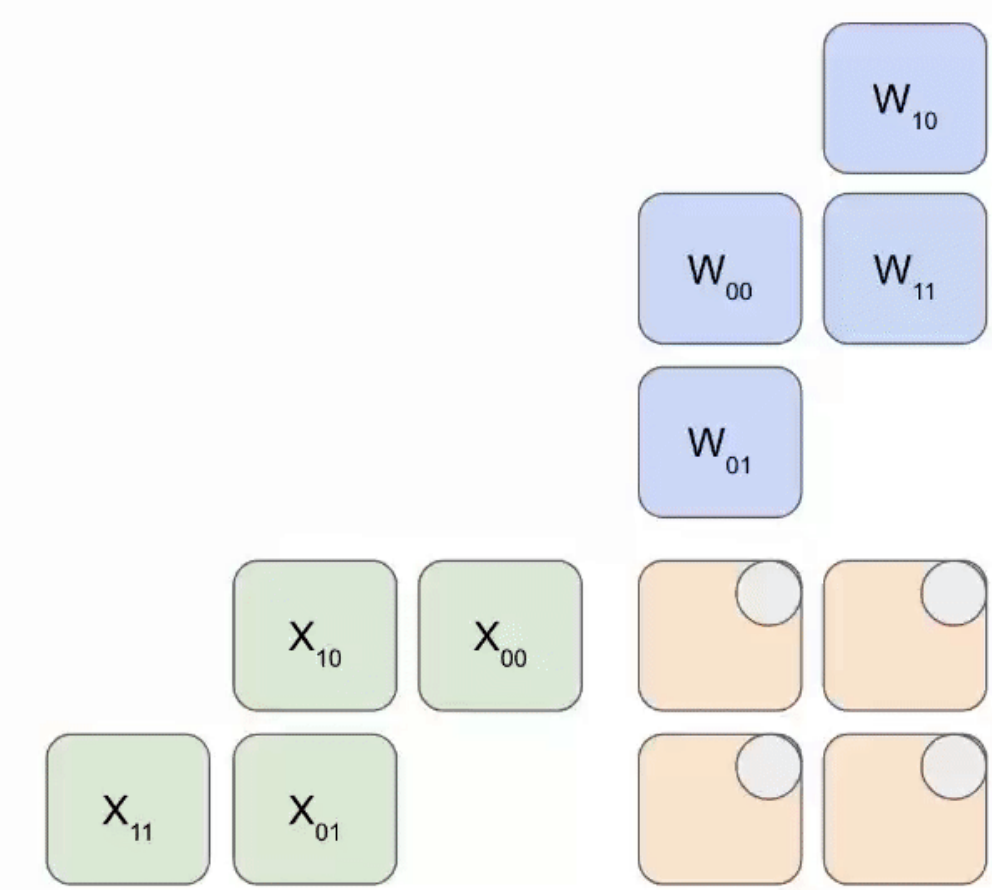
Appendix B: How does a systolic array work?

At the core of the TPU MXU is a `128x128` systolic array (`256x256` on TPU v6e). When fully saturated the systolic array can perform one `bf16[8,128] @ bf16[128,128] -> f32[8,128]`[29] multiplication per 8 clock cycles.

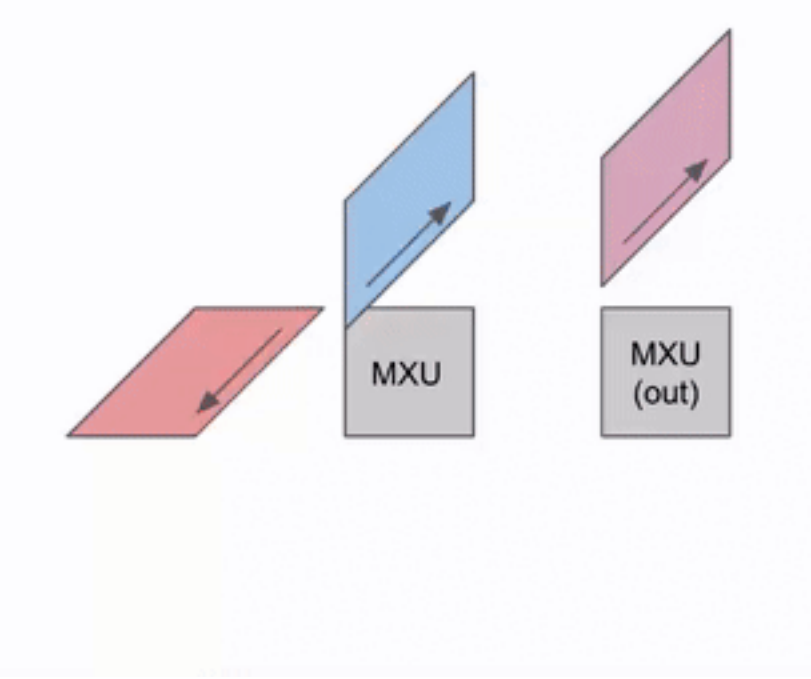
- At its core, the systolic array is a 2D `128x128` (`=16,384`) grid of ALUs each capable of performing a multiply and add operation.
- Weights (**W**, the `128x128` input) are passed down from above (called the RHS) while inputs (**X**, the `8x128` input) are passed in from the left (called the LHS).

Here is a simplified animation of multiplying a set of weights (blue) with a set of activations (green). You'll notice that the weights (RHS) are partially loaded first, diagonally, and then the activations are fed in, also diagonally. In each frame below, we multiply all the overlapped green and blue units, sum the result with any residual passed in from above, and then pass the result in turn down one unit.

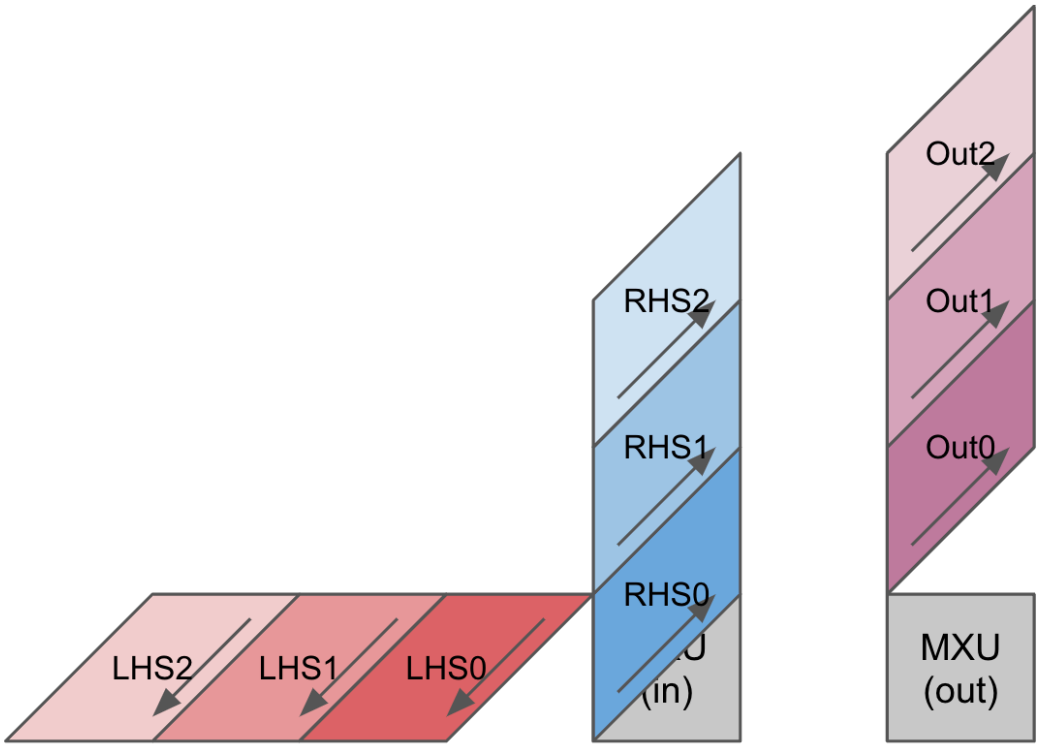
[29] If you are not familiar with this notation, it means: multiplying a `8x128` matrix with bfloat16 elements by a `128x128` matrix with bfloat16 elements and storing the results in a `8x128` matrix with float32 elements.



Here's a more general version of this animation showing the output being streamed out of computation:



Here's a diagram showing how this can be pipelined across multiple RHS and LHS arrays:



There is an initial pipeline bubble as the weights (RHS) and activations (LHS) are loaded. After that initial bubble, new inputs and weights can be loaded in without an additional bubble.

Here's a bad animation of a bf16[2, 3] x bf16[3, 3] matrix multiplication, which you could imagine as a matmul

of a 2x3 weight matrix with an input activation of batch 1 and size 3. This is rotated compared to the previous slides and inputs flow out to the right instead of down, but you can roughly see the structure.



We can efficiently pipeline this to multiply large matrices without too large a pipeline bubble. With that said, it's important that our matrices have shapes larger than the side dimension of the MXU, which is generally 128x128. Some TPUs (since TPU v3) have multiple MXUs, either 2 for TPU v3 or 4 for TPU v4/5, so we need to ensure tiling dimensions are larger than 128 * number of MXUs. Here's a good animation for this.

Trillium (TPU v6e) has a `256x256` systolic array, which means it can perform 4x more FLOPs / cycle. This also means the dimensions of your tensors need to be twice as large to utilize the MXU fully.

This blog post has another excellent animation of a systolic array multiplication for a fixed weight matrix.

Chapter 3

Sharded Matrices and How to Multiply Them

Partitioning Notation and Collective Operations

When we train an LLM on ten thousand TPUs or GPUs, we're still doing abstractly the same computation as when we're training on one. The difference is that **our arrays don't fit in the HBM of a single TPU/GPU**, so we have to split them.[30] We call this "*sharding*" or "*partitioning*" our arrays. The art of scaling is figuring out how to shard our models so computation remains efficient.

Here's an example 2D array **A** sharded across 4 TPUs:

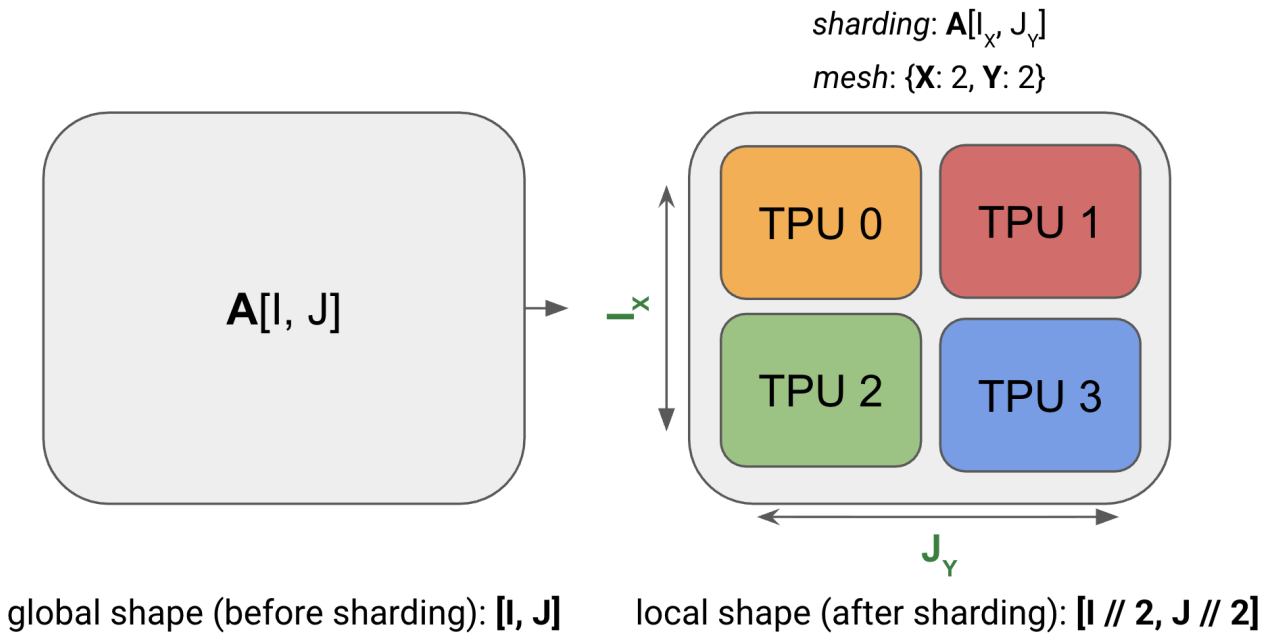


Figure 7: Figure: an example array of shape A[I, J] gets sharded across 4 devices. Both dimensions are evenly sharded across 2 devices with a sharding A[IX, JY]. Each TPU holds 1/4 of the total memory.

Note how the sharded array still has the same *global* or *logical shape* as the unsharded array, say `(4, 128)`, but it also has a *device local shape*, like `(2, 64)`, which gives us the actual size in bytes that each TPU is holding (in the figure above, each TPU holds ¼ of the total array). Now we'll generalize this to arbitrary arrays.

[30] It's worth noting that we may also choose to parallelize for speed. Even if we could fit on a smaller number of chips, scaling to more simply gives us more FLOPs/s. During inference, for instance, we can sometimes fit on smaller topologies but choose to scale to larger ones in order to reduce latency. Likewise, during training we often scale to more chips to reduce the step time.

A unified notation for sharding

We use a variant of *named-axis notation* to describe *how* the tensor is sharded in blocks across the devices: we assume the existence of a 2D or 3D grid of devices called the **device mesh** where each axis has been given **mesh axis names e.g. X**, **Y, and Z.** We can then specify how the matrix data is laid out across the device mesh by describing how each named dimension of the array is partitioned across the physical mesh axes. We call this assignment a **sharding**.

Example (the diagram above): For the above diagram, we have: * **Mesh:** the device mesh above `Mesh(devices=((0, 1), (2, 3)), axis_names=('X', 'Y'))`, which tells us we have 4 TPUs in a 2x2 grid, with axis names X and Y. * **Sharding:** $A[I_X, J_Y]$, which tells us to shard the first axis, I, along the mesh axis X, and the second axis, J, along the mesh axis Y. This sharding tells us that each shard holds $1/(|X| \cdot |Y|)$ of the array.

Taken together, we know that the local shape of the array (the size of the shard that an individual device holds) is $(|I|/2, |J|/2)$, where $|I|$ is the size of A's first dimension and $|J|$ is the size of A's second dimension.

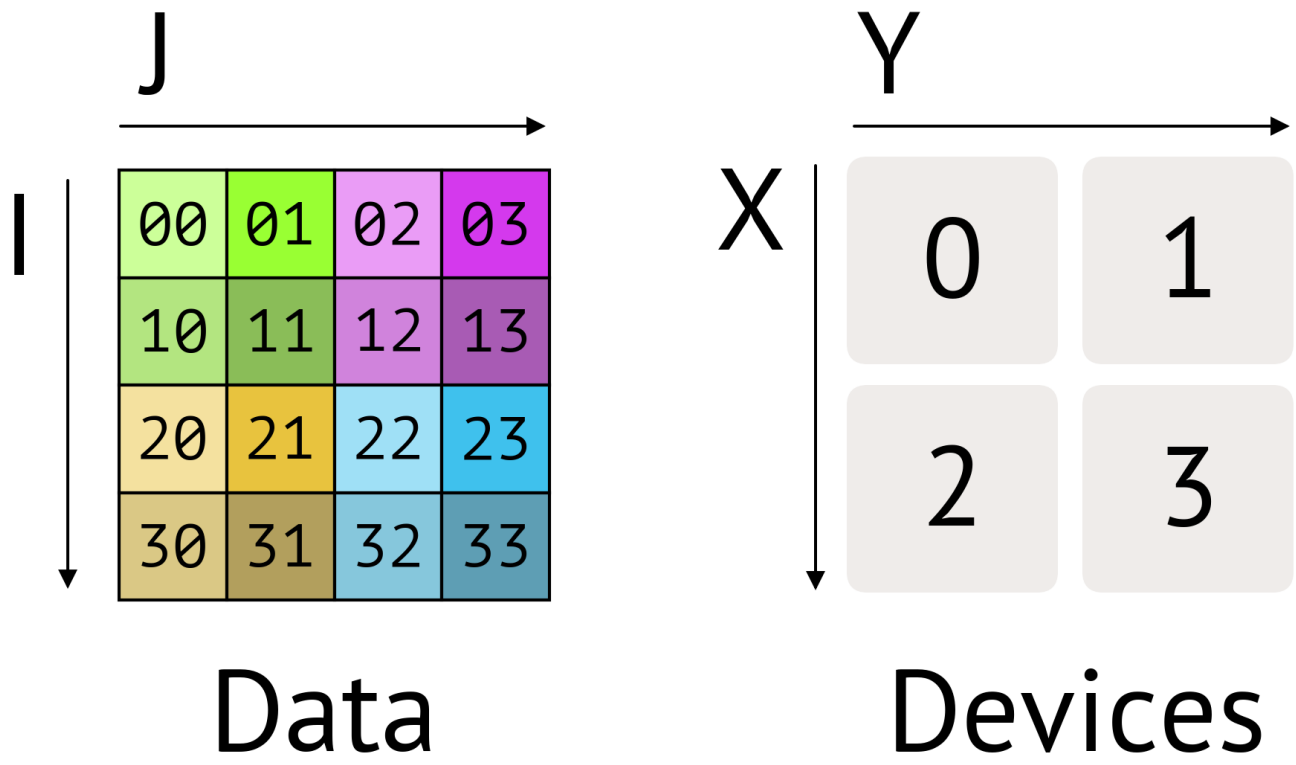
Pop Quiz [2D sharding across 1 axis]: Consider an array `fp32[1024, 4096]` with sharding $A[I_{XY}, J]$ and mesh `{'X': 8, 'Y': 2}`. How much data is held by each device? How much time would it take to load this array from HBM on H100s (assuming `3.4e12` memory bandwidth per chip)?

Click here for the answer.

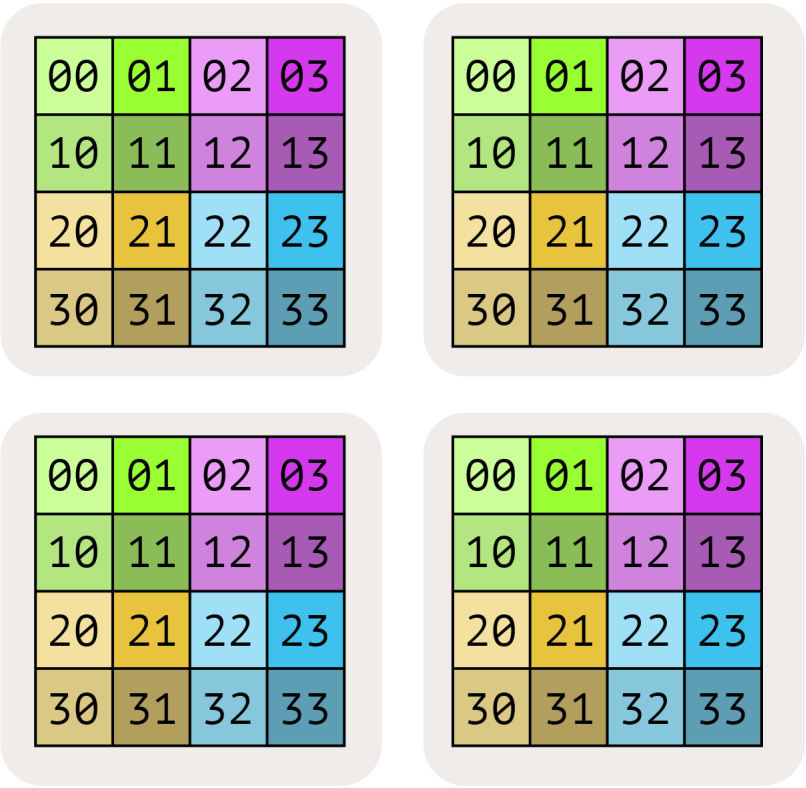
$A[I_{XY}, J]$ shards the first dimension (I) along both the X and Y hardware axes. In this example, the local shape is $(|I|/(|X| \cdot |Y|), |J|)$. For the given example, the global shape is `fp32[1024, 4096]`, so the local shape is `fp32[64, 4096]`.

Since each GPU has `4 * 64 * 4096 = 1MiB` bytes, this would take about `1e6 / 3.4e12 = 294ns`, although likely significantly more due to various overheads since this is so small.

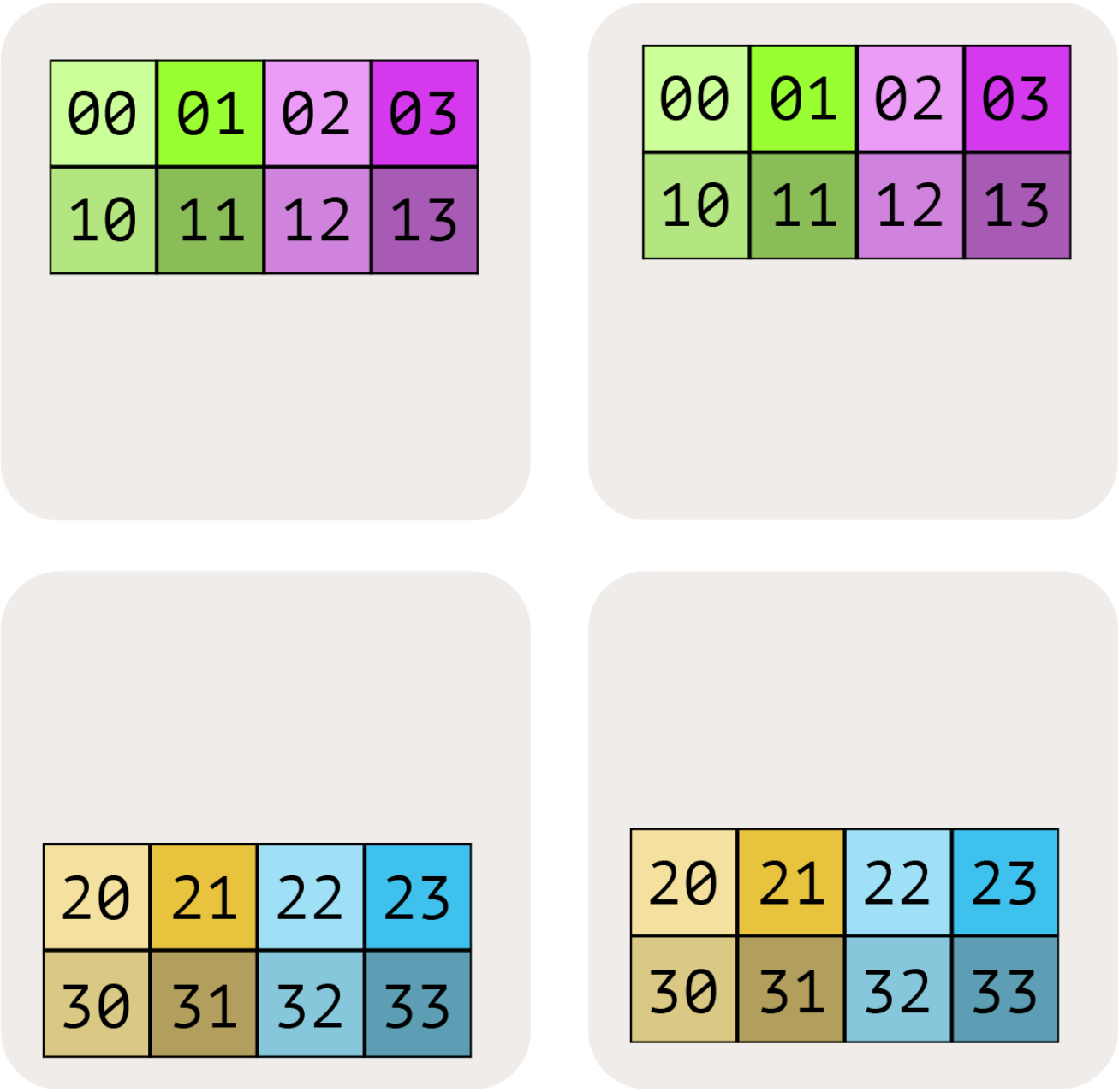
Visualizing these shardings: Let's try to visualize these shardings by looking at a 2D array of data split over 4 devices:



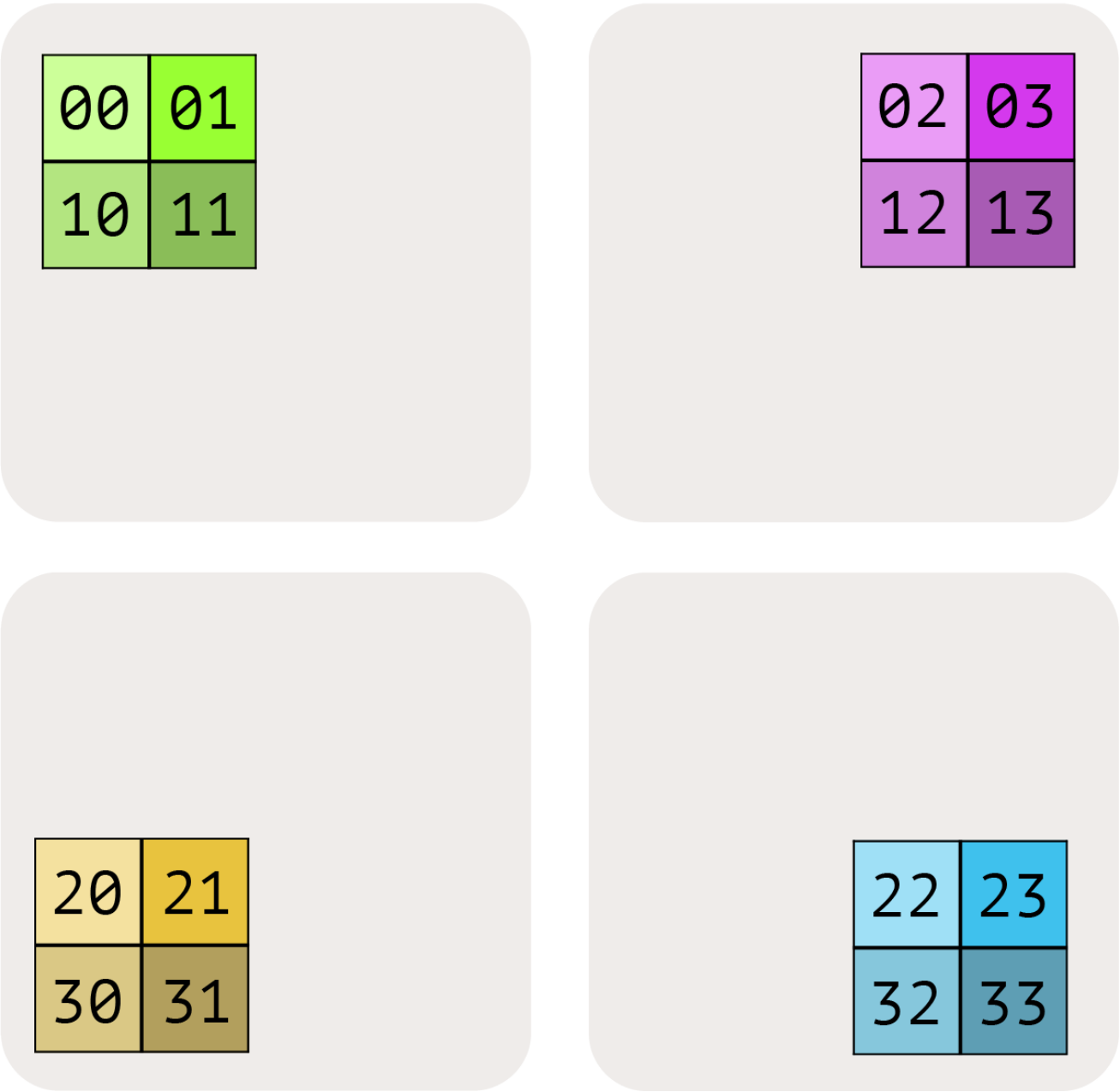
We write the *fully-replicated* form of the matrix simply as $A[I, J]$ with no sharding assignment. This means that *each* device contains a full copy of the entire matrix.



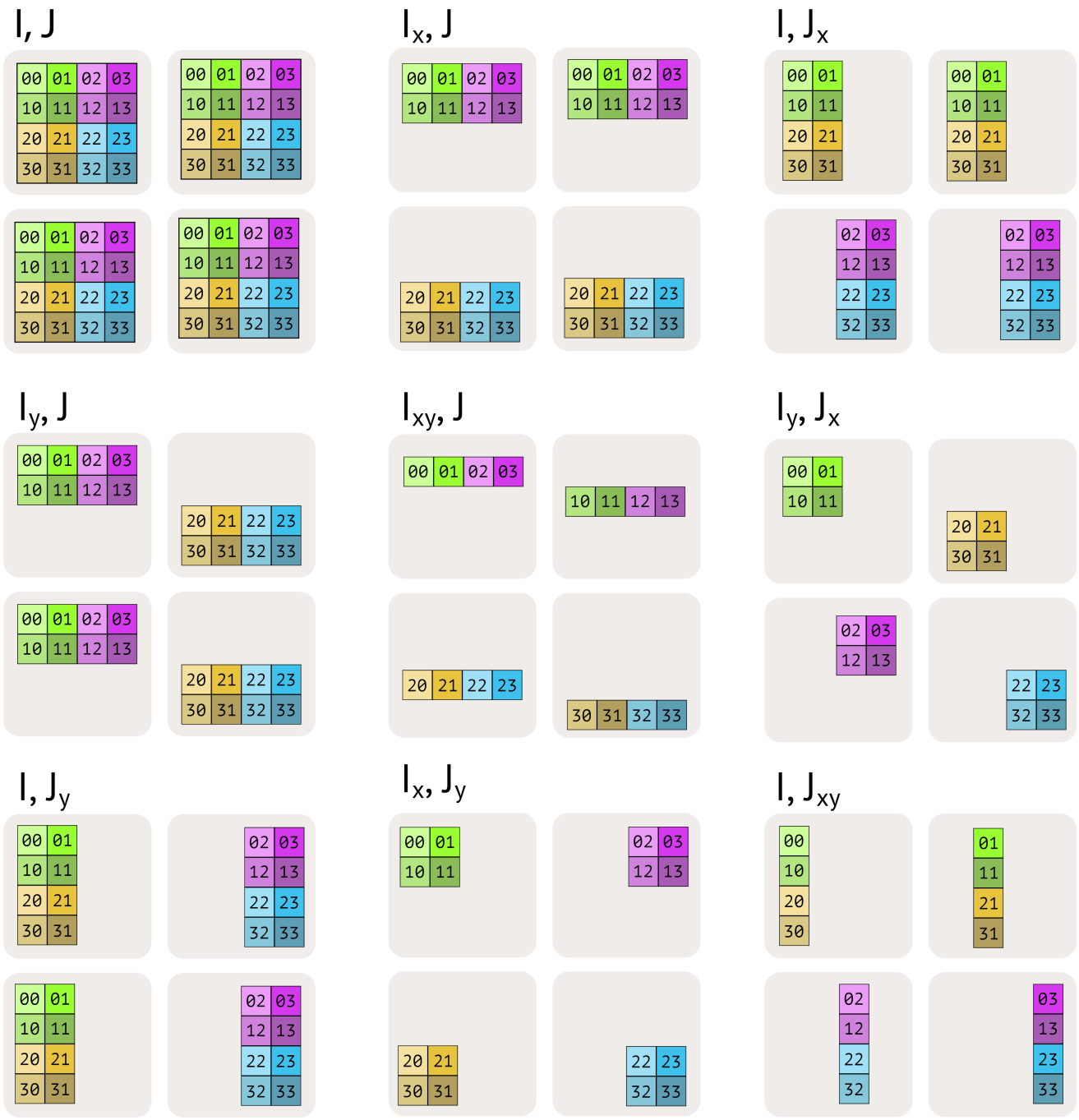
We can indicate that one of these dimensions has been partitioned across a mesh axis with a subscript mesh axis. For instance $A[I_X, J]$ would mean that the **I** logical axis has been partitioned across the **X** mesh dimension, but that the **J** dimension is *not* partitioned, and the blocks remain *partially-replicated* across the **Y** mesh axis.



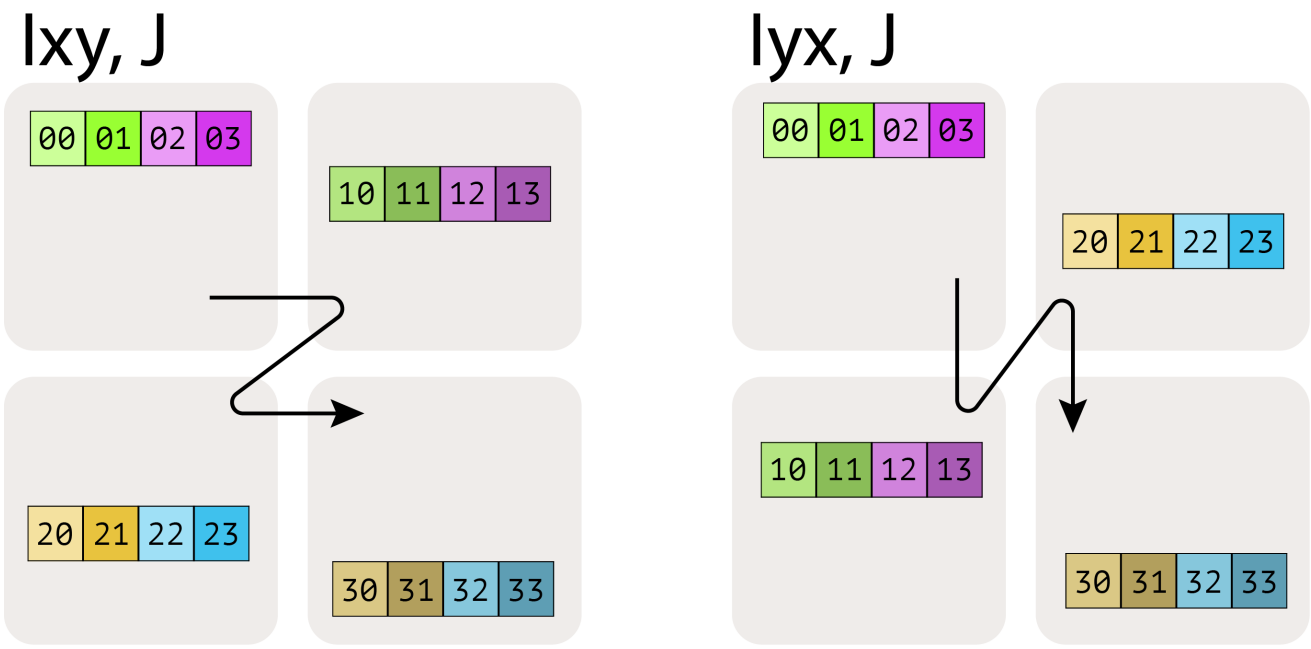
$A[I_X, J_Y]$ means that the **I** logical axis has been partitioned across the **X** mesh axis, and that the **J** dimension has been partitioned across the **Y** mesh axis.



We illustrate the other possibilities in the figure below:



Here $A[I_{XY}, J]$ means that we treat the **X** and **Y** mesh axes as a larger flattened dimension and partition the **I** named axis across all the devices. The order of the multiple mesh-axis subscripts matters, as it specifies the traversal order of the partitioning across the grid.



Lastly, note that we *cannot* have multiple named axes sharded along the *same* mesh dimension. e.g. $A[I_X, J_X]$ is a nonsensical, forbidden sharding. Once a mesh dimension has been used to shard one dimension of an array, it is in a sense "spent".

Pop Quiz: Let **A** be an array with shape `int8[128, 2048]`, sharding $A[I_{XY}, J]$, and mesh `Mesh({'X': 2, 'Y': 8, 'Z': 2})` (so 32 devices total). How much memory does **A** use per device? How much total memory does **A** use across all devices?

Click here for the answer.

Answer: Our array **A** is sharded over X and Y and replicated over Z, so per device it has shape `int8[128 / (2 * 8), 2048] = int8[8, 2048]`, with size `8 * 2048 = 16,384` bytes. Because it's replicated over Z, while within a Z-plane it's fully sharded over X and Y, there are 2 complete copies of the original array (one per Z-plane). So the total size across all devices is: original array size × Z replicas = 128 * 2048 * 2 = 512 KiB total. Alternatively, we can verify this as: 32 devices × 16,384 bytes per device = 512 KiB total.

How do we describe this in code?

So far we've avoided talking about code, but now is a good chance for a sneak peek. JAX uses a named sharding syntax that very closely matches the abstract syntax we describe above. We'll talk more about this in Section 10, but here's a quick preview. You can play with this in a Google Colab here and profile the result to see how JAX handles different shardings. This snippet does 3 things:

1. Creates a **jax.Mesh** that maps our 8 TPUs into a 4x2 grid with names 'X' and 'Y' assigned to the two axes.
2. Creates matrices A and B where A is sharded along both its dimensions and B is sharded along the output dimension.
3. Compiles and performs a simple matrix multiplication that returns a sharded array.

```
import jax
import jax.numpy as jnp

# Create our mesh! We're running on a TPU v2-8 4x2 slice with names 'X' and 'Y'.
# The Auto axis type tells JAX to let the XLA compiler infer intermediate shardings.
assert len(jax.devices()) == 8
Auto = jax.sharding.AxisType.Auto
mesh = jax.make_mesh(axis_sizes=(4, 2), axis_names=('X', 'Y'), axis_types=(Auto, Auto))
```

```
# A little utility function to help define our sharding. A PartitionSpec is our
# sharding (a mapping from axes to names).
def P(*args):
  return jax.NamedSharding(mesh, jax.sharding.PartitionSpec(*args))

# We shard both A and B over the non-contracting dimension and A over the contracting dim.
A = jnp.zeros((8, 2048), dtype=jnp.bfloat16, device=P('X', 'Y'))
B = jnp.zeros((2048, 8192), dtype=jnp.bfloat16, device=P(None, 'Y'))

# We can perform a matmul on these sharded arrays! out_shardings tells us how we want
# the output to be sharded. JAX/XLA handles the rest of the sharding for us.
y = jax.jit(lambda A, B: jnp.einsum('BD,DF->BF', A, B), out_shardings=P('X', 'Y'))(A, B)
```

The cool thing about JAX is that these arrays behave as if they're unsharded! `B.shape` will tell us the global or logical shape (2048, 8192). We have to actually look at `B.addressable_shards` to see how it's locally sharded. We can perform operations on these arrays and JAX will attempt to figure out how to broadcast or reshape them to perform the operations. For instance, in the above example, the local shape of **A** is `[2, 1024]` and for **B** is `[2048, 4096]`. JAX/XLA will automatically add communication across these arrays as necessary to perform the final multiplication.

Computation With Sharded Arrays

If you have an array of data that's distributed across many devices and wish to perform mathematical operations on it, what are the overheads associated with sharding both the data and the computation?

Obviously, this depends on the computation involved.

- For *elementwise* operations, there is **no overhead** for operating on a distributed array.
- When we wish to perform operations across elements resident on many devices, things get complicated. Thankfully, for most machine learning nearly all computation takes place in the form of matrix multiplications, and they are relatively simple to analyze.

The rest of this section will deal with how to multiply sharded matrices. To a first approximation, this involves moving chunks of a matrix around so you can fully multiply or sum each chunk. **Each sharding will involve different communication.** For example, $A[I_X, J] \cdot B[J, K_Y] \to C[I_X, K_Y]$ can be multiplied without any communication because the *contracting dimension* (J, the one we're actually summing over) is unsharded. However, if we wanted the output unsharded (i.e. $A[I_X, J] \cdot B[J, K_Y] \to C[I, K]$), we would either need to copy A and B or C to every device (using an *AllGather*). These two choices have different communication costs, so we need to calculate this cost and pick the lowest one.

You can think of this in terms of "block matrix multiplication".

To understand this, it can be helpful to recall the concept of a "block matrix", or a nested matrix of matrices:

$$\begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix} = \left(\begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \\ a_{20} & a_{21} \\ a_{30} & a_{31} \end{bmatrix} \begin{bmatrix} a_{02} & a_{03} \\ a_{12} & a_{13} \\ a_{22} & a_{23} \\ a_{32} & a_{33} \end{bmatrix} \right) = \begin{pmatrix} \mathbf{A_{00}} & \mathbf{A_{01}} \\ \mathbf{A_{10}} & \mathbf{A_{11}} \end{pmatrix} \tag{10}$$

Matrix multiplication has the nice property that when the matrix multiplicands are written in terms of blocks, the product can be written in terms of block matmuls following the standard rule:

$$\begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} \cdot \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} + A_{01}B_{10} & A_{00}B_{01} + A_{01}B_{11} \\ A_{10}B_{00} + A_{11}B_{10} & A_{10}B_{01} + A_{11}B_{11} \end{pmatrix} \tag{11}$$

What this means is that implementing distributed matrix multiplications reduces down to moving these sharded blocks over the network, performing *local* matrix multiplications on the blocks, and summing their results. **The question then is what communication to add, and how expensive it is.**

Conveniently, we can boil down all possible shardings into roughly 4 cases we need to consider, each of which has a rule for what communication we need to add 1. **Case 1:** neither input is sharded along the contracting dimension. *We can multiply local shards without any communication.* 2. **Case 2:** one input has a sharded contracting dimension. *We typically "AllGather" the sharded input along the contracting dimension.* 3. **Case 3:** both inputs are sharded along the contracting dimension. *We can multiply the local shards, then "AllReduce" the result.* 4. **Case 4:** both inputs have a non-contracting dimension sharded along the same axis. We cannot proceed without AllGathering one of the two inputs first.

You can think of these as rules that simply need to be followed, but it's also valuable to understand why these rules hold and how expensive they are. We'll go through each one of these in detail now.

Case 1: neither multiplicand has a sharded contracting dimension

Lemma: when multiplying sharded matrices, the computation is valid and the output follows the sharding of the inputs *unless* the contracting dimension is sharded or both matrices are sharded along the same axis. For example, this works fine

$$\mathbf{A}[I_X, J] \cdot \mathbf{B}[J, K_Y] \to \mathbf{C}[I_X, K_Y]$$

with no communication whatsoever, and results in a tensor sharded across both the X and Y hardware dimensions. Try to think about why this is. Basically, the computation is *independent* of the sharding, since each batch entry has some local chunk of the axis being contracted that it can multiply and reduce. Any of these cases work fine and follow this rule:

$$\begin{aligned}
\mathbf{A}[I, J] \cdot \mathbf{B}[J, K] &\to \mathbf{C}[I, K] \\
\mathbf{A}[I_X, J] \cdot \mathbf{B}[J, K] &\to \mathbf{C}[I_X, K] \\
\mathbf{A}[I, J] \cdot \mathbf{B}[J, K_Y] &\to \mathbf{C}[I, K_Y] \\
\mathbf{A}[I_X, J] \cdot \mathbf{B}[J, K_Y] &\to \mathbf{C}[I_X, K_Y]
\end{aligned}$$

Because neither **A** nor **B** has a sharded contracting dimension **J**, we can simply perform the local block matrix multiplies of the inputs and the results will *already* be sharded according to the desired output shardings. When both multiplicands have non-contracting dimensions sharded along the same axis, this is no longer true (see the invalid shardings section for details).

Case 2: one multiplicand has a sharded contracting dimension

Let's consider what to do when one input **A** is sharded along the contracting **J** dimension and **B** is fully replicated:

$\mathbf{A}[I, J_X] \cdot \mathbf{B}[J, K] \to \mathbf{C}[I, K]$

We cannot simply multiply the local chunks of **A** and **B** because we need to sum over the full contracting dimension of **A**, which is split across the X axis. Typically, we first "**AllGather**" the shards of **A** so every device has a full copy, and only then multiply against **B:**

$\mathbf{AllGather}_X[I, J_X] \to \mathbf{A}[I, J]$

$\mathbf{A}[I, J] \cdot \mathbf{B}[J, K] \to \mathbf{C}[I, K]$

This way the actual multiplication can be done fully on each device.

Takeaway: When multiplying matrices where one of the matrices is sharded along the contracting dimension, we generally AllGather it first so the contraction is no longer sharded, then do a local matmul.

Note that when **B** is not also sharded along X, we could also do the local partial matmul and then sum (or *AllReduce*) the sharded partial sums, which lets us shard the compute but usually has a higher communication cost. This can be faster in some cases, although it's usually true in practice that **B** will be sharded. Question 4 below works through when this is better.

What is an AllGather? An AllGather is the first core MPI communication primitive we will discuss. An AllGather *removes the sharding* along an axis and reassembles the shards spread across devices onto *each* device along that axis. Using the notation above, an AllGather removes a subscript from a set of axes, e.g.

$$\textbf{AllGather}_{XY}(A[I_{XY}, J]) \to A[I, J]$$

We don't have to remove all subscripts for a given dimension, e.g. $A[I_{XY}, J] \to A[I_Y, J]$ is also an AllGather, just over only a single axis. Also note that we may also wish to use an AllGather to remove *non-contracting* dimension sharding, for instance in the matrix multiply:

$$A[I_X, J] \cdot B[J, K] \to C[I, K]$$

We could either AllGather **A** initially to remove the input sharding, or we can do the sharded matmul and then AllGather the result **C**.

How is an AllGather actually performed? To perform a 1-dimensional AllGather around a single TPU axis (a ring), we basically have each TPU pass its shard around a ring until every device has a copy.[31] Here is an animation:

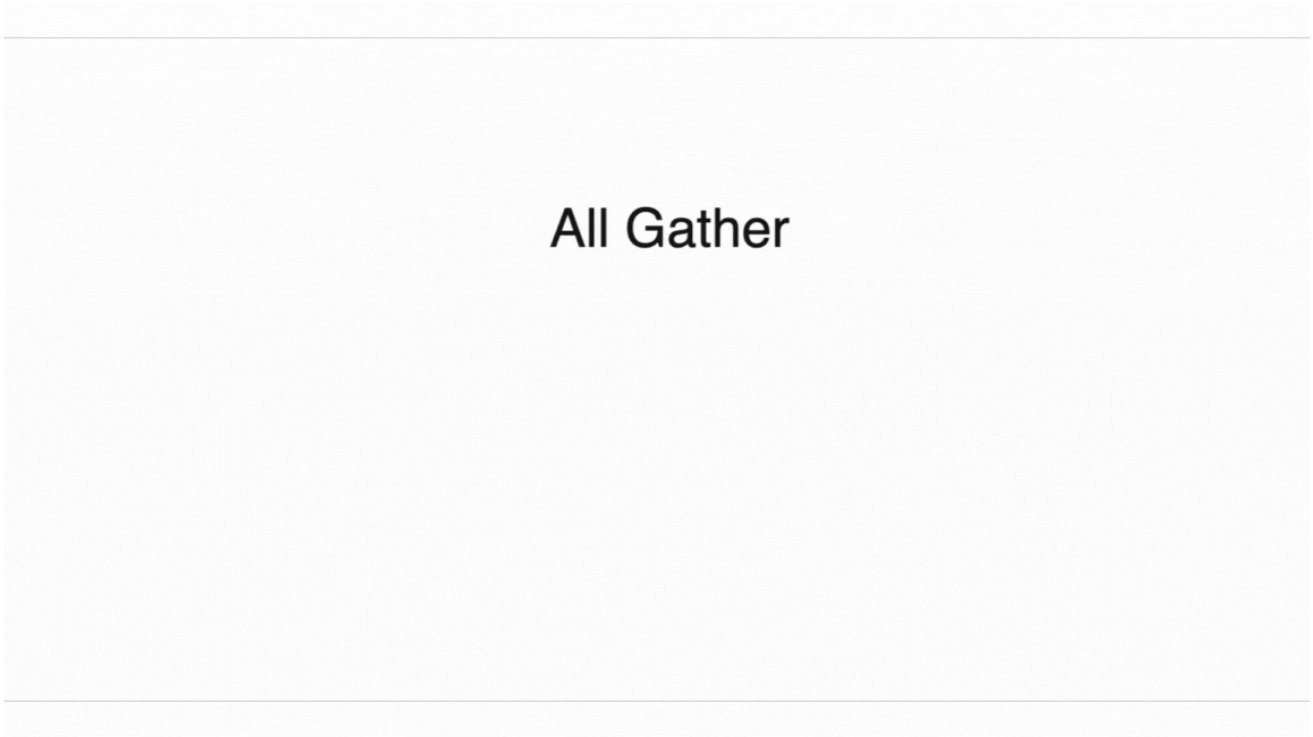


Figure 8: Figure: An animation showing how to perform an AllGather around a set of 8 TPU or GPU devices. Each device starts with 1 / 8th of the array and ends up with a full copy.

We can either do an AllGather in one direction or both directions (two directions are shown above). If we do one direction, each TPU sends chunks of size bytes/N over $N - 1$ hops around the ring. If we do two directions, we have $\lfloor \frac{N}{2} \rfloor$ hops of size $2 \cdot \text{bytes}/N$.

How long does this take? Let's take the bidirectional AllGather and calculate how long it takes. Let V be the number of bytes in the array, and X be the number of shards on the contracting dimension. Then from the above diagram, each hop sends $V/|X|$ bytes in each direction, so each hop takes

$$T_{hop} = \frac{2 \cdot V}{|X| \cdot W_{\text{ici}}}$$

[31] A GPU AllGather can also work like this, where you create a ring out of the GPUs in a node and pass the chunks around in that (arbitrary) order.

where W_{ici} is the **bidirectional** ICI bandwidth.[32] We need to send a total of $|X|/2$ hops to reach every TPU[33], so the total reduction takes

$T_{total} = \frac{2 \cdot V \cdot X}{2 \cdot X \cdot W_{\text{ici}}}$

$T_{total} = \frac{V}{W_{\text{ici}}}$

Note that this **doesn't depend on X!** That's kind of striking, because it means even though our TPUs are only locally connected, the locality of the connections doesn't matter. We're just bottlenecked by the speed of each link.

Takeaway: when performing an AllGather (or a ReduceScatter or AllReduce) in a throughput-bound regime, the actual communication time depends only on the size of the array and the available bandwidth, not the number of devices over which our array is sharded!

A note on ICI latency: Each hop over an ICI link has some intrinsic overhead regardless of the data volume. This is typically around 1us. This means when our array A is very small and each hop takes less than 1us, we can enter a "latency-bound" regime where the calculation *does* depend on X.

For the full details, click here.

Let T_{min} be the minimum time for a single hop. Then

$T_{hop} = \max\left[T_{min}, \frac{2 \cdot V}{X \cdot W_{\text{ici}}}\right]$

$T_{total} = \max\left[\frac{T_{min} \cdot X}{2}, \frac{V}{W_{\text{ici}}}\right]$

since we perform $X/2$ hops. For large reductions or gathers, we're solidly bandwidth bound. We're sending so much data that the overhead of each hop is essentially negligible. But for small arrays (e.g. when sampling from a model), this isn't negligible, and the ICI bandwidth isn't relevant. We're bound purely by latency. Another way to put this is that given a particular TPU, e.g. TPU v5e with `4.5e10` unidirectional ICI bandwidth, sending any buffer under `4.5e10 * 1e-6 = 45kB` will be latency bound.

Here is an empirical measurement of AllGather bandwidth on a TPU v5e 8x16 slice. The array is sharded across the 16 axis so it has a full bidirectional ring.

Note that we not only achieve about 95% of the peak claimed bandwidth (`4.5e10`) but also that we achieve this peak at about 10MB, which when 16-way sharded gives us about 625kB per device (*aside*: this is much better than GPUs).

What happens when we AllGather over multiple axes? When we gather over multiple axes, we have multiple dimensions of ICI over which to perform the gather. For instance, AllGatherXY([B, DXY]) operates over two hardware mesh axes. This increases the available bandwidth by a factor of N_{axes}.

When considering latency, we end up with the general rule:

$T_{total} = \max\left[\frac{T_{min} \cdot \sum_i |X_i|}{2}, \frac{V}{W_{\text{ici}} \cdot N_{\text{axes}}}\right]$

where $\sum_i |X_i|/2$ is the length of the longest path in the TPU mesh.

Pop Quiz 2 [AllGather time]: Using the numbers from Part 2, how long does it take to perform the AllGatherY([EY, F]) $\to$ [E, F] on a TPU v5e with a 2D mesh `{'X': 8, 'Y': 4}`, $E = 2048$, $F = 8192$ in bfloat16? What about with $E = 256, F = 256$?

Click here for the answer.

Answer: Let's start by calculating some basic quantities:

1) TPU v5e has 4.5e10 bytes/s of unidirectional ICI bandwidth for each of its 2 axes.
2) In bfloat16 for (a), we have $A[E_Y, F]$ so each device holds an array of shape bf16[512, 8192] which has 512 * 8192 * 2 = 8.4MB. The total array has size 2048 * 8192 * 2 = 34MB.

[32] The factor of 2 in the numerator comes from the fact that we're using the bidirectional bandwidth. We send V/X in each direction, or $2V/X$ total.

[33] technically, $\lfloor X/2 \rfloor$

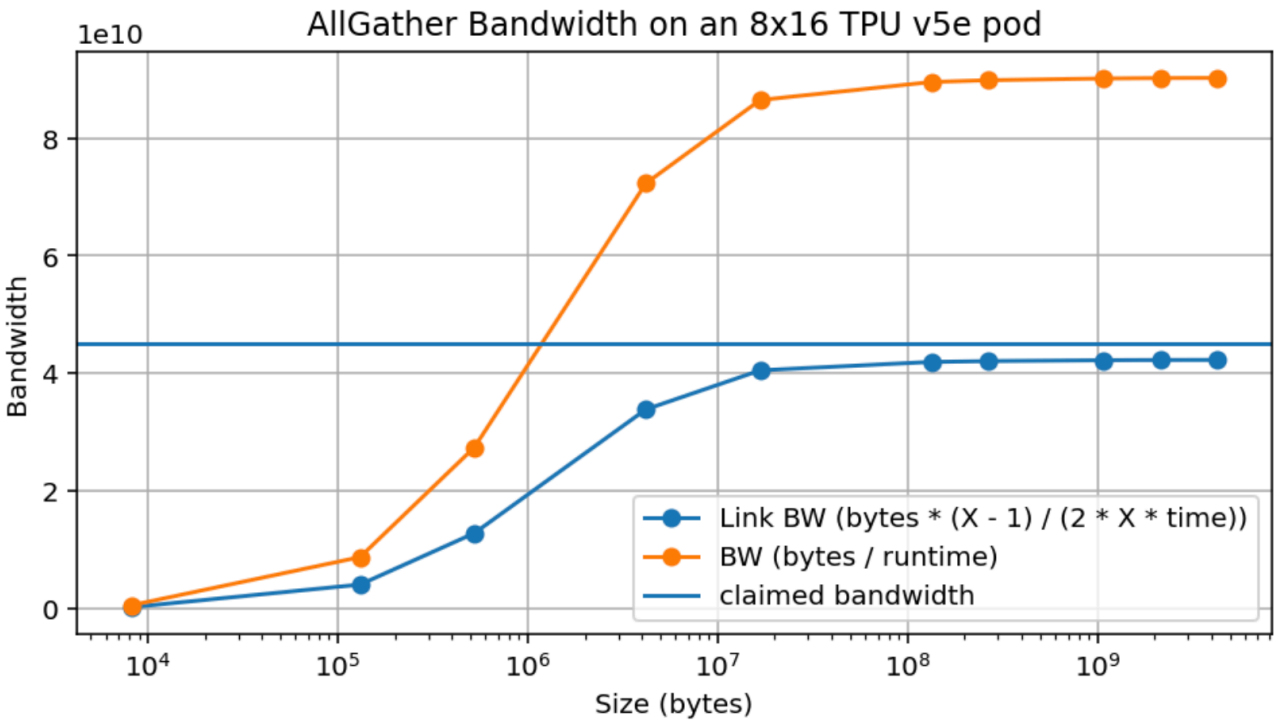


Figure 9: Figure: empirical bandwidth and estimated link bandwidth for TPU v5e during an AllGather. BW in orange is the actual bytes per second AllGathered, while the blue curve shows the empirical unidirectional link bandwidth calculated according to the known cost of the collective.

For part (1), we can use the formula above. Since we're performing the AllGather over one axis, we have $T_{\text{comms}} = 34\text{e}6/9\text{e}10 = 377\text{us}$. To check that we're not latency-bound, we know over an axis of size 4, we'll have at most 3 hops, so our latency bound is something like 3us, so we're not close. However, TPU v5e only has a wraparound connection when one axis has size 16, so here *we actually can't do a fully bidirectional AllGather*. We have to do 3 hops for data from the edges to reach the other edge, so in theory we have more like $T_{\text{comms}} = 3 * 8.4\text{e}6/4.5\text{e}10 = 560\mu s$. **Here's an actual profile** from this Colab, which shows $680\mu s$, which is reasonable since we're likely not getting 100% of the theoretical bandwidth! *For part (2)* each shard has size `64 * 256 * 2 = 32kB. 32e3 / 4.5e10 = 0.7us`, so we're latency bound. Since we have 3 hops, this will take roughly 3 * 1us = 3us. In practice, it's closer to 8us.

Note: when we have a 2D mesh like `{'X': 16, 'Y': 4}`, it is not necessary for each axis to correspond to a specific *hardware* axis. This means for instance the above could describe a 4x4x4 TPU v5p cube with 2 axes on the X axis. This will come into play later when we describe data parallelism over multiple axes.

Case 3: both multiplicands have sharded contracting dimensions

The third fundamental case is when both multiplicands are sharded on their contracting dimensions, along the same mesh axis:

$$\mathbf{A}[I, J_X] \cdot \mathbf{B}[J_X, K] \to C[I, K]$$

In this case the *local* sharded block matrix multiplies are at least *possible* to perform, since they will share the same sets of contracting indices. But each product will only represent a *partial sum* of the full desired product, and each device along the **X** dimension will be left with different *partial sums* of this final desired product. This is so common that we extend our notation to explicitly mark this condition:

$$\mathbf{A}[I, J_X] \cdot_{\text{LOCAL}} \mathbf{B}[J_X, K] \to C[I, K]\{\ U_X\}$$

The notation **{ UX }** reads "**unreduced** along X mesh axis" and refers to this status of the operation being

"incomplete" in a sense, in that it will only be finished pending a final sum. The $\cdot_{\text{LOCAL}}$ syntax means we perform the local sum but leave the result unreduced.

This can be seen as the following result about matrix multiplications and outer products:

$A \cdot B = \sum_{i=1}^{P} \underbrace{A_{:,i} \otimes B_{i,:}}_{\in \mathbb{R}^{n \times m}}$

where $\otimes$ is the outer product. Thus, if TPU **i** on axis **X** has the **i**th column of **A**, and the **i**th row of **B**, we can do a local matrix multiplication to obtain $A_{:,i} \otimes B_{i,:} \in \mathbb{R}_{n \times m}$. This matrix has, in each entry, the **i**th term of the sum that **A** $\bullet$ **B** has at that entry. We still need to perform that sum over **P**, which we sharded over mesh axis **X**, to obtain the full **A** $\bullet$ **B**. This works the same way if we write **A** and **B** by blocks (i.e. shards), and then sum over each resulting shard of the result.

We can perform this summation using a full **AllReduce** across the **X** axis to remedy this:

$$A[I, J_X] \cdot_{\text{LOCAL}} B[J_X, K] \to C[I, K]\{U_X\}$$
$$\textbf{AllReduce}_X C[I, K]\{U_X\} \to C[I, K]$$

AllReduce removes partial sums, resulting in *each* device along the axis having the same fully-summed value. AllReduce is the second of several key communications we'll discuss in this section, the first being the AllGather, and the others being ReduceScatter and AllToAll. An AllReduce takes an array with an unreduced (partially summed) axis and performs the sum by passing those shards around the unreduced axis and accumulating the result. The signature is

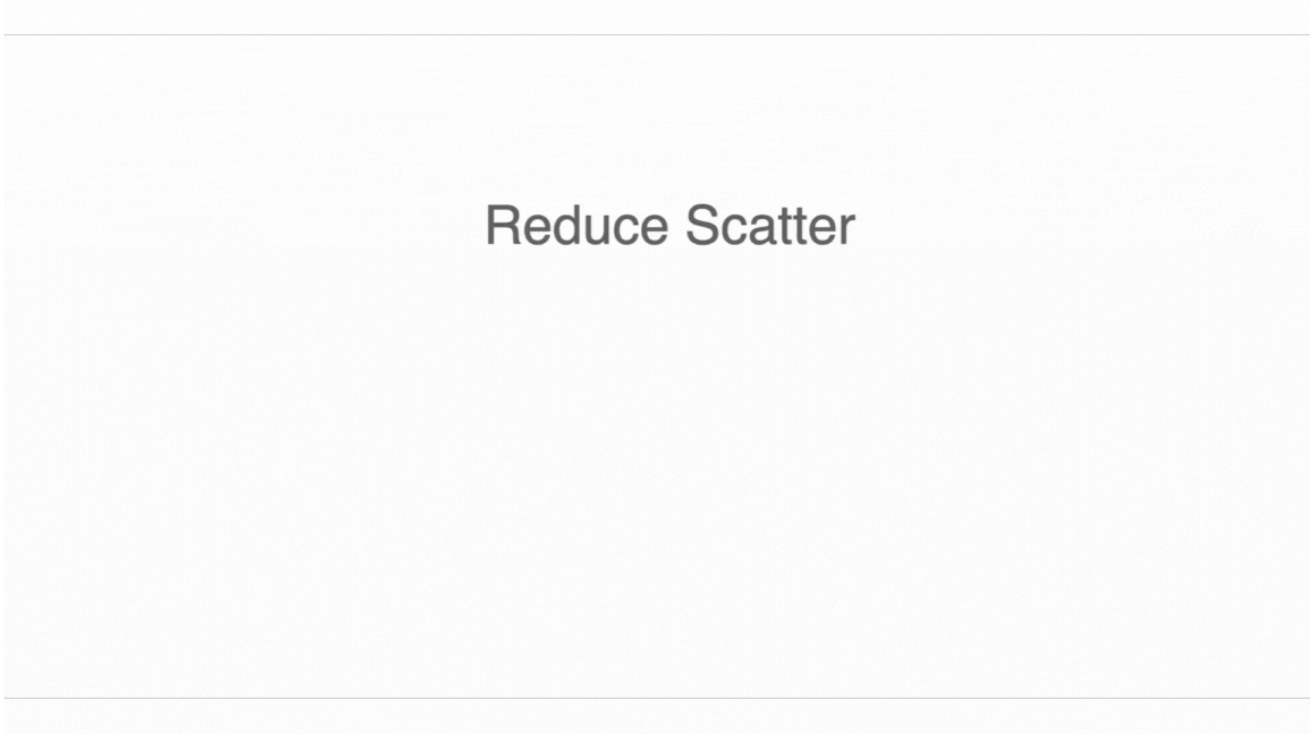
$\textbf{AllReduce}_Y A[I_X, J]\{U_Y\} \to A[I_X, J]$

This means it simply removes the $\{U_Y\}$ suffix but otherwise leaves the result unchanged.

How expensive is an AllReduce? One mental model for how an AllReduce is performed is that every device sends its shard to its neighbors, and sums up all the shards that it receives. Clearly, this is more expensive than an AllGather because each "shard" has the same shape as the full array. Generally, **an AllReduce is twice as expensive as an AllGather.** One way to see this is to note that an **AllReduce** can be expressed as a composition of two other primitives: a **ReduceScatter** and an **AllGather**. Like an AllReduce, a ReduceScatter resolves partial sums on an array but results in an output 'scattered' or partitioned along a given dimension. AllGather collects all those pieces and 'unpartitions/unshards/replicates' the logical axis along that physical axis.

$$\textbf{ReduceScatter}_{Y,J} : A[I_X, J]\{U_Y\} \to A[I_X, J_Y]$$
$$\textbf{AllGather}_Y : A[I_X, J_Y] \to A[I_X, J]$$

What about a ReduceScatter? Just as the AllGather reassembles a sharded array (removing a subscript), a ReduceScatter sums an unreduced/partially summed array and then scatters (shards) a different logical axis along the same mesh axis. $X[F]\{U_Y\} \to X[F_Y]$. The animation shows how this is done: note that it's very similar to an AllGather but instead of retaining each shard, we sum them together. Thus, its latency is roughly the same, excluding the time taken to perform the reduction.



The communication time for each hop is simply the per-shard bytes V/Y divided by the bandwidth W_{ici}, as it was for an AllGather, so we have

$T_{\text{comms per AllGather or ReduceScatter}} = \frac{V}{W_{\text{ici}}}$

$T_{\text{comms per AllReduce}} = 2 \cdot \frac{V}{W_{\text{ici}}}$

where W_{ici} is the bidirectional bandwidth, so long as we have a full ring to reduce over.

Case 4: both multiplicands have a non-contracting dimension sharded along the same axis

Each mesh dimension can appear at most once when sharding a tensor. Performing the above rules can sometimes lead to a situation where this rule is violated, such as:

$A[I_X, J] \cdot B[J, K_X] \to C[I_X, K_X]$

This is invalid because a given shard, say **i**, along dimension **X**, would have the **(i, i)**th shard of **C**, that is, a diagonal entry. There is not enough information among all shards, then, to recover anything but the diagonal entries of the result, so we cannot allow this sharding.

The way to resolve this is to AllGather some of the dimensions. Here we have two choices:

$$\begin{aligned} \textbf{AllGather}_X A[I_X, J] &\to A[I, J] \\ A[I, J] \cdot B[J, K_X] &\to C[I, K_X] \end{aligned}$$

or

$$\begin{aligned} \textbf{AllGather}_X B[J, K_X] &\to B[J, K] \\ A[I_X, J] \cdot B[J, K] &\to C[I_X, K] \end{aligned}$$

In either case, the result will only mention **X** once in its shape. Which one we pick will be based on what sharding the following operations need.

A Deeper Dive into TPU Communication Primitives

The previous 4 cases have introduced several "core communication primitives" used to perform sharded matrix multiplications:

1. **AllGather:** removes a subscript from a sharding, gathering the shards.
2. **ReduceScatter:** removes an "un-reduced" suffix from an array by summing shards over that axis, leaving the array sharded over a second axis.
3. **AllReduce:** removes an "un-reduced" suffix, leaving the array unsharded along that axis.

There's one more core communication primitive to mention that arises in the case of Mixture of Experts (MoE) models and other computations: the **AllToAll**.

Our final communication primitive: the AllToAll

A final fundamental collective which does not occur naturally when considering sharded matrix multiplies, but which comes up constantly in practice, is the **AllToAll** collective, or more precisely the special case of a *sharded transposition* or resharding operation. e.g.

$\textbf{AllToAll}_{X,J} A[I_X, J] \to A[I, J_X]$

AllToAlls are typically required to rearrange sharded layouts between different regions of a sharded computation that don't have compatible layout schemes. They arise naturally when considering sharded mixture-of-experts models. *You can think of an AllToAll as moving a subscript from one axis to another.* Because an all to all doesn't need to replicate all of the data of each shard across the ring, it's actually *cheaper* than an AllGather (by a factor of ¼)[34].

If we generalize to an ND AllToAll, the overall cost for an array of V total bytes (summed across all devices) on an AxBxC mesh is

$T_{\text{comms per AllToAll}} = \frac{V \cdot \max(A,B,C,...)}{4 \cdot N \cdot W_{\text{ici}}}$

where as usual W_{ici} is the bidirectional ICI bandwidth and $N = A \cdot B \cdot C \cdot ...$ is the total number of devices. Equivalently, in terms of the per-device bytes V/N, the cost is $(V/N) \cdot \max(A, B, C, ...)/(4 \cdot W_{\text{ici}})$. For a 1D mesh, this reduces to $V/(4 \cdot W_{\text{ici}})$, which is 1 / 4 the cost of an AllGather. In 2D, the cost actually scales down with the size of the smallest axis.

[34] For even-sized bidirectional rings, each device will send $(N/2 + (N/2 - 1) + ... + 1)$ chunks right and $((N/2 - 1) + ... + 1)$ chunks left $= 0.5 \cdot (N/2) \cdot (N/2 + 1) + 0.5 \cdot (N/2) \cdot (N/2 - 1) = N^2/4$. The size of each chunk (aka shard of a shard) is bytes$/N^2$ so the per-device cost is (bytes$/N^2) \cdot N^2/4$ = bytes$/4$. This result scales across all devices as the total bandwidth scales with device number.

Aside: If you want a hand-wavy derivation of this fact, start with a 1D torus $\mathbb{Z}/N\mathbb{Z}$. If we pick a source and target node at random, they are on average N / 4 hops from each other, giving us a cost of $(V \cdot N)/(4 * N)$. Now if we consider an ND torus, each axis is basically independent. Each node has $1/N$ bytes and on average has to hop its data $\max(A, B, C, ...)/4$ hops. You can also derive this from bisection bandwidth: in an AllToAll, each half of the mesh sends half its data ($V/4$ bytes) to the other half. The narrowest bisection cuts perpendicular to the longest axis, crossing $2 \cdot N/\max(A, B, ...)$ links (two cut planes, counting wraparound), for a one-directional bandwidth of $N \cdot W_{ici}/\max(A, B, ...)$. Dividing gives the formula above.

More about the ReduceScatter

ReduceScatter is a more fundamental operation than it first appears, as it is actually the derivative of an AllGather, and vice versa. i.e. if in the forward pass we have:

$\mathbf{AllGather}_X A[I_X] \to A[I]$

Then we ReduceScatter the reverse-mode derivatives **A'** (which will in general be different on each shard) to derive the sharded **A'**:

$\mathbf{ReduceScatter}_X A'[I]\{U_X\} \to A'[I_X]$

Likewise, $\text{ReduceScatter}_X(A[I]\{U_X\}) \to A[I_X]$ in the forward pass implies $\text{AllGather}_X(A'[I_X]) \to A'[I]$ in the backwards pass.

For details on how AllGather and ReduceScatter are derivatives of each other, click here.

This stems from the fact that broadcasts and reductions are transposes of each other as linear operators, and AllGather and ReduceScatter are outer products (also known as Kronecker products) of broadcast and reduce, respectively. Concretely, if we have a vector $x \in \mathbb{R}^n$, any number of devices $p \in \mathbb{N}$, and we let $u = (1, ..., 1) \in \mathbb{R}^p$, we can define broadcast and reduce in the following way, which should match your intuitive understanding of them:

$$\begin{aligned} \text{broadcast} &: \mathbb{R}^n \to \mathbb{R}^{pn} \\ \text{broadcast} &= u \otimes \mathbf{I}_n \\ \text{reduce} &: \mathbb{R}^{pn} \to \mathbb{R}^n \\ \text{reduce} &= u^T \otimes \mathbf{I}_n \end{aligned}$$

Let's see how this looks in an example, where $n = 1$, $p = 2$. If $x = (7)$, we have $\text{broadcast}(x) = \left(\begin{pmatrix}1\\1\end{pmatrix} \otimes (1)\right) x = \begin{pmatrix}1\\1\end{pmatrix} x = \begin{pmatrix}7\\7\end{pmatrix} \in \mathbb{R}^{pn}$. This matches what we'd expect, broadcasting a vector in $\mathbb{R}^n$ to $\mathbb{R}^{pn}$. Now letting $y = (8, 9)$, we have $\text{reduce}(y) = ((1 \quad 1) \otimes (1))\, y = (1 \quad 1) \begin{pmatrix}8\\9\end{pmatrix} = (17)$. This again matches what we'd expect, reducing a vector in $\mathbb{R}^{pn}$ to a vector in $\mathbb{R}^n$. Since $(A \otimes B)^T = A^T \otimes B^T$ for any two matrices A and B, we see that $\text{reduce} = \text{broadcast}^T$. We recover AllGather and ReduceScatter as the following outer products:

$$\begin{aligned} \text{AllGather} &: \mathbb{R}^{pn} \to \mathbb{R}^{p^2 n} \\ \text{AllGather} &= \text{broadcast} \otimes \mathbf{I}_p \\ \text{ReduceScatter} &= \mathbb{R}^{p^2 n} \to \mathbb{R}^{pn} \\ \text{ReduceScatter} &= \text{reduce} \otimes \mathbf{I}_p \end{aligned}$$

Here we think of $\mathbb{R}^{p^2 n}$ as $\mathbb{R}^{p \times pn}$, so one $\mathbb{R}^{pn}$ vector for each of our p devices. We suggest playing around with small examples, say $n = 2$, $p = 3$, to see what these operators look like as matrices. Using the same transposition property, we once more obtain $\text{AllGather}^T = \text{ReduceScatter}$, and of course $\text{ReduceScatter}^T = \text{AllGather}$. This

transposition will arise during backpropagation, since if we have $y = Ax$ for some linear operator A, such as AllGather or ReduceScatter, then during backpropagation we will have the derivative of the loss with respect to y, $\frac{\partial L}{\partial y}$, and we obtain $\frac{\partial L}{\partial x}$ as $\frac{\partial L}{\partial x} = A^T \frac{\partial L}{\partial y}$. This shows how the derivative of AllGather will be ReduceScatter, and vice versa.

Turning an AllReduce into an AllGather and ReduceScatter also has the convenient property that we can defer the final AllGather until some later moment. Very commonly we'd rather not pay the cost of reassembling the full matrix product replicated across the devices. Rather we'd like to preserve a sharded state even in this case of combining two multiplicands with sharded contracting dimensions:

$A[I, J_X] \cdot B[J_X, K] \to C[I, K_X]$

In this case, we can also perform a ReduceScatter instead of an AllReduce, and then optionally perform the AllGather at some later time, i.e.

$$A[I, J_X] \cdot_{LOCAL} B[J_X, K] \to C[I, K]\{U_X\}$$
$$\textbf{ReduceScatter}_{X,K} C[I, K]\{U_X\} \to C[I, K_X]$$

Note that ReduceScatter *introduces* a sharded dimension, and so has a natural freedom to shard along either the **I** or **K** named dimensions in this case. We generally need to choose *which* named dimension to introduce a new sharding to when using a ReduceScatter (though the choice is usually forced by the larger modeling context). This is why we use the syntax **ReduceScatterX,K** to specify the axis to shard.

How to overlap matmul communication with compute

As we discussed in Part 1, we generally assume we can always overlap communication with some useful computation if the comms are fast enough. The collectives in this section generally can be overlapped with the matrix multiplication compute itself, but doing so is non-trivial. The algorithm we use is something called a **collective matmul**, first described in Wang et al.. Here is a simplified animation of how this overlap can be implemented:

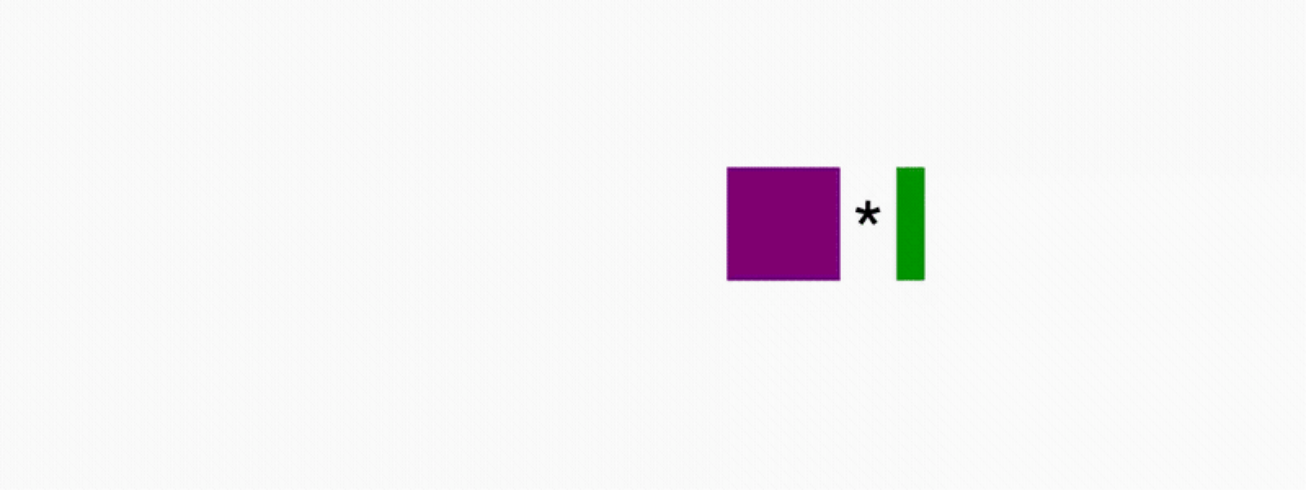
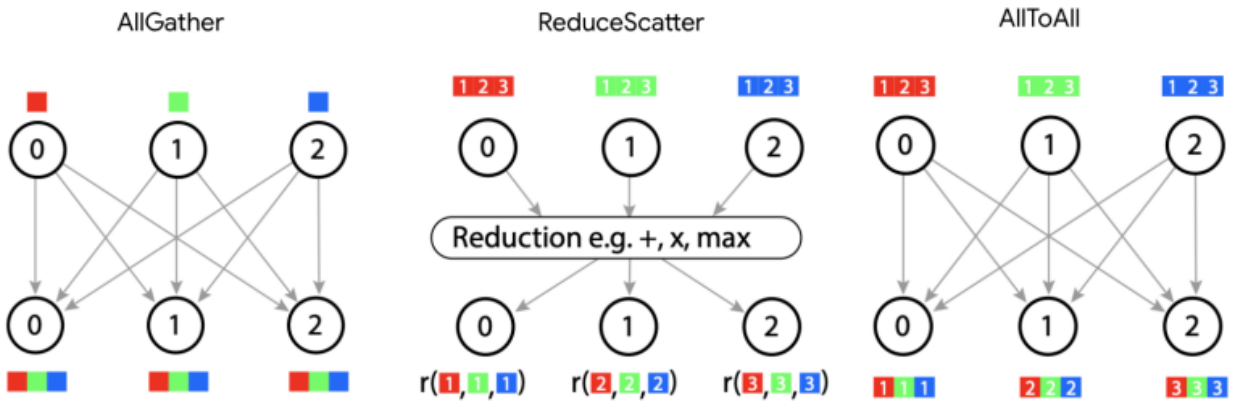


Figure 10: Figure: an animation showing how a single sharded matrix-vector product can be overlapped with the resulting AllReduce (case 3 above). A full matmul is composed of multiple matrix-vector products.

To put it simply, we can do the matmul for one chunk of the matrix while starting the ring reduction for previous chunks. In some cases we can also tile over the batch dimension or matrix input dimension. We work through a simple JAX implementation in Part 10 and the Mosaic docs also give a good example on GPU. We encourage you to implement a version of this at some point.

What Have We Learned?

- The sharding of an array is specified by a **Mesh** that names the physical, hardware axes of our TPU mesh and a **Sharding** that assigns mesh axis names to the logical axes of the array.
  - For example, **A**[IXY, J] describes an abstract array **A** with its first dimension sharded along two mesh axes X and Y. Combined with Mesh(mesh_shape=(4, 8), axis_names=('X', 'Y')) or the abbreviated Mesh({'X': 4, 'Y': 8}), this tells us our array is sharded 32 ways along the first dimension.
- **Arithmetic with sharded arrays works exactly like with unsharded arrays unless you perform a contraction along a sharded axis**. In that case, we have to introduce some communication. We consider four cases:
  1. *Neither array is sharded along the contracting dimension*: no communication is needed.
  2. *One array is sharded along the contracting dimension* (or the contracting dimensions are sharded along different axes): we AllGather one of the inputs before performing the operation.
  3. *Both arrays are identically sharded along the contracting dimension:* we multiply the shards locally then perform an AllReduce or ReduceScatter.
  4. *Both arrays are sharded along the same mesh axis along a non-contracting dimension:* we AllGather one of the inputs first.
- TPUs use roughly **4 core communication primitives**:
  1. AllGather: $[A_X, B] \to [A, B]$
  2. ReduceScatter: $[A, B]\{U_X\} \to [A_X, B]$
  3. AllToAll: $[A, B_X] \to [A_X, B]$
  4. AllReduce: $[A_X, B]\{U_Y\} \to [A_X, B]$ (technically not a primitive since it combines a ReduceScatter + AllGather)



- The cost and latency of each of these operations **doesn't depend on the size of the axis (as long as they're bandwidth bound)**, but only on the size of the input arrays and the bandwidth of the link. For a unidirectional AllGather/ReduceScatter:

$$T_{\text{comm per AllGather or ReduceScatter}} = \frac{\text{Data volume}}{\text{bandwidth}} \cdot \frac{\text{Axis} - 1}{\text{Axis}} \longrightarrow \frac{\text{Data volume}}{\text{bandwidth (bidirectional)}}$$

- An AllReduce is composed of a ReduceScatter followed by an AllGather, and thus has 2x the above cost. An AllToAll only has to pass shards part-way around the ring and is thus ¼ the cost of an AllGather. Here's a summary:

Operation	Description	Syntax	Runtime
AllGather	Gathers all the shards of a sharded array along an axis, removing a subscript.	$[A_X, B] \to [A, B]$	bytes / (bidirectional ICI bandwidth * num_axes)

Operation	Description	Syntax	Runtime
ReduceScatter	Sums a partially summed array along an axis and shards it along another axis (adding a subscript).	$[A, B]\{U_X\} \to [A_X, B]$	Same as AllGather
AllReduce	Sums a partially summed array along an axis. Removes a { Ux }. Combines an AllGather and ReduceScatter.	$[A_X, B]\{U_Y\} \to [A_X, B]$	2 * AllGather
AllToAll	Gathers (replicates) an axis and shards a different dimension along the same axis.	$[A, B_X] \to [A_X, B]$	AllGather / 4 for a bidirectional ring

Some Problems to Work

Here are some instructive problems based on content in this section. We won't include all answers at the moment but we'll write up more answers as we can.

Question 1 [replicated sharding]: An array is sharded $A[I_X, J, K, ...]$ (i.e., only sharded across X), with a mesh `Mesh({'X': 4, 'Y': 8, 'Z': 2})`. What is the ratio of the total number of bytes taken up by A across all chips to the size of one copy of the array?

Click here for the answer.

Our array is only sharded along X, which has size 4, so effectively each shard has size $[I/4, J, K, ...] = \text{sizeof}(A)/4$. Since our array is replicated across Y and Z, the total size is $Y \cdot Z \cdot \text{sizeof}(A)$, so the ratio of total size to single chip size is $Y \cdot Z \cdot \text{sizeof}(A)/\text{sizeof}(A) = 16$.

Question 2 [AllGather latency]: How long should $\text{AllGather}_X([B_X, D_Y])$ take on a TPU v4p 4x4x4 slice with mesh `Mesh({'X': 4, 'Y': 4, 'Z': 4})` if $B = 1024$ and $D = 4096$ in bfloat16? How about $\text{AllGather}_{XY}([B_X, D_Y])$? How about $\text{AllReduce}_Z([B_X, D_Y]\{U_Z\})$?

Click here for the answer.

We have a wraparound link on all axes because we have a full `4x4x4` cube, so we have 9e10 bidirectional bandwidth to work with.

1. Because we're just gathering over one axis and the other is sharded, we're effectively gathering $2BD/Y$ bytes over 1 axis. *If you think about just a single shard along the Y-axis, the AllGather along X looks like an unsharded AllGather with 1 / Y of the bytes.* Since our ICI bandwidth for TPU v4p is 9e10 bytes/second bidirectional, this will take $2BD/(9\text{e}10 \cdot Y) = 2 \cdot 1024 \cdot 4096/(9\text{e}10 \cdot 4) = 23\mu s$.
2. We have twice the bandwidth as before but we're AllGathering the full array, so `T = 2BD / (2 * W) = 2*1024*4096 / (2 * 9e10) = 46us`. This is far from the latency bound of 4us (1us per hop), so we're fine.
3. The cost of an AllReduce is twice that of an AllGather. Each shard has size $2BD/(X * Y)$, so the cost is about $4BD/(X * Y * W)$, or roughly `4 * 1024 * 4096 / (16 * 9e10) = 11.6us`.

Fun fact: parts (1) and (2) aren't actually optimal, because the array is also replicated along the unused Z axis, and we can exploit those idle links: we can first re-shard $[B_X, D_Y] \to [B_{XZ}, D_Y]$ for free (each device just drops part of its shard) and then perform AllGather_{XZ} (or AllGather_{XYZ}), reaching the same final state while gathering over more axes. This cuts part (1) to 11.5us and part (2) to 31us — in practice you'd get this by just sharding over more axes up front, which is one reason to shard arrays as finely as possible.

Question 3 [latency-bound AllGather]: Let's say we're performing an $\text{AllGather}_X([B_X])$ but B is very small (say 128). How long should this take on a TPU v4p 4x4x4 slice with mesh `Mesh({'X': 4, 'Y': 4, 'Z': 4})` in bfloat16? *Hint: you're probably latency bound.*

Click here for the answer.

Our array in bfloat16 uses only 256 bytes total, and only 64 per device. Since we have an axis of size 4 on a TPU v4p, we have a wraparound link, so we can send the array in both directions. With `4.5e10` of unidirectional bandwidth, each hop would take roughly `64 / 4.5e10 ~ 0`, so we're definitely latency bound. Counting the number of hops, we can do the full gather in only 2 hops, so roughly 2us a good estimate.

Question 4 [matmul strategies]: To perform $X[B, D] \cdot_D Y[D_X, F] \to Z[B, F]$, in this section we tell you to perform $\text{AllGather}_X(Y[D_X, F])$ and multiply the fully replicated matrices (Case 2, *Strategy 1*). Instead, you could multiply the local shards like $X[B, D_X] \cdot_D Y[D_X, F] \to Z[B, F]\{U_X\}$ (Case 3, *Strategy 2*), and then $\text{AllReduce}_X(Z[B, F]\{U_X\})$. How many FLOPs and comms does each of these perform? Which is better and why?

Click here for the answer.

Let's start with our baseline (*Strategy 1*). As we've shown, the cost of the AllGather is $2DF/W_{\text{ici}}$. Once we have the fully replicated arrays, the total compute time is $2BDF/C$ (where C is our accelerator FLOPs/s, since each TPU does the same FLOPs). So we have

$$T_{\text{total (Strategy 1)}} = \max\left(\frac{2BDF}{C}, \frac{2DF}{W_{\text{ici}}}\right)$$

By comparison, the new strategy (Strategy 2) does an AllReduce over $2BF$ bytes, which has cost $4BF/W_{\text{ici}}$ but does $1/X$ fewer FLOPs (since the computation is sharded). This means we do $2 \cdot B \cdot D \cdot F/X$ FLOPs and the resulting AllReduce communicates $2 \cdot 2 \cdot B \cdot F$ bytes in bfloat16. Thus, our total time for *Strategy 2* (no AllGather, just an AllReduce later on) is roughly

$$T_{\text{total}} = \max\left(\frac{2BDF}{X \cdot C}, \frac{4BF}{W_{\text{ici}}}\right)$$

The question is: *which of these is bigger?* Strategy (2) is compute bound when $D/(X \cdot C) > 2/W_{\text{ici}}$, or when $D/2X > C/W_{\text{ici}} \approx 2550 \to X < D/(2 * 2550)$. We might reasonably expect $D \approx 8k$, so this would mean roughly $X < 2$ which is unlikely – hence we're basically always comms bound with Strategy 2. With the baseline (Strategy 1), we're comms bound when $B < C/W_{\text{ici}} = 2550$ which is often but not always true.

So if $B < 2550$, we're comms-bound in both cases and we have

$$T_{\text{comms for Strategy 2}} < T_{\text{comms for Strategy 1}} \Leftrightarrow \frac{4BF}{W_{\text{ici}}} < \frac{2DF}{W_{\text{ici}}}$$

which is true when $D > 2B$ where $2B < 5100$. This is often true, so Strategy 2 can sometimes be better if our batch is small. When our batch is large ($B > 2550$), we have

$$T_{\text{comms for Strategy 2}} < T_{\text{math for Strategy 1}} \Leftrightarrow \frac{4BF}{W_{\text{ici}}} < \frac{2BDF}{C}$$

This is true when $2/W_{\text{ici}} < D/C$, or when $D > 2 * 2550 = 5100$, which is usually true for large models. So this alternative strategy is typically better for large models, unless D is small.

Why don't we always do this? Well, in practice we may do this sometimes, but it's typically rare to have the contracting dimension of one of the inputs to a matmul sharded along an axis that the other input isn't sharded over. For instance, if we're doing FSDP (explained in Section 5), we'll shard our parameters over the data dimension but our activations will *also be sharded along data*. So in this sense this doesn't show up much.

Question 5 [minimum latency]: Let's say I want to do a matmul $A[I, J] \cdot_J B[J, K] \to C[I, K]$ on a TPU v4p 4x4x4 with the lowest possible latency. Assume the inputs can be sharded arbitrarily but the result should be fully replicated. How should my inputs be sharded? What is the total FLOPs and comms time?

Click here for the (partial) answer.

We won't provide a full answer here, but we'll start by describing the four most likely options:

1. $A[I_{XYZ}, J] \cdot B[J, K]$ + AG at the end
2. $A[I, J] \cdot B[J, K_{XYZ}]$ + AG at the end
3. $A[I, J_{XYZ}] \cdot B[J_{XYZ}, K]$ + AR at the end
4. $A[I, J] \cdot B[J, K]$ (fully replicated)

We could also consider sharding different axes along different mesh axes, but that isn't likely to change the final cost. For all but (4), the total FLOPs per TPU is the same, but comms are different for each. We then simply need to calculate the comms cost for each and see which is lowest. The TLDR is that (1) and (2) are equally good.

Question 6: Let's say we want to perform $A[I_X, J_Y] \cdot_J B[J_Y, K] \to C[I_X, K]$ on TPU v5e 4x4. What communication do we perform? How much time is spent on communication vs. computation?

- What about $A[I_X, J] \cdot_J B[J_X, K_Y] \to C[I_X, K_Y]$? This is the most standard setting for training where we combine data, tensor, and ZeRO sharding.
- What about $A[I_X, J] \cdot_J B[J, K_Y] \to C[I_X, K_Y]$? This is standard for inference, where we do pure tensor parallelism (+data).

Question 7: A typical Transformer block has two matrices $W_{\text{in}}[D, F]$ and $W_{\text{out}}[F, D]$ where $F \gg D$. Say we have a batch size B. Then the full block is $In[B, D] \cdot W_{\text{in}}[D, F] \cdot W_{\text{out}}[F, D]$. Let's pick $D = 8192$, $F = 32768$, and $B = 128$ and assume everything is in bfloat16. Assume we're running on a TPU v5e 2x2 slice but let's pretend each TPU only has 300MB of free memory. How should In, W_{in}, W_{out}, and Out be sharded to stay below the memory limit while minimizing overall time? How much time is spent on comms and FLOPs? *Hint: the final output doesn't need to be fully replicated, but it should be sharded the same as the input so the "layer" can be repeated.*

Click here for the (partial) answer.

First let's think about memory. Each of our two big matrices uses `2 * 8192 * 32768 = 536MB`. Our activations `In` have size `2 * 128 * 8192 = 2MB` (small enough not to worry about). Since we only have 300MB of spare memory in each device, we clearly need to shard our matmuls.

1. $In[B_X, D] * W_{\text{in}}[D_{XY}, F] * W_{\text{out}}[F, D_{XY}] \to Out[B_X, D]$ (this is often called FSDP)
2. $In[B, D_{XY}] * W_{\text{in}}[D, F_{XY}] * W_{\text{out}}[F_{XY}, D] \to Out[B, D_{XY}]$ (this is called tensor parallelism)

The first is pretty bad because we need to AllGather our big weights or our activations first. The second requires an AllGather at the beginning and a ReduceScatter at the end (which is cheaper than an AllReduce). I'll leave it as an exercise to do the rest of the math.

Question 8 [challenge]: Using the short code snippet above as a template, allocate a sharded array and benchmark each of the 4 main communication primitives (AllGather, AllReduce, ReduceScatter, and AllToAll) using pmap or shard_map. You will want to use `jax.lax.all_gather`, `jax.lax.psum`, `jax.lax.psum_scatter`, and `jax.lax.all_to_all`. Do you understand the semantics of these functions? How long do they take?

Question 9 [another strategy for sharded matmuls?]: Above we claimed that when only one input to a matmul is sharded along its contracting dimension, we should AllGather the sharded matrix and perform the resulting contraction locally. Another strategy you might think of is to perform the sharded matmul and then AllReduce the result (as if both inputs were sharded along the contracting dimension), i.e. $A[I, J_X] *_J B[J, K] \to C[I, K]$ by way of

1. $C[I, K]\{U_X\} = A[I, J_X] \cdot B[J_X, K]$
2. $C[I, K] = \text{AllReduce}(C[I, K]\{U_X\})$

Answer the following:

1. Explicitly write out this algorithm for matrices $A[N, M]$ and $B[M, K]$, using indices to show exactly what computation is done on what device. Assume A is sharded as $A[I, J_X]$ across ND devices, and you want your output to be replicated across all devices.
2. Now suppose you are ok with the final result not being replicated on each device, but instead sharded (across either the N or K dimension). How would the algorithm above change?
3. Looking purely at the communication cost of the strategy above (in part 2, not 1), how does this communication cost compare to the communication cost of the algorithm in which we first AllGather A and then do the matmul?

Click here for the answer.

1. First compute the outer products, storing the result in $O[N, K] : o_{kj} = \sum_i a_{ki} b_{ij}$. Note that the repeated index is not the one being contracted, as we are doing an outer product. Here the sum ranges across the set of i values stored on the particular device we are using. So, for example, if we have a contracting axis of size 16, and 4 devices, then on device 0, i would range from {0, 1, 2, 3}; on device 1, i would range from {4, 5, 6, 7}; on device 2, i would range from {8, 9, 10, 11}; and on device 3, i would range from {12, 13, 14, 15}. Then AllReduce the partial-sums of $O[N, K]$ which live on each device, to form the full $O[N, K]$.
2. Instead of doing an AllReduce in step 2, we could get away with a cheaper ReduceScatter, along either axis: $[N, K]\{U_X\} \to [N_X, K]$ or $[N, K]\{U_X\} \to [N, K_X]$.

3. As described in the main text above, the cost of doing an AllGather (when we are throughput-bound) is the same as that of a ReduceScatter; it is simply given by the size of the full matrix we are processing. So in the gather-then-matmul algorithm, this scales as NM (since we are AllGather-ing A); in the matmul-then-reduce-scatter algorithm, this scales as NK (since we are reduce-scattering O). So the communication cost ratio of the two algorithms is `M/K`.

Question 10: Fun with AllToAll: In the table above, it was noted that the time to perform an AllToAll is a factor of 4 lower than the time to perform an AllGather or ReduceScatter (in the regime where we are throughput-bound). In this problem we will see where that factor of 4 comes from, and also see how this factor would change if we only had single-direction ICI links, rather than bidirectional ICI links.

1. Let's start with the single-direction case first. Imagine we have D devices in a ring topology and want to do either an AllGather or a ReduceScatter on an N x N matrix $A[I_X, J]$ (say D divides N for simplicity). Describe the comms involved in these two collectives, and calculate the total number of scalars (floats or ints) which are transferred across **a single** ICI link during the entirety of this algorithm.
2. Now let's think about an AllToAll, still in the single-directional ICI case. How is the algorithm different in this case than the all-gather case? Calculate the number of scalars that are transferred across a single ICI link in this algorithm.
3. You should have found that the ratio between your answers to part (a) and part (b) is a nice number. Explain where this factor comes from in simple terms.
4. Now let's add bidirectional communication. How does this affect the total time needed in the all-gather case?
5. How does adding bidirectional communication affect the total time needed in the AllToAll case?
6. Now simply explain the ratio between AllGather time and AllToAll time in a bidirectional ring.

Click here for the answer.

(1) **Solution:** The process is simple: in each step of the algorithm, each device will send a single-shard "strip" of the matrix (totalling $\frac{N}{D} \times N$ elements in size) to its nearest neighbor. This occurs $D-1$ times, since each shard needs to be communicated to all of the devices except the one it starts out on. So in total, $\frac{N^2(D-1)}{D}$ scalars are transferred by each device, i.e. flow across a single ICI link.

Answer: $N^2(1-\frac{1}{D})$, or simply N^2 when $D >> 1$.

(2) **Solution:** The key difference between an AllToAll and an AllGather, from the perspective of communications, is that in an AllToAll, the entirety of the shard that lives on a particular device does not need to be communicated to every other device. Imagine the shard stored on a particular device (call it device 0) is $[A, B, C, D]$ (here A,B,C,D are matrices and we are imagining a ring with 4 devices for illustration). Now the matrix A does not need to be communicated anywhere, the matrix B needs to end up on device 1; matrix C ends up on device 2; and matrix D ends up on device 3. So in the first step of the algorithm, we send B, C, and D to device 1; in the next step, device 1 sends C and D onwards to device 2; in the final step, device 2 sends just D on to device 3. The total number of parameters transferred in this case is (size of A/B/C/D) $*$ $(3+2+1)$. The size of A/B/C/D is (in the general case now) $\frac{N^2}{D^2}$, and again in the general case the $(3+2+1)$ term becomes $((D-1)+(D-2)+...+1)$, or $\frac{(D)(D-1)}{2}$. So the total number of bytes transferred across a single ICI link is $\frac{N^2(D-1)}{D\times 2}$.

Answer: $\frac{N^2}{2}(1-\frac{1}{D})$, or simply $\frac{N^2}{2}$ when $D >> 1$.

(3) **Solution:** The factor is simply $\frac{1}{2}$, i.e. an AllToAll is half as costly as an all-gather/ReduceScatter on a unidirectional ring topology. Looking over the derivations above, this ultimately came from the fact that in the all-gather case, we are transferring the same sized block each of $(D-1)$ times, i.e. we're doing the sum tiny block size $*$ $(D+D+D+...+D)$, whereas in the AllToAll case, we're doing the sum tiny block size $*$ $(D+D-1+D-2+...+1)$. The factor of two thus essentially comes from the fact that $1+2+...+n = n(n+1)/2$.

(4) **Solution**: The total number of scalars that any one link has to carry now reduces by a factor of 2, since in a bidirectional ring, each "sharded strip" can be sent two ways simultaneously.

(5) **Solution**: In this case, we win a factor of 4 compared to the unidirectional case. This is easiest to see by considering the fate of each of the size-(N2/D2) blocks in a single sharded strip, say the one which

originates on device 0. Instead of (as in the unidirectional case) sending one of these blocks a distance of D-1, another block a distance D - 2, etc. all the way to 1, we now divide the strip into blocks which move right or left, moving a maximum distance of floor(D/2). So the corresponding sum now becomes $D/2 + D/2 - 1 + D/2 - 2 + ... = D/2 \cdot (D/2 + 1)/2$, or $D^2/8$ in the limit of large D. Compare this to $D^2/2$ in the unidirectional case, and we see that we've won a factor of 4.

(6) **Solution:** In a unidirectional ring, we saw that the AllToAll time was already twice as fast as the all-gather time; this comes from the fact that we don't need to send our full strip to every single device. Then, when we added bidirectionality, we saw that it was a 4x win for AllToAll, and only a 2x win for all-gathers. Putting these ratios together, we get our sought after factor of 4.

That's it for Part 3! For Part 4 (about Transformer math), click here!

Chapter 4

All the Transformer Math You Need to Know

Counting Dots

Let's start with vectors x, y and matrices A, B of the following shapes:

$$

array	shape
x	[P]
y	[P]
A	[N P]
B	[P M]

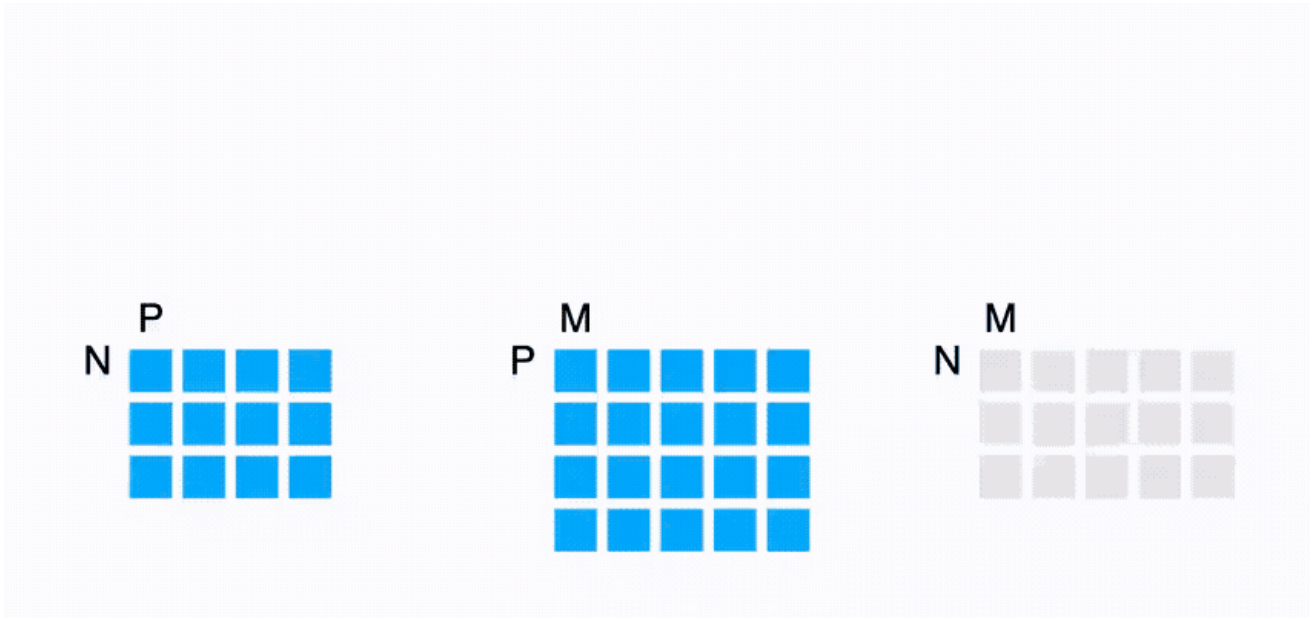
$$

- A dot product of $x \cdot y$ requires P *adds* and *multiplies*, or $2P$ floating-point operations total.
- A matrix-vector product Ax does N dot-products along the rows of A, for $2NP$ FLOPs.
- A matrix-matrix product AB does a matrix-vector product for each of the M columns of B, for $2NPM$ FLOPs total.
- In general, if we have two higher-dimensional arrays C and D, where some dimensions are CONTRACTING and some are BATCHING (e.g. $C[GHIJKL], D[GHMNKL]$), then the FLOPs cost of this contraction is two times the product of all of the C and D dimensions where the batch and contraction dimensions are only counted once (e.g. $2GHIJMNKL$). Note that a dimension is only batching if it occurs in both multiplicands. (Note also that the factor of 2 won't apply if there are no contracting dimensions and this is just an elementwise product.)[35]

Operation	FLOPs	Data
$x \cdot y$	$2P$	$2P$
Ax	$2NP$	$NP + P$
AB	$2NPM$	$NP + PM$
$[c_0, ..., c_N] \cdot [d_0, ..., d_N]$	$2\prod c_i \times \prod_{\substack{d_j \notin BATCH \\ d_j \notin CONTRACT}} d_j$	$\prod c_i + \prod d_j$

Make note of the fact that for a matrix-matrix multiply, the *compute* scales cubically $O(N^3)$ while the data transfer only scales quadratically $O(N^2)$ — this means that as we scale up our matmul size, it becomes *easier* to hit the compute-saturated limit. This is extremely unusual, and explains in large part why we use architectures dominated by matrix multiplication — they're amenable to being scaled!

[35] Contracting dimensions are axes that are summed over during the operation (they appear in both inputs but not in the output), like the inner dimension in a matrix multiply. Batching dimensions are shared axes that appear in both inputs and are carried unchanged to the output; they index independent subproblems and aren't multiplied together in FLOP counts. In einsum terms: labels present on both inputs and the output are batching; labels present on both inputs but absent from the output are contracting.



Forward and reverse FLOPs

During training, we don't particularly care about the result of a given matrix multiply; we really care about its derivative. It turns out that calculating that derivative costs about 3x more than just doing the matmul itself.

If we imagine **B** is just one matrix in a larger network and **A** are our input activations with $\mathbf{C} = \mathbf{A}\ \mathbf{B}$, the derivative of the loss **L** with respect to **B** is given by the chain rule:

$\frac{\partial L}{\partial B} = \frac{\partial L}{\partial C}\frac{\partial C}{\partial B} = A^T\left(\frac{\partial L}{\partial C}\right)$

which requires $2NPM$ FLOPs to compute (since it contracts over the N dimension). Likewise, the derivative of the loss with respect to **A** is

$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial C}\frac{\partial C}{\partial A} = \left(\frac{\partial L}{\partial C}\right)B^T$

which is again $2NPM$ FLOPs since **dL/dC** is a matrix of size $[N, M]$. While this quantity isn't the derivative w.r.t. a parameter, it's used to compute derivatives for previous layers of the network (e.g. just as dL/dC is used to compute dL/dB above).

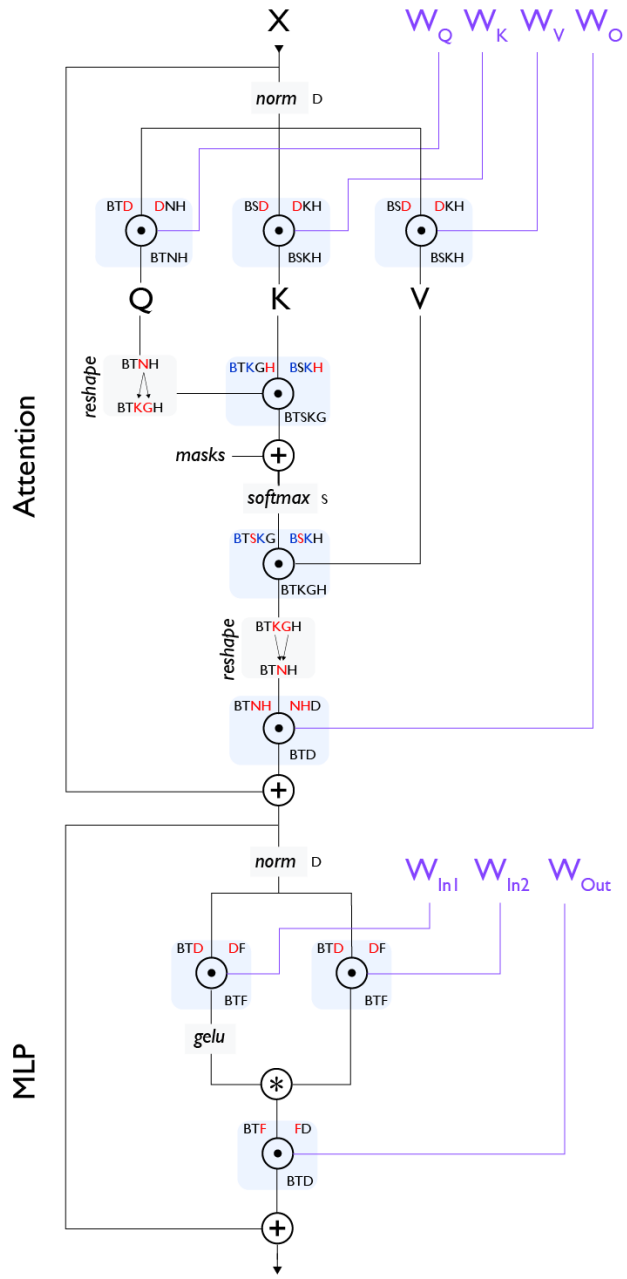
Adding these up, we see that **during training, we have a total of 6NPM FLOPs**, compared to 2NPM during inference: 2NPM in the forward pass, 4NPM in the backward pass. Since PM is the number of parameters in the matrix, this is the simplest form of the famous $6 *$ num parameters $*$ num tokens approximation of Transformer FLOPs during training: each token requires $6 *$ num parameters FLOPs. We'll show a more correct derivation below.

Transformer Accounting

Transformers are the future. Well, they're the present at least. Maybe a few years ago, they were one of many architectures. But today, it's worth knowing pretty much every detail of the architecture. We won't reintroduce the architecture, but this blog and the original Transformer paper may be helpful references.

Here's a basic diagram of the Transformer decoder architecture:

Note [gating einsum]: The diagram above uses a "gating einsum" where we split the up-projection matrix into two matrices (W_{In1} and W_{In2} above) whose outputs are elementwise multiplied as a kind of "gating function". Not all LLMs use this, so you will sometimes see a single W_{In} matrix and a total MLP parameter count of 2DF instead of 3DF. Typically in this case, D and F will be scaled up to keep the parameter count the same as the 3 matrix case. With that said, some form of gating einsum is used by LLaMA, DeepSeek, and many other models.



symbol	dimension
B	batch
L	number of layers
T	sequence length (query)
S	sequence length (key value)
V	vocab
D	d_model, embedding dimension
F	MLP hidden dimension
H	attention head dimension
N	number of query heads
K	number of key/value heads
G	q heads per kv head = N // K

Figure 11: Figure: this diagram shows one layer of a standard Transformer and flows from top-to-bottom. We use a single-letter convention to describe the shapes and layouts of arrays in a Transformer, again showing contracting dimensions in red, and batched dimensions in blue. In a given operation, the input shape is given on top-left and the parameter shape is given on the top-right, with the resulting shape below, e.g. BTD is the input shape for the gating einsum and DF is the weight shape.

Note 2 [MHA attention]: With self-attention, T and S are the same but for cross-attention they may be different. With vanilla Multi-Head Attention (MHA), N and K are the same while for Multi-Query Attention (MQA) K=1 and for Grouped MQA (GMQA), K merely has to divide N.

Note 3 [pre-norm vs. post-norm]: The above diagram shows what is known as a "pre-norm" architecture in which the norm occurs before the residual connection, usually as `x + attn(norm(x))`. Models like LLaMA-3 use this today. The original Transformer paper used a "post-norm" architecture in which the layernorm occurs after the residual connection, i.e. `norm(x + attn(x))`.

Global FLOPs and Params Calculation

Let's calculate the per-layer FLOPs of a Transformer (so we can avoid having to stick factors of **L** everywhere). Note that the training FLOPs below are almost always 3x the inference FLOPs, so you can divide any total by 3 to get the cost of just the forward pass.

MLPs

The MLPs of a Transformer typically consist of 2 input matmuls that are element-wise combined and a single output matmul:

operation	train FLOPs	params
$A[B,T,D] \cdot W_{in1}[D,F]$	$6BTDF$	DF
$A[B,T,D] \cdot W_{in2}[D,F]$	$6BTDF$	DF
$\sigma\left(A_{in1}\right)[B,T,F] * A_{in2}[B,T,F]$	$O(BTF)$	
$A[B,T,F] \cdot W_{out}[F,D]$	$6BTDF$	DF
	$\approx 18BTDF$	$3DF$

Attention

For the generic grouped-query attention case with different **Q** and **KV** head numbers, let us assume equal head dimension H for **Q**,**K**,**V** projections, and estimate the cost of the **QKVO** matmuls:

operation	train FLOPs	params
$A[B,T,D] \cdot W_Q[D,N,H]$	$6BTDNH$	DNH
$A[B,T,D] \cdot W_K[D,K,H]$	$6BTDKH$	DKH
$A[B,T,D] \cdot W_V[D,K,H]$	$6BTDKH$	DKH
$A[B,T,N,H] \cdot W_O[N,H,D]$	$6BTDNH$	DNH
	$12BTD(N+K)H$	$2D(N+K)H$

The dot-product attention operation is more subtle, effectively being a $TH \cdot HS$ matmul batched over the B, K dimensions, a softmax, and a $TS \cdot SH$ matmul again batched over the B, K dimensions. We highlight the batched dims in blue:

operation	train FLOPs
$Q[B,T,K,G,H] \cdot K[B,S,K,H]$	$6BTSKGH = 6BTSNH$
$\text{softmax}_S \;\; L[B,T,S,K,G]$	$O(BTSKG) = O(BTSN)$
$S[B,T,S,K,G] \cdot V[B,S,K,H]$	$6BTSKGH = 6BTSNH$
	$\approx 12BTSNH = 12BT^2NH$

Note [causal masking]: Most recent transformers use a causal mask as opposed to full bidirectional attention. In this case the useful FLOPs of the dot product operations are reduced by half. To achieve this reduction in practice we need to make use of an attention kernel, rather than a naive einsum.

Other Operations

There are several other operations happening in a Transformer. Layernorms are comparatively cheap and can be ignored for first-order cost estimates. Note that each layer typically has two of them (one before attention and one before the MLP). There is also the final enormous (though not per-layer) unembedding matrix multiply.

operation	train FLOPs	params
$2 \times \text{layernorm}_D \;\; A[B,T,D]$	$O(BTD)$	$2D$
$A[B,T,D] \cdot W_{unembed}[D,V]$	$6BTDV$	DV

General rule of thumb for Transformer FLOPs

If we neglect the cost of dot-product attention (which is reasonable for shorter-context training), then the total FLOPs across all layers is

$$\begin{aligned}(18BTDF + 12BTD(N+K)H)L &= 6 * BT * (3DF + 2D(N+K)H)L \\ &= 6 * \text{num tokens} * \text{parameter count}\end{aligned}$$

This leads to a famous rule of thumb for estimating dense Transformer FLOP count, ignoring the attention FLOPs. (Unembedding is another simple matmul with $6BTDV$ FLOPs and DV params, and follows the same rule of thumb.)

Fractional cost of attention with context length

If we do account for dot-product attention above and assume $F = 4D$, $D = NH$ (as is typical) and $N = K$, the ratio of dot-product attention FLOPs to all matmul FLOPs (including the attention projections) is:

$$\frac{\text{attention FLOPs}}{\text{matmul FLOPs}} = \frac{12BT^2NH}{18BTDF+24BTDNH} = \frac{12BT^2D}{4*18BTD^2+24BTD^2} = \frac{12BT^2D}{96BTD^2} = \frac{T}{8D}$$

The upshot is that **dot-product attention FLOPs only become dominant during training once T>8D**. For D ~ 8k, this would be ~64K tokens. This makes some sense, since it means as the MLP size increases, the attention FLOPs become less critical. For large models, the quadratic cost of attention is not actually a huge obstacle to longer-context training. However, for smaller models, e.g. Gemma-27B with D=4608, attention becomes dominant around 37k sequence lengths.[36] Flash Attention also helps alleviate the cost of long-context, which we discuss briefly in Appendix A.

[36] Note that some modern OSS models introduce local attention or other optimizations that reduce the cost of attention and change this roofline.

Miscellaneous Math

Sparsity and Mixture-of-Experts

We'd be remiss not to briefly discuss Mixture of Experts (MoE) models, which replace the single dense MLP blocks in a standard Transformer with a set of independent MLPs that can be dynamically routed between. To a first approximation, **an MoE is just a normal dense model with E MLP blocks per layer**, instead of just one. Each token activates k of these experts, typically $k \ll E$. The ratio E/k is called the sparsity and is usually between 8 and 64 (e.g. DeepSeek v3 has effectively $k = 8$, $E = 256$). This increases the parameter count by $O(E)$, while multiplying the total number of activated parameters per token by k, compared with the dense version.

![Figure: an example MoE layer with n experts. The gating expert routes each token to k of them, and the output of those k MLPs get summed. Our parameter count is n times the size of each expert, but only k are used for each token. <a href=](assets/img/moe.png)

Compared to a dense model, an MoE introduces new comms, primarily two AllToAlls (one before and one after the MoE block) that route tokens to the correct expert and bring them back to their home device.[37] However, as we saw in the previous section, the cost of each AllToAll is only 1/4 that of a comparable AllGather along a single axis (for a bidirectional ring).

Gradient checkpointing

Backpropagation as an algorithm trades compute for memory. Instead of a backward pass requiring $O(n^2_{\text{layers}})$ FLOPs, **it requires $O(n_{\text{layers}})$ memory**, saving all intermediate activations generated during the forward pass. While this is better than quadratic compute, it's incredibly expensive memory-wise: a model with $B * T = 4M$ (4M total tokens per batch), L=64, and D=8192 that avoids all unnecessary backward pass compute would have to save roughly $2 * 20 * B * T * D * L = 84TB$ of activations in bfloat16. The 20 comes from (roughly) counting every intermediate node in the Transformer diagram above, since e.g.

$f(x) = \exp(g(x))$

$\frac{df}{dx} = \exp(g(x)) \cdot \frac{dg}{dx}$

so to avoid recomputing we need to save $g(x)$ and $\exp(g(x))$ from the forward pass. To avoid saving this much memory, we can choose to only save some fraction of the intermediate activations. Here are a few strategies we use.

- **Block remat**: only save the input to each layer. This is the most aggressive method we use and only saves 1 checkpoint per layer, meaning we'd only save 4.2TB in the example above. This forces us to repeat essentially all forward pass FLOPs in the backward pass, meaning we increase our FLOPs from $6 \cdot \text{num params} \cdot \text{num tokens}$ to roughly $8 \cdot \text{num params} \cdot \text{num tokens}$.
- **Big matmuls only:** another simple policy is to only save the outputs of large matmuls. This lets us avoid recomputing any large matmuls during the backward pass, but still makes us recompute other activation functions and parts of attention. This reduces the 20 per layer above to closer to 7 per layer.

This is by no means comprehensive. When using JAX, these are typically controlled by `jax.remat/jax.checkpoint` (you can read more here).

Key-Value (KV) caching

As we'll see in Section 7, LLM inference has two key parts, prefill and generation.

- **Prefill** processes a long prompt and saves its attention activations in a Key-Value Cache (KV Cache) for use in generation, specifically the key-value projections in the attention block.
- **Generation** batches several of these KV caches together and samples tokens from each of them.

Each KV cache is then effectively an array of size $[2, S, L, K, H]$ where the 2 accounts for the keys and values. This is quite large! The total size of the Key-Value cache in int8 is $2SLKH$. For a moderately sized model with

[37] Technically, this only happens if we are data or sequence sharded along the same axis as our experts.

8k context length, 64 layers, and $KH = NH = D = 8192$, this is $2 \cdot 8192 \cdot 64 \cdot 8192 = 8$GiB. You can see why we would want to use GMQA with $K \ll N$.

What Should You Take Away from this Section?

- The overall parameters and FLOPs of a Transformer are fairly easy to calculate, and are summarized here, assuming MHA (with batch size B, vocab size V, a sequence of length T, D=dmodel, and F=dff):

Component	Params per layer	Training FLOPs per layer
MLP	3DF	18BTDF
Attention	4DNH	24BTDNH + 12BT2NH
Other	2D	BTD
Vocab	DV (total, not per-layer)	12BTDV

- The parameter count of the MLP block dominates the total parameter count and the MLP block also dominates the FLOPs budget as long as the sequence length $T < 8D$.
- The total FLOPs budget during training is well approximated by 6·num_params·num_tokens for reasonable context lengths.
- During inference, our KV caches are roughly $2 \cdot S \cdot L \cdot K \cdot H$ per cache (where K is the number of KV heads), although architectural modifications can often reduce this.

A Few Problems to Work

Question 1: How many parameters does a model with $D = 4096$, $F = 4 \cdot D$, $V = 32,000$, and $L = 64$ have? What fraction of these are attention parameters? How large are our KV caches per token? *You can assume $N \cdot H = D$ and multi-head attention with int8 KVs.*

Click here for the answer.

1. The total parameters is roughly $L \cdot (3DF + 4DNH + 2D) + 2DV$ (counting the two layernorms per layer). For the given numbers, this is $64 \cdot (3 \cdot 4e3 \cdot 16e3 + 4 \cdot 4e3 \cdot 4e3 + 2 \cdot 4e3) + 2 \cdot 4e3 \cdot 32e3 = 16e9$, or 16B parameters.
2. The ratio of attention parameters to total parameters in general is $4DNH/(4DNH + 3DF) = 4D^2/(4D^2 + 12D^2) = 1/4$. This means roughly 1/4 of the parameters are used in attention.
3. Per token, our KV caches are $2 \cdot L \cdot N \cdot H = 2 \cdot 64 \cdot 4096$ in int8, which is `512 KiB / token`.

Question 2: How many total FLOPs are required to perform A[BX, DY] *D W[DY, F] on `{'X': 4, 'Y': 8, 'Z': 4}`? How many FLOPs are performed by each TPU?

Click here for the answer.

The total "theoretical" FLOPs of the operation is $2 \cdot B \cdot D \cdot F$. However, because the computation isn't sharded across the Z dimension, we're actually doing Z extra FLOPs, meaning $2 \cdot B \cdot D \cdot F \cdot Z$ total FLOPs. Since the computation is sharded across the other dimensions, the total per-device is roughly $2 \cdot B \cdot D \cdot F/(X \cdot Y)$.

Question 3: How many FLOPs are involved in performing $A[I, J, K, L] * B[I, J, M, N, O] \to C[K, L, M, N, O]$?

Click here for the answer.

Following the rule above, we have I and J as contracting dimensions and K, L, M, N, and O as non-contracting dimensions. We have no "batching dimensions", so this is just $2 \cdot I \cdot J \cdot K \cdot L \cdot M \cdot N \cdot O$, the product of all the axes. If we had a shared axis, it would only be counted once.

Question 4: What is the arithmetic intensity of grouped multi-query attention (ignoring the Q/K/V/O projections)? *Give the answer as a function of the Q and KV sequence lengths T and S and the multi-query factor G.* At what context length is attention FLOPs-bound? Given the HBM bandwidth of our TPUs, plot the effective relative cost of attention to the FFW block as the context length grows. *Hint: assume we're using an efficient*

attention implementation that doesn't do any unnecessary reads/writes. Consider both the limits where $T = S$ and $T \ll S$.

Click here for the answer.

Self-attention requires loading the Q, K, and V activations, then computing $\text{softmax}(Q \cdot K) \cdot V$, then writing the result back to HBM. This will be done with Flash Attention so there are some caveats to this math, but basically in bf16 self-attention performs

$\text{Q[B,T,N,H]} \rightarrow_{\text{reshape}} \text{Q[B, T, K, G, H]} \cdot \text{K[B, S, K, H]} \rightarrow \text{O[B, T, S, K, G]}$

$U = \text{softmax}_S(\text{O[B, T, S, K, G]})$

$\text{U[B, T, S, K, G]} \cdot \text{V[B, S, K, H]} \rightarrow \text{X[B, T, K, G, H]}$

So our total bytes is $2 * \text{sizeof}(Q) + 2 * \text{sizeof(K or V)} = 4BTNH + 4BSKH = 4BHK * (TG + S)$, total FLOPs is $4BTSNH + O(BTSN)$ and the arithmetic intensity is $4BTSKGH / (4BHK * (TG + S))$.

So basically, during prefill we have $S = T$ so we have an arithmetic intensity of $4BT^2KGH / 4BHKT \cdot (G + 1) = TG/(G + 1) = O(T)$. During generation, $T = 1$ so we have $4BSKGH/(4BHK \cdot (G + S)) = SG/(G + S) \rightarrow G$ assuming S is very large. Depending on how you interpret the question, during prefill or training self-attention is compute-bound at S=240 assuming no sequence sharding. During generation, we are never compute-bound because G is small. Nonetheless, you can see that increasing G leads to us being closer to compute-bound.

Question 5: At what sequence length are self-attention FLOPs equal to the QKVO projection FLOPs?

Click here for the answer.

This is purely a question of when $24BTDNH = 12BT^2NH$. Simplifying we get $2D = T$, so e.g. for $D = 4096$, this is 8192. This tells us that for most reasonable context lengths, matmul FLOPs are greater.

Question 6: Say we only save the output of each of the 7 main matmuls in a Transformer layer during our forward pass (Q, K, V, O + the three FFW matrices). How many extra FLOPs do we need to "rematerialize" during the backward pass?

Click here for the answer.

Saving only the seven matmul outputs (Q, K, V, O, W_1, W_2, W_3) means the backward pass must recompute the two attention matmuls

$QK^\top \quad \text{and} \quad \text{softmax}(QK^\top)V$

in order to obtain $\frac{\partial L}{\partial W_O}$.

Each is a $T \times T$ matmul batched over B sequences and N heads, so the additional FLOPs are

$4\,B\,T^2\,N\,H.$

Other recomputed operations are: 1. $O(BTD)$ for $\frac{\partial L}{\partial W_{\text{In1}}}$ and $\frac{\partial L}{\partial W_{\text{In2}}}$. 2. And $O(BTF)$ for $\frac{\partial L}{\partial W_{\text{Out}}}$.

Question 7: DeepSeek v3 says it was trained for 2.79M H800 hours on 14.8T tokens (source). Given that it has 37B activated parameters, roughly what hardware utilization did they achieve? *Hint: note that they used FP8 FLOPs without structured sparsity.*

Click here for the answer.

From the spec sheet here, we find 3,026 TFLOPs/s of FP8 performance with sparsity, or typically half this (`1.513e15` FLOPs/s) without sparsity. 2.79M H800 hours means `2.79e6 * 1.513e15 * 60 * 60 = 1.52e25` total FLOPs. Given the activated parameter count of 37B, this training run should have used about `6 * 37e9 * 14.8e12 = 3.3e24` FLOPs. That means the FLOPs utilization is about `3.3e24 / 1.52e25 = 21.7%`.

Question 8: Mixture of Experts (MoE) models have E copies of a standard dense MLP block, and each token activates k of these experts. What batch size in tokens is required to be compute-bound for an MoE with weights in int8 on TPU v5e? For DeepSeek, which has 256 (routed) experts and $k = 8$, what is this number?

Click here for the answer.

Because we have E copies of each expert, in int8, for each weight matrix we need to load $E \cdot D \cdot F$ bytes. Because each token activates k experts, for each weight matrix we have $2 \cdot k \cdot B \cdot D \cdot F$ FLOPs. To be compute-bound with int8 weights and bfloat16 FLOPs, we need the arithmetic intensity (FLOPs per byte loaded) to exceed the TPU's ~240 FLOPs/byte, which happens when $(2 \cdot k \cdot BDF)/EDF > 240$ or $k \cdot B/E > 120$.

Therefore, we need $B > 120 \cdot E/k$ to be compute-bound. For DeepSeek, this gives us $B > 120 \cdot 256/8 = 3840$. This is a remarkably large batch size at generation time.

That's it for Part 4! For Part 5 (about scaling Transformer training), click here!

Appendix

Appendix A: How does Flash Attention work?

The traditional objection to scaling Transformers to very long context is that the attention FLOPs and memory usage scale quadratically with context length. While it's true that the attention QK product has shape $[B, T, S, N]$ where B is the batch size, T and S are the Q and K sequence dims, and N is the number of heads, this claim comes with some serious caveats:

1. As we noted earlier, even though this is quadratic, the attention FLOPs only dominate when $T > 8 \cdot D$, and during training the memory of a single attention matrix is small compared to all of the weights and activation checkpoints living in memory, especially when sharded.
2. We don't need to materialize the full attention matrix in order to compute attention! We can compute local sums and maxes and avoid ever materializing more than a small chunk of the array. While the total FLOPs is still quadratic, we drastically reduce memory pressure.

This second observation was first made by Rabe et al. 2021 and later in the Flash Attention paper (Dao et al. 2022). The basic idea is to compute the attention in chunks of K/V, where we compute the local softmax and some auxiliary statistics, then pass them on to the next chunk which combines them with its local chunk. Specifically, we compute

1. **M:** The running max of $q \cdot k$ over the sequence dimension
2. **O:** The running full attention softmax over the sequence dimension
3. **L:** The running denominator $\sum_i \exp(q \cdot k_i - \text{running max})$

With these, we can compute the new max, the new running sum, and the new output with only a constant amount of memory. To give a sketchy description of how this works, attention is roughly this operation:

$\text{Attn}(Q, K, V) = \sum_i \frac{\exp(Q \cdot K_i - \max_j Q \cdot K_j) V_i}{\sum_l \exp(Q \cdot K_l - \max_j Q \cdot K_j)}$

The max is subtracted for numerical stability and can be subtracted without affecting the outcome since $\sum_i \exp(a_i + b) = \exp(b) \sum \exp(a)$. Looking just at the denominator above, if we imagine having two contiguous chunks of key vectors, K^1 and K^2 and we compute the local softmax sums L^1 and L^2 for each

$L^1 = \sum_i \exp(Q \cdot K_i^1 - \max_j Q \cdot K_j^1)$

$L^2 = \sum_i \exp(Q \cdot K_i^2 - \max_j Q \cdot K_j^2)$

Then we can combine these into the full softmax sum for these two chunks together by using

$L^{\text{combined}} = \exp(M^1 - \max(M^1, M^2)) \cdot L^1 + \exp(M^2 - \max(M^1, M^2)) \cdot L^2$

where

$M^1 = \max_j Q \cdot K_j^1$ and $M^2 = \max_j Q \cdot K_j^2$

This can be done for the full softmax as well, giving us a way of accumulating arbitrarily large softmax sums. Here's the full algorithm from the Flash Attention paper.

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.

Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.

for $1 \le j \le T_c$ **do**

  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

  for $1 \le i \le T_r$ **do**

    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.

    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.

    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.

    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.

  end for

end for

Return $\mathbf{O}$.

From a hardware standpoint, this lets us fit our chunk of Q into VMEM (what the algorithm above calls on-chip SRAM) so we only have to load the KV chunks on each iteration, increasing the arithmetic intensity. We can also keep the running statistics in VMEM.

One last subtle point worth emphasizing is an attention softmax property that's used to make the Flash VJP (reverse mode derivative) calculation practical for training. We define an intermediate softmax array:

$S_{ij} = \frac{e^{\tau q_i \cdot k_j}}{\sum_l e^{\tau q_i \cdot k_l}}$

In attention, we obtain dS from reverse-mode dO and V arrays:

$dS_{ij} = dO_{id} \cdot_d V_{jd} = \sum_d dO_{id} V_{jd}$

During the backpropagation of this gradient to Q and K

$d(q_i \cdot k_j) = (dS_{ij} - S_{ij} \cdot_j dS_{ij}) S_{ij}$

We exploit an identity that allows us to exchange a contraction along the large key **length** dimension with a local contraction along the feature **depth** dimension.

$$
\begin{aligned}
S_{ij} \cdot_j dS_{ij} &= \sum_j \frac{e^{\tau q_i \cdot k_j}}{\sum_k e^{\tau q_i \cdot k_k}} \sum_d dO_{id} V_{jd} \\
&= \sum_d dO_{id} \sum_j \frac{e^{\tau q_i \cdot k_j}}{\sum_k e^{\tau q_i \cdot k_k}} V_{jd} \\
&= \sum_d dO_{id} O_{id} \\
&= dO_{id} \cdot_d O_{id}
\end{aligned}
$$

This replacement is crucial for being able to implement a sequence-block *local* calculation for the VJP, and enables further clever sharding schemes like ring attention.

Chapter 5

How to Parallelize a Transformer for Training

What Do We Mean By Scaling?

The goal of "model scaling" is to be able to increase the number of chips used for training or inference while achieving a proportional, linear increase in throughput (we call this *strong scaling*). While performance on a single chip depends on the trade-off between memory bandwidth and FLOPs, performance at the cluster level depends on hiding inter-chip communication by overlapping it with useful FLOPs. This is non-trivial, because increasing the number of chips increases the communication load while reducing the amount of per-device computation we can use to hide it. As we saw in Section 3, sharded matrix multiplications often require expensive AllGathers or ReduceScatters that can block the TPUs from doing useful work. The goal of this section is to find out when these become *too expensive.*

In this section, we'll discuss four common parallelism schemes: (pure) **data parallelism, fully-sharded data parallelism** (FSDP / ZeRO sharding), **tensor parallelism** (also known as model parallelism), and (briefly) **pipeline parallelism**. For each, we'll show what communication cost we incur and at what point that cost starts to bottleneck our compute cost.[38] For this section, you can focus solely on inter-chip communication costs, since as long as we have a large enough single-chip batch size, the transfer of data from HBM to MXU is already overlapped with computation.

We'll use the following notation to simplify calculations throughout this section.

Notation	Meaning (model parameters)
D	**d**model (the hidden dimension/residual stream dim)
F	**d**ff (the feed-forward dimension)
B	Batch dimension (number of tokens in the batch; total, not per-device)
T	Sequence length
L	Number of layers in the model

Notation	Meaning (hardware characteristic)
C	FLOPS/s per chip
W	Network bandwidth (bidirectional, often subscripted as e.g. W_{ici} or W_{dcn})
X	Number of chips along mesh axis X
Y	Number of chips along an alternate mesh axis, labeled Y
Z	Number of chips along a third mesh axis, labeled Z

For simplicity's sake, **we'll approximate a Transformer as a stack of MLP blocks** — attention is a comparatively small fraction of the FLOPs for larger models as we saw in Section 4. We will also ignore the gating matmul, leaving us with the following simple structure for each layer:

[38] We'll focus on communication bounds — since while memory capacity constraints are important, they typically do not bound us when using rematerialization (activation checkpointing) and a very large number of chips during pre-training. We also do not discuss expert parallelism here for MoEs — which expands the design space substantially, only the base case of a dense Transformer.

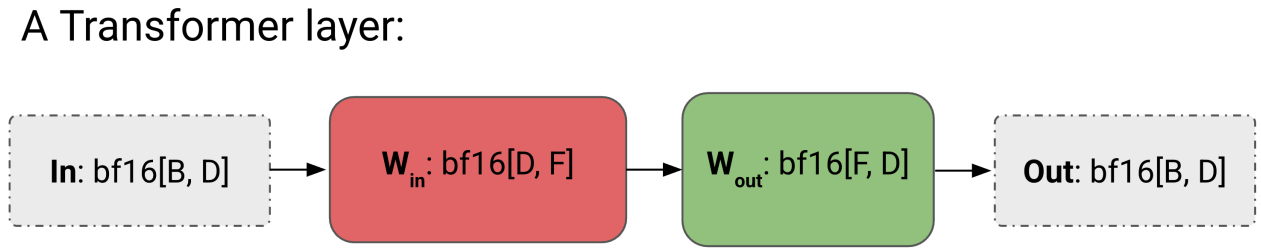


Figure 12: Figure: a simplified Transformer layer. We treat each FFW block as a stack of two matrices Win: bf16[D, F] (up-projection) and Wout: bf16[F, D] (down-projection) with an input In: bf16[B, D].

Here's the full algorithm for our little Transformer with no parallelism.

Forward pass: need to compute Loss[B]

1. Tmp[B, F] = In[B, D] *D Win[D, F]
2. Out[B, D] = Tmp[B, F] *F Wout[F, D]
3. Loss[B] = …

Backward pass: need to compute dWout[F, D], dWin[D, F]

1. dOut[B, D] = …
2. dWout[F, D] = Tmp[B, F] *B dOut[B, D]
3. dTmp[B, F] = dOut[B, D] *D Wout[F, D]
4. dWin[D, F] = In[B, D] *B dTmp[B, F]
5. dIn[B, D] = dTmp[B, F] *F Win[D, F] (*needed for previous layers*)

We provide this for comparison to the algorithms with communication added.

Here are the 4 parallelism schemes we will discuss. Each scheme can be thought of as uniquely defined by a sharding for **In**, **Win, Wout, and Out** in the above diagram.

1. Data parallelism: *activations sharded along batch, parameters and optimizer state are replicated on each device. Communication only occurs during the backwards pass.*

$$\text{In}[B_X, D] \cdot_D W_{\text{in}}[D, F] \cdot_F W_{\text{out}}[F, D] \to \text{Out}[B_X, D]$$

2. Fully-sharded data parallelism (FSDP or ZeRO-3): *activations sharded along batch (like pure data parallelism), parameters sharded along same mesh axis and AllGathered just-in-time before use in forward pass. Optimizer state also sharded along batch. Reduces duplicated memory.*

$$\text{In}[B_X, D] \cdot_D W_{\text{in}}[D_X, F] \cdot_F W_{\text{out}}[F, D_X] \to \text{Out}[B_X, D]$$

3. Tensor parallelism (also called Megatron sharding or model parallelism): *activations sharded along D (d_{model}), parameters sharded along F (d_{ff}). AllGather and ReduceScatter activations before and after each block. Compatible with FSDP.*

$$\text{In}[B, D_Y] \cdot_D W_{\text{in}}[D, F_Y] \cdot_F W_{\text{out}}[F_Y, D] \to \text{Out}[B, D_Y]$$

4. Pipeline parallelism: *weights sharded along the layer dimension, activations microbatched and rolled along the layer dimension. Communication between pipeline stages is minimal (just moving activations over a single hop). To abuse notation:*

$$\text{In}[L_Z, B, D][i] \cdot_D W_{\text{in}}[L_Z, D, F][i] \cdot_F W_{\text{out}}[L_Z, F, D][i] \to \text{Out}[L_Z, B, D][i]$$

Data Parallelism

Syntax: $\text{In}[B_X, D] \cdot_D W_{\text{in}}[D, F] \cdot_F W_{\text{out}}[F, D] \to \text{Out}[B_X, D]$

When your model fits on a single chip with even a tiny batch size (>240 tokens, so as to be compute-bound), **you should always use simple data parallelism.** Pure data parallelism splits our activations across any number of TPUs so long as the number of TPUs is smaller than our batch size. The forward pass involves no communication, but at the end of every step, **each TPU performs an AllReduce on its local gradients to synchronize them before updating the parameters.**

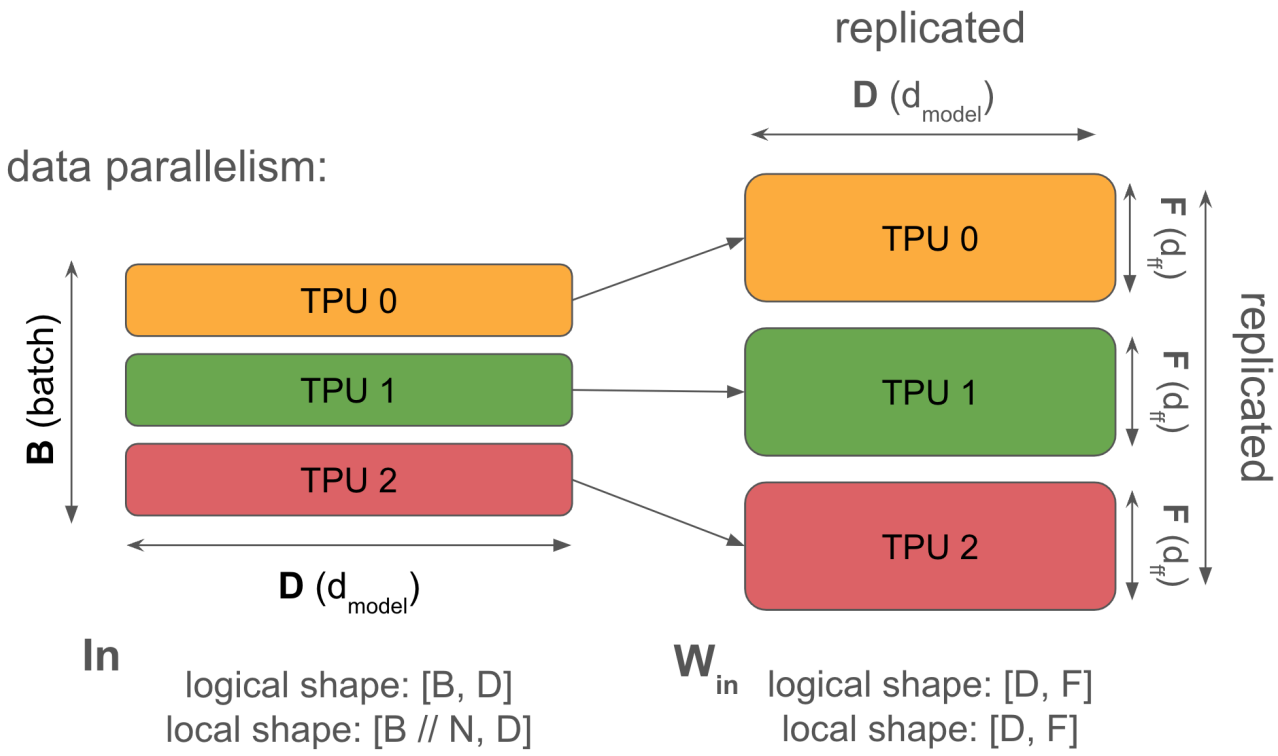


Figure 13: Figure: a diagram of pure data parallelism (forward pass). Our activations (left) are fully sharded along the batch dimension and our weights are fully replicated, so each TPU has an identical copy of the weights. This means the total memory of our weights is increased by a factor of N, but no communication is required on the forward-pass.

Here's the full algorithm for the forward and backwards pass. We abuse notation to write dL/dOut as dOut, purely for compactness.

Pure Data Parallelism Algorithm:

Forward pass: need to compute Loss[BX]

1. Tmp[BX, F] = In[BX, D] *D Win[D, F]
2. Out[BX, D] = Tmp[BX, F] *F Wout[F, D]
3. Loss[BX] = …

Backward pass: need to compute dWout[F, D], dWin[D, F]

1. dOut[BX, D] = …
2. dWout[F, D] {UX} = Tmp[BX, F] *B dOut[BX, D]
3. dWout[F, D] = **AllReduce**(dWout[F, D] {UX}) (*not on critical path, can be done async*)
4. dTmp[BX, F] = dOut[BX, D] *D Wout[F, D]
5. dWin[D, F] {UX} = In[BX, D] *B dTmp[BX, F]
6. dWin[D, F] = **AllReduce**(dWin[D, F] {UX}) (*not on critical path, can be done async*)
7. dIn[BX, D] = dTmp[BX, F] *F Win[D, F] (*needed for previous layers*)

We ignore the details of the loss function and abbreviate Tmp = $W_{\text{in}} \cdot$ In. Note that, although our final loss is the average **AllReduce**(Loss[BX]), we only need to compute the AllReduce on the backward pass when averaging

weight gradients.

Note that the forward pass has no communication — **it's all in the backward pass**! The backward pass also has the great property that the AllReduces aren't in the "critical path", meaning that each AllReduce can be performed whenever it's convenient and doesn't block you from performing subsequent operations. The overall communication cost *can still bottleneck us* if it exceeds our total compute cost, but it is much more forgiving from an implementation standpoint. We'll see that model/tensor parallelism doesn't have this property.

Why do this? Pure data parallelism reduces activation memory pressure by splitting our activations over the batch dimension, allowing us to almost arbitrarily increase batch size as long as we have more chips to split the batch dimension over. Especially during training when our activations often dominate our memory usage, this is very helpful.

Why not do this? Pure data parallelism does nothing to reduce memory pressure from model parameters or optimizer states, which means pure data parallelism is rarely useful for interesting models at scale where our parameters + optimizer state don't fit in a single TPU. To give a sense of scale, if we train with parameters in bf16 and optimizer state in fp32 with Adam[39], the largest model we can fit has TPU memory/10 parameters, so e.g. on a TPUv5p chip with 96GB of HBM and pure data parallelism this is about 9B parameters.

Takeaway: the largest model we can train with Adam and pure data parallelism has num_params = HBM per device/10. For TPU v5p this is roughly 9B parameters.[40]

To make this useful for real models during training, we'll need to at least partly shard the model parameters or optimizer.

When do we become bottlenecked by communication? As we can see above, we have two AllReduces per layer, each of size $2DF$ (for bf16 weights). When does data parallelism make us communication bound?

As in the table above, let C = per-chip FLOPs, W_{ici} = **bidirectional** network bandwidth, and X = number of shards across which the batch is partitioned[41]. Let's calculate the time required to perform the relevant matmuls, T_{math}, and the required communication time T_{comms}. Since this parallelism scheme requires no communication in the forward pass, we only need to calculate these quantities for the backwards pass.

Communication time: From a previous section we know that the time required to perform an AllReduce in a 1D mesh depends only on the total bytes of the array being AllReduced and the ICI bandwidth W_{ici}; specifically the AllReduce time is $2 \cdot \text{total bytes}/W_{\text{ici}}$. Since we need to AllReduce for both W_{in} and W_{out}, we have 2 AllReduces per layer. Each AllReduce is for a weight matrix, i.e. an array of DF parameters, or $2DF$ bytes. Putting this all together, the total time for the AllReduce in a single layer is

$$T_{\text{comms}} = \frac{2 \cdot 2 \cdot 2 \cdot D \cdot F}{W_{\text{ici}}}. \tag{12}$$

$$\tag{13}$$

Matmul time: Each layer comprises two matmuls in the forward pass, or four matmuls in the backwards pass, each of which requires $2(B/X)DF$ FLOPs. Thus, for a single layer in the backward pass, we have

$$T_{\text{math}} = \frac{2 \cdot 2 \cdot 2 \cdot B \cdot D \cdot F}{X \cdot C} \tag{14}$$

$$\tag{15}$$

Since we overlap, the total time per layer is the max of these two quantities:

[39] Adam stores parameters, first order and second order accumulators. Since the params are in bfloat16 and optimizer state is in float32, this gives us `2 + 8 = 10` bytes per parameters.

[40] Note that this doesn't include gradient checkpoints, so this wouldn't actually be useful. This is an absolute lower bound with a batch of 1 token.

[41] We assume this partitioning is done over an ICI mesh, so the relevant network bandwidth is W_{ici}

$$T \approx \max(\frac{8 \cdot B \cdot D \cdot F}{X \cdot C}, \frac{8 \cdot D \cdot F}{W_{\text{ici}}})$$
$$T \approx 8 \cdot D \cdot F \cdot \max(\frac{B}{X \cdot C}, \frac{1}{W_{\text{ici}}})$$

We become compute-bound when $T_{\text{math}}/T_{\text{comms}} > 1$, or when

$$\frac{B}{X} > \frac{C}{W_{\text{ici}}}. \tag{16}$$

The upshot is that, to remain compute-bound with data parallelism, we need the per-device batch size B/X to exceed the ICI operational intensity, C/W_{ici}. This is ultimately a consequence of the fact that the computation time scales with the per-device batch size, while the communication time is independent of this quantity (since we are transferring model weights). Note the resemblance of the $B/X > C/W_{\text{ici}}$ condition to the single-device compute-bound rule $B > 240$; in that case as well, the rule came from the fact that computation time scaled with batch size while data-transfer size was (in the $B \ll F, D$ regime) independent of batch size.

Let's put in some real numbers to get a sense of scale. For TPUv5p, `C=4.6e14` and `W=2 * 9e10` for 1D data parallelism over ICI, so **our batch size per chip must be at least 2,550 to avoid being communication-bound**. Since we can do data parallelism over multiple axes, if we dedicate all three axes of a TPUv5p pod to pure data parallelism, we 3x our bandwidth W_{ici} and can scale down to only BS=850 per TPU or 7.6M tokens per batch per pod (of 8960 chips)! **This tells us that it's fairly hard to become bottlenecked by pure data parallelism!**

Note [context parallelism]: Throughout this section, B always refers to the total batch size **in tokens**. Clearly, however, our batch is made up of many different sequences, so how does this work? As far as the MLP is concerned, **tokens are tokens**! It doesn't matter if they belong to the same sequence or two different sequences. So we are more or less free to do data parallelism over both the batch and sequence dimension: we call this context parallelism or sequence parallelism, but you can think of it as simply being another kind of data parallelism. Attention is trickier than the MLP since we do some cross-sequence computation, but this can be handled by gathering KVs or Qs during attention and carefully overlapping FLOPs and comms (typically using something called "ring attention"). Throughout this section, we will just ignore our sequence dimension entirely and assume some amount of batch or sequence parallelism.

Note on multiple mesh axes: We should quickly note how multiple axes affects the available bandwidth. When we use multiple mesh axes for a given parallelism strategy, we get more bandwidth.

- **Definition:** M_X (M_Y, M_Z, etc.) is the number of hardware mesh axes that a given parallelism strategy spans.
- **Effect (bandwidth-bound):** Using M axes provides ($\approx M$ times) aggregate link bandwidth, so collective time scales $\propto 1/M_X$.

Fully-Sharded Data Parallelism (FSDP)

Syntax: $\text{In}[B_X, D] \cdot_D W_{\text{in}}[D_X, F] \cdot_F W_{\text{out}}[F, D_X] \to \text{Out}[B_X, D]$

Fully-sharded data parallelism (often called FSDP or ZeRO-sharding) splits the model optimizer states and weights across the data parallel shards and efficiently gathers and scatters them as needed. **Compared to pure data parallelism, FSDP drastically reduces per-device memory usage and saves on backward pass FLOPs, with very minimal overhead.**

You'll remember (from Section 3) that an AllReduce can be decomposed into an AllGather and a ReduceScatter. This means that, instead of doing the full gradient AllReduce for standard data parallelism, we can shard the weights and optimizer states across chips, AllGather them at each layer during the forward pass and ReduceScatter across the weights during the backward pass at no extra cost.

Here's the full algorithm for FSDP.

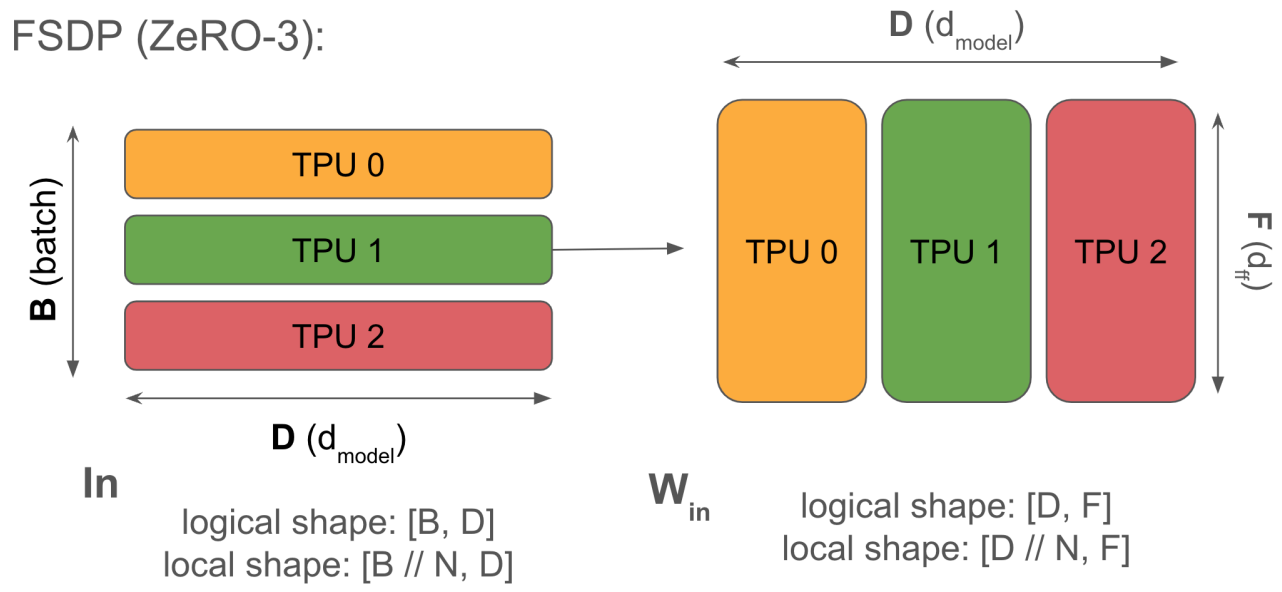


Figure 14: Figure: FSDP shards the contracting dimension of Win and the output dimension of Wout along the data dimension. This reduces memory but (from Section 3) requires us to gather the weights for W before we perform the matmul. Note that the activations (left) are not sharded along the contracting dimension, which is what forces us to gather. Note that our weight optimizer state is likewise sharded along the contracting dimension.

Fully-Sharded Data Parallelism (FSDP):

Forward pass: need to compute Loss[BX]

1. Win[D, F] = **AllGather**(Win[DX, F]) (*not on critical path, can do it during previous layer*)
2. Tmp[BX, F] = In[BX, D] *D Win[D, F] (*can throw away Win[D, F] now*)
3. Wout[F, D] = **AllGather**(Wout[F, DX]) (*not on critical path, can do it during previous layer*)
4. Out[BX, D] = Tmp[BX, F] *F Wout[F, D]
5. Loss[BX] = ...

Backward pass: need to compute dWout[F, DX], dWin[DX, F]

1. dOut[BX, D] = ...
2. dWout[F, D] {UX} = Tmp[BX, F] *B dOut[BX, D]
3. dWout[F, DX] = **ReduceScatter**(dWout[F, D] {UX}) (*not on critical path, can be done async*)
4. Wout[F, D] = **AllGather**(Wout[F, DX]) (*can be done ahead of time*)
5. dTmp[BX, F] = dOut[BX, D] *D Wout[F, D] *(can throw away Wout[F, D] here)*
6. dWin[D,F] {UX} = In[BX, D] *B dTmp[BX, F]
7. dWin[DX, F] = **ReduceScatter**(dWin[D, F] {UX}) *(not on critical path, can be done async)*
8. Win[D, F] = **AllGather**(Win[DX, F]) (*can be done ahead of time*)
9. dIn[BX, D] = dTmp[BX, F] *F Win[D, F] (*needed for previous layers) (can throw away Win[D, F] here*)

This is also called "ZeRO Sharding", from "Zero Redundancy Optimizer" since we don't perform any unnecessary compute or store any unnecessary state. ZeRO-{1,2,3} are used to refer to sharding the optimizer states, gradients, and weights in this way, respectively. Since all have the same communication cost[42], we can basically always do ZeRO-3 sharding, which shards the parameters, gradients, and optimizer states across a set of devices.

Why would we do this? Standard data parallelism involves a lot of duplicated work. Each TPU AllReduces the full gradient, then updates the full optimizer state (identical work on all TPUs), then updates the parameters

[42]Technically, FSDP adds communication in the forward pass that pure DP doesn't have, but this is in the same proportion as the backward pass so it should have no effect on the comms roofline. The key here is that ZeRO-3 turns a backward-pass AllReduce into an AllGather and a ReduceScatter, which have the same total comms volume.

(again, fully duplicated). For ZeRO sharding (sharding the gradients/optimizer state), instead of an AllReduce, you can ReduceScatter the gradients, update only your shard of the optimizer state, update a shard of the parameters, then AllGather the parameters as needed for your forward pass.

When do we become bottlenecked by communication? Our relative FLOPs and comms costs are exactly the same as pure data parallelism, since each AllReduce in the backward pass has become an AllGather + ReduceScatter. Recall that an AllReduce is implemented as an AllGather and a ReduceScatter, each with half the cost. Here we model the forward pass since it has the same FLOPs-to-comms ratio as the backward pass:

$$
\begin{aligned}
T_{\text{math}} &= \frac{2 \cdot 2 \cdot B \cdot D \cdot F}{X \cdot C} \\
T_{\text{comms}} &= \frac{2 \cdot 2 \cdot D \cdot F}{W_{\text{ici}}} \\
T &\approx \max\left(\frac{4 \cdot B \cdot D \cdot F}{X \cdot C}, \frac{4 \cdot D \cdot F}{W_{\text{ici}}}\right) \\
T &\approx 4 \cdot D \cdot F \cdot \max\left(\frac{B}{X \cdot C}, \frac{1}{W_{\text{ici}}}\right)
\end{aligned}
$$

Therefore, as with pure data-parallelism, we are compute bound when $B/X > C/W_{\text{ici}}$, i.e. when the per-device batch size B/X exceeds the "ICI operational intensity" C/W_{ici} (`4.59e14 / 1.8e11 = 2550` for v5p). This is great for us, because it means if our per-device batch size is big enough to be compute-bound for pure data-parallelism, we can — without worrying about leaving the compute-bound regime — simply upgrade to FSDP, saving ourselves a massive amount of parameter and optimizer state memory! Though we did have to add communication to the forward pass, this cost is immaterial since it just overlaps with forward-pass FLOPs.

Takeaway: Both FSDP and pure Data Parallelism become bandwidth bound on TPUv5 when the batch size per device is less than $2550/M_X$, where M_X is the number of mesh axes.

For example, DeepSeek-V2 (one of the only recent strong models to release information about its training batch size) used a batch size of ~40M tokens. **This would allow us to scale to roughly 47,000 chips, or around 5 TPUv5 pods, before we hit a bandwidth limit.**

For LLaMA-3 70B, which was trained for approximately `6.3e24 (15e12 * 70e9 * 6)` FLOPs, we could split a batch of 16M tokens over roughly `16e6 / (2550 / 3) = 18,823` chips (roughly 2 pods of 8960 chips), each with `4.59e14` FLOPs running at 50% peak FLOPs utilization (often called MFU), and **train it in approximately 17 days**. Not bad! But let's explore how we can do better.

Note on critical batch size: somewhat unintuitively, we become more communication bottlenecked as our total batch size decreases (with fixed chip number). Data parallelism and FSDP let us scale to arbitrarily many chips so long as we can keep increasing our batch size! However, in practice, as our batch size increases, we tend to see diminishing returns in training since our gradients become almost noise-free. We also sometimes see training instability. Thus, the game of finding an optimal sharding scheme in the "unlimited compute regime" often starts from a fixed batch size, determined by scaling laws, and a known (large) number of chips, and then aims to find a partitioning that allows us to fit that small batch size on so many chips.

Tensor Parallelism

Syntax: $\text{In}[B, D_Y] \cdot_D W_{\text{in}}[D, F_Y] \cdot_F W_{\text{out}}[F_Y, D] \to \text{Out}[B, D_Y]$ (we use Y to eventually combine with FSDP)

In a fully-sharded data-parallel AllReduce we move the weights across chips. We can also shard the feedforward dimension of the model and move the activations during the layer — this is called "1D model parallelism" or Megatron sharding. This can unlock a smaller efficient batch size per pod. The figure below shows an example of a single matrix sharded in this way:

As noted, **In[B, DY] *D Win[D, FY] *F Wout[FY, D] -> Out[B, DY] means we have to gather our activations before the first matmul. This is cheaper than ZeRO sharding when the activations are smaller than the weights.** This is typically true only with some amount of ZeRO sharding added (which reduces the size of the gather). This is one of the reasons we tend to mix ZeRO sharding and tensor parallelism.

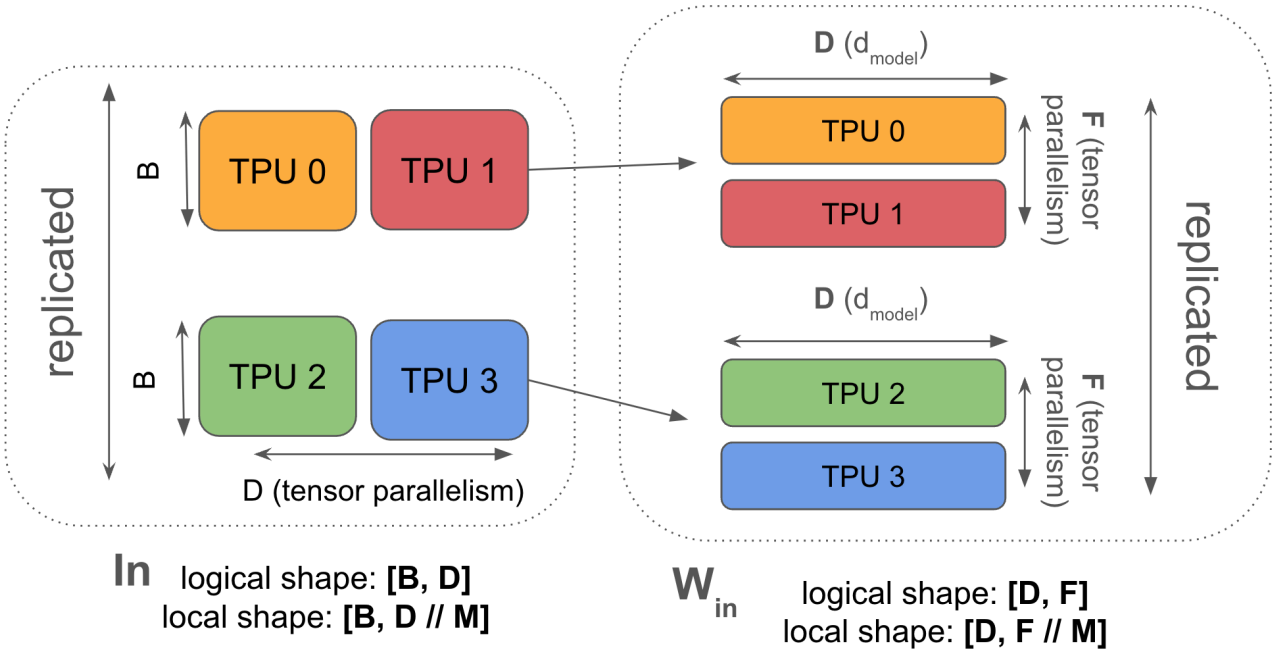


Figure 15: Figure: an example of basic tensor parallelism. Since we're only sharding our activations over Y (unlike in FSDP where we shard over X), we replicate our activations over X. Using our standard syntax, this is A[B, DY] * B[D, FY] -> C[B, FY]. Because we're only sharding over one of the contracting dimensions, we typically AllGather the activations A before the matmul.

Here's the algorithm for tensor parallelism!

Tensor Parallelism:

Forward pass: need to compute Loss[B]

1. In[B, D] = **AllGather**(In[B, DY]) *(on critical path)*
2. Tmp[B, FY] = In[B, D] *D Win[D, FY] *(not sharded along contracting, so no comms)*
3. Out[B, D] {UY} = Tmp[B, FY] *F Wout[FY, D]
4. Out[B, DY] = **ReduceScatter**(Out[B, D] {UY}) *(on critical path)*
5. Loss[B] = …

Backward pass: need to compute dWout[FY, D], dWin[D, FY]

1. dOut[B, DY] = …
2. dOut[B, D] = **AllGather**(dOut[B, DY]) *(on critical path)*
3. dWout[FY, D] = Tmp[B, FY] *B dOut[B, D]
4. dTmp[B, FY] = dOut[B, D] *D Wout[FY, D] *(can throw away dOut[B, D] here)*
5. In[B, D] = **AllGather**(In[B, DY]) *(this can be skipped by sharing with (1) from the forward pass)*
6. dWin[D, FY] = In[B, D] *B dTmp[B, FY]
7. dIn[B, D] {UY} = dTmp[B, FY] *F Win[D, FY] *(needed for previous layers)*
8. dIn[B, DY] = **ReduceScatter**(dIn[B, D] {UY}) *(on critical path)*

One nice thing about tensor parallelism is that it interacts nicely with the two matrices in our Transformer forward pass. Naively, we would do an AllReduce after each of the two matrices. But here we first do **In[B, DY] * Win[D, FY] -> Tmp[B, FY]** and then **Tmp[B, FY] * Wout[FY, D] -> Out[B, DY]**. This means we AllGather **In** at the beginning, and ReduceScatter **Out** at the end, rather than doing an AllReduce.

How costly is this? Let's only model the forward pass - the backwards pass is just the transpose of each operation here. In 1D tensor parallelism we AllGather the activations before the first matmul, and ReduceScatter them after the second, sending two bytes at a time (bf16). Let's figure out when we're bottlenecked by communication.

$$T_{\text{math}} = \frac{4 \cdot B \cdot D \cdot F}{Y \cdot C} \tag{17}$$

$$T_{\text{comms}} = \frac{2 \cdot 2 \cdot (B \cdot D)}{W_{\text{ici}}} \tag{18}$$

$$T \approx \max\left(\frac{4 \cdot B \cdot D \cdot F}{Y \cdot C}, \frac{2 \cdot 2 \cdot (B \cdot D)}{W_{\text{ici}}}\right) \tag{19}$$

Noting that we want compute cost to be greater than comms cost, we get:

$$\frac{4 \cdot B \cdot D \cdot F}{Y \cdot C} > \frac{2 \cdot 2 \cdot (B \cdot D)}{W_{\text{ici}}} \tag{20}$$

$$\frac{F}{Y \cdot C} > \frac{1}{W_{\text{ici}}} \tag{21}$$

$$F > Y \cdot \frac{C}{W_{\text{ici}}} \tag{22}$$

Thus for instance, for TPUv5p, $C/W_{ici} = 2550$ in bf16, so we can only do tensor parallelism up to $Y < F/2550$. When we have multiple ICI axes, our T_{comms} is reduced by a factor of M_Y, so we get $Y < M_Y \cdot F/2550$.

Takeaway: Tensor Parallelism becomes communication bound when $Y > M_Y \cdot F/2550$. For most models this is between 8 and 16-way tensor parallelism.

Note that this doesn't depend on the precision of the computation, since e.g. for int8, on TPUv5p, C_{int8}/W_{ici} is 5100 instead of 2550 but the comms volume is also halved, so the two factors of two cancel.

Let's think about some examples:

- On TPUv5p with LLaMA 3-70B with $D = 8192$, $F \approx 30,000$, we can comfortably do 8-way tensor parallelism, but will be communication bound on 16 way tensor parallelism. The required F for 8-way model sharding is 20k.
- For Gemma 7B, $F \approx 50k$, so we become communication bound with 19-way tensor parallelism. That means we could likely do 16-way and still see good performance.

Combining FSDP and Tensor Parallelism

Syntax: $\text{In}[B_X, D_Y] \cdot_D W_{\text{in}}[D_X, F_Y] \cdot_F W_{\text{out}}[F_Y, D_X] \to \text{Out}[B_X, D_Y]$

The nice thing about FSDP and tensor parallelism is that they can be combined. By sharding **Win** and **Wout** along both axes we both save memory and compute. Because we shard B along X, we reduce the size of the model-parallel AllGathers, and because we shard F along Y, we reduce the communication overhead of FSDP. This means a combination of the two can get us to an even lower effective batch size than we saw above.

Here's the full algorithm for mixed FSDP + tensor parallelism. While we have a lot of communication, all our AllGathers and ReduceScatters are smaller because we have batch-sharded our activations and tensor sharded our weights much more!

Forward pass: need to compute Loss[B]

1. In[BX, D] = **AllGather**Y(In[BX, DY]) *(on critical path)*
2. Win[D, FY] = **AllGather**X(Win[DX, FY]) *(can be done ahead of time)*
3. Tmp[BX, FY] = In[BX, D] *D Win[D, FY]

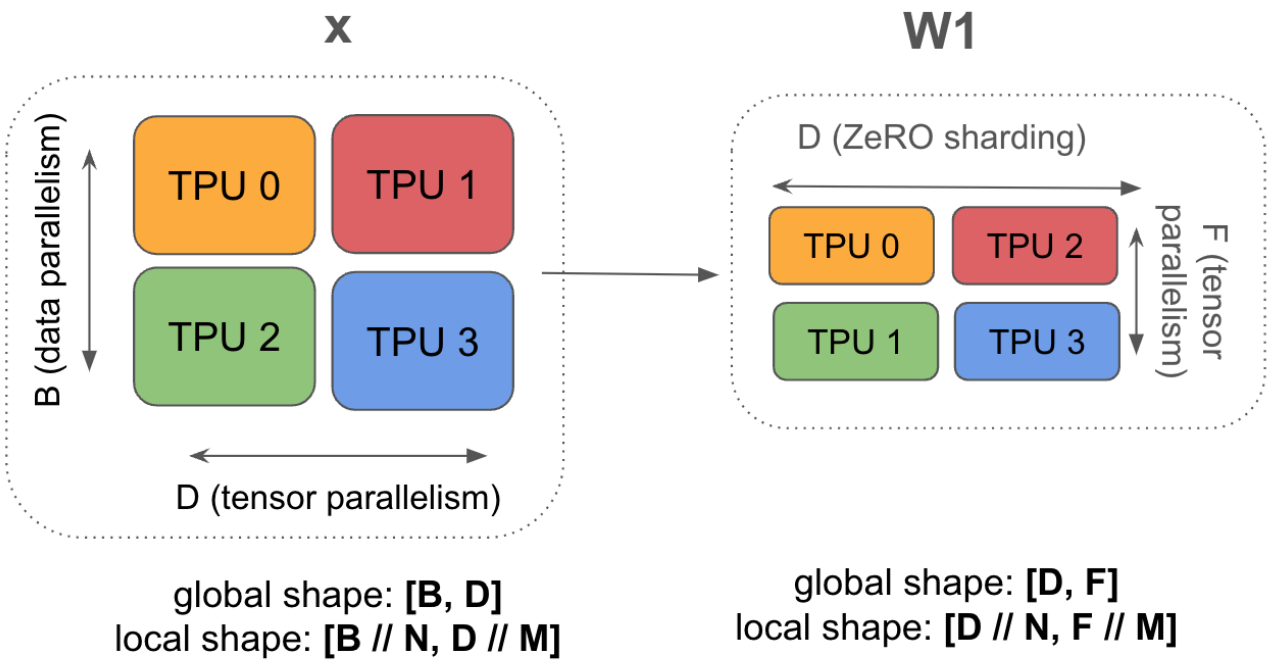


Figure 16: Figure: a diagram combining FSDP and tensor parallelism. Unlike the other cases, there is no duplication of model parameters.

4. Wout[FY, D] = **AllGather**X(Wout[FY, DX]) *(can be done ahead of time)*
5. Out[BX, D] {UY} = Tmp[BX, FY] *F Wout[FY, D]
6. Out[BX, DY] = **ReduceScatter**Y(Out[BX, D] {UY}) *(on critical path)*
7. Loss[BX] = …

Backward pass: need to compute dWout[FY, DX], dWin[DX, FY]

1. dOut[BX, DY] = …
2. dOut[BX, D] = **AllGather**Y(dOut[BX, DY]) *(on critical path)*
3. dWout[FY, D] {UX} = Tmp[BX, FY] *B dOut[BX, D]
4. dWout[FY, DX] = **ReduceScatter**X(dWout[FY, D] {UX})
5. Wout[FY, D] = **AllGather**X(Wout[FY, DX]) *(can be done ahead of time)*
6. dTmp[BX, FY] = dOut[BX, D] *D Wout[FY, D] *(can throw away dOut[B, D] here)*
7. In[BX, D] = **AllGather**Y(In[BX, DY]) *(not on critical path + this can be shared with (2) from the previous layer)*
8. dWin[D, FY] {UX} = In[BX, D] *B dTmp[BX, FY]
9. dWin[DX, FY] = **ReduceScatter**X(dWin[D, FY] {UX})
10. Win[D, FY] = **AllGather**X(Win[DX, FY]) *(can be done ahead of time)*
11. dIn[BX, D] {UY} = dTmp[BX, FY] *F Win[D, FY] *(needed for previous layers)*
12. dIn[BX, DY] = **ReduceScatter**Y(dIn[BX, D] {UY}) *(on critical path)*

What's the right combination of FSDP and TP? A simple but key maxim is that FSDP moves weights and tensor parallelism moves activations. That means as our batch size shrinks (especially as we do more data parallelism), tensor parallelism becomes cheaper because our activations per-shard are smaller.

- Tensor parallelism performs $\textbf{AllGather}_Y([B_X, D_Y])$ which shrinks as X grows.
- FSDP performs $\textbf{AllGather}_X([D_X, F_Y])$ which shrinks as Y grows.

Thus by combining both we can push our minimum batch size per replica down even more. We can calculate the optimal amount of FSDP and TP in the same way as above:

Let X be the number of chips dedicated to FSDP and Y be the number of chips dedicated to tensor parallelism. Let N be the total number of chips in our slice with $N = XY$. Let M_X and M_Y be the number of mesh axes over which we do FSDP and TP respectively (these should roughly sum to 3). We'll purely model the forward pass since it has the most communication per FLOP. Then adding up the comms in the algorithm above, we have

$T_{\text{FSDP comms}}(B, X, Y) = \frac{2 \cdot 2 \cdot D \cdot F}{Y \cdot W_{\text{ici}} \cdot M_X}$

$T_{\text{TP comms}}(B, X, Y) = \frac{2 \cdot 2 \cdot B \cdot D}{X \cdot W_{\text{ici}} \cdot M_Y}$

And likewise our total FLOPs time is

$T_{\text{math}} = \frac{2 \cdot 2 \cdot B \cdot D \cdot F}{N \cdot C}$.

To simplify the analysis, we make two assumptions: first, we allow X and Y to take on non-integer values (as long as they are positive and satisfy $XY = N$); second, we assume that we can fully overlap comms on the X and Y axis with each other. Under the second assumption, the total comms time is

$T_{\text{comms}} = \max\left(T_{\text{FSDP comms}}, T_{\text{TP comms}}\right)$

Before we ask under what conditions we'll be compute-bound, let's find the optimal values for X and Y to minimize our total communication. Since our FLOPs is independent of X and Y, the optimal settings are those that simply minimize comms. To do this, let's write T_{comms} above in terms of X and N (which is held fixed, as it's the number of chips in our system) rather than X and Y:

$T_{\text{comms}}(X) = \frac{4D}{W_{\text{ici}}} \max\left(\frac{F \cdot X}{N \cdot M_X}, \frac{B}{X \cdot M_Y}\right)$

Because $T_{\text{FSDP comms}}$ is monotonically increasing in X, and $T_{\text{TP comms}}$ is monotonically decreasing in X, the maximum must be minimized when $T_{\text{FSDP comms}} = T_{\text{TP comms}}$, which occurs when

$$\frac{FX_{opt}}{M_X} = \frac{BN}{X_{opt} M_Y} \rightarrow$$
$$X_{opt} = \sqrt{\frac{B}{F} \frac{M_X}{M_Y} N}$$

This is super useful! This tells us, for a given B, F, and N, what amount of FSDP is optimal. Let's get a sense of scale. Plugging in realistic values, namely $N = 64$ (corresponding to a 4x4x4 array of chips), $B = 48,000$, $F = 32768$, gives roughly $X \approx 13.9$. So we would choose X to be 16 and Y to be 4, close to our calculated optimum.

Takeaway: in general, during training, the optimal amount of FSDP is $X_{opt} = \sqrt{\frac{B}{F} \frac{M_X}{M_Y} N}$.

Now let's return to the question we've been asking of all our parallelism strategies: **under what conditions will we be compute-bound?** Since we can overlap FLOPs and comms, we are compute-bound when

$\max\left(T_{\text{FSDP comms}}, T_{\text{TP comms}}\right) < T_{\text{math}}$

By letting $\alpha \equiv C / W_{\text{ici}}$, the ICI arithmetic intensity, we can simplify:

$\max\left(\frac{F}{Y \cdot M_X}, \frac{B}{X \cdot M_Y}\right) < \frac{B \cdot F}{N \cdot \alpha}$

Since we calculated X_{opt} to make the LHS maximum equal, we can just plug it into either side (noting that $Y_{opt} = N / X_{opt}$), i.e.

$\frac{F}{N \cdot W_{\text{ici}} \cdot M_X} \sqrt{\frac{B}{F} \frac{M_X}{M_Y} N} < \frac{B \cdot F}{N \cdot C}$

Further simplifying, we find that

$\sqrt{\frac{B \cdot F}{M_X \cdot M_Y \cdot N}} < \frac{B \cdot F}{N \cdot \alpha}$,

where the left-hand-side is proportional to the communication time and the right-hand-side is proportional to the computation time. Note that while the computation time scales linearly with the batch size (as it does regardless

of parallelism), the communication time scales as the square root of the batch size. The ratio of the computation to communication time thus also scales as the square root of the batch size:

$\frac{T_{\text{math}}}{T_{\text{comms}}} = \frac{\sqrt{BF}\sqrt{M_X M_Y}}{\alpha\sqrt{N}}$.

To ensure that this ratio is greater than one so we are compute bound, we require

$\frac{B}{N} > \frac{\alpha^2}{M_X M_Y F}$

To get approximate numbers, again plug in $F = 32,768$, $\alpha = 2550$, and $M_X M_Y = 2$ (as it must be for a 3D mesh). This gives roughly $B/N > 99$. This roughly wins us a factor of eight compared to the purely data parallel (or FSDP) case, where assuming a 3D mesh we calculate that B/N must exceed about 850 to be compute bound.

Takeaway: combining tensor parallelism with FSDP allows us to drop to a B/N of $2550^2/2F$. This lets us handle a batch of as little as 100 per chip, which is roughly a factor of eight smaller than we could achieve with just FSDP.

Below we plot the ratio of FLOPs to comms time for mixed FSDP + TP, comparing it both to only tensor parallelism (TP) and only data parallelism (FSDP), on a representative 4x4x4 chip array. While pure FSDP parallelism dominates for very large batch sizes, in the regime where batch size over number of chips is between roughly 100 and 850, a mixed FSDP + TP strategy is required in order to be compute-bound.

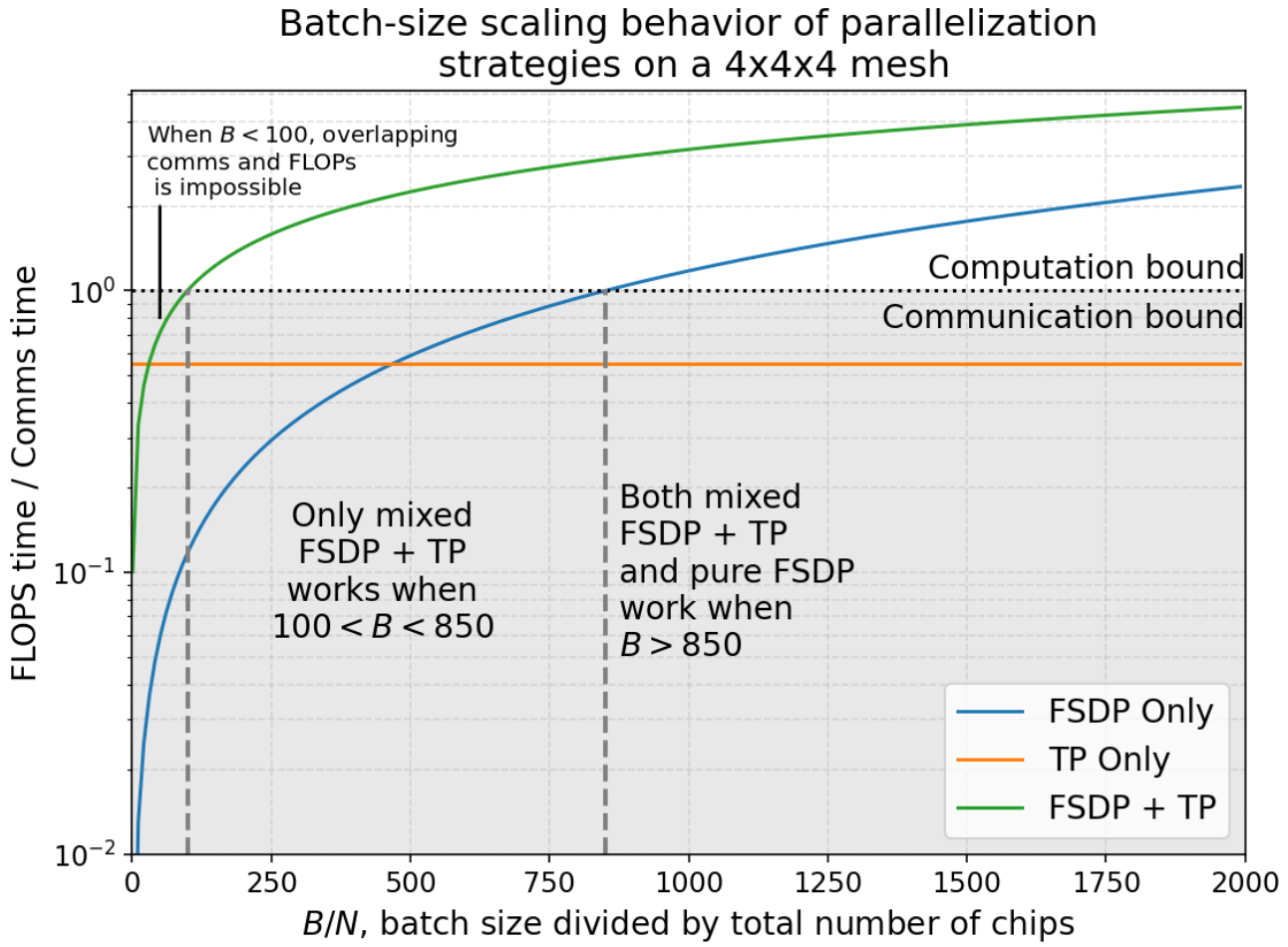


Figure 17: Figure: ratio of FLOPs to comms time for optimal mixed FSDP/TP on a TPUv5p 4x4x4 slice with F=30k. As expected, tensor parallelism has a fixed ratio with batch size; ideal mixed FSDP + TP scales with $\sqrt{B}$, and FSDP scales with B. However, in intermediate batch size regimes, only FSDP + TP achieves a ratio greater than unity.

Here's another example of TPU v5p 16x16x16 showing the FLOPs and comms time as a function of batch size for different sharding schemes.

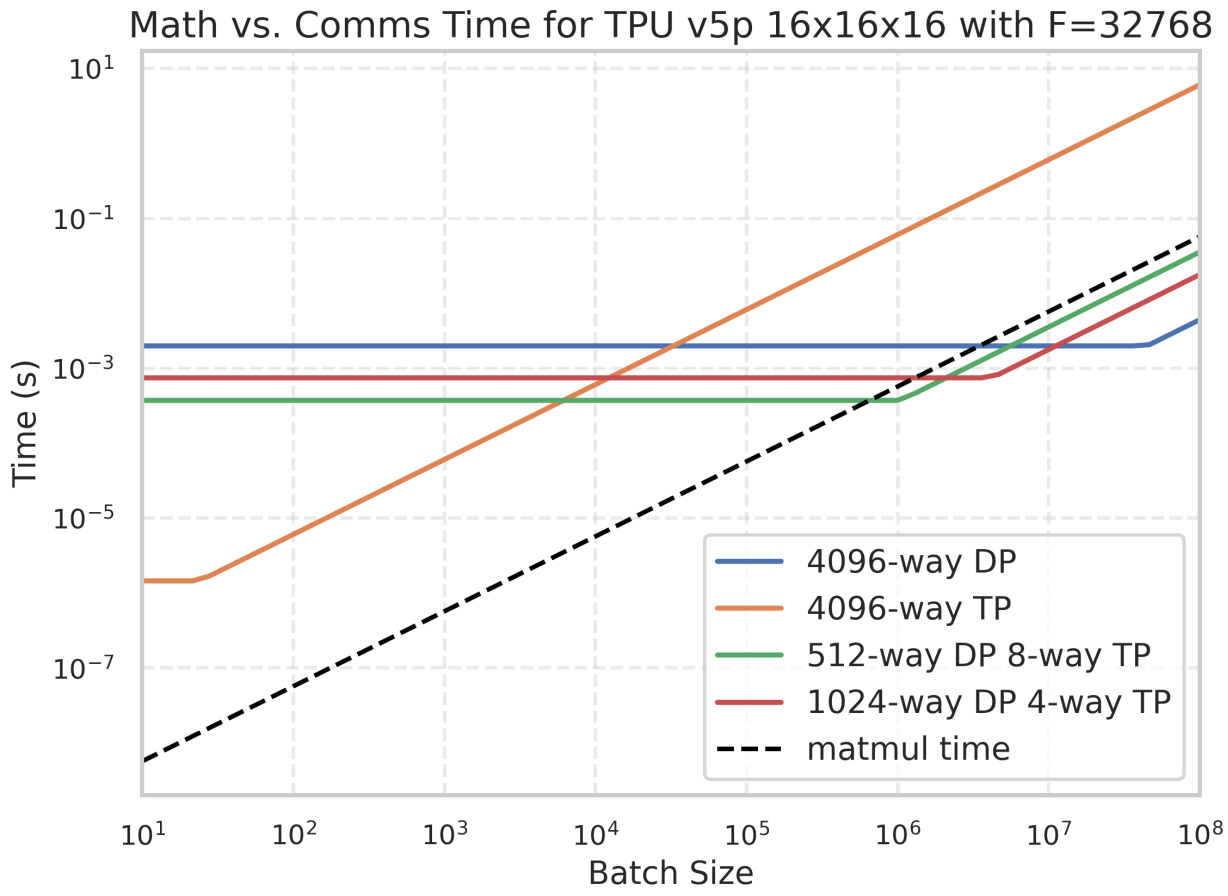


Figure 18: Figure: time taken for communication with different parallelism schemes. The black dashed line is the time taken by the matrix multiplication FLOPs, so any curve above this line is comms-bound. We note that all strategies become comms-bound below batch size 6e5, which is in line with our expected 4096 * 2550^2 / (2 * 8192 * 4) = 4e5.

The black curve is the amount of time spent on model FLOPs, meaning any batch size where this is lower than all comms costs is strictly comms bound. You'll notice the black curve intersects the green curve at about `4e5`, as predicted.

Here's an interactive animation to play with this, showing the total compute time and communication time for different batch sizes:

Figure:** forward-pass MLP compute and communication time on a TPU v5p 16x16x16 ($D = 8192$, $F = 32768$) as a function of batch size, using $T_{\text{FSDP comms}}$, $T_{\text{TP comms}}$ and T_{math} from above. Drag the slider to change the FSDP/TP split. Any batch size in the shaded region is comms-bound.

You'll notice this generally agrees with the above: the best split, FSDP=512 and TP=8, becomes compute-bound above a batch size of about 4e5, while every split with more tensor parallelism stays comms-bound at all batch sizes. Splits with less tensor parallelism need an even larger batch.

Pipelining

You'll probably notice we've avoided talking about pipelining at all in the previous sections. Pipelining is a dominant strategy for GPU parallelism that is somewhat less essential on TPUs. Briefly, pipelined training involves splitting the layers of a model across multiple devices and passing the activations between pipeline stages during the forward and backward pass. The algorithm is something like:

1. Initialize your data on TPU 0 with your weights sharded across the layer dimension ($W_{\text{in}}[L_Z, D_X, F_Y]$ for pipelining with FSDP and tensor parallelism).
2. Perform the first layer on TPU 0, then copy the resulting activations to TPU 1, and repeat until you get to the last TPU.
3. Compute the loss function and its derivative $\partial L/\partial x_L$.
4. For the last pipeline stage, compute the derivatives $\partial L/\partial W_L$ and $\partial L/\partial x_{L-1}$, then copy $\partial L/\partial x_{L-1}$ to the previous pipeline stage and repeat until you reach TPU 0.

Here is some (working) Python pseudo-code

This pseudocode should run on a Cloud TPU VM. While it's not very efficient or realistic, it gives you a sense how data is being propagated across devices.

```
batch_size = 32
d_model = 128
d_ff = 4 * d_model

num_layers = len(jax.devices())

key = jax.random.PRNGKey(0)

# Pretend each layer is just a single matmul.
x = jax.random.normal(key, (batch_size, d_model))
weights = jax.random.normal(key, (num_layers, d_model, d_model))

def layer_fn(x, weight):
  return x @ weight

# Assume we have num_layers == num_pipeline_stages
intermediates = [x]
for i in range(num_layers):
  x = layer_fn(x, weights[i])
  intermediates.append(x)

  if i != num_layers - 1:
    x = jax.device_put(x, jax.devices()[i+1])

def loss_fn(batch):
  return jnp.mean(batch ** 2)  # make up some fake loss function

loss, dx = jax.value_and_grad(loss_fn)(x)

for i in range(num_layers - 1, -1, -1):
  _, f_vjp = jax.vjp(layer_fn, intermediates[i], weights[i])
  dx, dw = f_vjp(dx)  # compute the jvp dx @ J(L)(x[i], W[i])
  weights[i] = weights[i] - 0.01 * dw  # update our weights

  if i != 0:
    dx = jax.device_put(dx, jax.devices()[i-1])
```

Why is this a good idea? Pipelining is great for many reasons: it has a low communication cost between pipeline stages, meaning you can train very large models even with low bandwidth interconnects. This is often

very useful on GPUs since they are not densely connected by ICI in the way TPUs are.

Why is this difficult/annoying? You might have noticed in the pseudocode above that TPU 0 is almost always idle! It's only doing work on the very first and last step of the pipeline. The period of idleness is called a pipeline bubble and is very annoying to deal with. Typically we try to mitigate this first with microbatching, which sends multiple small batches through the pipeline, keeping TPU 0 utilized for at least a larger fraction of the total step time.

A second approach is to carefully overlap the forward matmul $W_i @ x_i$, the backward dx matmul $W_i @ \partial L/\partial x_{i+1}$, and the dW matmul $\partial L/\partial x_{i+1} @ x_i$. Since each of these requires some FLOPs, we can overlap them to fully hide the bubble. Here's a plot from the recent DeepSeek v3 paper showing their "bubble-free" pipeline schedule:

![Figure: the DeepSeek v3 pipeline schedule (from their recent paper). Orange is the forward matmul, green is the dL/dx matmul, and blue is the dL/dW matmul. By prioritizing the backwards dL/dx multiplications, we can avoid "stranding](assets/img/deepseek-pipeline.png)

Because it is less critical for TPUs (which have larger interconnected pods), we won't delve into this as deeply, but it's a good exercise to understand the key pipelining bottlenecks.

Scaling Across Pods

The largest possible TPU slice is a TPU v5p SuperPod with 8960 chips (and 2240 hosts). When we want to scale beyond this size, we need to cross the Data-Center Networking (DCN) boundary. Each TPU host comes equipped with one or several NICs (Network Interface Cards) that connect the host to other TPU v5p pods over Ethernet. As noted in the TPU Section, each host has about 200Gbps (25GB/s) of full-duplex DCN bandwidth, which is about 6.25GB/s full-duplex (egress) bandwidth per TPU.

Typically, when scaling beyond a single pod, we do some form of model parallelism or FSDP within the ICI domain, and then pure data parallelism across multiple pods. Let N be the number of TPUs we want to scale to and M be the number of TPUs per ICI-connected slice. To do an AllReduce over DCN, we can do a ring-reduction over the set of pods, giving us (in the backward pass):

$T_{\text{math}} = \frac{2 \cdot 2 \cdot 2 \cdot BDF}{N \cdot C}$

$T_{\text{comms}} = \frac{2 \cdot 2 \cdot 2 \cdot DF}{M \cdot W_{\text{dcn}}}$

The comms bandwidth scales with M, since unlike ICI the total bandwidth grows as we grow our ICI domain and acquire more NICs. Simplifying, we find that $T_{\text{math}} > T_{\text{comms}}$ when

$\frac{B}{\text{slice}} > \frac{C}{W_{\text{dcn}}}$

For TPU v5p, the $\frac{C}{W_{\text{dcn}}}$ is about `4.59e14 / 6.25e9 = 73,440`. This tells us that to efficiently scale over DCN, there is a minimum batch size per ICI domain needed to egress each node.

How much of a problem is this? To take a specific example, say we want to train LLaMA-3 70B on TPU v5p with a BS of 2M tokens. LLaMA-3 70B has $F \approx 30,000$. From the above sections, we know the following:

- We can do Tensor Parallelism up to $Y = M_Y \cdot F/2550 \approx 11 \cdot M_Y$.
- We can do FSDP so long as $B/N > 2550/M_X$. That means if we want to train with BS=2M and 3 axes of data parallelism, we'd at most be able to use ≈ 2400 chips, roughly a quarter of a TPU v5p pod.
- When we combine FSDP + Tensor Parallelism, become comms-bound when we have $B/N < 2550^2/(2 \cdot 30000) = 108$, so this lets us scale to roughly 18k chips! However, the maximum size of a TPU v5p pod is 8k chips, so beyond that we have to use DCN.

The TLDR is that we have a nice recipe for training with BS=1M, using roughly X (FSDP) = 1024 and Y (TP) = 8, but with BS=2M we need to use DCN. As noted above, we have a DCN arithmetic intensity of 73,440, so we just need to make sure our batch size per ICI domain is greater than this. This is trivial for us, since with 2 pods we'd have a per-pod BS of 1M, and a per TPU batch size of 111, which is great (maybe cutting it a bit close, but theoretically sound).

Takeaway: Scaling across multiple TPU pods is fairly straightforward using pure data parallelism so long as our per-pod batch size is at least 73k tokens.

Takeaways from LLM Training on TPUs

- Increasing parallelism or reducing batch size both tend to make us more communication-bound because they reduce the amount of compute performed per chip.
- Up to a reasonable context length (~32k) we can get away with modeling a Transformer as a stack of MLP blocks and define each of several parallelism schemes by how they shard the two/three main matmuls per layer.
- During training there are 4 main parallelism schemes we consider, each of which has its own bandwidth and compute requirements (data parallelism, FSDP, tensor parallelism, and mixed FSDP + tensor parallelism).

Strategy	**Description**
Data Parallelism	Activations are batch sharded, everything else is fully-replicated, we all-reduce gradients during the backward pass.
FSDP	Activations, weights, and optimizer are batch sharded, weights are gathered just before use, gradients are reduce-scattered.
Tensor Parallelism (aka Megatron, Model)	Activations are sharded along d_{model}, weights are sharded along d_{ff}, activations are gathered before Win, the result reduce-scattered after Wout.
Mixed FSDP + Tensor Parallelism	Both of the above, where FSDP gathers the model sharded weights.

And here are the "formulas" for each method:

Strategy	Formula
DP	$\text{In}[B_X, D] \cdot_D W_{\text{in}}[D, F] \cdot_F W_{\text{out}}[F, D] \to \text{Out}[B_X, D]$
FSDP	$\text{In}[B_X, D] \cdot_D W_{\text{in}}[D_X, F] \cdot_F W_{\text{out}}[F, D_X] \to \text{Out}[B_X, D]$
TP	$\text{In}[B, D_Y] \cdot_D W_{\text{in}}[D, F_Y] \cdot_F W_{\text{out}}[F_Y, D] \to \text{Out}[B, D_Y]$
TP + FSDP	$\text{In}[B_X, D_Y] \cdot_D W_{\text{in}}[D_X, F_Y] \cdot_F W_{\text{out}}[F_Y, D_X] \to \text{Out}[B_X, D_Y]$

- Each of these strategies has a limit at which it becomes network/communication bound, based on their per-device compute and comms. Here's compute and comms per-layer, assuming X is FSDP and Y is tensor parallelism.

Strategy	Compute per layer (ignoring gating einsum)	Comms per layer (bytes, forward + backward pass)
DP	$4BDF/X + 8BDF/X$	$0 + 8DF$
FSDP	$4BDF/X + 8BDF/X$	$4DF + 8DF$
TP	$4BDF/Y + 8BDF/Y$	$4BD + 4BD$
FSDP + TP	$4BDF/(XY) + 8BDF/(XY)$	$(4BD/X + 4DF/Y) + (8BD/X + 8DF/Y)$

- Pure data parallelism is rarely useful because the model and its optimizer state use bytes = 10x parameter count. This means we can rarely fit more than a few billion parameters in memory.
- Data parallelism and FSDP become comms bound when the batch size per shard $< C/W$, the arithmetic intensity of the network. For ICI this is 2,550 and for DCN this is about 71,000. This can be increased with more parallel axes.
- Tensor parallelism becomes comms bound when $|Y| > F/2550$. **This is around 8-16 way for most models.** This is independent of the batch size.
- Mixed FSDP + tensor parallelism allows us to drop the batch size to as low as $2550^2/2F \approx 100$. This is remarkably low.

- Data parallelism across pods requires a minimum batch size per pod of roughly 71,000 before becoming DCN-bound.
- Basically, if your batch sizes are big or your model is small, things are simple. You can either do data parallelism or FSDP + data parallelism across DCN. The middle section is where things get interesting.

Some Problems to Work

Let's use LLaMA-2 13B as a basic model for this section. Here are the model details:

hyperparam	value
L	40
D	5,120
F	13824
N	40
K	40
H	128
V	32,000

LLaMA-2 has separate embedding and output matrices and a gated MLP block.

Question 1: How many parameters does LLaMA-2 13B have (I know that's silly but do the math)? *Note that, as in Transformer Math, LLaMA-3 has 3 big FFW matrices, two up-projection and one down-projection. We ignored the two "gating" einsum matrices in this section, but they behave the same as Win in this section.*

Click here for the answer.

- FFW parameters: $3LDF$ = `8.5e9`
- Attention parameters: $4DNHL$ = `4.2e9`
- Vocabulary parameters: $2VD$ = `0.33e9`
- Total: `8.5e9 + 4.2e9 + 0.33e9 = 13.0e9`, as expected!

Question 2: Let's assume we're training with BS=16M tokens and using Adam. Ignoring parallelism for a moment, how much total memory is used by the model's parameters, optimizer state, and activations? *Assume we store the parameters in bf16 and the optimizer state in fp32 and checkpoint activations three times per layer (after the three big FFW matmuls).*

Click here for the answer.

The total memory used for the parameters (bf16) and the two optimizer states (fp32, the first and second moment accumulators) is `(2 + 4 + 4) * 13e9 ~ 130GB`. The activations after the first two matmuls are shaped BF and after the last one BD (per the Transformer diagram above), so the total memory for bf16 is $2 \cdot L \cdot (BD + 2 * BF) = 2LB \cdot (D + 2F)$ or `2 * 40 * 16e6 * 5,120 * (1 + 2 * 2.7) ~ 4.2e13 = 42TB`, since `B=16e6`. All other activations are more or less negligible.

Question 3: Assume we want to train with 32k sequence length and a total batch size of 3M tokens on a TPUv5p 16x16x16 slice. Assume we want to use bfloat16 weights and a float32 optimizer, as above.

1. Can we use pure data parallelism? Why or why not?
2. Can we use pure FSDP without being comms-bound? Why or why not? With pure FSDP, how much memory will be used per device (assume we do gradient checkpointing only after the 3 big FFW matrices).
3. Can we use mixed FSDP + tensor parallelism? Why or why not? If so, what should X and Y be? How much memory will be stored per device? Using only roofline FLOPs estimates and ignoring attention, how long will each training step take at 40% MFU?

Click here for the answer.

First, let's write down some numbers. With 32k sequence length and a 3M batch size, we have a sequence batch size of 96. On a TPU v5p 16x16x16 slice, we have `393TB` of HBM.

1. We can't use pure data parallelism, because it replicates the parameters and optimizer states on each chip, which are already around 130GB (from Q2) which is more HBM than we have per-chip (96GB).
2. Let's start by looking purely at memory. Replacing BS=16M with 3M in Q2, we get `~7.86e12` total checkpoint activations, and with the 1.3e11 optimizer state this brings us to almost exactly 8e12 = 8TB. The TPUv5p slice has `393TB` of HBM in total, so we are safely under the HBM limit. Next let's look at whether we'll be comms or compute-bound. With 4096 chips and 3 axes of parallelism, we can do a minimum batch size of `850 * 4096 = 3.48M` tokens. That's slightly above our 3M batch size. So we're actually comms-bound, which is sad. So the general answer is **no, we cannot do FSDP alone without being comms-bound**.
3. Now we know our primary concern is being comms-bound, so let's plug in some numbers. First of all, we know from above that our per-chip batch size with mixed FSDP + tensor parallelism needs to be above $2550^2/2F = 235$ here. That means we can in theory do this! Let's figure out how much of each.

We have the rule $X_{opt} = \sqrt{(B/F) \cdot (M_X/M_Y) \cdot N}$, so here we have `sqrt(3e6 * 2 * 4096 / 13824) = 1333`, meaning we'll do roughly 1024 way DP and 4 way TP. Per TPU memory will be as in (2), and step time will just be `6 * 3e6 * 13e9 / (4096 * 4.6e14 * 0.4) = 300ms`.

That's it for Part 5! For Part 6, which applies this content to real LLaMA models, click here!

Appendix

Appendix A: Deriving the backward pass comms

Above, we simplified the Transformer layer forward pass as Out[B, D] = In[B, D] *D Win[D, F]* F Wout[F, D]. How do we derive the comms necessary for the backwards pass?

This follows fairly naturally from the rule in the previous section for a single matmul $\mathbf{Y = X * A}$:

$\frac{dL}{dA} = \frac{dL}{dY}\frac{dY}{dA} = X^T\left(\frac{dL}{dY}\right)$

$\frac{dL}{dX} = \frac{dL}{dY}\frac{dY}{dX} = \left(\frac{dL}{dY}\right)A^T$

Using this, we get the following formulas (letting Tmp[B, F] stand for In[B, D] * Win[D, F]):

1. dWout[F, D] = Tmp[B, F] *B dOut[B, D]
2. dTmp[B, F] = dOut[B, D] *D Wout[F, D]
3. dWin[D, F] = In[B, D] *B dTmp[B, F]
4. dIn[B, D] = dTmp[B, F] *F Win[D, F]

Note that these formulas are mathematical statements, with no mention of sharding. The job of the backwards pass is to compute these four quantities. So to figure out the comms necessary, we just take the shardings of all the quantities which are to be matmulled in the four equations above (Tmp, dOut, Wout, Win), which are specified by our parallelization scheme, and use the rules of sharded matmuls to figure out what comms we have to do. Note that dOut is sharded in the same way as Out.

Chapter 6

Training LLaMA 3 on TPUs

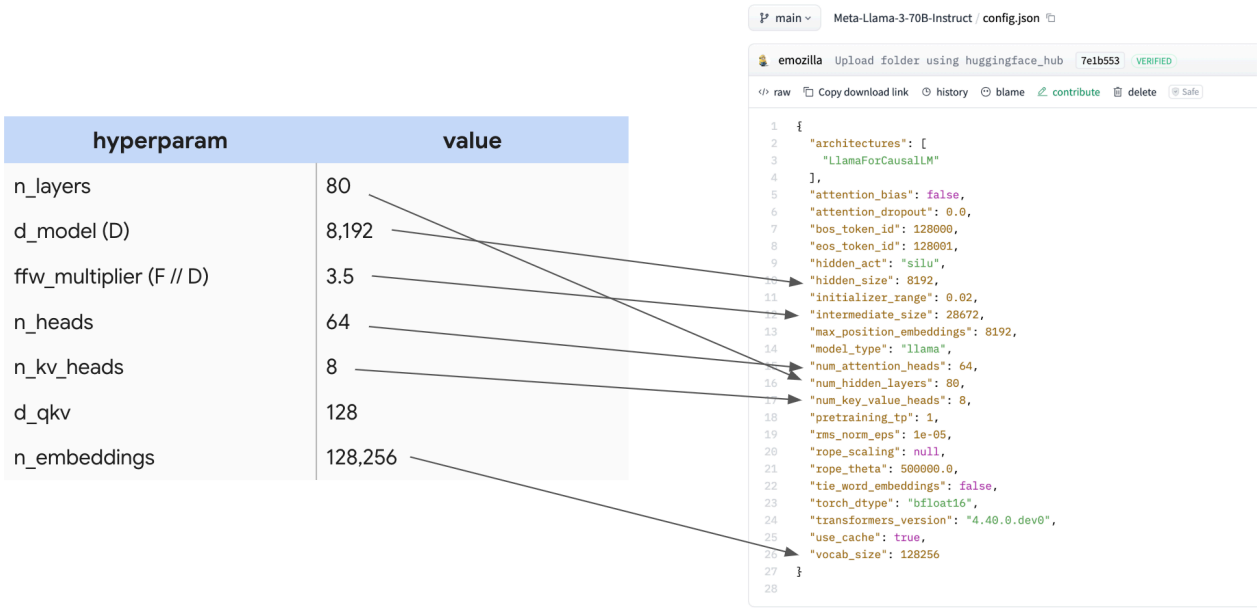
Our goal in this section is to apply results from the previous section to a very practical problem: training the LLaMA 3 family (herd) of models. Unlike the previous sections we want you to do a lot of this work yourself. For this reason, we've hidden the answers to each section so you can try to answer it first. Try grabbing a pen and doing it by hand!

What does LLaMA 3 look like?

The LLaMA-3 model family includes 3 main models: LLaMA 3 8B, 70B, and 405B. We'll mostly focus on 70B, and leave 8B and 405B for you to explore in the problem section at the end. Here's the architecture for LLaMA 3-70B, taken from the LLaMA HuggingFace page.

hyperparam	value
n_{layers} (L)	80
d_{model} (D)	8,192
d_{ff} (F)	28,672
n_{heads} (N)	64
$n_{\text{kv_heads}}$ (K)	8
d_{qkv} (H)	128
$n_{\text{embeddings}}$ (V)	128,256

To highlight how easy this is to find, here's the config itself, along with a mapping:



It's useful to make a big table with these numbers for many different open-source LLMs, so you can quickly compare the design decisions they've made.

Counting parameters and FLOPs

Question: From this table, can we calculate the LLaMA 3-70B parameter count? Let's apply the content of Section 4 and see if we can get 70B!

param	formula	count
FFW params	d_model * d_ff * 3 (for SwiGLU gate, up, and down projections) * n_layers	8,192 * 8,192 * 3.5 * 3 * 80 = **56.3e9**
Vocab params	2 (input and output embeddings) * n_embeddings * d_model	2 * 128,256 * 8,192 = **2.1e9**
Attention params	n_layers * [2 (for q embedding and concatenated output projection) * d_model * n_heads * d_qkv + 2 (for k and v) * d_model * n_kv_heads * d_qkv]	80 * (2 * 8,192 * 64 * 128 + 2 * 8,192 * 8 * 128) = **12e9**
		56.3e9 + 2.1e9 + 12e9 = **70.4e9**

That's great! We get the number we expect. You'll notice as expected that the FFW parameters totally dominate the overall parameter count, although attention is non-trivial.

Takeaway: The 3 big weight matrices in the MLP block are so much larger than all the other arrays in the Transformer that we can typically almost ignore all other parameters when reasoning about model memory or FLOPs. For LLaMA 3-70B, they represent 56B of 70B parameters.

Let's look at FLOPs now! *Remember the general rules for training from Section 4.*

Question: How many FLOPs does LLaMA-3 perform per token per training step? *This helps us determine how expensive the whole training process will be.*

Click here for the answer, once you've thought about it!

Answer: As shown in Section 4, we do roughly 6·param count FLOPs per token, so here that's roughly `6 * 70e9 = 4.2e11` FLOPs / token. That's about half a TFLOP per token per step. Assuming we're compute-bound, this should take roughly `4.2e11 / 4.59E+14 = 1ms` on a single TPU v5p chip, assuming perfect FLOPs utilization.

Question: LLaMA 3 was trained for about 15 trillion tokens. How many FLOPs is that total?

Click here for the answer, once you've thought about it!

Answer: That's easy, it's just `4.2e11 * 15e12 = 6.3e24 FLOPs` total. 6.3 yottaFLOPs. That's a lot! On a single TPU this would take `6.3e24 / 4.59E+14 = 435 years`. That's also a lot!

Question: Let's say we wanted to train on a full TPU v5p pod with 16x20x28 = 8960 chips. How long would this take to train at 40% MFU in bfloat16, assuming we are compute-bound?

Click here for the answer, once you've thought about it!

Answer: We know that each TPU v5p can perform 4.59e14 FLOPs / second. At 40% MFU, this will take about `T = 6.3e24 / (8960 * 4.59e14 * 0.4) = 3.8e6 seconds`. **This is about 44 days!** That's fairly reasonable, assuming we can actually achieve 40% MFU.

Question: LLaMA 3-70B was pretrained with a batch size of about 4M tokens. How many TPUs do we need at minimum to train with this batch size? *You can assume bfloat16 parameters and float32 optimizer state, and that you checkpoint activations 2 times per layer (at the beginning of the layer and after attention).*

Click here for the answer, once you've thought about it!

Answer: This question is primarily asking about memory usage, since that's the only strict constraint on available compute. During training, we have three primary uses of HBM: model parameters, optimizer state, and

activation checkpoints. If we assume bfloat16 weights, float32 optimizer state, and a *very* conservative activation checkpointing scheme (2 times per layer), we have:

Params | 2 * 70GB | ~140GB |
Optimizer State | 8 * 70GB | ~560GB |
Activation Checkpoints | 2 * 8192 * 4e6 * 2 * 80 | ~10.5TB |
Total | | ~11.2TB |

The total here is about 11.2TB. You notice that activation checkpointing strongly dominates the memory picture, even with a very conservative checkpointing scheme. We could technically go to 1 checkpoint per layer, or do microbatching, but this is a reasonable picture. With these assumptions, since each TPU v5p has 96GB of HBM, we need `11.2e12 / 96e9 = 117` TPUs. That's not very much actually!

Why wouldn't we do this? Well, because it would take us `44 days * 8960 / 117 = 3369 days` to train. That's nearly ten years. **That's a lot.** Still, this makes it clear that we're using these large clusters not because we're bound by memory but rather because we need the extra FLOPs. Also, since we're checkpointing so infrequently, we're doing close to 8ND FLOPs instead of 6ND.

Question: Under the same assumptions as the question above, if we use 8960 TPU v5p chips, how much memory will we use per-chip?

Click here for the answer, once you've thought about it!

Answer: Our total memory is still about 11.2TB, so per-chip we'll be using about 1.3GB per chip, which is basically nothing. If we did much more aggressive checkpointing, e.g. 12 checkpoints per layer, we'd still only be at 8GB per chip. We're nowhere near being memory bound during training at these scales.

Takeaways: It is technically possible to train even very large models on very small topologies, with the caveat that they will likely take a long time. Being able to calculate the total FLOPs of a training run allows us to ballpark its training time by assuming a modest MFU and a known topology.

How to shard LLaMA 3-70B for training

Let's stick to our setting from above and say we want to train LLaMA 3-70B with 4M token batch size (1024 sequences of length 4096 per batch) on a TPU v5p pod of 8960 chips. Let's discuss what the best sharding strategy is for this model.

Question: Under the assumptions above, can we train our model with FSDP alone? To start, let's say we can't do any sequence/context parallelism. *This should be the first idea you have, since it's simple and will introduce no extra communication if it works.*

Click here for the answer, once you've thought about it!

Answer: This answer will be a little pedantic. As noted above, LLaMA 3-70B is initially trained with sequences of length 4K, so the batch size of 4M tokens gives us a *sequence batch size* of 1024. That means we can only really do pure data parallelism/FSDP up to 1024 chips *because that's how many sequences we have to do data parallelism over.* So the answer in the simple sense of "fully data parallelism with no extra communication" is no. The next question will answer a slightly less pedantic version of this.

Question: Let's relax the requirement of not doing any sequence sharding. If we allow ourselves to do FSDP over both the batch *and* sequence axes, can we train LLaMA 3-70B with only FSDP on 8960 chips?

Click here for the answer, once you've thought about it!

Answer: Now that we're allowing ourselves to do sequence/context parallelism as well, we can scale up way more. First let's calculate our per-device batch size. If we do 8960-way FSDP, we end with a per-TPU batch size of `4 * 1024 * 1024 / 8960 = 468 tokens`. We know from the previous section that we become ICI-bound by FSDP when per device batch size $< 2550/M_X$. Since we can dedicate 3 axes here with a full 3D pod, this would give us a lower bound of 850, which we're well below. **So the answer is no, even with 3 axes. We would be solidly communication-bound.**

Question: Now let's look at mixed tensor parallelism and FSDP. Does there exist some combination that lets us remain compute-bound? What amount of FSDP and tensor parallelism should we do if so?

Click here for the answer, once you've thought about it!

Answer: First let's check to see if this will even fit. We know that we'll be comms-bound if our per-chip batch size is less than $2550^2/2F = 113$. As we saw above, we're slightly above this. So that's great! Now to pick the optimal amount of FSDP, we can use the formula

$$X_{opt} = \sqrt{\frac{2BN}{F}} = \sqrt{\frac{2 \cdot 4.19e6 \cdot 8960}{28672}} = 1618$$

Rounding to a reasonable multiple of 2, that gives us roughly 2048-way FSDP and 4-way tensor parallelism. That should work well!

Takeaways: We can train LLaMA-3 with a 4M token batch size on a full TPU v5p pod with a mixture of data parallelism (1024-way), sequence parallelism (2-way), and tensor parallelism (4-way) without being communication-bound. We will be comms-bound if we try to do pure FSDP or FSDP + sequence parallelism. The equations we've cooked up in the previous section are very practical.

Worked Problems

Question 1 [Scaling LLaMA 70B to more chips]: say we want to train LLaMA 3-70B on 4 pods with the same batch size. What parallelism scheme would we use? Would we be compute or communication bound? Roughly how long would it take to train? *Make sure to use the correct roofline bound.*

Question 2 [LLaMA 405B]:

(a) Using the LLaMA 3-405B config (a gated model, so you may need to log in and request access to view it), write a table with all the key hyperparameters as above. How many total parameters does this model have? How many FLOPs per training step? How many FLOPs do we perform if we train for 15T tokens?

(b) Assume we want to train on 8 TPU v5p pods. What parallelism scheme would we use? How long would training take? Would we be compute or comms bound?

That's all for Section 6. For Section 7, about Transformer inference, click here.

Chapter 7

All About Transformer Inference

The Basics of Transformer Inference

So you've trained a Transformer, and you want to use it to generate some new sequences. *At the end of the day, benchmark scores going up and loss curves going down are only proxies for whether something interesting is going to happen once the rubber hits the road!*[43]

Sampling is conceptually simple. We put a sequence in and our favorite Transformer will spit out $\log p(\text{next token}_i | \text{previous tokens})$, i.e. log-probabilities for all possible next tokens. We can sample from this distribution and obtain a new token. Append this token and repeat this process and we obtain a sequence of tokens which is a continuation of the prompt.

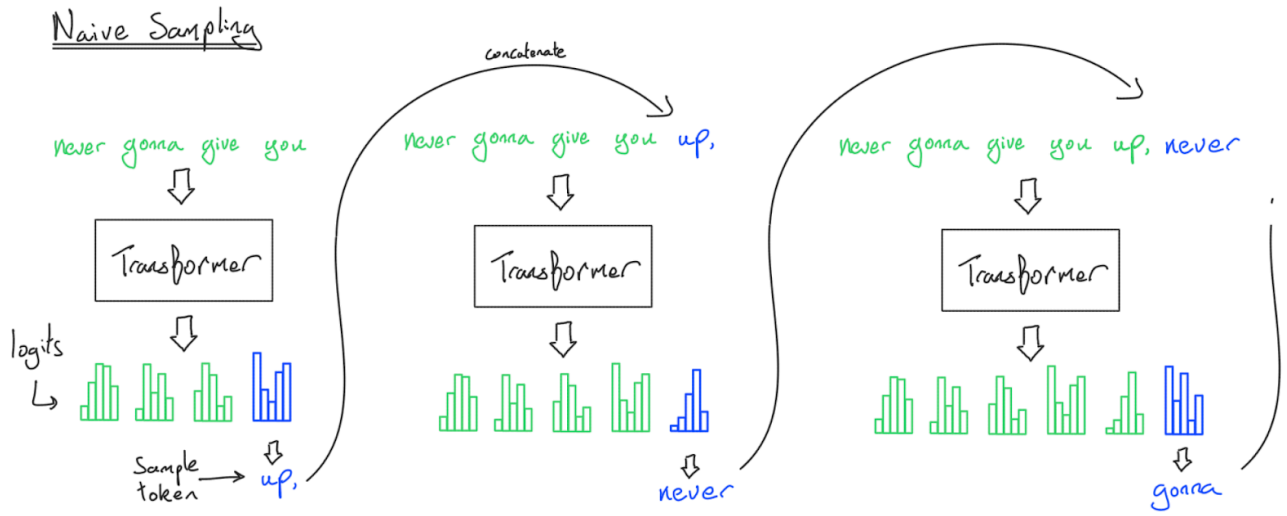


Figure 19: Figure: naive sampling from a Transformer. The blue logits give us a distribution over the next token that we can sample from. Note that each step re-processes the entire prefix, leading to a $\Theta(n^2)$ runtime for the algorithm.

We have just described the naive implementation of Transformer sampling, and while it works, **we never do it in practice** because we are re-processing the entire sequence every time we generate a token. This algorithm is $O(n^2)$ on the FFW and $O(n^3)$ on the attention mechanism to generate n tokens!

How do we avoid this? Instead of doing the full forward pass every time, it turns out we can save some intermediate activations from each forward pass that let us avoid re-processing previous tokens. Specifically, since a given token only attends to previous tokens during dot-product attention, we can simply write each token's key and value projections into a new data structure called a **KV cache**. Once we've saved these key/value projections

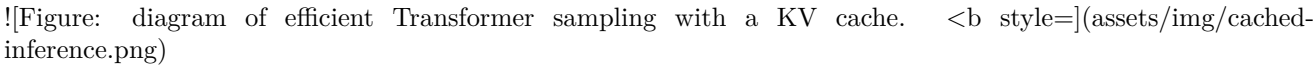
[43] Historically, you can do a surprising amount of research on Transformers without ever touching inference — scoring-based multiple choice benchmarks can be run efficiently without a proper KV cache or generation loop implementation. This meant, especially in research codebases, there's often a lot of low hanging fruit in the inference codepath.

for past tokens, future tokens can simply compute their $q_i \cdot k_j$ products without performing any new FLOPs on the earlier tokens. Amazing!

With this in mind, inference has two key parts:

- Prefill**: Given a long prompt, we process all the tokens in the prompt at the same time and save the resulting activations (specifically, the key-value projections) in a** "KV cache"**. We also save the logits for the last token.
- Generation**: Given a KV cache and the previous logits, we incrementally sample one token from the logits, feed that token back into the Transformer, and produce a new set of logits for the next step. We also append the KV activations for that new token to the KV cache. We repeat this until we hit a special `<EOS>` token or reach some maximum length limit.

Here's a diagram of sampling with a KV cache:



By sampling with a KV cache, we've reduced our time complexity to generate n tokens to $O(n)$ on the FFW and $O(n^2)$ on the attention, since we never reprocess a previous token. However, many forward passes are still needed to generate a sequence — that's what's happening when you query Gemini or ChatGPT and the result streams back to you. Every token is (usually) a separate (but partially cached) Transformer call to a massive model.

We will soon see that prefill** and generation** are very different beasts — Transformer inference is two tasks in disguise! Compared to training, the KV cache is also a novel and significant source of complexity.

What do we actually want to optimize?

Before we proceed further, it's worth highlighting one aspect of inference that's totally new: latency. While during training we only care about throughput (total tokens processed per second **per chip**), during inference we have to worry about how fast we're producing tokens (both the **Time To First Token (TTFT)** and the **per-token latency**). For example:

- **Offline batch inference** for evals and data generation only cares about bulk cost of inference and is blind to the latency of individual samples.
- **Chat interfaces/streaming tasks** need to run cheaply at scale while having low TTFT and generating tokens fast enough to exceed human reading speed.
- **Edge inference** (e.g. `llama.cpp` on your laptop) only needs to service one user at a time at the lowest possible latency, potentially with heavy hardware constraints.

Maximizing hardware utilization is still critical and helps with cost and TTFT, but unlike training, it does not *necessarily* translate to better experience for individual users in all contexts. Many optimizations at the accelerator, systems and model architectural level make tradeoffs between latency, throughput, context length and even model quality.

A more granular view of the Transformer

So far we've mostly treated a Transformer as a stack of feedforward blocks. While this is often reasonable from a FLOPs and memory standpoint, it's not sufficient to properly model inference.[44] As we saw in Part 4, the major components of a Transformer forward pass are:

1. **A bunch of linear operations**, including the MLP (W_{in}, W_{out}) and the attention QKV projections and output projections (W_Q, W_K, W_V, and W_O). These all involve reading parameters and a batch of activations from HBM, doing some FLOPs, and writing the result back to HBM.
2. **Dot-product attention**. We need to read a batch of key-value projections and a batch of query activations from HBM, do a few inner products and some softmax operations, and write the attention result back to HBM.

[44] One thing you'll notice throughout this section is that inference is much less forgiving than training. We typically have far fewer FLOPs, less opportunity for batching, and a much greater sensitivity to latency. KV caches dramatically complicate inference as well.

3. **Everything else**, including applying layer norms, activation functions, token sampling, updating KV caches, and positional embeddings. These do take some FLOPs, but are dominated by, or fused into, the above.

For the next couple of sections, we're going to look at each of these in the context of prefill and generation and ask what is likely to bottleneck our performance. Within a single accelerator, are we compute-bound or memory-bound? We want to emphasize how different the answers will be for prefill versus generation.

Linear operations: what bottlenecks us?

All our linear operations are conceptually the same, whether they live in the MLP block or attention. Their arithmetic intensity depends on the batch size. We did this math in Section 1 but it's worth repeating. Let's look at a single matrix multiply of a bf16[B, D] batch by a bf16[D, F] matrix. This could be the big MLP block (W_{in} or W_{out}) or one of the smaller attention projections (W_Q, W_K, W_V, W_O). To do this matmul, we need to load both of these arrays from HBM into the MXU, do the multiplication, then write the result back to HBM. As before, we have:

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}} = \frac{2BDF}{\text{Accelerator FLOPs/s}}$$

$$T_{\text{comms}} = \frac{\text{Communication Bytes}}{\text{Bandwidth Bytes/s}} = \frac{2BD+2FD+2BF}{\text{Bandwidth Bytes/s}}$$

A TPU or GPU can overlap these by loading as it does the compute, so to be compute-bound, we need $T_{\text{math}} \geq T_{\text{comms}}$, or:

$$\frac{2BDF}{2BD+2DF+2BF} \geq \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} \underset{\text{TPU v5e}}{=} \frac{1.97E+14}{8.20E+11} = 240$$

where the RHS is the arithmetic intensity of our hardware. Now let's assume D and F are very large compared to B (usually our batches are at most 500 and D and $F > 10k$), we can simplify the denominator by using the fact that $2BD + 2DF + 2BF \approx 2DF$ which gives us

$$\begin{aligned} \frac{2BDF}{2BD+2DF+2BF} \approx \frac{2BDF}{2DF} &\geq \frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} \\ \underset{\text{TPU v5e}}{=} \frac{1.97E+14}{8.20E+11} &\implies B \geq 240 = B_{\text{crit}} \end{aligned}$$

If we quantize our weights or use lower precision FLOPs for the matrix multiplication, this critical batch size can change. For instance, if we quantize our weights to int8 or fp8, B_{crit} decreases by 2x. If we do our FLOPs in int8 or fp8, B_{crit} increases by 2x. Thus if we let β = bits per param/bits per activation and $\alpha_{\text{hbm}} = C/W_{\text{hbm}}$, our critical batch size is actually $B_{\text{crit}} = \beta\alpha_{\text{hbm}}$.

Takeaway: Transformer matmuls are compute-bound *iff* the per-replica **token** batch size is greater than $B_{\text{crit}} = C/W_{\text{hbm}} \cdot (\text{bits per param/bits per activation}) = \beta \cdot \alpha_{\text{hbm}}$. For bf16 activations on TPU v5e, this is 240 tokens. For an H100, it is about 280 tokens.

During training, we'll have a high intensity during all our matrix multiplications because we reuse the same weights over a very large batch. **That high arithmetic intensity carries over to prefill, since user prompts are typically hundreds if not thousands of tokens long.** As we saw before, the hardware arithmetic intensity of a TPUv5e is 240, so if a sequence longer than 240 tokens is fed into a dense model running on this hardware at bf16, we would expect to be compute-bound and all is well. Prompts shorter than this can technically be batched together to achieve higher utilization, but this is typically not necessary.

Takeaway: During prefill, all matrix multiplications are basically always compute-bound. Therefore, simply maximizing hardware utilization or MFU (Model FLOPs Utilization) is enough to maximize throughput-per-chip (cost) and latency (in the form of TTFT). Unless prompts are extremely short, batching at a per-prompt level only adds latency for small improvements in prefill throughput.

However, during generation, for each request, we can only do our forward passes one token at a time since there's a sequential dependency between steps! Thus we can only (easily) achieve good utilization by batching multiple requests together, parallelizing over the batch dimension. We'll talk about this more later, but actually batching

many concurrent requests together without affecting latency is hard. For that reason, **it is much harder to saturate the hardware FLOPs with generation.**

Takeaway: During generation, the total token batch size must be greater than B_{crit} to be compute-bound on the linear/feed-forward operations (240 for bf16 params on TPU v5e). Because generation happens serially, token-by-token, this requires us to batch multiple requests together, which is hard!

It's worth noting just how large this is! Generate batch size of 240 means 240 concurrent requests generating at once, and 240 separate KV caches for dense models. That means this is difficult to achieve in practice, except in some bulk inference settings. In contrast, pushing more than 240 tokens through during a prefill is pretty routine, though some care is necessary as sparsity increases.

Note that this exact number will differ on the kind of quantization and hardware. Accelerators often can supply more arithmetic in lower precision. For example, if we have int8 parameters but do our computation in bf16, the critical batch size drops to 120. With int8 activations and int8 params, it jumps back up to 240 since the TPUv5e can supply 400 TOPs/s of int8 x int8.

What about attention?

Things get more complicated when we look at the dot-product attention operation, especially since we have to account for KV caches. Let's look at just one attention head with pure multi-headed attention. In a single Flash Attention fusion, we[45]:

1. Read the Q activations of shape bf16[B, T, D] from HBM.
2. Read the KV cache, which is a pair of bf16[B, S, D] tensors from HBM.
3. Perform $2BSTD$ FLOPs in the QK matmul. With Flash Attention, we don't need to write the bf16[B, S, T] attention matrix back into HBM.
4. Perform $2BSTD$ in the attention AV matmul.
5. Write the resulting bf16[B, T, D] tensor back into HBM.

Putting it all together, we get:

Multiheaded Attention Arithmetic Intensity $= \frac{4BSTD}{4BSD+4BTD} = \frac{ST}{S+T}$

For prefill, $S = T$ since we're doing self-attention, so this simplifies to $T^2/2T = T/2$. This is great because it means **the arithmetic intensity of attention during prefill is** $\Theta(T)$. That means it's quite easy to be compute-bound for attention. As long as our sequence length is fairly large, we'll be fine!

But since generation has a trivial sequence dim, and the B and D dims cancel, we can make the approximation:

$S \gg T = 1 \implies \frac{ST}{S+T} \approx 1$

This is bad, since it means we cannot do anything to improve the arithmetic intensity of attention during generation. We're doing a tiny amount of FLOPs while loading a massive KV cache. **So we're basically always memory bandwidth-bound during attention!**

Takeaway: during prefill, attention is usually compute bound for any reasonable sequence length (roughly > 480 tokens) while during generation our arithmetic intensity is low and constant, so we are always memory bandwidth-bound.

Why is this, conceptually? Mainly, we're compute-bound in linear portions of the model because the parameters (the memory bandwidth-heavy components) are reused for many batch items. However, every batch item has its own KV cache, so a bigger batch size means more KV caches. We will almost *always* be memory bound here unless the architecture is adjusted aggressively.

This also means you will get diminishing returns on throughput from increasing batch size once params memory becomes comparable to KV cache memory. The degree to which the diminishing returns hurt you depends on the ratio of parameter to KV cache bytes for a single sequence, i.e. roughly the ratio $2DF/SHK$. Since $HK \approx D$,

[45]We're simplifying a fair bit here by ignoring the non-matmul FLOPs in applying the softmax, masks etc. They should be overlapped with computation or HBM reads, but this can be non-trivial to do on certain TPU generations. While these details don't change the main message, which is that KV caches are usually memory bound, they are worth paying attention to.

this roughly depends on the ratio of F to S, the sequence length. This also depends on architectural modifications that make the KV cache smaller (we'll say more in a moment).

Theoretical estimates for LLM latency and throughput

From this math, we can get pretty good bounds on the step time we should aim for when optimizing. **(Note: if there is one thing we want the reader to take away from this entire chapter, it's the following).** For small batch sizes during generation (which is common), we can lower-bound our per-step latency by assuming we're memory bandwidth bound in both the attention and MLP blocks:

$$\text{Theoretical Min Step Time} = \frac{\text{Batch Size} \times \text{KV Cache Size} + \text{Parameter Size}}{\text{Total Memory Bandwidth}}$$

Similarly, for throughput:

$$\text{Theoretical Max Tokens/s} = \frac{\text{Batch Size} \times \text{Total Memory Bandwidth}}{\text{Batch Size} \times \text{KV Cache Size} + \text{Parameter Size}}$$

Eventually, as our batch size grows, FLOPs begin to dominate parameter loading, so in practice we have the more general equation:

$$\text{Theoretical Step Time (General)} = \underbrace{\frac{\text{Batch Size} \times \text{KV Cache Size}}{\text{Total Memory Bandwidth}}}_{\text{Attention (always bandwidth-bound)}} + \underbrace{\max\left(\frac{2 \times \text{Batch Size} \times \text{Parameter Count}}{\text{Total FLOPs/s}}, \frac{\text{Parameter}}{\text{Total Memory}}\right.}_{\text{MLP (can be compute-bound)}} \tag{23}$$

where the attention component (left) is never compute-bound, and thus doesn't need a FLOPs roofline. These are fairly useful for back-of-the-envelope calculations, e.g.

Pop Quiz: Assume we want to take a generate step with a batch size of 4 tokens from a 30B parameter dense model on TPU v5e 4x4 slice in int8 with bf16 FLOPs, 8192 context and 100 kB / token KV caches. What is a reasonable lower bound on the latency of this operation? What if we wanted to sample a batch of 256 tokens?

Click here for the answer.

Answer: in int8, our parameters will use 30e9 bytes and with the given specs our KV caches will use `100e3 * 8192 = 819MB` each. We have 16 chips, each with `8.2e11` bytes/s of bandwidth and `1.97e14` bf16 FLOPs/s. From the above equations, since we have a small batch size, we expect our step time to be at least `(4 * 819e6 + 30e9) / (16 * 8.2e11) = 2.5 ms`. At 256 tokens, we'll be well into the compute-bound regime for our MLP blocks, so we have a step time of roughly `(256 * 819e6) / (16 * 8.2e11) + (2 * 256 * 30e9) / (16 * 1.97e14) = 21ms`.

As you can see, there's a clear tradeoff between throughput and latency here. Small batches are fast but don't utilize the hardware well. Big batches are slow but efficient. Here's the latency-throughput Pareto frontier calculated for some older PaLM models (from the ESTI paper):

Not only do we trade off latency and throughput with batch size as knob, we may also prefer a larger topology to a smaller one so we can fit larger batches if we find ourselves limited by HBM. The next section explores this in more detail.

Takeaway: if you care about generation throughput, use the largest per-chip batch size possible. Any per-chip batch size above the TPU arithmetic intensity (B_{crit}, usually 120 or 240) will maximize throughput. You may need to increase your topology to achieve this. Smaller batch sizes will allow you to improve latency at the cost of throughput.

There are some caveats to this from a hardware standpoint. Click here for some nits.

This is all quite theoretical. In practice we often don't quite see a sharp roofline for a few reasons:

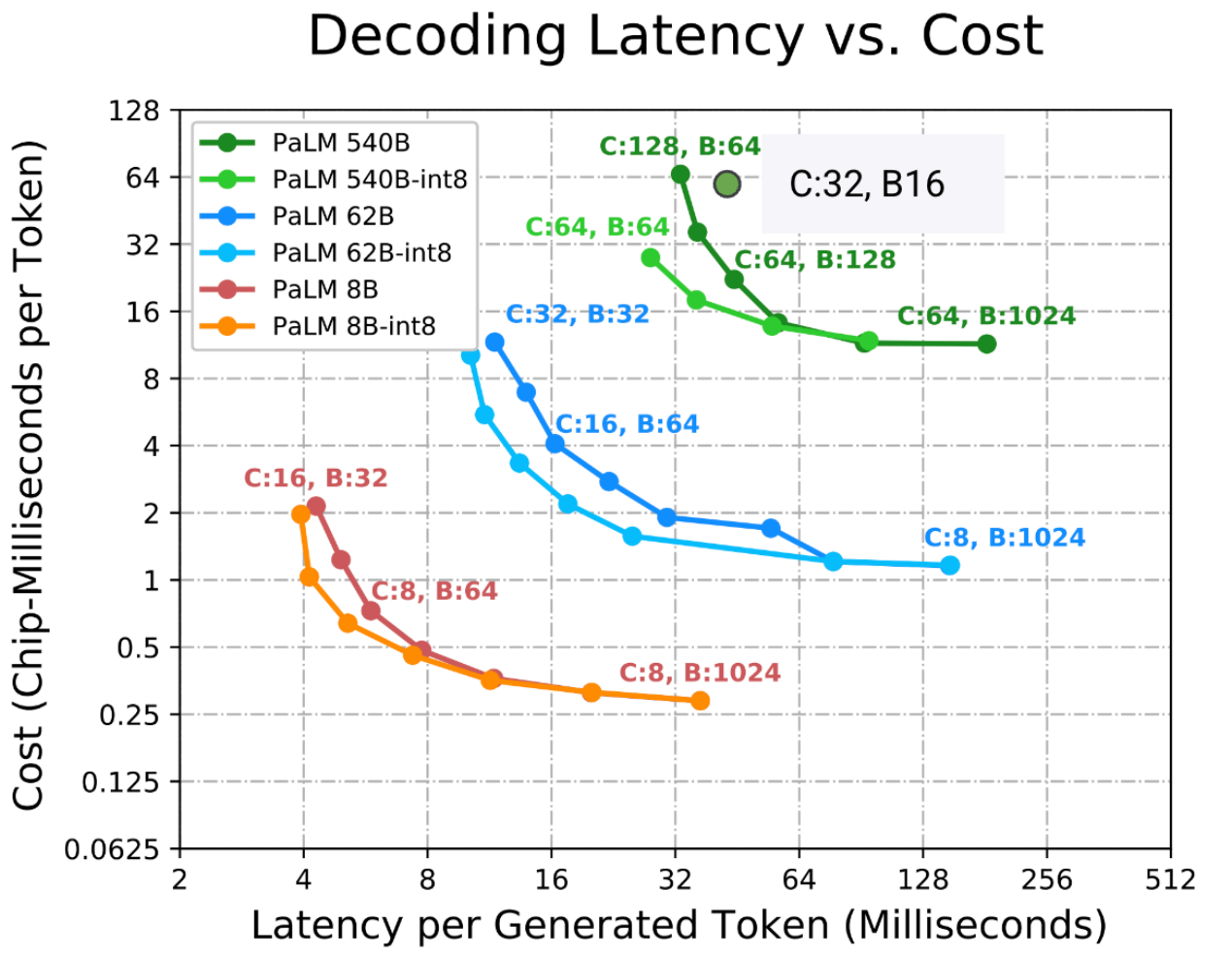


Figure 20: Figure: Pareto frontier of cost (read: throughput) versus latency for several PaLM models. Note how chip count (C) and batch size (B) moves you along the Pareto frontier, with the exception of the green dot (C:32 B:16 for PaLM 540B) where the available memory prevented the setup from supporting a good batch size and caused throughput to suffer. Note how throughput generally tends to flatten around after the batch size 240. int8 weights offers a better latency-throughput pareto optimal, but not a better max throughput.

- Our assumption that HBM reads will be perfectly overlapped with FLOPs is not realistic, since our compiler (XLA) is fallible.
- For sharded models, XLA also often fails to efficiently overlap the ICI communication of our model-sharded matrix multiples with the FLOPs themselves, so we often start taking a latency hit on linears over BS = 32.
- Batch sizes larger than the theoretical roofline will still see some improvement in throughput because of imperfect overlapping, but this is a good heuristic.

What about memory?

We've spent some time looking at bandwidth and FLOPs, but not at memory. The memory picture looks a lot different at inference time, thanks to our new data structure, the KV cache. For this section, let's pick a real model (LLaMA 2-13B) to demonstrate how different things look:

hyperparam	value
L (num_layers)	40
D (d_model)	5,120
F (ffw_dimension)	13,824
N (num_heads)	40
K (num_kv_heads)	40
H (qkv_dim)	128
V (num_embeddings)	32,000

What's using memory during inference? Well, obviously, our parameters. Counting those, we have:

param	formula	size (in bytes)
FFW params	d_model2 x ffw_multiplier x 3 (for SwiGLU gate, up, and down projections) x n_layers	5,120 x 5,120 x 2.7 x 3 x 40 = **8.5e9**
Vocab params	2 (input and output embeddings) x n_embeddings x d_model	2 x 32,000 x 5,120 = **0.3e9**
Attention params	[2 (*q and output*) x d_model x n_heads x d_qkv + 2 (*for k and v*) x d_model x n_kv_heads x d_qkv] x n_layers	(2 x 5,120 x 40 x 128 + 2 x 5,120 x 40 x 128) x 40 = **4.2e9**

Adding these parameters up, we get 8.5e9 + 4.2e9 + 0.3e9 = **13e9 total parameters**, just as expected. As we saw in the previous sections, during training we might store our parameters in bfloat16 with an optimizer state in float32. That may use around 100GB of memory. That pales in comparison to our gradient checkpoints, which can use several TBs.

How is inference different? During inference, we store one copy of our parameters, let's say in bfloat16. That uses 26GB — and in practice we can often do much better than this with quantization. There's no optimizer state or gradients to keep track of. Because we don't checkpoint (keep activations around for the backwards pass), our activation footprint is negligible for both prefill[46] and generate. If we prefill 8k tokens, a single activation only uses around `8,192 x 5,120 x 2 bytes = 80MB` of memory. Longer prefills can be broken down into many smaller forward passes, so it's not a problem for longer contexts either. Generation uses even fewer tokens than that, so activations are negligible.

The main difference is the KV cache. These are the keys and value projections for all past tokens, bounded in size only by the maximum allowed sequence length. The total size for T tokens is

$$\text{KV cache size} = 2 \cdot \text{bytes per float} \cdot H \cdot K \cdot L \cdot T$$

where H is the dimension of each head, K is the number of KV heads, L is the number of layers, and the 2 comes from storing both the keys and values.

[46] Particularly thanks to Flash Attention, which avoids materializing our attention matrix

This can get big very quickly, even with modest batch size and context lengths. For LLaMA-13B, a KV cache for a single 8192 sequence at bf16 is

8192 (T) $\times$ 40 (K) $\times$ 128 (H) $\times$ 40 (L) $\times$ 2 (bytes) $\times$ 2 = 6.7GB

Just 4 of these exceed the memory usage of our parameters! To be clear, LLaMA 2 was not optimized for KV cache size at longer contexts (it isn't always this bad, since usually K is much smaller, as in LLaMA-3), but this is still illustrative. We cannot neglect these in memory or latency estimates.

Modeling throughput and latency for LLaMA 2-13B

Let's see what happens if we try to perform generation perfectly efficiently at different batch sizes on 8xTPU v5es, up to the critical batch size (240) derived earlier for maximum theoretical throughput.

Batch Size	1	8	16	32	64	240
KV Cache Memory (GiB)	6.7	53.6	107.2	214.4	428.8	1608
Total Memory (GiB)	32.7	79.6	133.2	240.4	454.8	1634
Theoretical Step Time (ms)	4.98	12.13	20.30	36.65	69.33	249.09
Theoretical Throughput (tokens/s)	200.61	659.30	787.99	873.21	923.13	963.53

8x TPU v5es gives us 128GiB of HBM, 6.5TiB/s of HBM bandwidth (0.82TiB/s each) and 1600TF/s of compute.

For this model, increasing the batch size does give us better throughput, but we suffer rapidly diminishing returns. We OOM beyond batch size 16, and need an order of magnitude more memory to go near 240. A bigger topology can improve the latency, but we've hit a wall on the per chip throughput.

Let's say we keep the total number of params the same, but magically make the KV cache 5x smaller (say, with 1:5 GMQA, which means we have 8 KV heads shared over the 40 Q heads — see next section for more details).

Batch Size	1	8	16	32	64	240
KV Cache Memory (GiB)	1.34	10.72	21.44	42.88	85.76	321.6
Total Memory (GiB)	27.34	36.72	47.44	68.88	111.76	347.6
Theoretical Step Time (ms)	4.17	5.60	7.23	10.50	17.04	52.99
Theoretical Throughput (tokens/s)	239.94	1,429.19	2,212.48	3,047.62	3,756.62	4,529.34

With a smaller KV cache, we still have diminishing returns, but the theoretical throughput per chip continues to scale up to batch size 240. We can fit a much bigger batch of 64, and latency is also consistently better at all batch sizes. The latency, maximum throughput, and maximum batch size all improve dramatically! In fact, later LLaMA generations used this exact optimization — LLaMA-3 8B has 32 query heads and 8 KV heads (source).

Takeaway: In addition to params, the size of KV cache has a lot of bearing over the ultimate inference performance of the model. We want to keep it under control with a combination of architectural decisions and runtime optimizations.

Tricks for Improving Generation Throughput and Latency

Since the original Attention is All You Need paper, many techniques have been developed to make the model more efficient, often targeting the KV cache specifically. Generally speaking, a smaller KV cache makes it easier to increase batch size and context length of the generation step without hurting latency, and makes life easier for the systems surrounding the Transformer (like request caching). Ignoring effects on quality, we may see:

Grouped multi-query attention (aka GMQA, GQA): We can reduce the number of KV heads, and share them with many Q heads in the attention mechanism. In the extreme case, it is possible to share a single KV head across all Q heads. This reduces the KV cache by a factor of the Q:KV ratio over pure MHA, and it has been observed that the performance of models is relatively insensitive to this change.

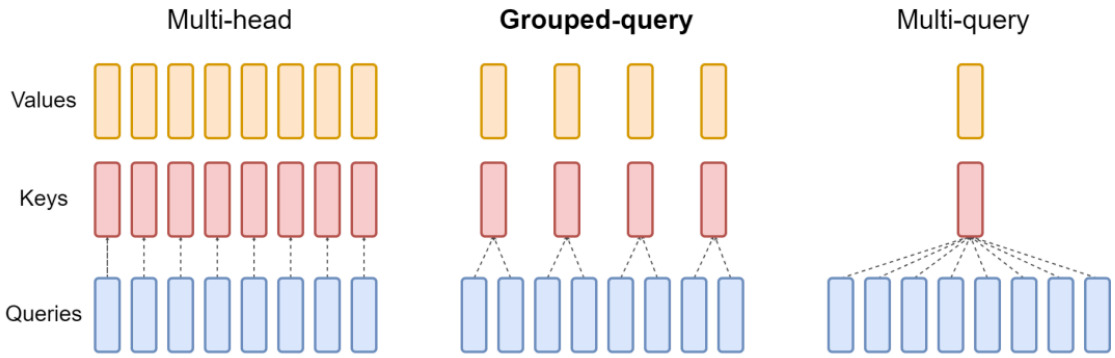


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

This also effectively increases the arithmetic intensity of the attention computation (see Question 4 in Section 4).

Mixing in some local attention layers: Local attention caps the context to a small to moderately sized max length. At training time and prefill time, this involves masking the attention matrix to a diagonal strip instead of a triangle. This effectively caps the size of the max length of the KV cache for the local layers. By mixing in some local layers into the model with some global layers, the KV cache is greatly reduced in size at contexts longer than the local window.

Sharing KVs across layers: The model can learn to share the same KV caches across layers in some pattern. Whilst this does reduce the KV cache size, and provide benefits in increasing batch size, caching, offline storage, etc., shared KV caches may need to be read from HBM multiple times, *so it does not necessarily improve the step time.*

![Left: Multiple layers of pure global attention. Right: An example of some global/local interleaving pattern with sharing with adjacent layers. Source: <a href=](assets/img/kv-sharing.png)

Quantization: Inference is usually less sensitive to the precision of parameters and KVs. By quantizing the parameters and KV cache (e.g. to int8, int4, `fp8` etc.), we can save on memory bandwidth on both, decrease the batch size required to reach the compute roofline and save memory to run at bigger batch sizes. Quantization has the added advantage that even if the model was not trained with quantization it can often be applied post training.

Using ragged HBM reads and Paged Attention: We allocated 8k of context for each KV cache in the calculations above but it is often not necessary to read the entire KV cache from memory — requests have a wide range of length distributions and don't use the max context of the model, so we can often implement kernels (e.g. Flash Attention variants) that only read the non-padding part of the KV cache.

Paged Attention is a refinement upon this that stores KV caches in OS-style page tables and mostly avoids padding the KV caches altogether. This adds a lot of complexity but means every batch only uses as much memory as it needs. This is a runtime optimization, so again it is indifferent to architecture.

![Figure: during generation, a single token (](assets/img/paged-attention.png)

Big Picture: All told, these KV cache optimizations can reduce KV cache sizes by over an order of magnitude compared to a standard MHA Transformer. This can lead to an order-of-magnitude improvement in the overall cost of the Transformer.

Distributing Inference Over Multiple Accelerators

So far we've handwaved how we're scaling beyond a single chip. Following Section 5, let's explore the different strategies available to us and their tradeoffs. As always, we will look at prefill and generation separately.

Prefill

From a roofline standpoint, **prefill is almost identical to training** and almost all the same techniques and tradeoffs apply — model (Megatron) parallelism, sequence sharding (for sufficiently long context), pipelining, even FSDP are all viable! You just have to keep the KVs kicking around so you can do generation later. As in training, increasing the number of chips gives us access to more FLOPs/s (for potentially lower TTFT), but adds communication overhead (potentially reducing throughput per chip).

The general rule for sharding prefill: here's a general set of rules for prefill. We'll assume we're doing prefill on a single sequence only (no batch dimension):

1. *Model sharding:* We typically do some amount of model parallelism first, up to the point we become ICI-bound. As we saw in Section 5, this is around $F/2200$ for 1 axis (usually around 4-8 way sharding).
2. *Sequence parallelism:* Beyond this, we do sequence parallelism (like data parallelism but sharding across the sequence dimension). While sequence parallelism introduces some extra communication in attention, it is typically fairly small at longer contexts. As with training, we can overlap the communication and computation (using collective matmuls for Megatron and ring attention respectively).

Takeaway: during prefill, almost any sharding that can work during training can work fine. Do model parallelism up to the ICI bound, then do sequence parallelism.

Generation

Generation is a more complicated beast than prefill. For one thing, it is harder to get a large batch size because we need to batch many requests together. Latency targets are lower. Together, these mean we are typically more memory-bound and more sensitive to communication overhead, which restrict our sharding strategies:

1. **FSDP is impossible:** since we are memory-bound in loading our parameters and KV caches from HBM to the MXU, we do not want to move them via ICI which is orders of magnitudes slower than HBM. *We want to move activations rather than weights.* This means methods similar to FSDP are usually completely unviable for generation.[47]
2. **There is no reason to do data parallelism:** pure data parallelism is unhelpful because it replicates our parameters and doesn't help us load parameters faster. You're better off spinning up multiple copies of the model instead.[48]
3. **No sequence = no sequence sharding.** Good luck sequence sharding.

This mostly leaves us with variants of model sharding for dense model generation. As with prefill, the simplest thing we can do is simple model parallelism (with activations fully replicated, weights fully sharded over hidden dimension for the MLP) up to 4-8 ways when we become ICI bound. However, since we are often memory bandwidth bound, we can actually go beyond this limit to improve latency!

Note on ICI bounds for generation: during training we want to be compute-bound, so our rooflines look at when our ICI comms take longer than our FLOPs. However, during generation, if we're memory bandwidth bound by parameter loading, we can increase model sharding beyond this point and improve latency at a minimal throughput cost (in terms of tokens/sec/chip). More model sharding gives us more HBM to load our weights over, and our FLOPs don't matter.[49] Let's look at how much model parallelism we can do before it becomes the bottleneck.

$$T_{\text{HBM comms}} = \frac{2DF}{Y \cdot W_{\text{hbm}}} \qquad T_{\text{ICI comms}} = \frac{2BD}{W_{\text{ici}}}$$

$$T_{\text{ICI comms}} > T_{\text{HBM comms}} \rightarrow \frac{W_{\text{hbm}}}{W_{\text{ici}}} > \frac{F}{Y \cdot B} \rightarrow Y > F/(B \cdot \beta)$$

[47] Accidentally leaving it on after training is an easy and common way to have order of magnitude regressions

[48] By this we mean, spin up multiple servers with copies of the model at a smaller batch size. Data parallelism at the model level is strictly worse.

[49] In the sense that FLOPs time isn't bottlenecking us, so the thing we need to worry about is ICI time exceeding parameter loading time.

where $\beta = W_{\text{hbm}}/W_{\text{ici}}$. This number is usually around 8 for TPU v5e and TPU v6e. That means e.g. if F is 16,384 and B is 32, we can in theory do model parallelism up to `16384 / (32 * 8) = 64` ways without a meaningful hit in throughput. This assumes we can fully shard our KV caches 64-ways which is difficult: we discuss this below.

For the attention layer, we also model shard attention W_Q and W_O over heads Megatron style. The KV weights are quite small, and replicating them is often cheaper than sharding beyond K-way sharding.

Takeaway: our only options during generation are variants of model parallelism. We aim to move activations instead of KV caches or parameters, which are larger. When our batch size is large, we do model parallelism up to the FLOPs-ICI bound (F/α). When our batch size is smaller, we can improve latency by model sharding more (at a modest throughput cost). When we want to model shard more ways than we have KV heads, we can shard our KVs along the batch dimension as well.

Sharding the KV cache

We also have an additional data structure that needs to be sharded — the KV cache. Again, we almost always prefer to avoid replicating the cache, since it is the primary source of attention latency. To do this, we first Megatron-shard the KVs along the head dimension. This is limited to K-way sharding, so for models with a small number of heads, we shard the head dimension as much as possible and then shard along the batch dimension, i.e. $\text{KV}[2, B_Z, S, K_Y, H]$. This means the KV cache is completely distributed.

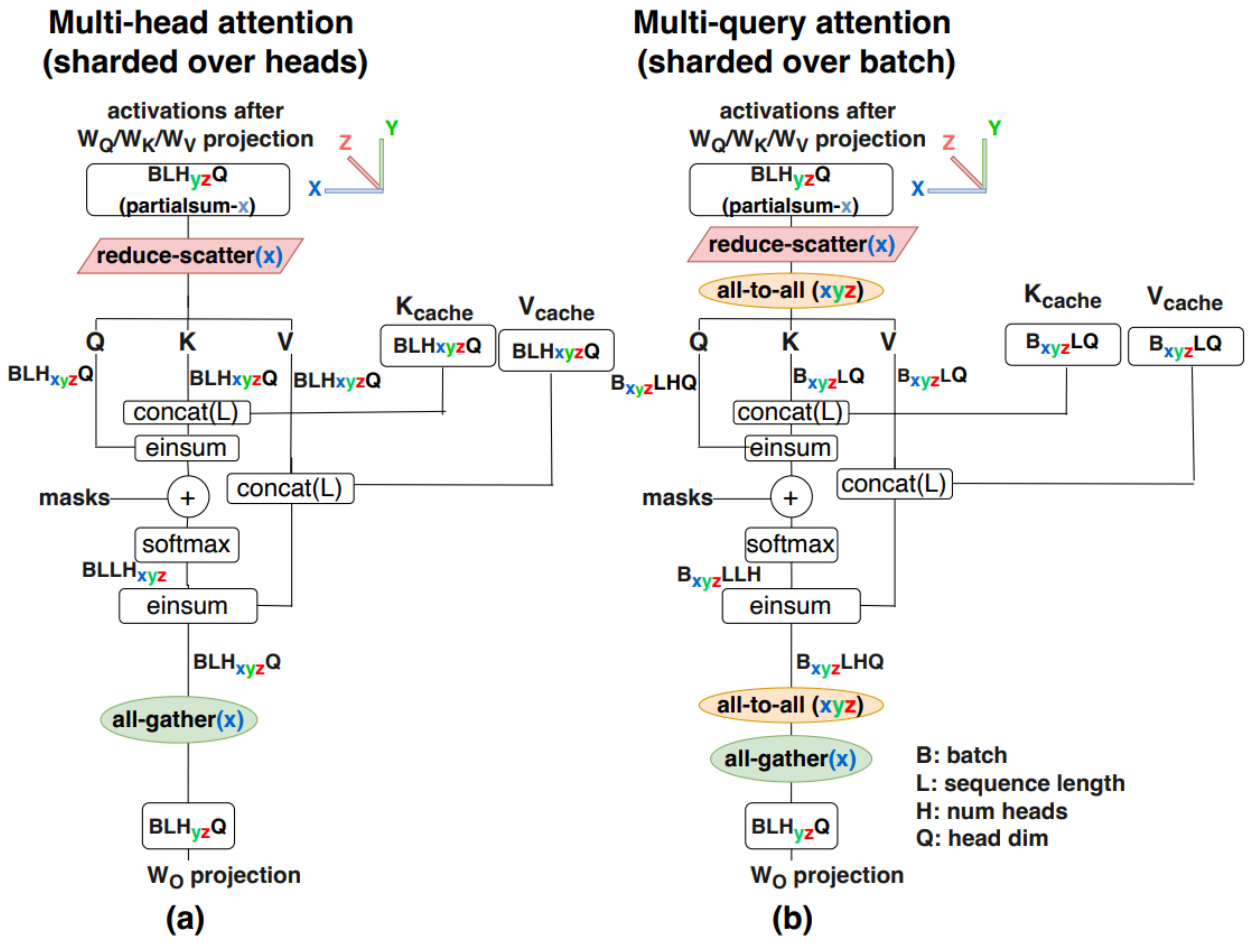


Figure 21: Figure: comparison of the attention mechanism with (a) Multi head attention with pure model sharding and (b) Multiquery attention with batch sharding of the KV cache. Notice how we need two extra AllToAlls to shift the activations from model sharding to batch sharding, so they can act on the KV caches.

The cost of this is two AllToAlls every attention layer — one to shift the Q activations to the batch sharding so we can compute attention with batch sharding, and one to shift the batch sharded attention output back to pure model sharded.

Here's the full algorithm!

Here we'll write out the full attention algorithm with model parallelism over both Y and Z. I apologize for using K for both the key tensor and the KV head dimension. Let $M = N/K$.

1. X[B, D] = … (existing activations, unsharded from previous layer)
2. K[BZ, S, KY, H], V[BZ, S, KY, H] = … (existing KV cache, batch sharded)
3. Q[B, NYZ, H] = X[B, D] * WQ[D, NYZ, H]
4. Q[BZ, NY, H] = **AllToAll**Z->B(Q[B, NYZ, H])
5. Q[BZ, KY, M, H] = **Reshape**(Q[BZ, NY, H])
6. O[BZ, S, KY, M] = Q[BZ, KY, M, H] *H K[BZ, S, KY, H]
7. O[BZ, S, KY, M] = **Softmax**S(O[BZ, S, KY, M])
8. O[BZ, KY, M, H] = O[BZ, S, KY, M] *S V[BZ, S, KY, H]
9. O[B, KY, MZ, H] = **AllToAll**Z->M(O[BZ, KY, M, H])
10. O[B, NYZ, H] = **Reshape**(O[B, KY, MZ, H])
11. X[B, D] {UYZ} = WO[NYZ, H, D] *N,H O[B, NYZ, H]
12. X[B, D] = **AllReduce**(X[B, D] { UYZ})

This is pretty complicated but you can see generally how it works. The new comms are modestly expensive since they operate on our small activations, while in return we save a huge amount of memory bandwidth loading the KVs (which are stationary).

- **Sequence sharding:** If the batch size is too small, or the context is long, we can sequence shard the KV cache. Again, we pay a collective cost in accumulating the attention across shards here. First we need to AllGather the Q activations, and then accumulate the KVs in a similar fashion to Flash Attention.

Designing an Effective Inference Engine

So far we've looked at how to optimize and shard the individual prefill and generate operations efficiently in isolation. To actually use them effectively, we need to design an inference engine which can feed these two operations at a point of our choosing on the latency/throughput Pareto frontier.

The simplest method is simply to run a batch of prefill, then a batch of generations:

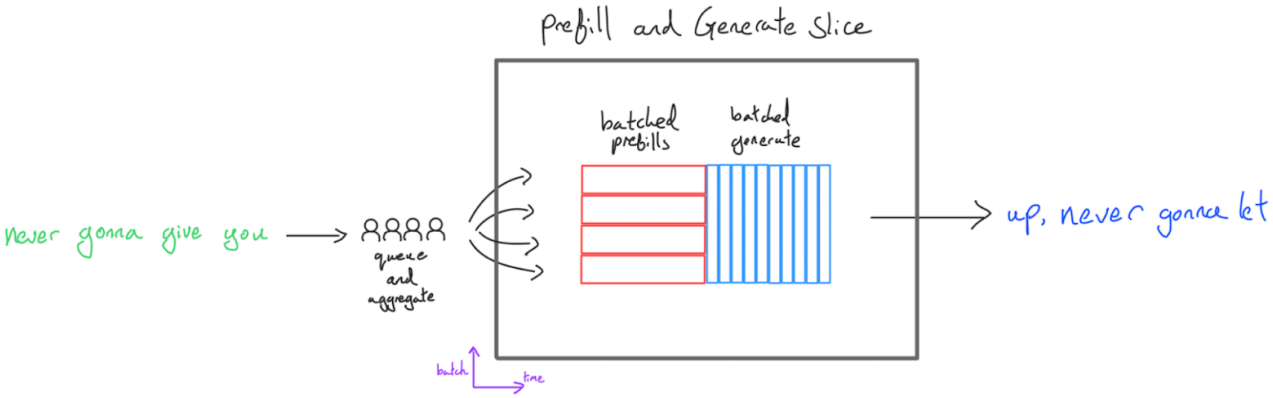


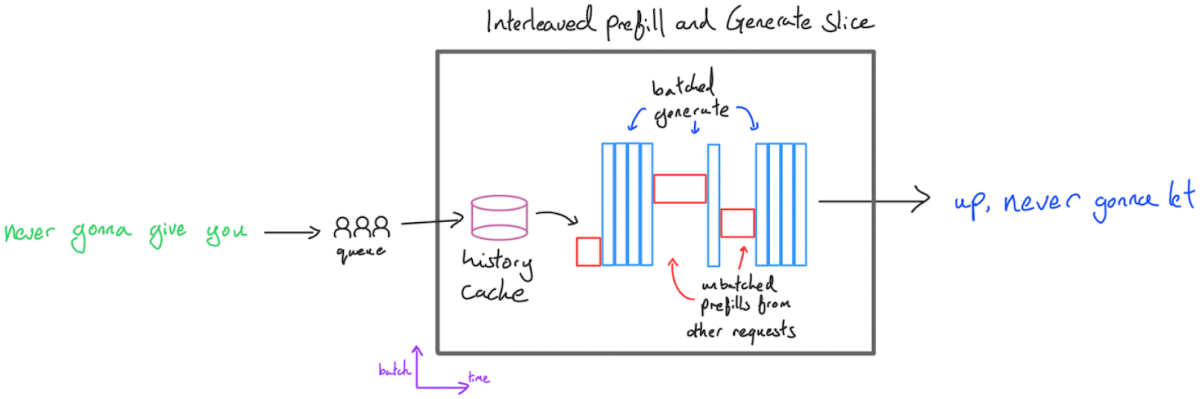
Figure 22: Figure: in the simplest setup, requests are aggregated, and the server alternates between running a batch of prefills and calling the generate function until completion for all sequences.

This is easy to implement and is the first inference setup in most codebases, but it has multiple drawbacks:

1. **Latency is terrible.** We couple the prefill and generate batch size. Time to first token (TTFT) is terrible at big prefill batch sizes — you need to finish all prefills before any users can see any tokens. Generate throughput is terrible at small batch sizes.
2. **We block shorter generations on longer ones.** Many sequences will finish before others, leaving empty batch slots during generation, hurting generate throughput further. The problem exacerbates as batch size and generation length increases.
3. **Prefills are padded.** Prefills are padded to the longest sequence and we waste a lot of compute. There are solutions for this, but historically XLA made it quite difficult to skip these FLOPs. Again this becomes worse the bigger the batch size and prefill sequence length.
4. **We're forced to share a sharding between prefill and generation.** Both prefill and generate live on the same slice, which means we use the same topology and shardings (unless you keep two copies of the weights) for both and is generally unhelpful for performance e.g. generate wants a lot more model sharding.

Therefore this method is only recommended for edge applications (which usually only cares about serving a single user and using hardware with less FLOPs/byte) and rapid iteration early in the lifecycle of a Transformer codebase (due to its simplicity).

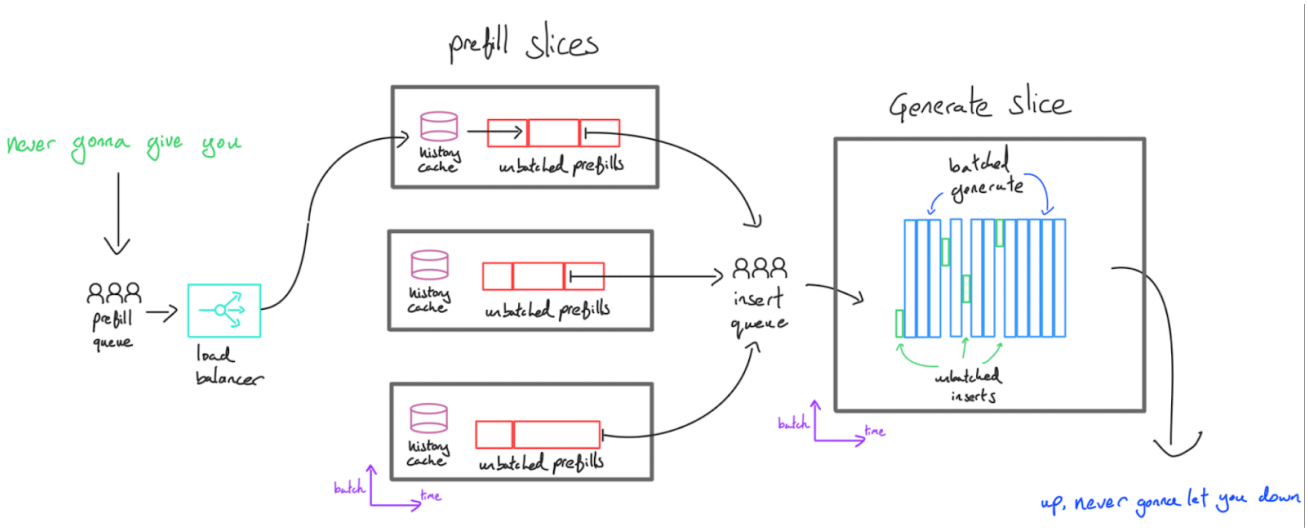
A slightly better approach involves performing prefill at batch size 1 (where it is compute-bound but has reasonable latency) but batch multiple requests together during generation:



This will avoid wasted TTFT from batched prefill while keeping generation throughput high. We call this an **interleaved** configuration, since we "interleave" prefill and generation steps. This is very powerful for bulk generation applications like evaluations where throughput is the main goal. The orchestrator can be configured to prioritise prefill the moment any generation slots open up, ensuring high utilisation even for very large generation batch sizes. We can also avoid padding our prefill to the maximum length, since it isn't batched with another request.

The main disadvantage is that when the server is performing a prefill, the generation of all other requests pauses since all the compute resources will be consumed by the prefill. User A whose response is busy decoding will be blocked by user B whose prefill is occurring. This means even though TTFT has improved, the token generation will be jittery and slow on average, which is not a good user experience for many applications — other user's prefills are on the critical path of the overall latency of a request.

To get around this, we separate decode and prefill. While Transformer inference can be done on one server, it is often better from a latency standpoint to execute the two different tasks on two sets of TPUs/GPUs. Prefill servers generate KV caches that get sent across the network to the generate servers, which batch multiple caches together and generate tokens for each of them. We call this **"disaggregated"** serving.



This provides a few advantages:

1. **Low latency at scale**: A user's request never blocks on another user's, except if there is insufficient prefill capacity. The request should be immediately prefilled, then sent to the generation server, then immediately slotted into the generation buffer. If we expect many concurrent requests to come in, we can scale the number of prefill servers independently from the number of generate servers so users are not left in the prefill queue for an extended period of time.
2. **Specialization:** Quite often, the latency-optimal parameter sharding strategy/hardware topology for prefill and generate is quite different (for instance, more model parallelism is useful for generate but not prefill). Constraining the two operations to use the same sharding hurts the performance of both, and having two sets of weights uses memory. Also, by moving prefill onto its own server, it doesn't need to hold any KV caches except the one it's currently processing. That means we have a lot more memory free for history caching (see the next section) or optimizing prefill latency.

One downside is that the KV cache now needs to be shifted across the network. This is typically acceptable but again provides a motivation for reducing KV cache size.

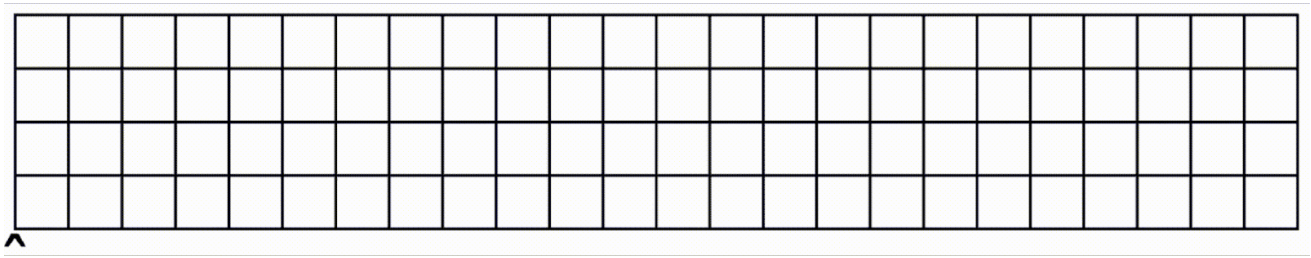
Takeaway: for latency-sensitive, high-throughput serving, we typically have to separate prefill and generation into separate servers, with prefill operating at batch 1 and generation batching many concurrent requests together.

Continuous batching

Problem (2) above motivates the concept of **continuous batching**. We optimize and compile:

- A prefill function that handles variable context lengths and inserts results into a KV buffer with some maximum batch size and context length/number of pages.
- A generate function which takes in the KV cache, and performs the generation step for all currently active requests.

We then combine these functions with an orchestrator which queues the incoming requests, calls prefill and generate depending on the available generate slots, handles history caching (see next section) and streams the tokens out.



Prefix caching

Since prefill is expensive and compute-bound (giving us less headroom), one of the best ways to reduce its cost is to do less of it. Because LLMs are autoregressive, the queries ["I", "like", "dogs"] and ["I", "like", "cats"] produce KV caches that are identical in the first two tokens. What this means is that, in principle, if we compute the "I like dogs" cache first and then the "I like cats" cache, we only need to do 1 / 3 of the compute. We can save most of the work by reusing the cache. This is particularly powerful in a few specific cases:

1. **Chatbots**: most chatbot conversations involve a back-and-forth dialog that strictly appends to itself. This means if we can save the KV caches from each dialog turn, we can skip computation for all but the newest tokens.
2. **Few-shot prompting**: if we have any kind of few-shot prompt, this can be saved and reused for free. System instructions often have this form as well.

The only reason this is hard to do is memory constraints. As we've seen, KV caches are big (often many GB), and for caching to be useful we need to keep them around until a follow-up query arrives. Typically, any unused HBM on the prefill servers can be used for a local caching system. Furthermore, accelerators usually have a lot of memory on their CPU hosts (e.g. a 8xTPUv5e server has 128GiB of HBM, but around 450GiB of Host DRAM). This memory is much slower than HBM — too slow to do generation steps usually — but is fast enough for a cache read. In practice:

- Because the KV cache is local to the set of TPUs that handled the initial request, we need some form of affinity routing to ensure follow-up queries arrive at the same replica. This can cause issues with load balancing.
- A smaller KV cache is helpful (again) — it enables us to save more KV caches in the same amount of space, and reduce read times.
- The KV cache and their lookups can be stored quite naturally in a tree or trie. Evictions can happen on an LRU basis.

![Figure: KV prefix cache implemented as an LRU trie. We can avoid duplicating KV memory by sharing prefixes. Source: <a href=](assets/img/prefix-caching-trie.png)

Let's look at an implementation: JetStream

Google has open-sourced a library that implements this logic called JetStream. The server has a set of "prefill engines" and "generate engines", usually on different TPU slices, which are orchestrated by a single controller. Prefill happens in the "prefill thread", while generation happens in the "generate thread". We also have a "transfer thread" that orchestrates copying the KV caches from the prefill to generate slices.

The Engine interface (implemented here) is a generic interface that any LLM must provide. The key methods are:

- **prefill:** takes a set of input tokens and generates a KV cache.
- **insert:** takes a KV cache and inserts it into the batch of KV caches that generate is generating from.
- **generate:** takes a set of batched KV caches and generates one token per batch entry, appending a single token's KV cache to the decode state for each token.

We also have a PyTorch version of JetStream available here.

Worked Problems

I'm going to invent a new model based on LLaMA-2 13B for this section. Here are the details:

hyperparam	value
L (num_layers)	64
D (d_model)	4,096
F (ffw_dimension)	16,384
N (num_heads)	32
K (num_kv_heads)	8

hyperparam	value
H (qkv_dim)	256
V (num_embeddings)	32,128

Question 1: How many parameters does the above model have? How large are its KV caches per token in int8? *You can assume we share the input and output projection matrices.*

Click here for the answer.

Parameter count:

- MLP parameter count: $L * D * F * 3$
- Attention parameter count: $L * 2 * D * H * (N + K)$
- Vocabulary parameter: $D * V$ (since we share these matrices)

Our total parameter count is thus $L * D * (3F + 2H * (N + K)) + D * V$. Plugging in the numbers above, we have `64 * 4096 * (3*16384 + 2 * 256 * (32 + 8)) + 4096 * 32128 = 18.4e9`. Thus, this model has about 18.4 billion parameters.

The KV caches are $2 * L * K * H$ per token in int8, which is `2 * 64 * 8 * 256 = 262kB` per token.

Question 2: Say we want to serve this model on a TPUv5e 4x4 slice and can fully shard our KV cache over this topology. What's the largest batch size we can fit, assuming we use int8 for everything and want to support 128k sequences? What if we dropped the number of KV heads to 1?

Click here for the answer.

Our KV caches have size $2 \cdot L \cdot K \cdot H$ per token in int8, or `2 * 64 * 8 * 256 = 262kB`. For 128k sequences, this means `262e3 * 128e3 = 33.5GB` per batch entry. Since each TPU has 16GB of HBM, including our parameters, the largest batch size we can fit is `(16 * 16e9 - 18.4e9) / 33.5e9 = 7`. If we had $K = 1$, we would have 8 times this, aka about 56.

Question 3: How long does it take to load all the parameters into the MXU from HBM assuming they're fully sharded on a TPU v5e 4x4 slice? Assume int8 parameters. *This is a good lower bound on the per-step latency.*

Click here for the answer.

We have a total of 18.4B parameters, or 18.4e9 bytes in int8. We have 8.2e11 HBM bandwidth per chip, so it will take roughly `18e9 / (8.2e11 * 16) = 1.4ms` assuming we can fully use our HBM bandwidth.

Question 4: Let's say we want to serve this model on a TPUv5e 4x4 slice using int8 FLOPs and parameters/activations. How would we shard it for both prefill and decode? *Hint: maybe answer these questions first:*

1. What does ICI look like on a 4x4?
2. What's the roofline bound on tensor parallelism?
3. How can we shard the KV caches?

For this sharding, what is the rough per-step latency for generation?

Question 5: Let's pretend the above model is actually an MoE. An MoE model is effectively a dense model with E copies of the FFW block. Each token passes through k of the FFW blocks and these `k` are averaged to produce the output. Let's use `E=16` and `k=2` with the above settings.

1. How many total and activated parameters does it have? *Activated means used by any given token.*
2. What batch size is needed to become FLOPs bound on TPU v5e?
3. How large are its KV caches per token?
4. How many FLOPs are involved in a forward pass with T tokens?

Click here for the answer.

(1) As an MoE, each MLP block now has $3 * E * D * F$ parameters, an increase of E over the dense variant. Thus it now has $L * D * (3EF + 2H * (N + K)) + D * V$ or `64 * 4096 * (3*16*16384 + 2 * 256 * (32 + 8)) + 4096 * 32128 = 212e9` total parameters, an increase of about 12x. For activated parameters, we

have k rather than E activated parameters, for a total of `64 * 4096 * (3*2*16384 + 2 * 256 * (32 + 8)) + 4096 * 32128 = 31.2e9`, an increase of less than 2x over the dense variant.

(2) Because we have E times more parameters for only k times more FLOPs, our HBM roofline increases by a factor of E/k. That means on a TPU v5e we need about `240 * (16 / 2) = 1920` tokens.

(3) The KV cache size stays the same as the MoE character doesn't change anything about the attention mechanism.

(4) This is still $2 \cdot$ activated params $\cdot T$. Thus this is $2 * 31.2\text{e}9 * T$.

Question 6: With MoEs, we can do "expert sharding", where we split our experts across one axis of our mesh. In our standard notation, our first FFW weight has shape `[E, D, F]` and we shard it as [EZ, DX, FY] where `X` is only used during training as our FSDP dimension. Let's say we want to do inference on a TPU v5e:

1. What's the HBM weight loading time for the above model on a TPU v5e 8x16 slice with Y=8, Z=16? How much free HBM is available per TPU?
2. What is the smallest slice we could fit our model on?

Question 7 [2D model sharding]: Here we'll work through the math of what the ESTI paper calls 2D weight-stationary sharding. We describe this briefly in Appendix B, but try doing this problem first to see if you can work out the math. The basic idea of 2D weight stationary sharding is to shard our weights along both the D and F axes so that each chunk is roughly square. This reduces the comms load and allows us to scale slightly farther.

Here's the algorithm for 2D weight stationary:

1. In[B, DX] = **AllGather**YZ(In[B, DXYZ])
2. Tmp[B, FYZ] {UX} = In[B, DX] *D Win[DX, FYZ]
3. Tmp[B, FYZ] = **AllReduce**X(Tmp[B, FYZ] {UX})
4. Out[B, DX] {UYZ} = Tmp[B, FYZ] *F Wout[FYZ, DX]
5. Out[B, DXYZ] = **ReduceScatter**YZ(Out[B, DX] {UYZ})

Your goal is to work out T_{math} and T_{comms} for this algorithm and find when it will outperform traditional 3D model sharding?

Click here for the answer!

Let's work out T_{math} and T_{comms}. All our FLOPs are fully sharded so as before we have $T_{\text{math}} = 4BDF/(N \cdot C)$ but our comms are now

$$T_{\text{2D comms}} = \frac{2BD}{2X \cdot W_{\text{ici}}} + \frac{4BF}{YZ \cdot W_{\text{ici}}} + \frac{2BD}{2X \cdot W_{\text{ici}}} = \frac{2BD}{X \cdot W_{\text{ici}}} + \frac{4BF}{YZ \cdot W_{\text{ici}}}$$

where we note that the AllReduce is twice as expensive and we scale our comms by the number of axes over which each operation is performed. Assuming we have freedom to choose our topology and assuming $F = 4D$ (as in LLaMA-2), we claim (by some basic calculus) that the optimal values for X, Y, and Z are $X = \sqrt{N/8}$, $YZ = \sqrt{8N}$ so the total communication is

$$T_{\text{2D comms}} = \frac{2B}{W_{\text{ici}}}\left(\frac{D}{X} + \frac{8D}{YZ}\right) = \frac{\sqrt{128}BD}{\sqrt{N} \cdot W_{\text{ici}}} \approx \frac{11.3BD}{\sqrt{N} \cdot W_{\text{ici}}}$$

Firstly, copying from above, normal 1D model parallelism would have $T_{\text{model parallel comms}} = 4BD/(3 \cdot W_{\text{ici}})$, so when are the new comms smaller? We have

$$\begin{aligned} T_{\text{model parallel comms}} > T_{\text{2D comms}} &\iff \frac{4BD}{3 \cdot W_{\text{ici}}} > \frac{\sqrt{128}BD}{\sqrt{N} \cdot W_{\text{ici}}} \\ &\iff N > 128 \cdot \left(\frac{3}{4}\right)^2 = 72 \end{aligned}$$

For a general F, we claim this condition is

$N > 32 \cdot \left(\frac{F}{D}\right) \cdot \left(\frac{3}{4}\right)^2$

So that tells us if we have more than 72 chips, we're better off using this new scheme. Now this is a slightly weird result because we've historically found ourselves ICI bound at around ~20 way tensor parallelism. But here, even if we're communication-bound, our total communication continues to decrease with the number of total chips! What this tells us is that we can continue to increase our chips, increase our batch size, do more parameter scaling, and see reduced latency.

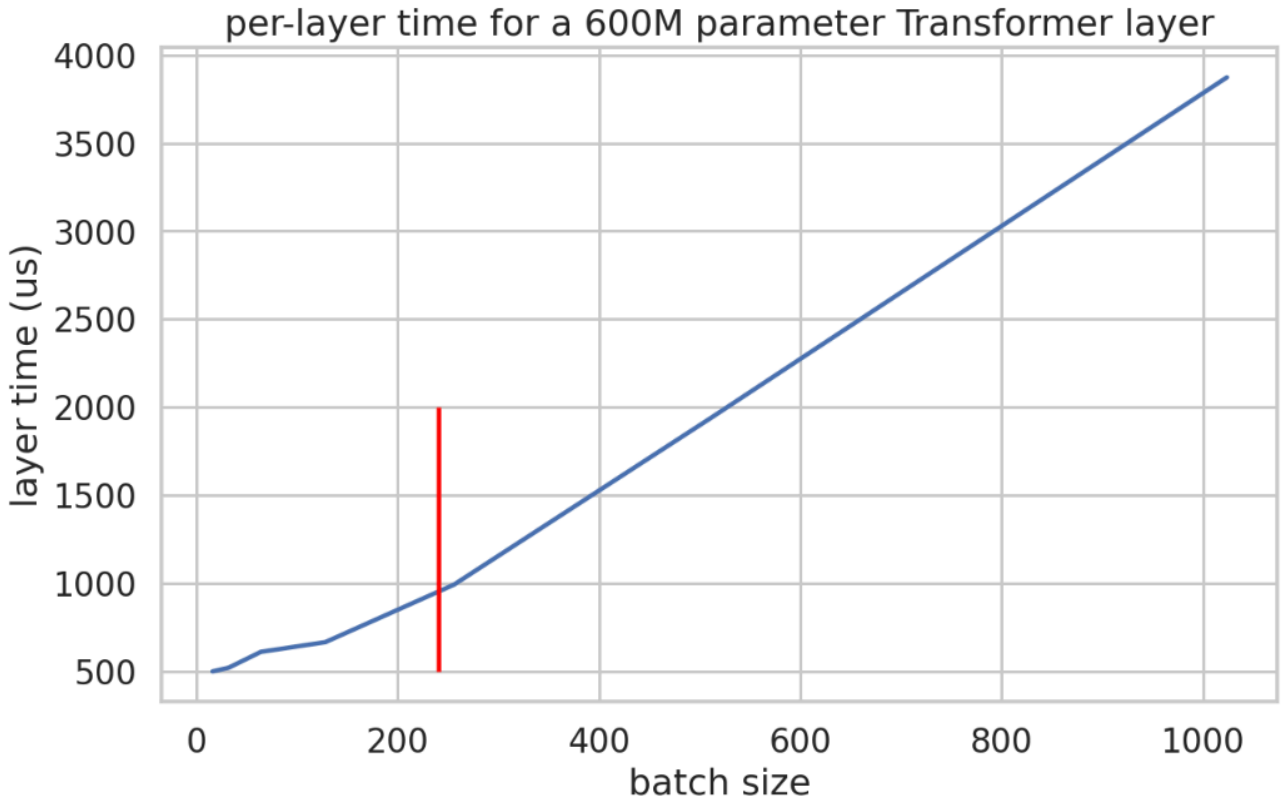
That's all for Part 7! For Part 8, with a look at how we might serve LLaMA 3 on TPUs, click here.

Appendix

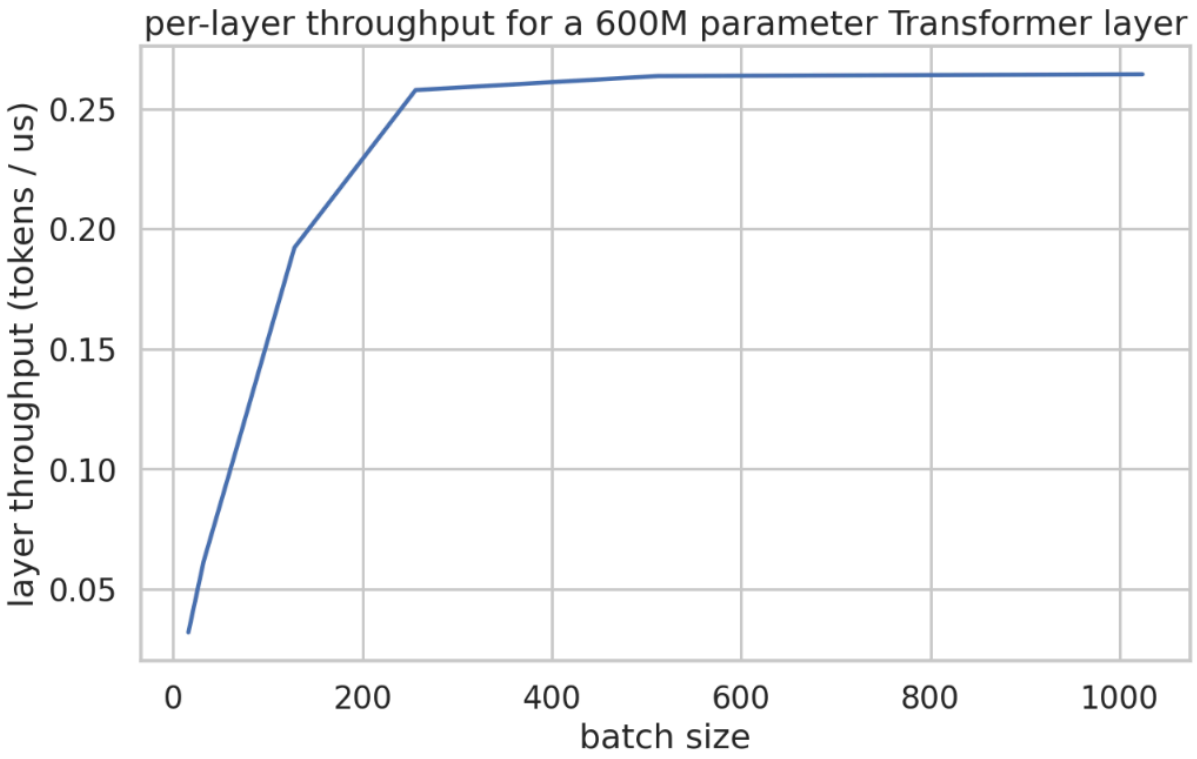
Appendix A: How real is the batch size > 240 rule?

The simple rule we provided above, that our batch size must be greater than 240 tokens to be compute-bound, is roughly true but ignores some ability of the TPU to prefetch the weights while other operations are not using all available HBM, like when doing inter-device communication.

Here's an empirical plot of layer time (in microseconds) for a small Transformer with dmodel 8192, dff 32768, and only 2 matmuls per layer. This comes from this Colab notebook. You'll see that step time increases very slowly up until around batch 240, and then increases linearly.



Here's the actual throughput in tokens / us. This makes the argument fairly clearly. Since our layer is about 600M parameters sharded 4 ways here, we'd expect a latency of roughly 365us at minimum.

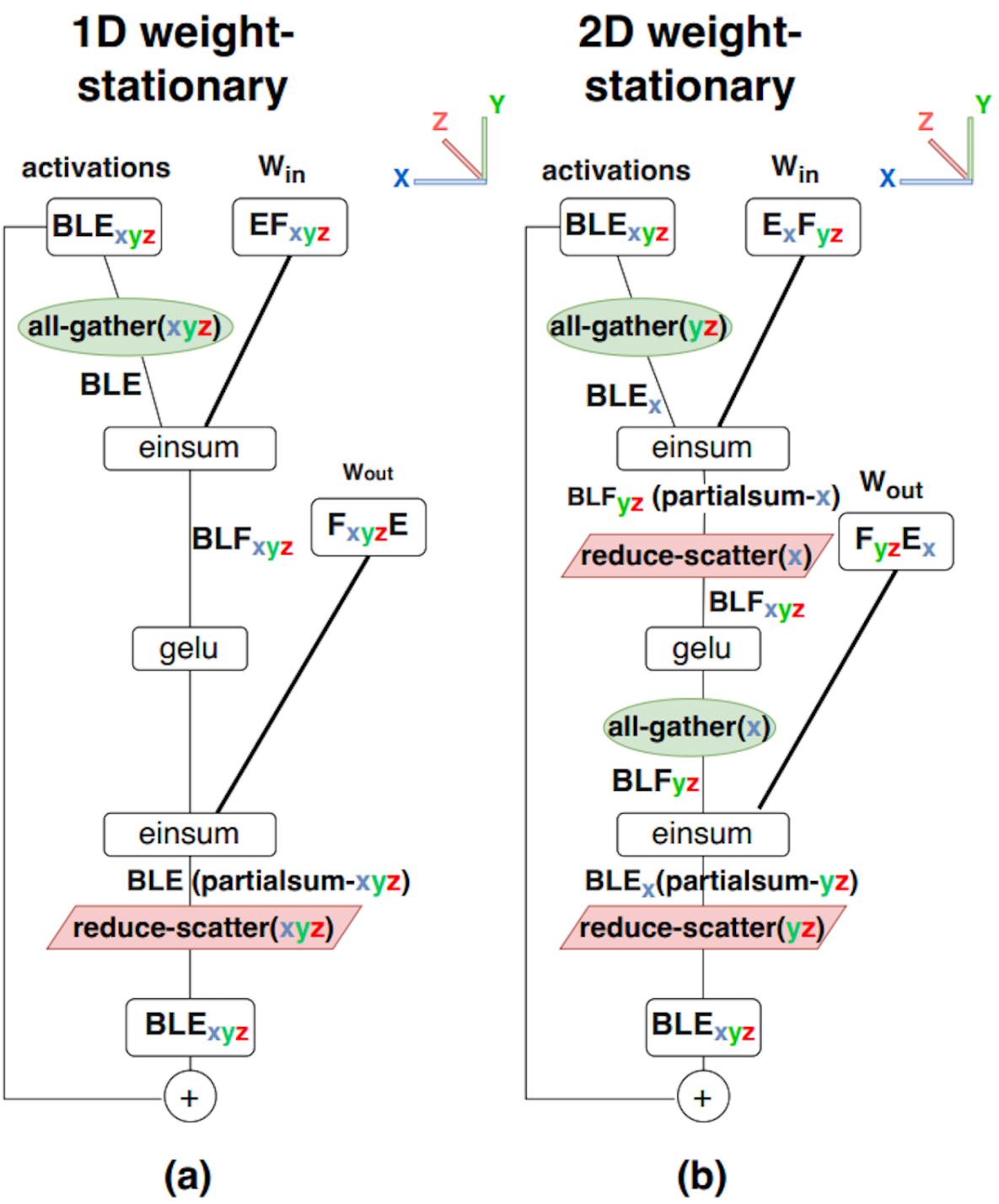


So at least in this model, we do in fact see throughput increase until about BS240 per data parallel shard.

Appendix B: 2D Weight Stationary sharding

As the topology grows, if we have access to higher dimensional meshes (like that of TPUs) it is possible to refine this further with "**2D Weight Sharding"** by introducing a second sharding axis. We call this"**2D Weight Stationary**", and was described in more detail in the Efficiently Scaling Transformer Inference paper.

Because we're only sharding the hidden F dimension in Megatron, it can become significantly smaller than E (the d_{model} dimension) once the number of chips grows large with 1D sharding. This means at larger batch sizes, it can be more economical to perform a portion of the collectives over the hidden dimension after the first layer of the MLP is applied.



This figure shows:

1. 1D weight-stationary sharding, a.k.a. Pure Megatron sharding, where activations are fully replicated after AllGather, and weights are fully sharded over the hidden F dimension.
2. 2D weight stationary sharding, where weights are sharded over both the hidden F and reduction E dimension, and activations are sharded over the E dimension. We perform an AllGather on the (yz) axis before the first layer, then ReduceScatter on the (x) axis.

For the attention layer, Megatron style sharding is also relatively simple for smaller numbers of chips. However,

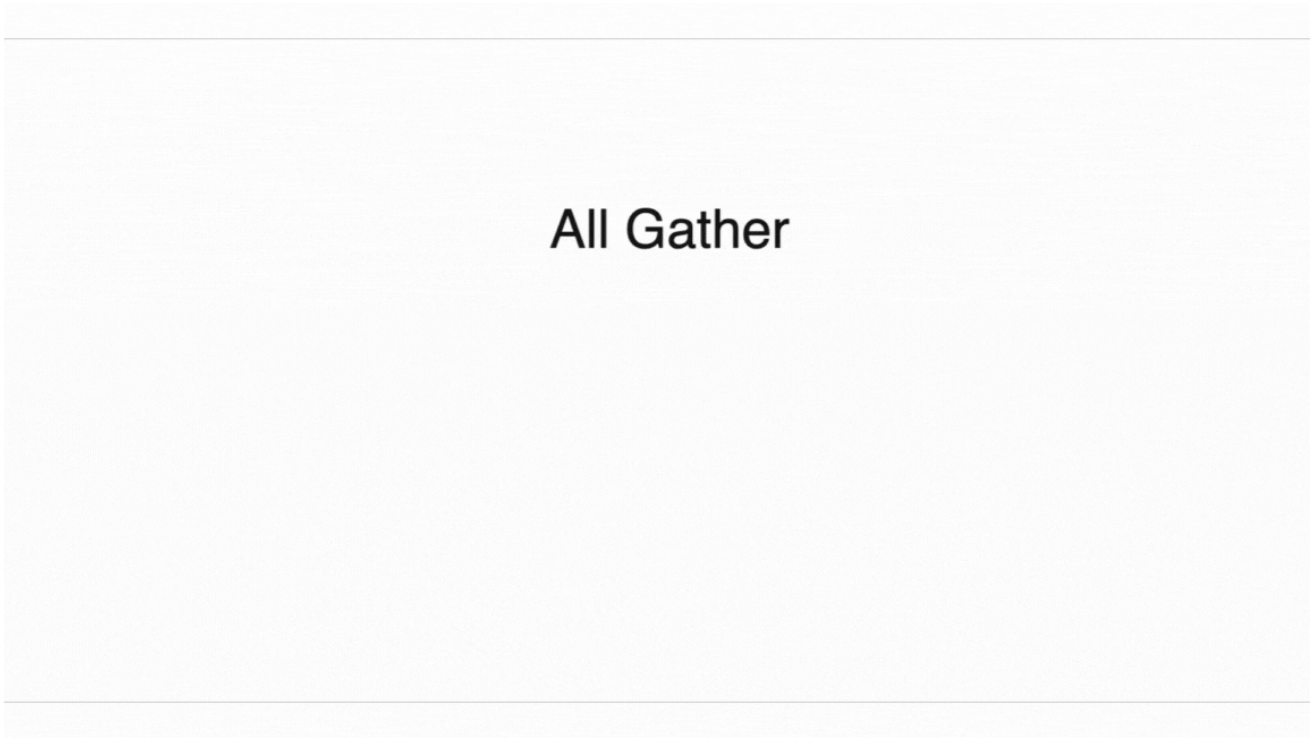
Megatron happens over the n_{heads} dimension, which puts a limit on the amount of sharding that is possible. Modifying the 2D sharding for attention (instead of sharding the hidden dimension, we shard the n_{heads} dimension), we gain the ability to scale further.

Appendix C: Latency bound communications

As a recap, in Section 3 we derived the amount of time it takes to perform an AllGather into a tensor of size B on each TPU, over X chips on a 1D ring with links of full-duplex bandwidth of WICI and latency Tmin.

$T_{total} = \max\left(\frac{T_{min} \cdot |X|}{2}, \frac{B}{W_{ICI}}\right)$

For large B, the wall clock stays relatively constant because as you add more chips to the system, you simultaneously scale the amount of data movement necessary to perform the operation and the total bandwidth available.



Because of the relatively low amounts of data being moved during latency optimized inference, collectives on activations are often bound by the latency term (especially for small batch sizes). One can visualise the latency quite easily, by counting the number of hops we need to complete before it is completed.

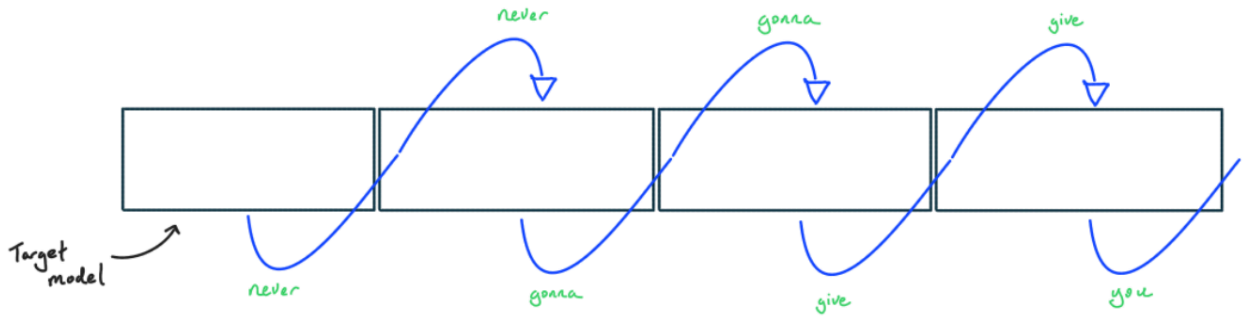
On TPUs, if the tensor size-dependent part of communication is less than 1 microsecond per hop (a hop is communication between two adjacent devices) we can be bottlenecked by the fixed overhead of actually dispatching the collective. With `4.5e10` unidirectional ICI bandwidth, ICI communication becomes latency bound when: $(\text{bytes}/n_{\text{shards}})/4.5e10 < 1e-6$. For 8-way Megatron sharding, this is when `buffer_size < 360kB`. **This actually is not that small during inference:** with `BS=16` and `D=8192` in int8, our activations will use `16*8192=131kB`, so we're already latency bound.

Takeaway: our comms become latency bound when total bytes $< W_{ICI} \times 1e-6$. For instance, with model parallelism over Y, we become bound in int8 when $Y > BD/45,000$.

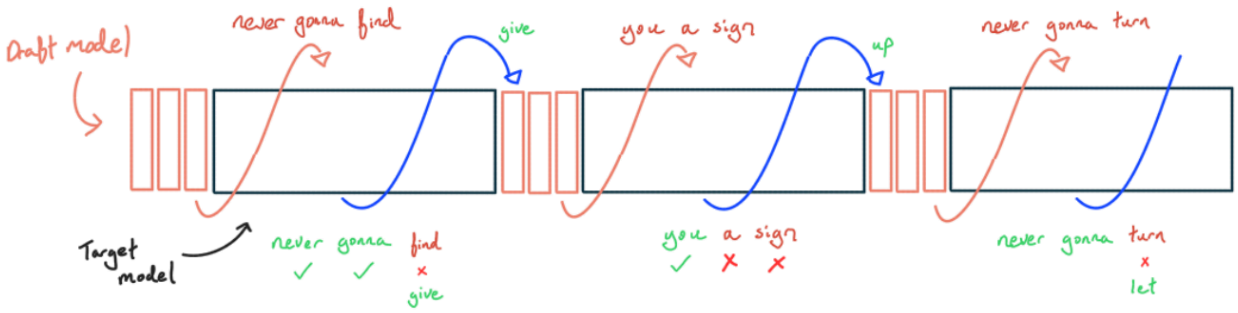
There's a parallel to be drawn here with the compute roofline — we are incurring the fixed cost of some small operations (latency for comms, memory bandwidth for matmuls).

Appendix D: Speculative Sampling

When we *really* care about end to end latency, there is one extra trick we can employ called speculative sampling. As a recap, we usually generate tokens from a large Transformer one by one:



With speculative sampling, we use a smaller, cheaper model to generate tokens and then check the result with the big model. This is easiest to understand with *greedy decoding*:



1. We sample greedily from some smaller, cheaper model. Ideally we use a model trained to match the larger model, e.g. by distillation, but it could be as simple as simply using n-grams or token matching a small corpus of text.
2. After we've generated K tokens, we use the big model to compute the next-token logits for all the tokens we've generated so far.
3. Since we're decoding greedily, we can just check if the token generated by the smaller model has the highest probability of all possible tokens. If one of the tokens is wrong, we take the longest correct prefix and replace the first wrong token with the correct token, then go back to (1). If all the tokens are correct, we can use the last correct logit to sample an extra token before going back to (1).

Why is this a latency win? This scheme still requires us to do the FLOPs-equivalent of one forward pass through the big model for every token, but because we can batch a bunch of tokens together, we can do all these FLOPs in one forward pass and take advantage of the fact that we're *not compute-bound* to score more tokens for free.

Every accepted token becomes more expensive in terms of FLOPs on average (since some will be rejected, and we have to call a draft model), but we wring more FLOPs out of the hardware, and the small model is cheap, so we win overall. We also share KV cache loads across multiple steps, so **speculative decoding can also be a throughput win for long context.** Since everything has been checked by the big model, we don't change the sampling distribution at all (though the exact trajectory will differ for non-greedy).

Traditionally, speculative decoding relies on the existence of a smaller model with a similar sampling distribution to the target model, e.g. LLaMA-2 2B for LLaMA-2 70B, which often doesn't exist. Even when this is available, the smaller drafter can still be too expensive if the acceptance rate is low. Instead, it can be helpful to embed a drafter within the main model, for instance by adding a dedicated drafter head to one of the later layers of the base model. Because this head shares most of its parameters with the main model, it's faster to run and matches the sampling distribution more closely.

For normal autoregressive sampling the token/s is the same as the step time. We are still beholden to the theoretical minimum step time according to the Arithmetic Intensity section here (in fact, Speculative Sampling step times are usually quite a bit slower than normal autoregressive sampling, but because we get more than 1 token out per step on average we can get much better tokens/s).

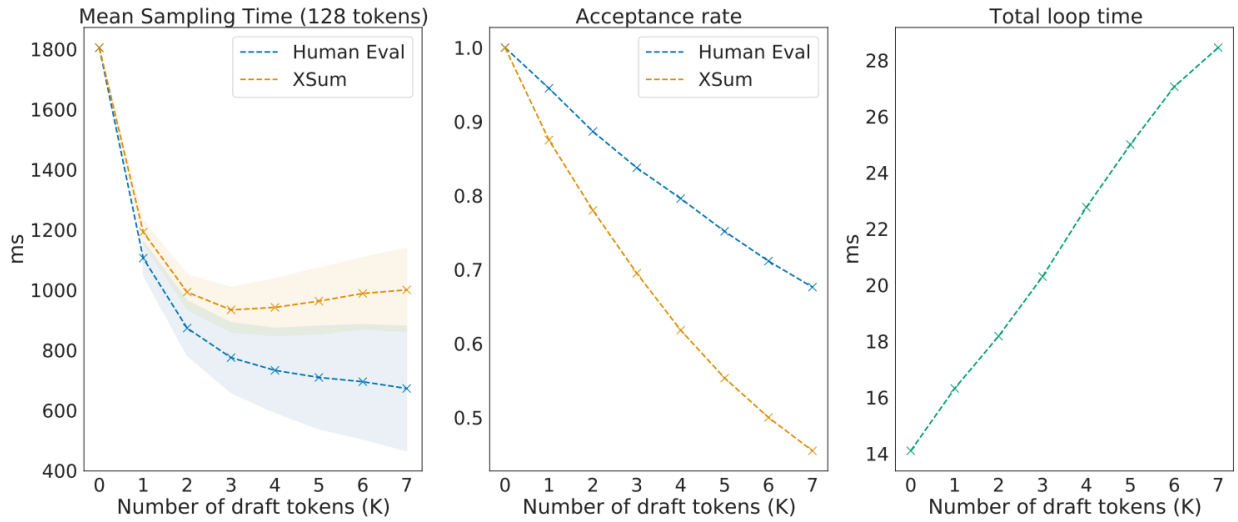


Figure 1 | **Left:** The average time to generate 128 tokens, with standard deviation. Note that as K increases, the overall speedup plateaus or even regresses, with XSum being optimal at $K = 3$. The variance consistently increases with K. **Middle:** The average number of tokens accepted divided by $K + 1$ – this serves as a measure of the overall efficiency of the modified rejection scheme, which decreases with the lookahead. **Right:** Average time per loop increases approximately linearly with K due to the increased number of model calls. Note that the gradient is slightly higher than the sampling speed of the draft model, due to additional overheads in nucleus decoding.

Figure 23: Figure: this figure shows the per-step latency and speculation success rate for Chinchilla (a 70B model from DeepMind) with a 4B parameter drafter (small model). For XSum (a natural language dataset), the ideal amount of speculation is about 3-4 tokens ahead, while HumanEval (a coding dataset) is more predictable and sees wins from more aggressive speculation.

How does this work for non-greedy decoding? This is a bit more complicated, but essentially boils down to a Metropolis-Hastings inspired algorithm where we have $P_{\text{draft model}}(\text{chosen token})$ and $P_{\text{target model}}(\text{chosen token})$ derived from the logits, and reject the chosen token probabilistically if the ratio of these probabilities is smaller than some threshold.

These two papers derived this concurrently and have good examples of how this works in practice.

Takeaway: Speculative sampling is yet another powerful lever for trading throughput for better per token latency. However, in the scenario where batch size is limited (e.g. small hardware footprint or large KV caches), it becomes a win-win.

Chapter 8

Serving LLaMA 3 on TPUs

This section will look at what it takes to serve LLaMA-3 and how efficiently it can be done. As in the previous "applied" section, try to work out the answers on your own with a pen and paper before looking them up!

What's the LLaMA Serving Story?

Let's remind ourselves what LLaMA 3-70B looks like (see Section 6 for reference):

hyperparam	value
n_{layers} (L)	80
d_{model} (D)	8,192
d_{ff} (F)	28,672
n_{heads} (N)	64
$n_{\text{kv heads}}$ (K)	8
d_{qkv} (H)	128
$n_{\text{embeddings}}$ (V)	128,256

Let's start with a simple question: **what hardware should we serve on?** The answer is basically, whichever is cheapest in FLOPs / dollar.[50] For this reason, we typically want to serve on TPU v5e, our current dedicated inference chip (cost comes from Google Cloud pricing as of February 2025):

TPU type	bfloat16 FLOPs/s	Google Cloud USD / hour	FLOPs / $
H100	9.9e14	$10.8	3.3e17
v5p	4.59e14	$4.2	3.9e17
v5e	1.97e14	$1.2	**5.8e17**

Each TPU v5e has 16GB of HBM which will require us to shard our model fairly aggressively. Let's start by thinking about some basic quantities that might matter for us:

Question: How large are LLaMA 3-70B's KV caches per token? *You can assume we store them in int8. This determines how large our batch size can be on a given topology.*

Click here once you've thought it through!

LLaMA 3-70B has 8 KV heads, so the size per token is `2 * K * H * L = 2 * 8 * 128 * 80 = 160kB`.

Note just how big this is! If we have a sequence length of 32k tokens (as is common), this uses `160e3 * 32,768 = 5.3GB / sequence`. For BS=240, this is 1.3TB! Since TPU v5e only have 16GB a piece, we would need about `(70e9 + 1.3e12) / 16e9 = 86` TPU v5e chips to even fit this much memory. Also note how large this is compared to the 70GB of model parameters.

[50] This isn't always true, sometimes more HBM or ICI bandwidth is critical rather than FLOPs, but this is a good heuristic.

Question: Let's say we want to serve L3 70B at batch size 32 and 8192 sequence length with everything (params and KVs) in int8. How much total memory will this use? What's the smallest slice we could serve this on?

Answer

Since our KVs are `160e3` bytes in int8, our total KV memory is `160e3 * 8192 * 32 = 41.9e9` bytes. Our parameters are `70e9` bytes, since we have 1 byte per parameter. Thus, our total memory usage is `41.9e9 + 70e9 = 112GB`.

The smallest slice we could use would have `112e9 / 16e9 = 7` TPUs, or (rounding to an even size), TPU v5e `4x2`. This will be a tight fit and we might not be able to quite fit this accounting for other overhead, so we might need a `4x4` at minimum (or to drop the batch size).

Question: At this batch size and quantization on a TPU v5e `4x2`, roughly what latency would we expect per decode step? What throughput (tokens / sec / chip). What about a `4x4`? *Assume we perform our FLOPs in bfloat16 and everything is fully sharded.*

Answer

We can invoke the formula from the previous section that

$$\text{Theoretical Step Time (General)} = \underbrace{\frac{\text{Batch Size} \times \text{KV Cache Size}}{\text{Total Memory Bandwidth}}}_{\text{Attention (always bandwidth-bound)}} + \underbrace{\max\left(\frac{2 \times \text{Batch Size} \times \text{Parameter Count}}{\text{Total FLOPs/s}}, \frac{\text{Parameter}}{\text{Total Memory}}\right.}_{\text{MLP (can be compute-bound)}}$$

Here our critical batch size will be about 120 since our parameters are in int8 but our FLOPs are in bfloat16. We could also manually calculate the RHS maximum, but that's basically a calculation we've already done several times. **So we're well into the memory-bound regime for both our matmul and our FLOPs.**

Strictly looking at memory bandwidth then, our step time is basically `(KV size + param size) / (8 * HBM bandwidth) = 112e9 / (8 * 8.2e11) = 17ms`. **So theoretically our step time is about 17ms.** Our throughput would be `32 / .017 = 1882 tokens / sec`, or `1882 / 8 = 235 tokens / sec / chip`.

There's one caveat here which is to check if we might be ICI bound on our matmuls. We could dedicate 2 axes to it here, so we're ICI bound in theory when $Y > 2 * F/2200 = 2 * 28672/2200 = 26$, so we're golden!

If we were to run on a `4x4`, we'd still be fine ICI-wise, so our latency would drop to `17 / 2 = 8.5ms`, but our throughput per-chip would remain the same.

Thinking about throughput

Let's spend a little time thinking purely about throughput. When we optimize for throughput, we want to be compute bound, meaning we come close to utilizing all the TPU MXU capacity. Typically that means we want the batch size to be as large as possible, so we are doing as much work as possible.

Question: On TPU v5e, using bfloat16 weights and activations, how large do our batch sizes need to be for us to be compute-bound in our matmuls? What if we do int8 weights but perform our FLOPs in bfloat16? What about int8 weights with int8 FLOPs?

Answer

As discussed in Section 7, for any bfloat16 matmul for which $B \ll D, F$ we have

$$T_{\text{math}} > T_{\text{comms}} \leftrightarrow \frac{2BDF}{2DF} \geq \frac{\text{TPU bfloat16 FLOPs/s}}{\text{HBM bandwidth}} = 240$$

When our weights are in int8, we lose a factor of 2 in the denominator, so we have $2BDF/DF = 2B > 240$, or equally $B > 120$, half the critical batch size from before. That's really helpful for us! When we do int8 weights and int8 FLOPs, we have to use the int8 value for TPU FLOPs/s, which goes from 1.97e14 for bfloat16 to 3.94e14, nearly double. That means we're back where we started at about $B > 240$.

The case of int8 weights and bfloat16 FLOPs is quite common, since quantizing parameters losslessly is often easier than doing low-precision arithmetic.

Question: What is the smallest TPU v5e topology we could serve LLaMA 3-70B on using bfloat16, int8, and int4 (both KVs and parameters) with 8k context? *You can think of KV caches as negligibly small for this one.*

Answer

This is easy! If we're OK with a tiny batch size then the only limit is fitting parameter memory in HBM, i.e. it is just `ceil(num_params * sizeof(dtype) / HBM per TPU)`, or `ceil(70e9 * sizeof(dtype) / 16e9)` rounded to the nearest reasonable topology (some multiple of 2):

dtype	param size	KV size / token (bytes)	min TPU v5es	actual min slice	remaining HBM for KV caches	num KV caches @ 8k
bf16	140GB	324kB	8.75	4x4 = 16 chips	116	43
int8	70GB	162kB	4.38	4x2 = 8 chips	58	43
int4	35GB	81kB	2.81	2x2 = 4 chips	29	43

That's pretty cool! It tells us we could fit LLaMA 70B on a TPU v5e 2x2 if we wanted to. Except you'll notice the number of KV caches is very small. That's our batch size! That means we'll be getting terrible FLOPs utilization. We'd be very happy to use a larger topology in order to push our batch size up to 240.

Question: Assume we use the largest batch size that fits on these topologies, what latency could we expect for each generate step?

Answer

This is also easy, since we're picking our batch size to fill up all our HBM! This is just a question of how long it takes to load a full TPU v5e's worth of bytes into the MXU. This is just `v5e HBM / v5e HBM memory bandwidth = 16GB / 8.2e11 = 19ms`, so this is **19ms / step**. Assuming our generations have a median length of 512 tokens, that is about 9s for each decode. Note that we could get marginally better latency with a smaller batch size, for instance if we only looked at model parameters in int4 our minimum latency is about 10ms / step, since HBM is no longer full.

Takeaway: we can always lower bound decode latency by asking how long it takes to load all the model's parameters from HBM into the MXU. When our KV caches are small, you can think about each layer as just loading the weights chunk-by-chunk and then discarding them. Unless we're using large batch sizes or lots of inter-device comms, this is often a reasonable bound (within 1.5x). When our batch size is bigger, we need to model the KV cache loading as well, since that dominates the parameters.

Likewise, in the FLOPs-bound regime (e.g. training or big-batch inference), we can use the Total FLOPs/$(N \cdot C)$ = $2 \cdot \text{param count} \cdot B/(N \cdot C)$ lower bound, which assumes no communication.

Question: For each of these, what throughput per chip does this give us (in terms of queries / chip)? *You can assume our median decode length is 512 tokens.*

Answer

This is an important question because it's exactly correlated with cost / token.

With our assumption about median decode length, our throughput is just $B/(\text{per-step latency} \cdot \text{median steps} \cdot N) \approx 43/(0.019 * 512 * N)$. This gives us roughly $(4.42/N)$ QPS, so plugging in N we get:

dtype	QPS / chip
bfloat16	0.27
int8	0.55
int4	1.11

Note that this is rather optimistic since it totally ignores the working memory of the forward pass (memory allocated to activations and attention). This is not ridiculous with Flash Attention, but it is also not realistic. The real numbers are likely around 1/2 of this. For absolutely maximum throughput we would probably want to more than double the number of chips and increase the batch size significantly as well.

Question: How would our peak throughput change if we doubled our topology for each of the above examples?

Answer

If we used a 4x8 slice in bfloat16, we would have 372GB remaining for KV caches, which would let us increase our batch size to 140. Then since our step time would remain the same, we would have a throughput of `14.39 / num_chips`, or

dtype	QPS / chip
bfloat16 (on 4x8)	0.44
int8 (on 4x4)	0.90
int4 (on 2x4)	1.80

A further increase would give an even bigger win! The big takeaway is that **the smallest topology is not the most performant topology** in all cases, if we're limited by KV cache size.

Question: Now let's dig into the question of sharding. Let's say we wanted to serve in bfloat16 on a TPU v5e 4x8. What sharding would we use for our model on a TPU v5e 4x8 during generation? Can we avoid being communication bound?

Answer

As discussed in the previous section, we only really have one option for sharding during generation: model parallelism. How much can we do before we become communication bound? As we've discussed in the previous section, our models become communication bound roughly when

$Y > \frac{F \cdot M_Y}{2200}$

For LLaMA 3-70B we have `F = 28,672`, so if we do 2 axes of model sharding this gives us roughly $Y = 28672 \cdot 2/2200 = 26$, so in general we could scale up to about 16 chips without being communication bound, which lets us use a `4x4` but not a `4x8`. Generally, since we do not perfectly overlap computation, even this estimate is overly optimistic.

Takeaway: we cannot actually serve on a 4x8 with pure model parallelism. The best we can do here is a 4x2 or *maybe* a 4x4.

However, as we've discussed, when our batch size is small we can often do more model parallelism without significantly hurting throughput, since our model is memory-bandwidth-bound and not FLOPs bound. We said before that this value is roughly $Y = F/(8 \cdot B)$, so if we did batch size 64, we could in theory go up to `Y = 28,672 / (8 * 64) = 56` way model parallelism before we become ICI-bound. To sanity check this, we can look at $T_{\text{ici comms}}$, $T_{\text{hbm comms}}$, and T_{math} for a single matmul. We clearly have:

$$T_{\text{ici comms}} = \frac{2BD}{W_{\text{ici}}} \qquad T_{\text{hbm comms}} = \frac{2DF}{Y \cdot W_{\text{hbm}}} \qquad T_{\text{math}} = \frac{2BDF}{Y \cdot C}$$

For a `4x8`, this would give us $T_{\text{ici comms}}$ = `(2 * 64 * 8192) / 9e10 = 11us`, $T_{\text{hbm comms}}$ = `(2 * 8192 * 28,672) / (32 * 8.2e11) = 18us`, and T_{math} = `(2 * 64 * 8192 * 28,672) / (32 * 1.97e14) = 4us`, so in theory we're still HBM bandwidth bound, which is great! *Note that scaling up from a `4x4` to a `4x8` probably isn't helpful from a throughput standpoint, but it'll reduce our latency!*

If we look at the int8 and int4 configs, we *can* do those with pure model parallelism. So we've hit a point at which quantization actually gives us a meaningful advantage beyond faster FLOPs: it lets us use a larger batch size before we become comms-bound. **So the end of this story is that we can't achieve peak throughput on a 4x8, but for the int8 and int4 configs we could do pure model parallelism.**

Tip: the maximum amount of useful model parallelism depends on d_{ff} and the number of axes over which you're sharding your model. The maximum value usually ranges between 8 and 32 depending on the model size. You can scale beyond this limit to improve latency at some throughput cost.

What about prefill?

We've mostly ignored prefill here because it's much simpler. Let's put a couple of concepts together and think about the end-to-end picture.

Question: Assume we achieve a 40% FLOPs utilization during prefill. How long will a prefill of length 8192 take on 16 TPU v5e chips?

Answer

At 8k tokens, we are solidly compute bound, so we just need to reason about FLOPs. We know our model has `70e9` parameters so each forward pass uses `2 * 70e9 * B` FLOPs. Assuming 40% MFU (FLOPs utilization), this gives us a runtime of about `2 * 70e9 * 8192 / (16 * 1.97e14 * 0.4) = 0.91s`. Compared to the numbers we've been looking at before, that's actually quite a lot!

Question: Assume we have a median prefill length of 8192 tokens and a median decode length of 4096 tokens. Say we have a generate batch size of 32. On average how many sequences finish decoding per step? On average how many tokens are evicted from our KV cache each step?

Answer

This is kind of straightforward. Since we have a median decode length of 4096 tokens, a sequence will finish roughly every 1 / 4096 tokens. Given a batch size of 32, this means we have `32 / 4096` sequences evicted per step. Since our KV cache length is roughly `8192 + 4096`, this is `32 * (8192 + 4096) / 4096 = 96` tokens evicted per step. The general formula is $B * (P + G)/G$ where P and G are the prefill and generate lengths.

Question: Assume we do disaggregated serving with a median prefill length of 8192 and a median decode length of 512. Assume the prefill and generate latencies calculated above in bfloat16. What ratio of prefill:generate servers will you need to keep both fully saturated.

Answer

This is kind of a fun question. Let P be the number of prefill servers and G be the number of generate servers. So generally speaking, this is a pipeline problem where we feed sequences in at a rate of `P / prefill_latency` and consume them at a rate of `B * G / (generate_latency * median_decode_length)`. We had calculated `910ms` per prefill step and `19ms` per decode step at batch size 43 (let's call that 32). Therefore we need `P / 0.91 = 32 * G / (0.019 * 512)` or `P = 3G`, i.e. we need about 3 times more prefill servers than generation servers!

Visualizing the Latency Throughput Tradeoff

Sticking with LLaMA 70B for a second, let's actually look at the latency and throughput for different batch sizes during generation. As we showed in the previous section for PaLM models, this gives us a Pareto frontier for throughput/latency. Let's assume 16-way tensor parallelism since that's a reasonable bound on what we can use while staying compute-bound in the MLP blocks. We'll use a TPU v5e 4x4 topology here. **The slider controls the sequence length so you can see the effect of larger KV caches.**

Figure:** throughput per chip against per-token latency for LLaMA 3-70B (int8 weights, 16-way tensor parallelism) on a TPU v5e 4x4, sweeping batch size along each curve. Drag the slider to change the context length; each curve stops at the largest batch whose KV cache fits in HBM.

- **See how dramatic the tradeoff is between cost and latency.** At the cost of doubling per-token latency, we can achieve a roughly 100x reduction in per-token cost. Also, our latency can range anywhere from 5.5ms with low batch size to 20 ms with very large batches.
- Note how at 2k context the throughput effectively plateaus at around 1 token / ms / chip when it hits the BS 120 roofline (120 here because we do int8 weights but bf16 FLOPs). As the sequence length increases, however, we can no longer fit this batch size in memory, so we never hit the point of full saturation.

- Note how much higher the latency is at large batch sizes for the same throughput, since KV loading becomes dominant (instead of parameter loading).

We can understand this better by breaking down the sources of cost and latency into param loading time, KV loading time, and FLOPs time. The shaded region is where we expect to be compute-bound in our MLP blocks.

Figure:** time per decode step for LLaMA 3-70B on a TPU v5e 4x4 as a function of batch size, split into parameter loading, KV cache loading, and FLOPs. The total is the KV loading time plus whichever of parameter loading or FLOPs is larger. Drag the slider to change the context length.

This tells quite a story. You can see that initially, parameter loading represents the vast majority of the latency, until the batch size becomes large enough that FLOPs and KV loading become more significant. Notably, at all sequence lengths greater than 2048, we spend more time on KV cache loading than we do on FLOPs! **So while we can improve our hardware utilization by increasing batch size, at long context lengths KV loading always dominates the total step time.**

Takeaway: for LLaMA 3-70B, we are strongly KV cache memory bandwidth-bound (and HBM-bound) in almost all of these configurations, highlighting just how important reducing KV cache size is for generation throughput. Also note just how dramatic the latency/throughput tradeoff remains here.

The code for this is quite simple.

Here's the code for computing these rooflines:

```
import numpy as np

num_chips = 16  # we fix 16 as the amount of total model parallelism we do
bytes_per_param = 1  # int8 means 1 byte per param
param_count = 70e9
param_size = bytes_per_param * param_count
sequence_length = 8192  # can vary this

hbm_bandwidth = 8.20E+11  # v5e
flops = 1.97E+14  # v5e

def kv_cache_size(bs):
    return 2 * bs * 128 * 8 * 80

def min_topology(bytes):
    return 2 ** np.ceil(np.log2(bytes / 16e9))

def get_max_batch_size(
    num_chips: int,
    sequence_length: int,
    param_size: float,
) -> int:
  batch_sizes = np.arange(1, 1024, 4)
  kv_sizes = kv_cache_size(sequence_length * batch_sizes)
  required_chips = min_topology(kv_sizes + param_size)
  max_idx = np.where(required_chips <= num_chips)[0][-1]
  return max_idx

max_idx = get_max_batch_size(
    num_chips=num_chips,
    sequence_length=sequence_length,
    param_size=param_size,
)  # get the largest batch size that can fit
batch_sizes = np.arange(1, 512, 1)[:max_idx]
kv_sizes = kv_cache_size(sequence_length * batch_sizes)
```

```
kv_comms_time = kv_sizes / (num_chips * hbm_bandwidth)

param_comms_time = param_size / (num_chips * hbm_bandwidth)
param_comms_time = np.asarray([param_comms_time] * batch_sizes.shape[0])

flops_time = 2 * param_size * batch_sizes / (num_chips * flops)  # roughly true in a 2ND sense

mlp_time = np.maximum(flops_time, param_comms_time)
attn_time = kv_comms_time  # always bandwidth-bound for generate

latency = 1000 * (mlp_time + attn_time)
throughput = batch_sizes / (latency * num_chips)
```

Note how we very explicitly break out latency into two sources: KV loading and param loading, and how the latency is either bound by FLOPs or comms, whichever is bigger.

Worked Problems

Here are a few worked problems. Some of these repeat things that are worked above, but might be pedagogically useful.

Question 1: How many FLOPs does each forward pass for LLaMA 3-405B use per-token? Assuming we're FLOPs bound, what is a lower bound on a single forward pass on N chips on TPU v5e? What if we're comms bound? *Ignore the fact that the model does not fit on a single chip.*

Question 2: Assume we want to serve LLaMA 3-8B with BS240 using int8 weights and int8 KV caches. How many bytes are used by (a) model parameters (b) KV caches and (c) peak working activations (roughly)? What's the smallest topology we can run this on?

Question 3: How would you serve LLaMA 3-405B on TPU v5e? Assume int8 weights and bfloat16 FLOPs. Let's say we have a firm limit of 15ms / token, what's the highest throughput configuration we could achieve? What is the theoretical minimum step time?

Question 4: The best way to learn about LLMs is to implement one from scratch. Building the full training pipeline is annoying and expensive, but turning trained weights you can download from HuggingFace into a working inference implementation is extremely instructive. Roughly speaking, you should try to do the following:

1. Download LLaMA 3 8B weights and load them in Colab. Visualize the weights (I like TreeScope for this) and see if you can identify each tensor. Count the # of parameters. Does it match what you expect? How many are in the MLP vs. attention?
2. Implement the full forward pass of the model. You don't need to worry about prefill/decode or anything fancy. Just get to the point where you can feed in a sequence and get plausible next-token probabilities out. You should be able to feed a prompt in, get probabilities out, pick the highest probability one, and repeat. This is slow but functional sampling. Try to only use the LLaMA-3 paper as a reference, but the `jax-ml/jax-llm-examples/llama3` repo can be a good reference for details (be careful of positional embeddings and causal masking). Your goal is to get coherent tokens out of the model. *You can run this all on a single TPU if you can get more than 16GiB of HBM.*
3. Now it's time to make this faster. Implement KV caching, where you can save the key/value activations from a forward pass and attend to them in a subsequent one. This will make your sampling loop much faster.
4. Implement separate prefill and decode servers. You can do these on different sets of chips. Prefill handles just a single prompt at once, then sends it to a batched decode server that handles batches of tokens.
5. Implement Flash Attention in Pallas. This will make attention much more efficient. *I think it's good to try and do this without a reference.*

That's all for Part 8! For Part 9, with a deep dive into XLA and TPU profiling, click here.

Chapter 9

How to Profile TPU Code

A Thousand-Foot View of the TPU Software Stack

Google exposes a bunch of APIs for programming TPUs, from high-level JAX code to low-level Pallas or HLO. Most programmers write JAX code exclusively, which lets you write abstract NumPy-style linear algebra programs that are compiled automatically to run efficiently on TPUs.

Here's a simple example, a JAX program that multiplies two matrices together:

```
import jax
import jax.numpy as jnp

def multiply(x, y):
  return jnp.einsum('bf,fd->db', x, y)

y = jax.jit(multiply)(jnp.ones((128, 256)), jnp.ones((256, 16), dtype=jnp.bfloat16))
```

By calling `jax.jit`, we tell JAX to trace this function and emit a lower-level IR called StableHLO, a platform-agnostic IR for ML computation, which is in turn lowered to HLO by the XLA compiler. The compiler runs many passes to determine fusions, layouts, and other factors that result in the HLO that is observable in a JAX profile. This HLO represents all the core linear algebra operations in the JAX code (matmuls, pointwise ops, convolutions, etc.) in an LLVM-style graph view. For instance, here is an abridged version of the above program as HLO[51]:

```
ENTRY %main.5 (Arg_0.1: f32[128,256], Arg_1.2: bf16[256,16]) -> f32[16,128] {
  %Arg_1.2 = bf16[256,16]{1,0} parameter(1), metadata={op_name="y"}
  %convert.3 = f32[256,16]{1,0} convert(bf16[256,16]{1,0} %Arg_1.2),
  %Arg_0.1 = f32[128,256]{1,0} parameter(0), metadata={op_name="x"}
  ROOT %dot.4 = f32[16,128]{1,0} dot(f32[256,16]{1,0} %convert.3, f32[128,256]{1,0} %Arg_0.1), lhs_contra
}
```

We'll explain the syntax of HLO in just a second, but for now just note that it actually matches the JAX code above fairly well. For instance,

```
ROOT %dot.4 = f32[16,128]{1,0} dot(f32[256,16]{1,0} %convert.3, f32[128,256]{1,0} %Arg_0.1), lhs_contracti
```

is the actual matmul above that multiplies two f32 matrices along the 0 and 1 dimensions, respectively.

To transform this HLO to code that can be executed on the TPU, the XLA compiler first lowers it to LLO (low-level optimizer) IR. LLO programs the TPU directly, scheduling copies between memories, pushing arrays onto the systolic array, etc. LLO code contains primitives that push buffers into the systolic array, pull results off, and schedule DMAs that communicate between different pieces of TPU memory. Once this has been lowered to LLO, it is then compiled to machine code that is loaded into the TPU IMEM and executed.

When a program is running slower than we'd like, we primarily work with the JAX level to improve performance. Doing so, however, often requires us to understand some of the semantics of HLO and how the code is actually

[51] To get this HLO, you can run `jax.jit(f).lower(*args, **kwargs).compile().as_text()`.

running on the TPU. When something goes wrong at a lower level, we pull yet another escape hatch and write custom kernels in Pallas. To view the HLO of a program and its runtime statistics, we use the JAX profiler.

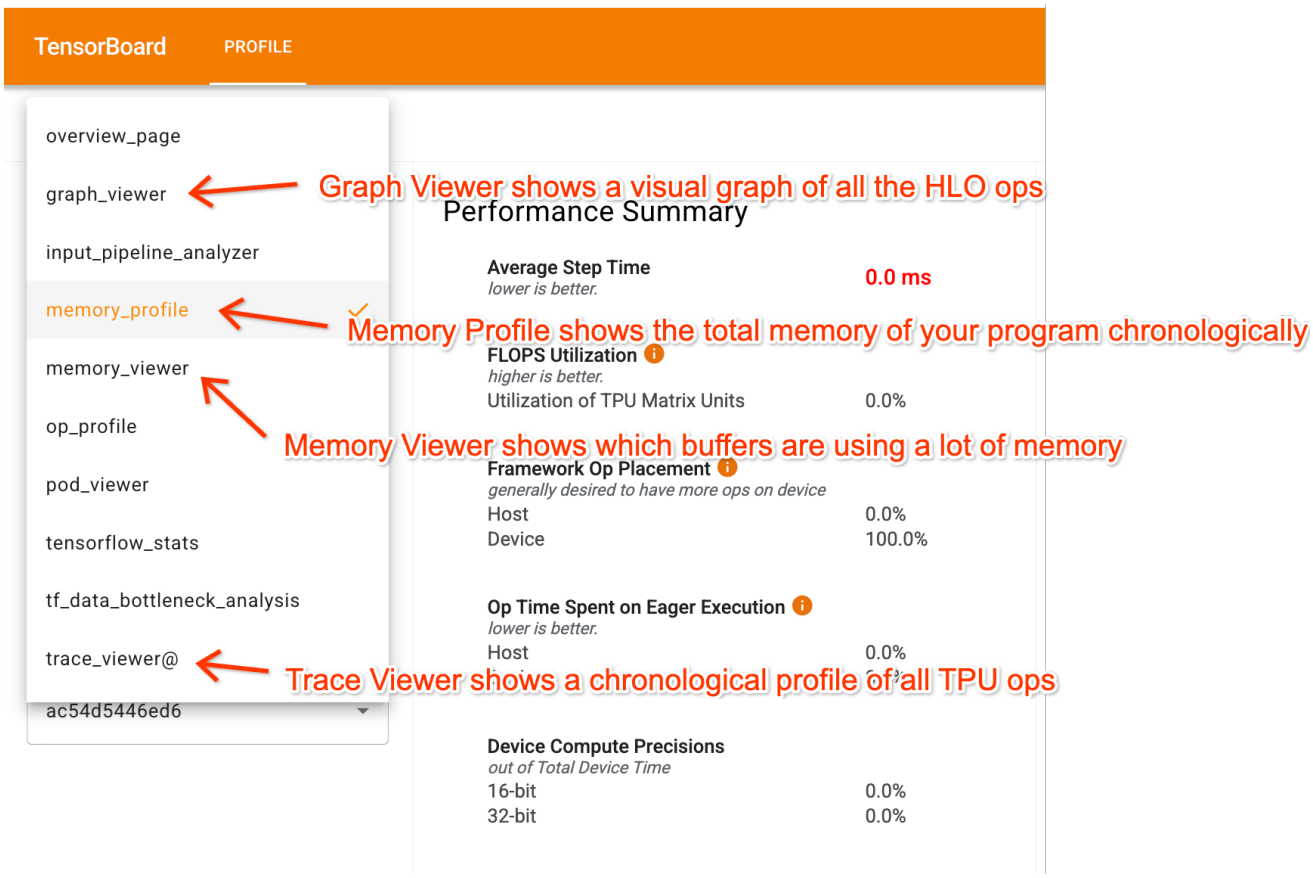
The JAX Profiler: A Multi-Purpose TPU Profiler

JAX provides a multi-purpose TPU profiler with a bunch of useful tools for understanding what's happening on the TPU when a program is run. You can use the `jax.profiler` module to trace a program as it's running and record everything from the duration of each subcomponent, the HLO of each program, memory usage, and more. For example, this code will dump a trace to a file in `/tmp/tensorboard` that can be viewed in TensorBoard (here is a step-by-step guide).

```
import jax
with jax.profiler.trace("/tmp/tensorboard"):
  key = jax.random.key(0)
  x = jax.random.normal(key, (1024, 1024))
  y = x @ x
  y.block_until_ready()

# Now you can load TensorBoard in a Google Colab with
#
# !pip install -U xprof
# !pip install -U protobuf
# %load_ext tensorboard
# %tensorboard --logdir=/tmp/tensorboard
#
# or externally with
#
# > tensorboard --logdir=/tmp/tensorboard
#
```

Here's an overview of what you can do in the profiler:



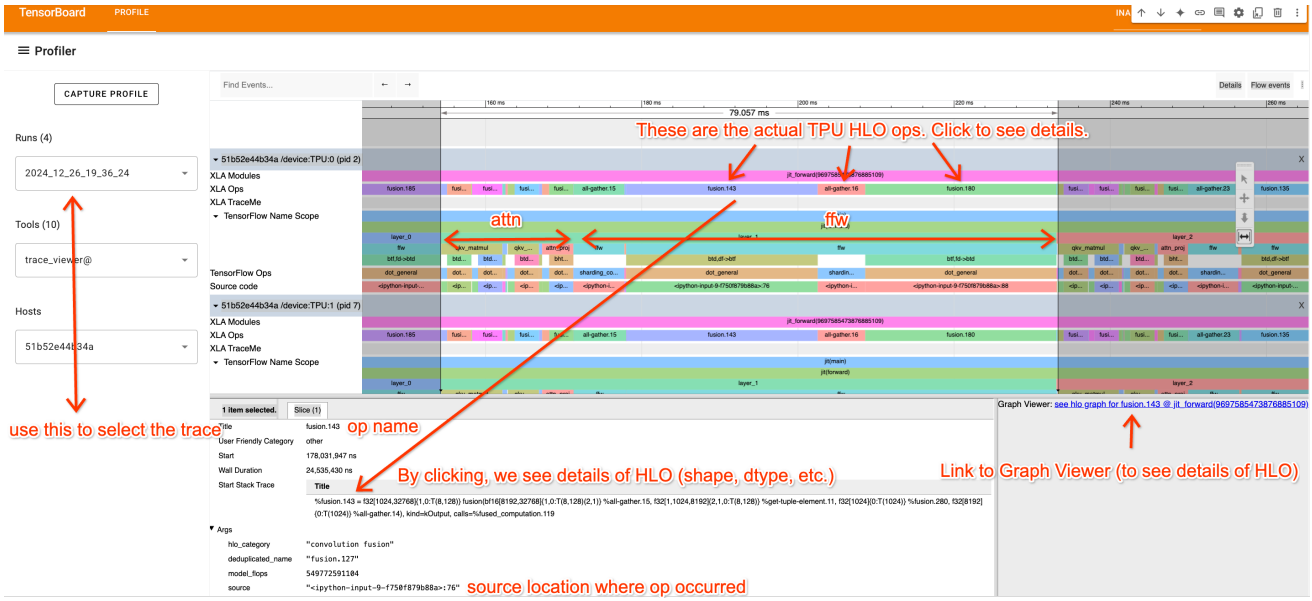
Once in TensorBoard, the profiler has a few key tabs that help you understand your program:

1. **Trace Viewer** shows a detailed timeline of what's actually happening on the TPU.
2. **Graph Viewer** shows the HLO graph, letting you see what parts of the program feed into each other and how things are sharded.
3. **Memory Profile and Memory Viewer:** these show how much memory your program is using.

While it's slightly difficult to share profiles, here is a Perfetto link that contains at least the Trace Viewer component for a simple Transformer. This Colab lets you generate the full JAX/TensorBoard trace and play around with it.

Trace Viewer

The Trace Viewer is probably the most useful part of the profiler. The example below shows a simple Transformer with pieces annotated. Names come from labels provided in the code.



The Trace Viewer shows a chronological timeline of all the actions on each TPU core. We're only looking at TPU:0 here, since typically all TPUs execute the same instructions. A few key notes:

1. The top row (XLA Ops) shows the actual TPU operations (the names are HLO names). Everything else is an approximate trace based on `jax.named_scope`, `jax.named_call`, and the Python stack trace.
2. Noting the repeated blocks, we can isolate a single layer here. We can also see (from looking at the code/understanding how a Transformer works) what parts are attention and what parts are MLPs.
3. By clicking on an XLA op, we can view where in the code it comes from (useful for understanding the trace) and see links to the Graph viewer.

Tip: you can navigate the Trace Viewer using "video game" style controls, with A/D panning left and right, and W/S zooming in and out. These controls make navigating a lot easier.

How to read an XLA op

HLO isn't actually very hard to read, and it's very helpful for understanding what a given part of the trace above corresponds to. Here's an example op called fusion.3.

```
%fusion.3 = bf16[32,32,4096]{2,1,0:T(8,128)(2,1)S(1)} fusion(bf16[32,32,8192]{2,1,0:T(8,128)(2,1)S(1)} %fu
```

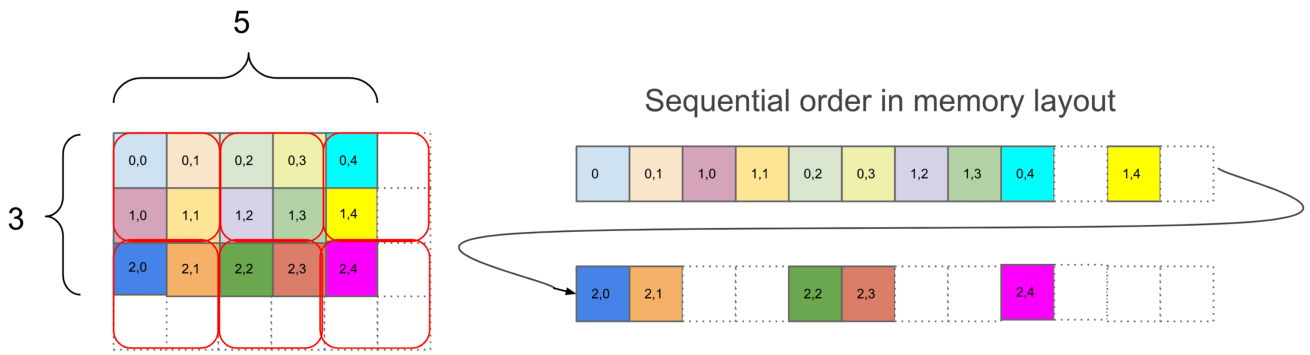
Let's break this down into its pieces.

- **Op Name**: fusion.3
  - A dot or fusion op is a set of operations containing at most 1 matrix multiplication and possibly a bunch of related pointwise VPU-ops.
- **Shape**: `bf16[32,32,4096]`
  - This is the output shape of the op. We can see the dtype is bf16 (2 bytes per element) and `[32,32,4096]` is the shape.
- **Layout:** `{2,1,0:T(8,128)(2,1)}`
  - `{2,1,0:T(8,128)(2,1)}` tells us the order of the axes in memory (column major, row major, etc.) and the array padding. More below.
- **Memory location:** S(1)
  - S(1) tells us this array lives in VMEM. S(0) (sometimes omitted) is HBM. S(2) and S(3) are other memory spaces.
- **Arguments**: `bf16[32,32,8192]{2,1,0:T(8,128)(2,1)S(1)} %fusion.32`
  - This op has one input, a bf16 array called fusion.32 with a particular shape. This tells us what function feeds into this one.

Let's try to understand this notation a little more. Let's take this as a simple example:

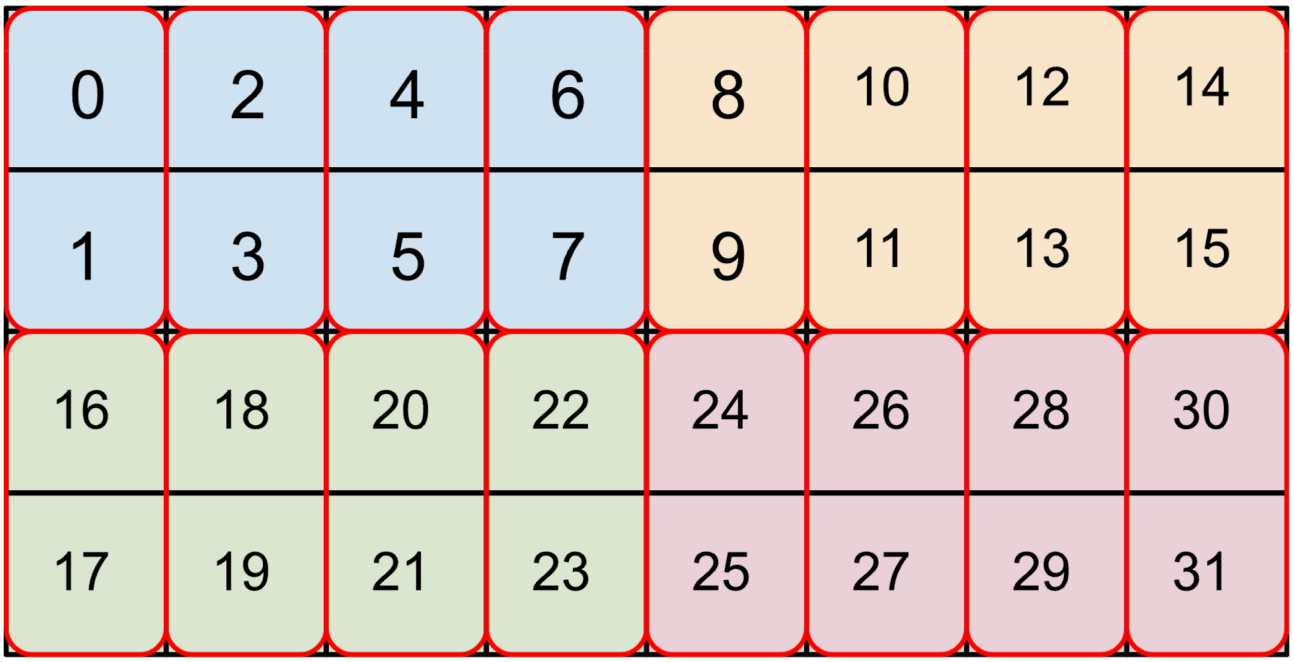
`f32[3,5]{1,0:T(2,2)}`

which again tells us that this Op returns a float32 array of shape `[3, 5]` with a particular tiling `{1,0:T(2,2)}`. While tilings don't matter *too* much, briefly, tilings tell us how an N-dimensional array is laid out sequentially in memory. Here's a diagram showing how this array is laid out:



F32[3,5] with tile size of 2x2

Within `{1,0:T(2,2)}`, the `1,0` part tells us the ordering of array dimensions in physical memory, from most minor to most major. You can read this part from right to left and pick out the corresponding dimensions in `f32[3,5]` to figure out the physical layout of the array. In this example, the physical layout is `[3,5]`, identical to the logical shape. After that, `T(2,2)` tells us that the array is tiled in chunks of `(2, 2)` where within each chunk, the array has rows first (**row-major**), then columns, i.e. `(0, 0)` is followed by `(0, 1)`, then `(1, 0)` and `(1, 1)`. Because of the `T(2, 2)` tiling, the array is padded to `[4, 6]`, expanding its memory usage by about 1.6x. For the big bf16 array given above, `bf16[32,32,8192]{2,1,0:T(8,128)(2,1)S(1)}`, we do `T(8,128)(2,1)` which tells us the array has two levels of tiling, an outer `(8, 128)` tiling and an inner `(2, 1)` tiling within that unit (used for bf16 so our loads are always multiples of 4 bytes). For example, here's `bf16[4,8]{1,0:T(2,4)(2,1)}` (colors are (2,4) tiles, red boxes are (2,1) tiles):

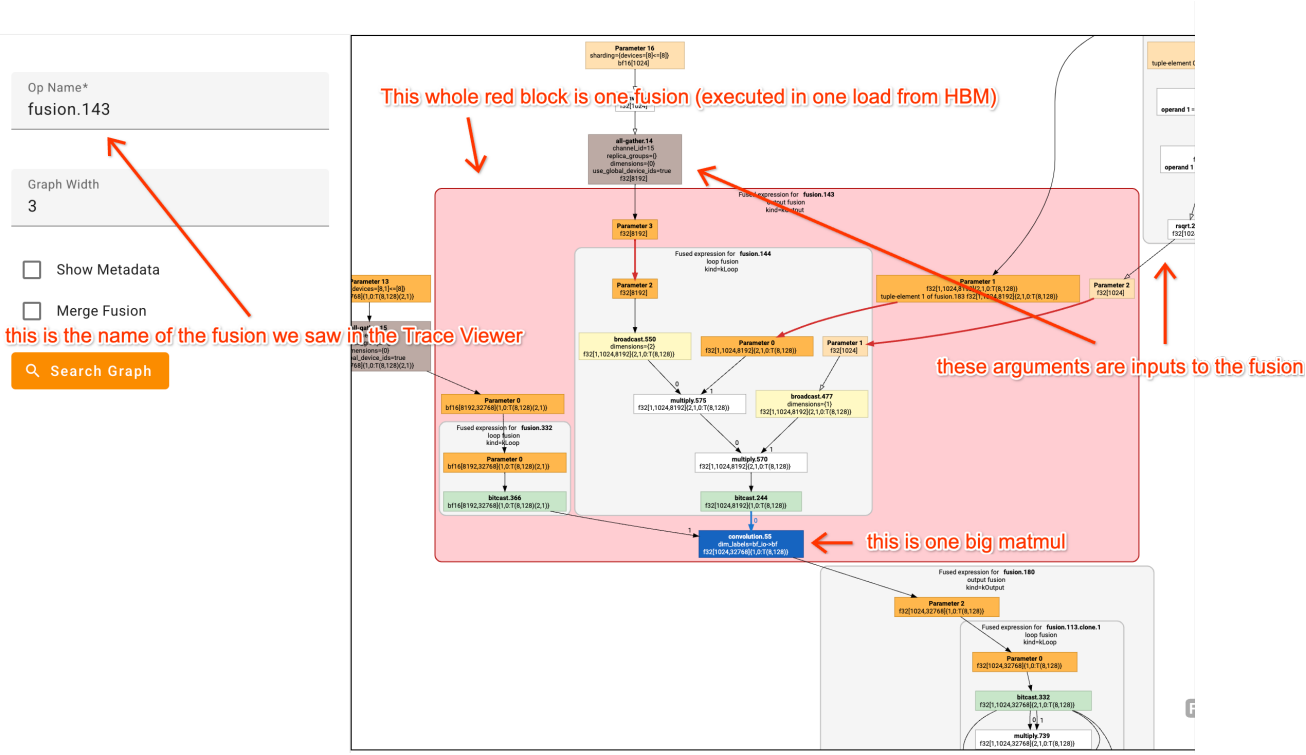


Tiling can affect how efficiently chunks of tensors can be loaded into VMEM and XLA will sometimes introduce copies that "retile" or "re-layout" a tensor inside a program, sometimes at non-trivial overhead.[52]

[52]JAX provides an experimental feature to work around this issue, by allowing XLA to compute its "preferred" layout for inputs

Graph Viewer

While some of the fusions above can seem complicated, the XLA Graph Viewer makes them easier to parse. For example here's the view of a fairly complicated fusion:



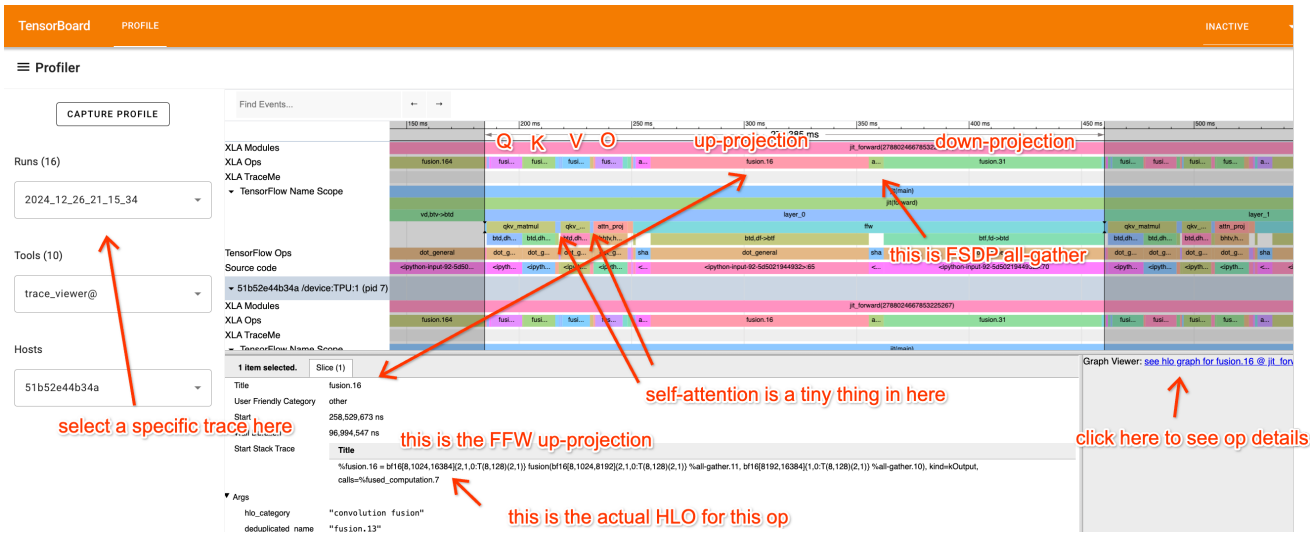
It's really helpful to stare at a bunch of HLO graphs and try to map HLO ops onto the code you're profiling. By hovering over a box you'll often see the line of code where the function was defined.

Looking at a real(ish) example profile

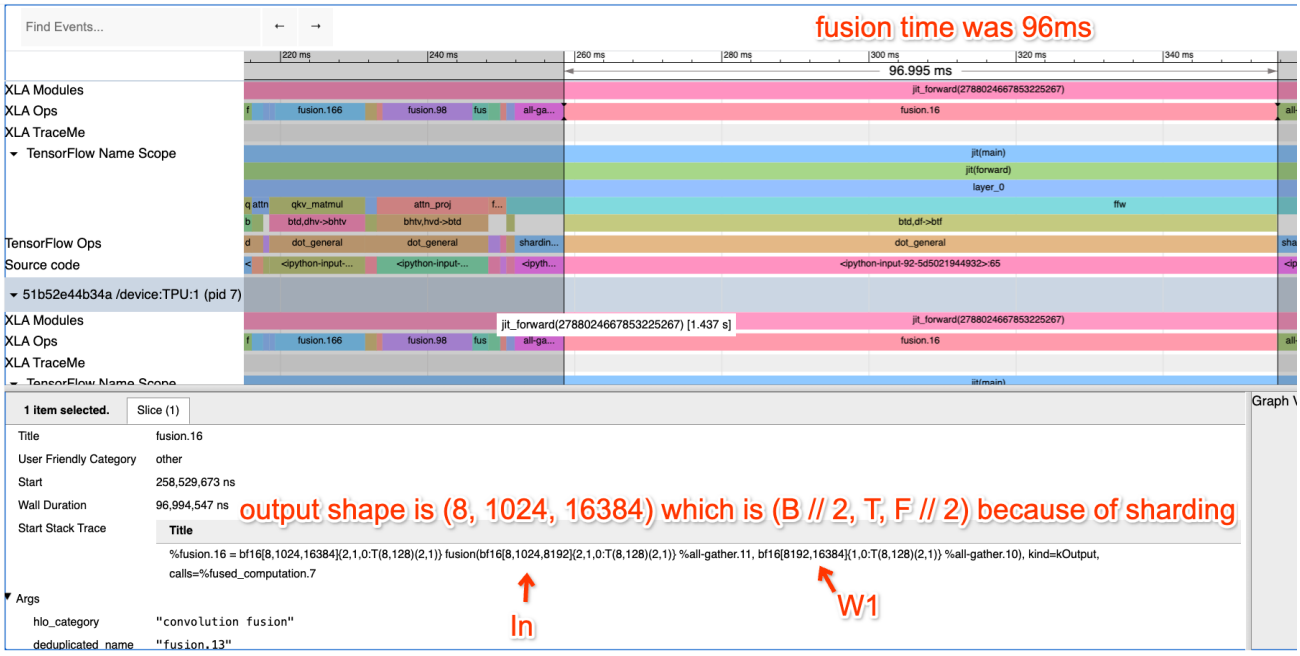
This Colab has an example profile for a fake Transformer. Here's a Perfetto link to at least see the Trace Viewer if you're in a hurry. I've gone to more effort than usual to annotate the trace with `jax.named_scope` calls so you can identify what's going on.

---

to a program. When you "just-in-time" compile a program with `jax.jit`, you typically pass in "mock" inputs that tell JAX what shape and dtype to expect. These typically also carry tiling information that may not be optimal. Instead, you can specify the input layouts as AUTO, and `jax.jit` will return a layout that the jitted program prefers. You can then explicitly load the tensor in that layout to avoid inducing copies within the program.



Take a look at the profile and try to really understand what each part is doing. Let's break it down a bit, starting with the FFW block:



Here we've zoomed into the FFW block. You'll see the up-projection Op is a fusion (matmul) with inputs `bf16[8, 1024, 8192]` and `bf16[8192, 16384]` and output `bf16[8, 1024, 16384]`. I know (because I wrote this code) that this is a local view of a 4-way DP, 2-way MP sharded matmul, so we're actually doing

X: `bf16[32, 1024, 8192]` * **Win**: `bf16[8192, 32768]` -> **Tmp**: `bf16[32, 1024, 32768]`

How long do we expect this to take? First of all, our batch size per data parallel shard is `8 * 1024 = 8192`, so we should be solidly compute-bound. This is on 8 TPU v2 cores, so we expect it to take about `2 * 32 * 1024 * 8192 * 32768 / (23e12 * 8) = 95.6ms` which is pretty much exactly how long it takes (96ms). That's great! That means we're getting fantastic FLOPs utilization!

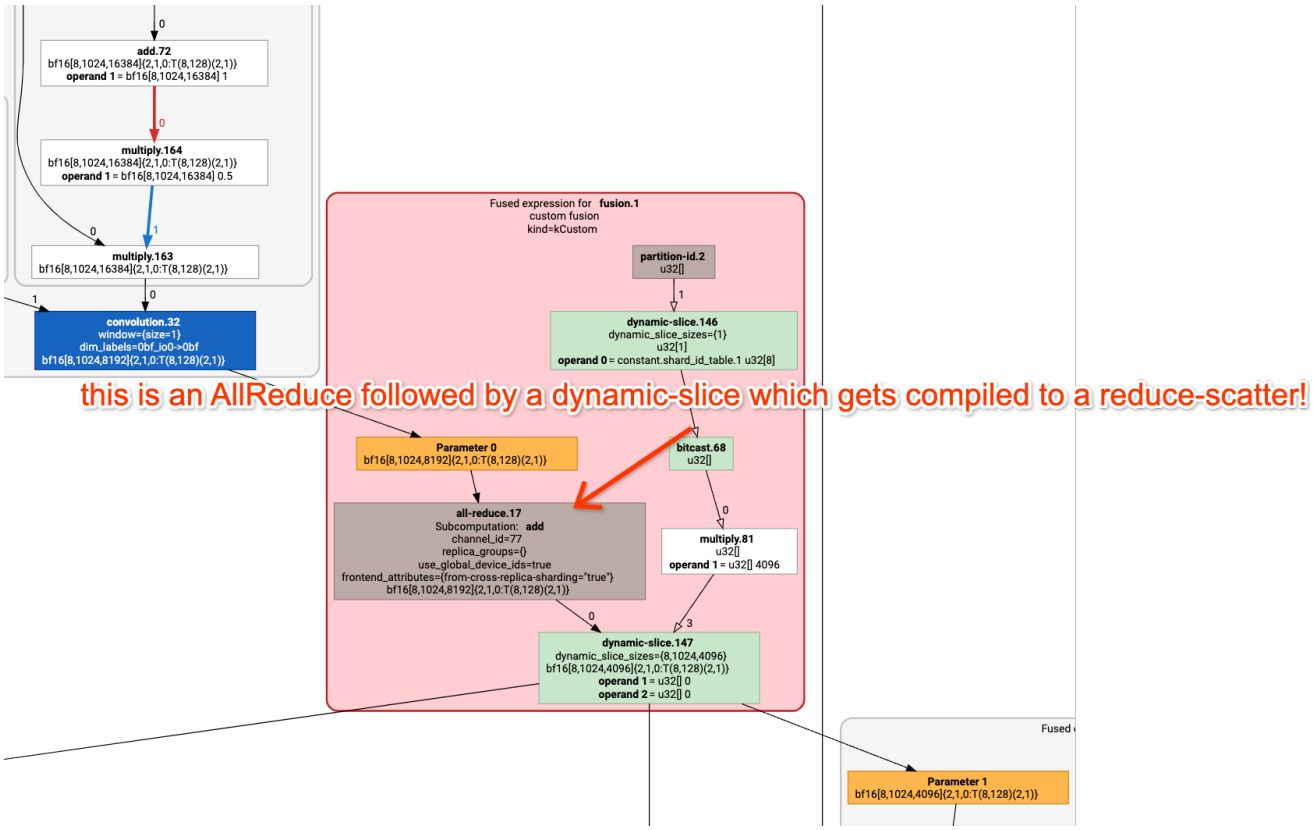
Note that Google Colab no longer hands out TPU v2-8 slices. To get a real 8-core slice to follow along, you can use Kaggle, which still provides them for free, or provision an 8-core slice on GCP.[53]

[53] If you only want to play around with sharding on fake problems, you can also fake 8 devices on CPU with `import jax; jax.config.update("jax_num_cpu_devices", 8)` (requires jax >= 0.4.27ish) and then `print(jax.devices())`. This only works for toy problems and won't reflect real performance.

What about communication? You'll notice the little fusion hidden at the end of the second matmul. If we click on it, you'll see

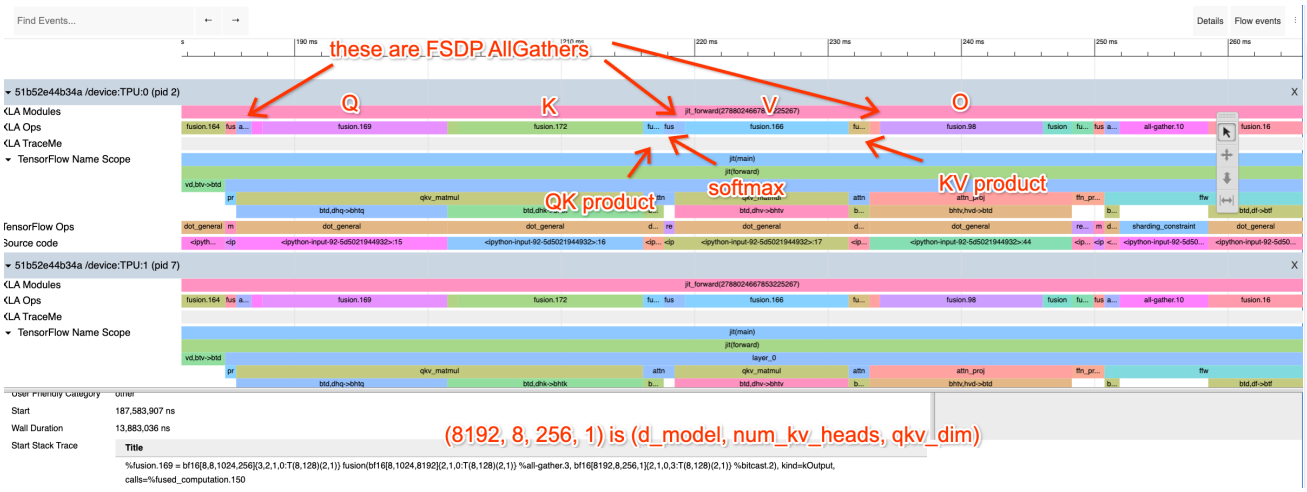
```
%fusion.1 = bf16[8,1024,4096]{2,1,0:T(8,128)(2,1)} fusion(bf16[8,1024,8192]{2,1,0:T(8,128)(2,1)} %fusion.3
```

which is basically a little ReduceScatter (here's the Graph Viewer);



How long do we expect this to take? Well, we're doing a ReduceScatter on a TPU v2 4x2, which should require only one hop on 1.2e11 bidirectional bandwidth. The array has size `2*32*1024*8192` with the batch axis sharded 4 ways, so each shard is `2*8*1024*8192=128MB`. So this should take roughly 1.1ms. **How long does it actually take?** 1.13ms reported in the profile. So we're really close to the roofline!

Let's look at attention too! Here's a profile of the attention component:

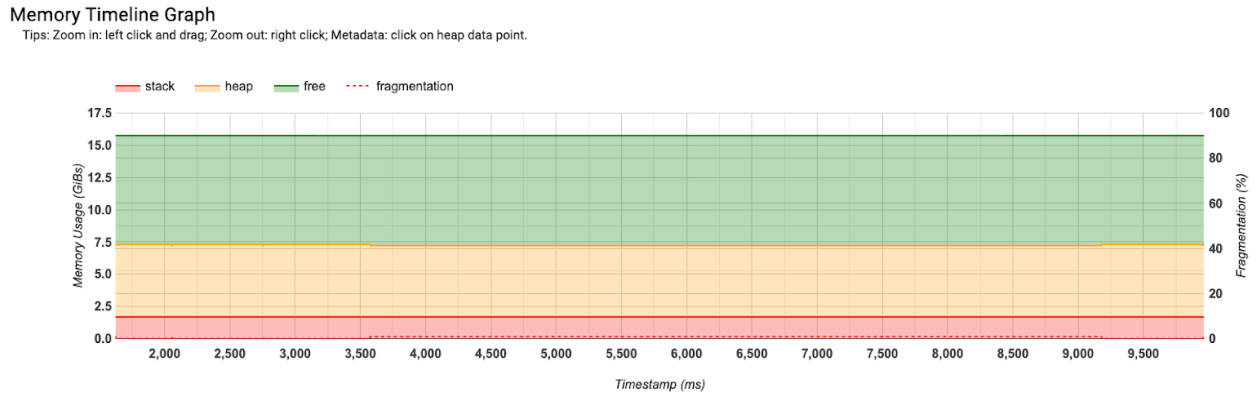


I've clicked on the Q projection op, which uses a matrix W_Q of shape [dmodel = 8192, nheads = 32, dqkv = 256]. We're Megatron sharding along the head dimension. Try to do the same exercise of calculating how long these

should take.

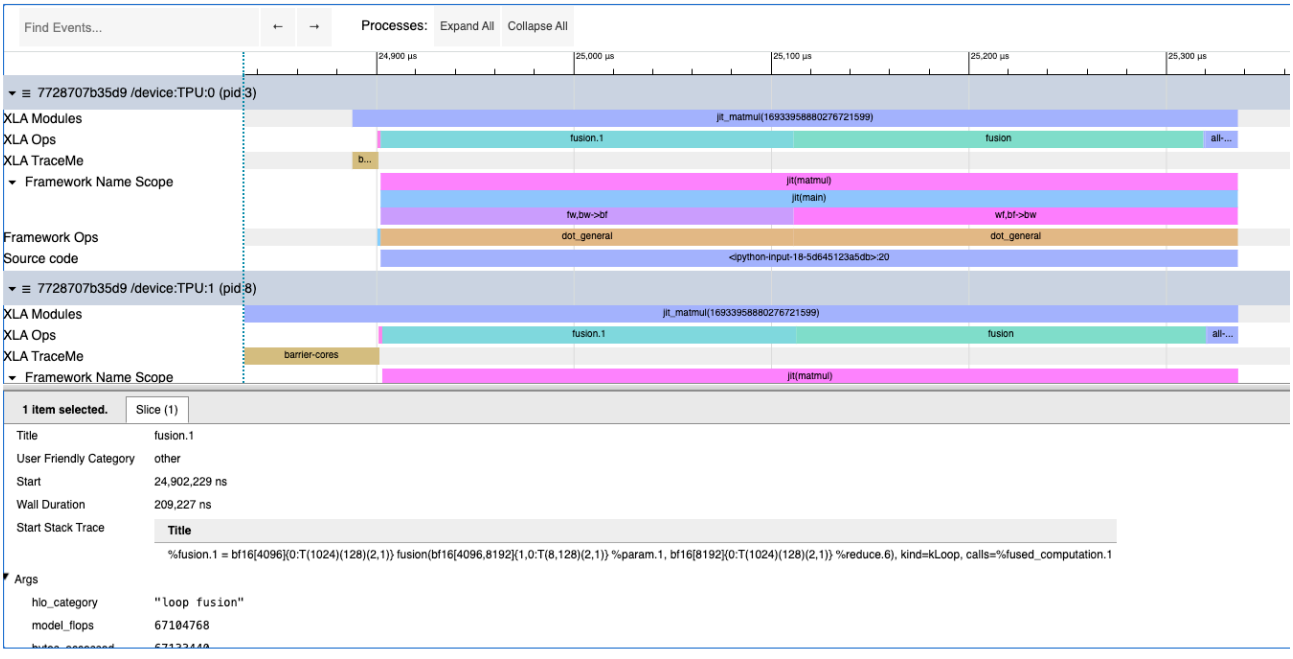
Memory Profile

The Memory Profile makes it easy to see the program memory as a function of time. This is helpful for debugging OOMs. You can see here about 7.5GB allocated to model parameters and about 8.5GB free. So we can fit a lot more into memory.



Worked Problems

Question 1: take a look at this Colab/profile and figure out what looks suspicious and what's going on here. Can you tell me exactly what computations are happening and what each operation is doing? What are the true shapes of each matrix involved and how are they sharded? *Try looking at the profile first without reading the code.*



Click here for the answer.

This is two matrix multiplications, i.e. specifically this:

```
def matmul(w1, w2, x):
  return jnp.einsum('wf,bf->bw', w2, jnp.einsum('fw,bw->bf', w1, x))
```

You can see a reduce, two big fusions, and an all-reduce. The first big fusion is:

```
%fusion.1 = bf16[4096]{0:T(1024)(128)(2,1)} fusion(bf16[4096,8192]{1,0:T(8,128)(2,1)} %param.1,
bf16[8192]{0:T(1024)(128)(2,1)} %reduce.6), kind=kLoop, calls=%fused_computation.1
```

which tells us the per-shard shape is `bf16[8192] * bf16[4096, 8192] -> bf16[4096]` (over the 8192 dimension). By observing the final AllReduce with {% raw %}`replica_groups={{0,16,32,48,64,80,96,112}, ...}`{% endraw %}, we can tell we're doing 8-way model parallelism, so the true shapes are `bf16[8, 8192] * bf16[32768, 8192] -> bf16[8, 32768]`.

Question 2: The Transformer Colab from earlier implements a simple mock Transformer. Since Colab no longer provides TPU v2-8 slices, you'll want to run this on Kaggle or an 8-core GCP slice to follow along. Follow the instructions in the Colab and get a benchmark of the naive Transformer with GSPMD partitioning. How long does each part take? How long should it take? What sharding is being used? Try fixing the sharding! *Hint: use `jax.lax.with_sharding_constraint` to constrain the behavior. With this fix, what's the best MFU you can get?*

For reference, the initial version gets roughly 184ms / layer and the optimized profile gets 67ms / layer. Once you've done this, try staring at the profile and see if you can answer these questions purely from the profile:

- What sharding strategy is this?
- What is the batch size, d_{model}, d_{ff}?
- What fraction of time is spent on attention vs. the MLP block?
- What fraction of time should be spent on each op at the roofline?

Note: since this problem was written, the XLA compiler has gotten better. The initial version is now at roughly 90ms / layer and the optimized profile is only about 10ms / layer better (80ms / layer). Still, it's worth playing with and seeing if you can do better.

That's all for Part 9. For Part 10, with a deep dive into JAX parallelism, click here.

Chapter 10

Programming TPUs in JAX

How Does Parallelism Work in JAX?

JAX supports three schools of thought for multi-device programming:

1. **Compiler, take the wheel!** Let the XLA compiler automatically partition arrays and decide what communication to add to facilitate a given program. This lets you take a program that runs on a single device and automatically run it on thousands without changing anything.
2. **JAX, take the wheel!** Automatic parallelism is great, but sometimes the compiler does something crazy. Explicit sharding lets you write single-device code like usual, but have JAX handle sharding propagation (not the compiler). This means JAX can ask you for clarification when it's unclear what you want.
3. **Just let me write what I mean, dammit!** While compilers are nice, they sometimes do the wrong thing and add communication you don't intend. Sometimes we want to be explicit about exactly what communication we intend to run.

Mode	View?	Explicit sharding?	Explicit Collectives?
Auto	Global	×	×
Explicit	Global	✓	×
Manual	Per-device	✓	✓

Correspondingly, JAX provides APIs for each of these modes:

1. `jax.jit` (with `Auto` mesh axes) lets you take any existing JAX function and call it with sharded inputs. JAX then uses XLA's Shardy compiler which automatically parallelizes the program. XLA will add communication for you (AllGathers, ReduceScatters, AllReduces, etc.) when needed to facilitate existing operations. While it isn't perfect, it usually does a decent job at automatically scaling your program to any number of chips without code changes.
2. `jax.jit` with `Explicit` mesh axes looks similar to (1), but lets JAX handle the sharding propagation instead of XLA. That means the sharding of an array is actually part of the JAX type system, and JAX can error out when it detects ambiguous communication and lets the user resolve it.
3. `jax.shard_map` is the more manual counterpart. You get a device-local view of the program and have to write any communication you want explicitly. Have a sharded array and want the whole thing on each device? Add a `jax.lax.all_gather`. Want to sum an array across your devices? Add a `jax.lax.psum` (an AllReduce). Programming is harder but far less likely to do something you don't want.

Auto sharding mode

`jax.jit` plays two roles inside JAX. As the name suggests, it "just-in-time" compiles a function from Python into bytecode (via XLA/HLO/LLO) so it runs faster. But if the input is sharded or the user specifies an `in_sharding` or `out_sharding`, it also lets XLA distribute the computation across multiple devices and add communication as needed. For example, here's how you could write a sharded matmul using `jax.jit`:

```
import jax
import jax.numpy as jnp

Auto = jax.sharding.AxisType.Auto
```

```
# This creates a fake set of 8 CPU devices so you can run this on a CPU without TPUs.
jax.config.update("jax_num_cpu_devices", 8)

# This creates a 2D 4x2 mesh with axis names X and Y that JAX uses by default.
# We explicitly tell JAX to let the XLA compiler infer sharding along these axes.
mesh = jax.make_mesh(axis_sizes=(4, 2), axis_names=('X', 'Y'), axis_types=(Auto, Auto))
jax.set_mesh(mesh)

# We create a matrix W and input activations In sharded across our devices.
In = jnp.zeros((8, 2048), dtype=jnp.bfloat16, device=jax.NamedSharding(mesh, jax.P('X', 'Y')))
W = jnp.zeros((2048, 8192), dtype=jnp.bfloat16, device=jax.NamedSharding(mesh, jax.P('Y', None)))

def matmul_square(In, W):
  return jnp.einsum('bd,df->bf', jnp.square(In), W)

# We can explicitly compile the sharded matmul function here. This adds all the
# necessary comms (e.g. an AllReduce after the matmul).
jit_matmul = jax.jit(matmul_square, out_shardings=jax.P('X', None)).lower(In, W).compile()

out = jit_matmul(In, W)
```

This will run automatically with any sharding and partition the computation across our devices. **But what's actually happening at the hardware level?**

1. First we create In and W sharded across our devices[54]. W is sharded 2-way along the contracting dimension, while In is sharded 8-way: 4-way along the input dimension and 2-way along the contracting dimension. This corresponds to a sharding W[DY, F] and In[BX, DY], aka a kind of model and data parallelism.
2. If we were running this locally (i.e. on one device), `matmul_square` would simply square the input and perform a simple matmul. But because we specify the `out_shardings` as `P('X', None)`, the output will be sharded along the batch but replicated across the model dimension and will require an AllReduce to compute.

Using our notation from previous sections, this will likely do something like

1. Out[BX, F] { UY } = In[BX, DY] *D W[DY, F]
2. Out[BX, F] = **AllReduce**(Out[BX, F] { UY })

`jax.jit` will add this for us automatically! We can actually print the HLO with `jit_matmul.as_text()` and see the following HLO (abbreviated dramatically):

```
# This fusion is the actual matmul of the sharded inputs and matrix
%fusion = bf16[2,8192]{1,0:T(4,128)(2,1)S(1)} fusion(bf16[2,1024]{1,0:T(4,128)(2,1)} %param, bf16[8192,10

# We reduce the partially summed results across devices
ROOT %AllReduce = bf16[2,8192]{1,0:T(4,128)(2,1)} AllReduce(bf16[2,8192]{1,0:T(4,128)(2,1)S(1)} %fusion)
```

We can see the matmul (the fusion) and the AllReduce above. Pay particular attention to the shapes. `bf16[2, 1024]` is a local view of the activations, since our `batch_size=8` is split across 4 devices and our `d_model=2048` is likewise split 2 ways.

This is pretty magical! No matter how complicated our program is, Shardy and jit will attempt to find shardings for all the intermediate activations and add communication as needed. With that said, Shardy has its flaws. It can make mistakes. Sometimes you'll look at a profile and notice something has gone wrong. A giant AllGather takes up 80% of the profile, where it doesn't need to. When this happens, we can try to correct the compiler by explicitly annotating intermediate tensors with `jax.lax.with_sharding_constraint`. For instance,

[54] Notice how we did this. This is one way to create an array with a particular sharding (i.e. by adding the device argument to the creation function). Another one is to create an array normally with `jnp.array(....)` and then do e.g. `jax.device_put(..., jax.P('X', 'Y'))`. Yet another is to write a function which creates the array you want, and jit-compile it with `out_shardings` being what you want.

with two matmuls I can force the intermediate activations to be sharded along the y dimension (not that this is a good idea) with the following:

```
import jax
import jax.numpy as jnp

Auto = jax.sharding.AxisType.Auto

mesh = jax.make_mesh((4, 2), ('X', 'Y'), (Auto, Auto))
jax.set_mesh(mesh)

def matmul(x, W_in, W_out):
  hidden = jnp.einsum('bd,df->bf', x, W_in)
  hidden = jax.lax.with_sharding_constraint(hidden, jax.P('X', 'Y'))
  return jnp.einsum('bf,df->bd', hidden, W_out)
```

This makes up about 60% of JAX parallel programming in the automatic partitioning world where you control the intermediate shardings via `jax.lax.with_sharding_constraint`. But "compiler tickling" is famously not a fun programming model. You could annotate every intermediate variable and still not know if you'll get the right outcome. Instead, what if JAX itself could handle and control sharding propagation?

Explicit sharding mode

Explicit sharding (or "sharding in types") looks a lot like automatic sharding, but sharding propagation happens at the JAX level! Each JAX operation has a sharding rule that takes the shardings of the op's arguments and produces a sharding for the op's result. You can see the resulting sharding using `jax.typeof`:

```
import jax
import jax.numpy as jnp
import numpy as np

Explicit = jax.sharding.AxisType.Explicit

# Running on a TPU v5e 2x2. This assigns names to the two physical axes of the hardware.
mesh = jax.make_mesh(axis_sizes=(2, 2), axis_names=('X', 'Y'), axis_types=(Explicit, Explicit))

# This tells JAX to use this mesh for all operations, so you can just specify the PartitionSpec P.
jax.set_mesh(mesh)

x = jax.device_put(np.arange(16, dtype=np.float32).reshape(8, 2), jax.P('X', 'Y'))

@jax.jit
def f(x):
  print(jax.typeof(x))  # float32[8@X,2@Y]
  out = x * 2
  print(jax.typeof(out))  # float32[8@X,2@Y]
  return out

f(x)
```

As you can see, JAX propagated the sharding from input (`x`) to output (`out`) which are inspectable at trace-time via `jax.typeof`. For most operations these rules are simple and obvious because there's only one reasonable choice (e.g. elementwise ops retain the same sharding). But for some operations it's ambiguous how to shard the result, in which case JAX throws a trace-time error and asks the programmer to provide an `out_sharding` argument explicitly (e.g. jnp.einsum, jnp.reshape, etc). Let's see another example where you have conflicts:

```
# We create a matrix W and input activations In sharded across our devices.
In = jnp.zeros((8, 2048), dtype=jnp.bfloat16, out_sharding=jax.P('X', 'Y'))
W = jnp.zeros((2048, 8192), dtype=jnp.bfloat16, out_sharding=jax.P('Y', None))
```

```
@jax.jit
def matmul_square(In, W):
  print(jax.typeof(In))  # bfloat16[8@X, 2048@Y]
  print(jax.typeof(W))  # bfloat16[2048@Y, 8192]
  return jnp.einsum('bd,df->bf', jnp.square(In), W)

matmul_square(In, W)  # This will error
```

This code errors with:

```
Contracting dimensions are sharded and it is ambiguous how the output should be sharded.
Please specify the output sharding via the `out_sharding` parameter.
Got lhs_contracting_spec=('Y',) and rhs_contracting_spec=('Y',)
```

This is awesome because how the output of einsum should be sharded is ambiguous. The output sharding can be: * P('X', 'Y') which will induce a ReduceScatter or * P('X', None) which will induce an AllReduce

Unlike Auto mode, explicit mode errors out when it detects ambiguous communication and requires the user to resolve it. So here you can do:

```
@jax.jit
def matmul_square(In, W):
  return jnp.einsum('bd,df->bf', jnp.square(In), W, out_sharding=jax.P('X', 'Y'))

out = matmul_square(In, W)
print(jax.typeof(out))  # bfloat16[8@X,8192@Y]
```

Auto mode and Explicit mode can be composed via `jax.sharding.auto_axes` and `jax.sharding.explicit_axes` APIs. This is a great doc to read for more information.

Manual sharding mode via shard_map

While Shardy is the "compiler take the wheel" mode, jax shard_map puts everything in your hands. You specify the sharding of the inputs, like in jax.jit, but then you write all communication explicitly. Whereas `jax.jit` leaves you with a global cross-device view of the program, `shard_map` gives you a local per-device view.

Here's an example. Try to reason about what this function does:[55]

```
import jax
import jax.numpy as jnp

Explicit = jax.sharding.AxisType.Explicit

mesh = jax.make_mesh((2, 4), ('x', 'y'), (Explicit, Explicit))
jax.set_mesh(mesh)

x = jnp.arange(0, 512, dtype=jnp.int32, out_sharding=jax.P(('x', 'y')))

# This function will operate on 1/8th of the array.
@jax.shard_map(in_specs=jax.P(('x', 'y')), out_specs=jax.P())
def slice_and_average(x):
  assert x.shape == (512 // 8,)
  return jax.lax.pmean(x[:4], axis_name=('x', 'y'))

out = slice_and_average(x)
assert out.shape == (4,)
```

[55] If you want to play with this yourself in a colab by emulating a mesh, you can do so using the following cell `import jax; jax.config.update('jax_num_cpu_devices', 8)`

What does this do? `slice_and_average` is run on each TPU with 1/8th of the array, from which we slice the first 4 elements and average them across the full mesh. This means we're effectively doing `mean(x[:4], x[64:68], x[128:132], …)`. This is pretty cool, because that's not an easy operation to express in JAX otherwise.

Why do this instead of jax.jit? If we'd used `jax.jit`, `slice_and_average` would have seen a global view of the array (the full `[512,]` array). We'd have had to slice out this non-uniform slice and then perform an average which XLA would have had to interpret correctly. XLA might have added the wrong communication or gotten confused. Here we see the local view and write only the communication we need.

Example [Collective Matmul]: To take a more realistic example, say we want to implement model parallelism where the activations are initially model sharded, i.e. A[BX, DY] *D W[D, FY] -> Out[BX, FY]. Naively, we would do this by AllGathering A first followed by a local matrix multiplication:

1. A[BX, D] = **AllGather**Y(A[BX, DY])
2. Out[BX, FY] = A[BX, D] *D W[D, FY]

Sadly, this is bad because it doesn't allow us to overlap the communication with the computation. Overlapping them can be done with a "collective matmul", as described in Wang et al. 2023. The algorithm is basically as follows:

- For each Y shard, perform a matmul of the local chunk of A with the local chunk of W, producing a result of shape `[B / X, F / Y]`. Simultaneously, permute A so you get the next chunk locally, perform the matmul, and sum the result.

We can implement that quite easily with `jax.shard_map`:

```
import functools

import jax
import jax.numpy as jnp
import numpy as np

Explicit = jax.sharding.AxisType.Explicit

# This is intended to run on a TPU v5e-8 runtime. If you can't get this,
# try setting jax.config.update('jax_num_cpu_devices', 8).
#
mesh = jax.make_mesh(axis_sizes=(2, 4), axis_names=('X', 'Y'), axis_types=(Explicit, Explicit))
jax.set_mesh(mesh)

B, D, F = 1024, 2048, 8192
A = jnp.arange(np.prod((B, D))).reshape((B, D))
W = jnp.arange(np.prod((D, F))).reshape((D, F))

A = jax.device_put(A, jax.P('X', 'Y'))
W = jax.device_put(W, jax.P(None, 'Y'))

@functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
def matmul(lhs, rhs):
  return lhs @ rhs

def collective_matmul_allgather_lhs_contracting(lhs, rhs):
  # lhs is the looped operand; rhs is the local operand
  axis_size = jax.lax.axis_size('Y')  # axis_size = 4 for this example
  idx = jax.lax.axis_index('Y')

  chunk_size = lhs.shape[1]
  assert rhs.shape[0] % chunk_size == 0
```

```
  def f(i, carrys):
    accum, lhs = carrys
    rhs_chunk = jax.lax.dynamic_slice_in_dim(rhs, (idx + i) % axis_size * chunk_size, chunk_size)
    # Matmul for a chunk
    update = lhs @ rhs_chunk
    # Circular shift to the left
    lhs = jax.lax.ppermute(
        lhs,
        axis_name='Y',
        perm=[(j, (j - 1) % axis_size) for j in range(axis_size)]
    )
    return accum + update, lhs

  accum = jnp.zeros((lhs.shape[0], rhs.shape[1]), dtype=lhs.dtype)
  accum = jax.lax.pcast(accum, ('X', 'Y'), to='varying')
  accum, lhs = jax.lax.fori_loop(0, axis_size - 1, f, (accum, lhs), unroll=True)

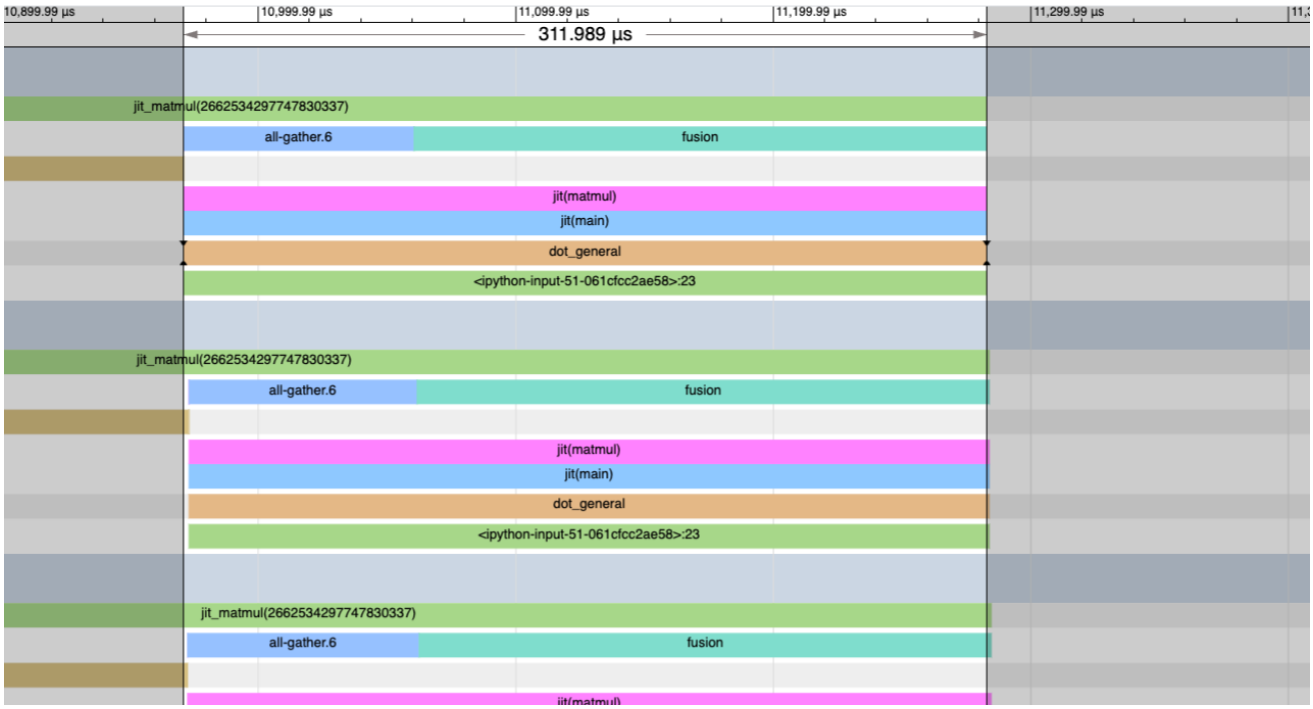
  # Compute the last chunk after the final permute to leave lhs in the state we found it
  i = axis_size - 1
  rhs_chunk = jax.lax.dynamic_slice_in_dim(rhs, (idx + i) % axis_size * chunk_size, chunk_size)
  update = lhs @ rhs_chunk
  return accum + update

jit_sharded_f = jax.jit(jax.shard_map(
  collective_matmul_allgather_lhs_contracting,
  in_specs=(jax.P('X', 'Y'), jax.P(None, 'Y')), out_specs=jax.P('X', 'Y')))

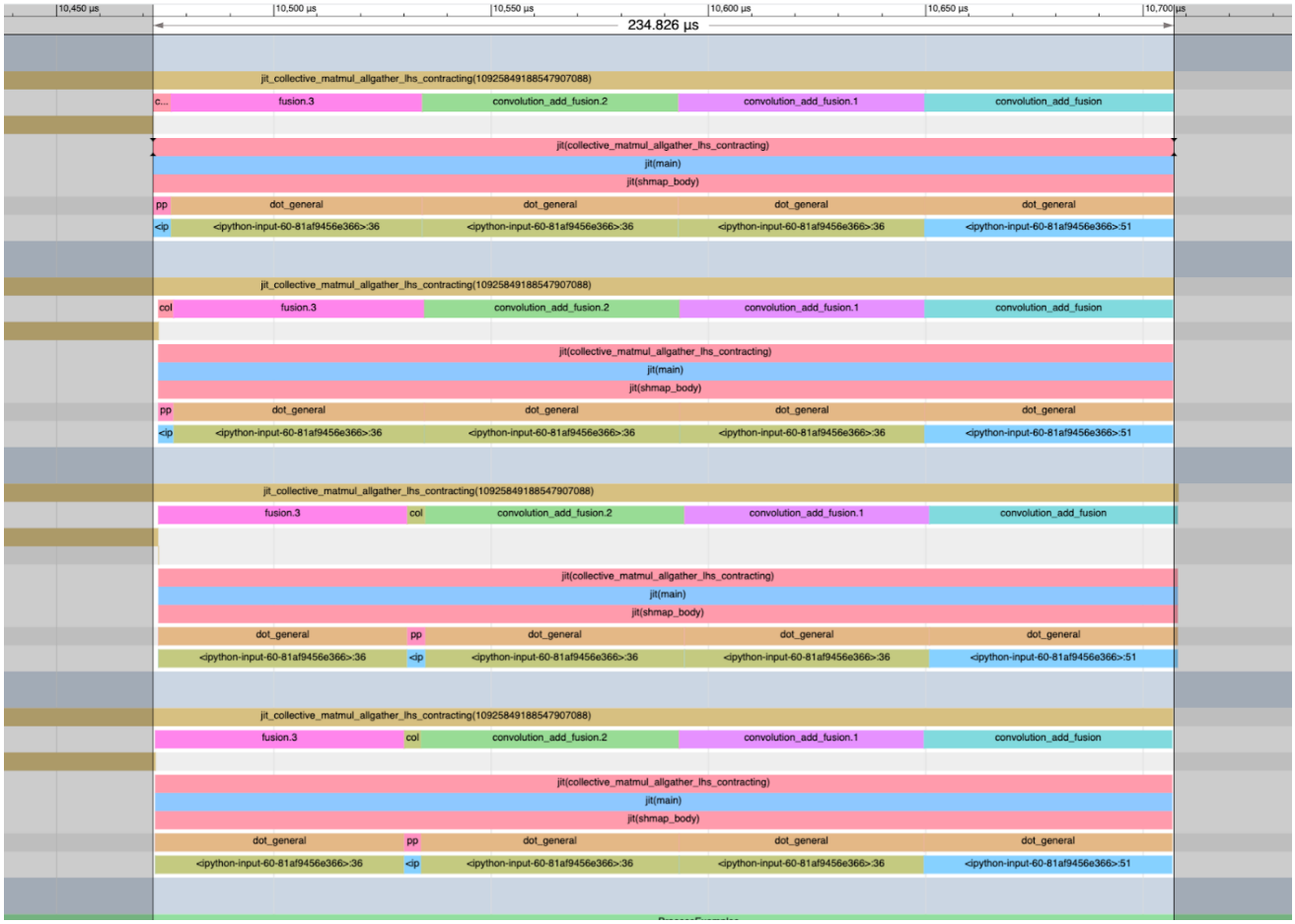
shmapped_out = jit_sharded_f(A, W)
expected_out = matmul(A, W)

np.testing.assert_array_equal(shmapped_out, expected_out)
```

This is pretty neat! We can benchmark this and see that it's also a lot faster! Here's the profile with the default jit matmul which takes 311us with a big blocking AllGather at the beginning:



And here's the version above that takes 244us. You can see the profile doesn't have the AllGather. It's all useful work! Our FLOPs utilization is also a lot higher.



It's also worth noting that the matmul time with no sharding on the contracting dimension is 224us, so we're remarkably close to the unsharded baseline here. This is a good example of the kind of performance engineering you might end up doing to improve TPU utilization. For more `shard_map` examples, this note is great.

Now here are a couple of useful worked problems to try and implement using `jax.jit` or `shard_map`!

Worked Problems

Here are some random JAX-related problems. I'll add some more later. For all of these, you'll need some number of TPUs. Colab no longer hands out TPU v2-8 slices, so use Kaggle (which still provides them for free) or an 8-core GCP slice.[56] From now on, we'll assume you have N devices available.

Question 1: Let **A** be an array of activations of shape float32[SX, DY] with `X * Y = N`. Do the following:

1. Write a function in JAX that computes the average within each `(X, Y)` shard, i.e. it returns an array of size [X, Y] where `arr[i, j]` is the average over shard `(i, j)`. Do this with both `jax.jit` and `shard_map`. Profile each and see how long they took. Was there any communication added? *Hint: there shouldn't be, but sometimes XLA adds it anyway.*
2. Write a function in JAX that returns `roll(x, shift, axis=0) - x` for some shift **within each shard along X**. I'm not enough of a masochist to make you do this in jax.jit, so just do this with `shard_map`.

Click here for the answer.

Part 1: Here is a solution to part 1. Note the fairly complex reshapes we have to do for the `jax.jit` solution.

[56] If you just want to emulate a mesh on fake problems, you can also fake 8 devices on CPU with `import jax; jax.config.update('jax_num_cpu_devices', 8)` (requires jax >= 0.4.27ish), though this won't reflect real performance.

```
import numpy as np

import jax
import jax.numpy as jnp

Auto = jax.sharding.AxisType.Auto

mesh = jax.make_mesh((4, 2), ('X','Y'), (Auto, Auto))

average_shmap = jax.shard_map(
    lambda x: x.mean(keepdims=True),
    mesh=mesh,
    in_specs=jax.P('X','Y'), out_specs=jax.P('X','Y')
)

def average(x):
  X, Y = mesh.axis_sizes
  return x.reshape(X, x.shape[0] // X, Y, x.shape[1] // Y).mean(axis=(1, 3))

average_jit = jax.jit(average, out_shardings=jax.NamedSharding(mesh, jax.P('X','Y')))

x = jnp.arange(8 * 64 * 8, dtype=jnp.float32).reshape(8 * 64, 8)
x = jax.device_put(x, jax.NamedSharding(mesh, jax.P('X','Y')))

y1 = average_shmap(x)
y2 = average_jit(x)

np.testing.assert_array_equal(y1, y2)
```

Part 2: Here is a similar solution to Part 2. The `shard_map` version is the one you were asked for, but a `jax.jit` version is included for comparison.

```
import numpy as np

import jax
import jax.numpy as jnp

import functools

Auto = jax.sharding.AxisType.Auto

mesh = jax.make_mesh((4, 2), ('X','Y'), (Auto, Auto))

def shift_shmap(x, shift: int):
  shmapped = jax.shard_map(
      lambda x: jnp.roll(x, shift, axis=0) - x,
      mesh=mesh,
      in_specs=jax.P('X','Y'), out_specs=jax.P('X','Y')
  )
  return shmapped(x)

@functools.partial(jax.jit, static_argnames=['shift'], out_shardings=jax.NamedSharding(mesh, jax.P('X','Y
def shift_jit(x, shift: int):
  X, Y = mesh.axis_sizes
  reshaped = x.reshape(X, x.shape[0] // X, -1)
  return (jnp.roll(reshaped, shift, axis=1) - reshaped).reshape(x.shape[0], x.shape[1])
```

```
x = jnp.arange(8 * 64 * 8, dtype=jnp.float32).reshape(8 * 64, 8)
x = jax.device_put(x, jax.NamedSharding(mesh, jax.P('X','Y')))

y1 = shift_shmap(x, 5)
y2 = shift_jit(x, 5)

np.testing.assert_array_equal(y1, y2)
```

Question 2: Here we'll make a basic "mixture of experts" model together. Let **W**: float32[EX, D, F] be a set of E "expert" matrices. Let **A**: float32[SX, D] (our activations) and let **B**: int32[SX] be a set of "routing assignments" where B[i] is an integer in the range `[0, E)` telling us which matrix we want to process that activation. We want to write a function in JAX that returns `Out[i] = A[i] @ W[B[i]]`.

1. Let's start by ignoring sharding altogether. Make all of these tensors small enough so they fit in one device. Write a local implementation of this function. *Make sure you don't materialize an array of shape `[S, D, F]`! Hint: there are a few ways to do this. The simplest is to loop over the experts, multiply all of **A** by `W[e]`, and mask out the tokens not routed to expert `e`. Alternatively, you can sort the tokens by expert into a padded buffer of shape `[E, S, D]` and do a single batched matmul, masking out the padding (why does the second dimension need to have size S?). `jax.lax.ragged_dot` does the sorted version without the padding.*
2. If you just `jax.jit` the above method, something will happen. Profile this and see what communication it decided to do. How long does it take?
3. One problem you'll notice with the above is that it likely gathers the full set of activations **A** locally, i.e. AllGatherX([SX, D]). Not only is this expensive communication-wise, it's also incredibly expensive memory-wise if we can't fit the full set of activations locally. Implement the above using `shard_map` and explicit communication.
    1. For a first pass, it might be easiest to use a `jax.lax.all_gather` and reorder as in step 1.
    2. For a second pass, try to avoid materializing any array of size `[E, S, D]`, i.e. try to perform the computation in a ragged fashion using a `jax.lax.all_to_all` inside a `jax.lax.while_loop`. This way, you can avoid materializing the full activations and wasting compute on padding. How much faster is this than your original implementation?
4. Most MoEs route to multiple (k) experts and then average the result. Refactor the above to implement this. Let **B**: int32[SX, k] in this case for the k experts to route to.

Click here for the (partial) answer.

Part 1: You have a lot of choices here. Here's the simplest option, which just iterates over the experts with masking and never sorts the tokens:

```
def moe_local(W: jnp.ndarray, A: jnp.ndarray, B: jnp.ndarray) -> jnp.ndarray:
    S, _ = A.shape
    E, _, F = W.shape

    def expert_forward(carry, e):
        output = carry  # [S, F]
        mask = (B == e)[:, None]  # [S, 1]
        expert_result = A @ W[e]  # [S, F] - this expert's transform of ALL tokens
        output = output + expert_result * mask  # Only keep results for assigned tokens
        return output, None

    output = jnp.zeros((S, F))
    output, _ = jax.lax.scan(expert_forward, output, jnp.arange(E))

    return output
```

This wastes compute, since every expert processes every token. A more efficient option is to sort the tokens by expert and then use `jax.lax.ragged_dot`, which takes the sorted activations along with the number of tokens

per expert and does the matmul without any padding or masking.

Part 3: I'm only going to sketch the pseudocode here (if you have a clean solution feel free to add it):

```
chunk_size = 128
def matmul(W, x, B):
  i = 0
  outs = []
  x = # sort x according to assignments
  while (chunk := x[i:i+chunk_size]).any():
     chunk = all_to_all(chunk)
     outs.append(matmul_local(W, chunk))
     i += chunk_size
  return concat(outs)
```

The basic idea is to iterate over chunks of the array, sort them and do an all_to_all, then do the local FLOPs.

Question 3: The collective matmul example above is actually super relevant for real LLMs. Let's tweak the example to do the full Transformer stack.

1. As an exercise, let's start by implementing an AllReduce collective matmul, i.e. A[BX, DY] *D W[DY, F] -> Out[BX, F]. Note that the output is sharded only along X, i.e. it is replicated across Y. The naive algorithm is discussed above, basically just a local matmul followed by an AllReduce. Try to make a comms overlapped "collective" version of this operation. *Hint: tile over the output dimension and feel free to use `jax.lax.psum` (aka AllReduce). Note: due to the way XLA handles this, it may not actually be faster than the baseline.*
2. The complement to the AllReduce collective matmul above is a ReduceScatter collective matmul, as in Tmp[BX, FY] *F W2[FY, D] -> Out[BX, DY]. This occurs in the down-projection matrix in a Transformer. Implement a collective, overlapped version of this in JAX. Be careful about passing only the minimal amount of data you need. *Hint: try permuting the result as you accumulate it.*
3. Put the AllGather collective matmul from the main text together with the ReduceScatter collective matmul from (2) into an end-to-end Transformer block that performs In[BX, DY] *D Win[D, FY] *F Wout[FY, D] -> Out[BX, DY] with overlapped communication.[57] How much faster is this than a `jax.jit` implementation?

Question 4: All of the collective matmuls implemented above are unidirectional: they only permute in one direction. Rewrite the collective AllReduce matmul and the collective ReduceScatter matmuls to use bidirectional communication. How much faster are these?

That's all for Part 10. That's basically it! For final conclusions and further reading, click here.

Chapter 11

[57] As before, we can't do $W_{in} \cdot W_{out}$ first because of a non-linearity we've omitted here.

Conclusions and Further Reading

Thank you for reading the whole thing and congratulations on making it all the way to the end. Before we conclude, a few acknowledgments:

Acknowledgments

This document represents a significant collective investment from many people at Google DeepMind, who we'd like to briefly acknowledge!

- James Bradbury, Reiner Pope, Noam Shazeer, and Blake Hechtman originally derived many of the ideas in this manuscript, and were early to understand the systems view of the Transformer.
- Sholto Douglas wrote the first version of this doc and is responsible for kicking off the project. He is more than anyone responsible for the overall narrative of this doc.
- Jacob Austin led the work of transforming this first version from rough notes into a more polished and comprehensive artifact. He did much of the work of editing, formatting, and releasing this document, and coordinated contributions from other authors.
- Most of the figures and animations were made by Anselm Levskaya and Charlie Chen.
- Charlie Chen wrote the inference section and drew many of the inference figures.
- Roy Frostig helped with publication, editing, and many other steps of the journey.

We'd also like to thank many others who gave critical feedback throughout the process, in particular Zak Stone, Nikhil Sethi, Caitlin Stanton, Alek Dimitriev, Sridhar Lakshmanamurthy, Albert Magyar, Diwakar Gupta, Jeff Dean, Corry Wang, Matt Johnson, Peter Hawkins, and many others. Thanks to Ruiqi Gao for help with the HTML formatting.

Thank you all!

Before you go, you might also enjoy reading the new Part 12 on NVIDIA GPUs!

Further Reading

There is a bunch of related writing, including the following:

- **TPU Deep Dive**: a wonderful in-depth look at the TPU architecture in the spirit of this book.
- **Domain specific architectures for AI inference**: a hardware and model deep dive in the spirit of this book.
- **A Domain-Specific Supercomputer for Training Deep Neural Networks**: one of the OG TPU papers, this has a lot of great details about the Google TPU program not covered here.
- **Making Deep Learning Go Brrrr From First Principles**: a more GPU and PyTorch-focused tutorial on LLM rooflines and performance engineering.
- **Writing TPU Kernels with Pallas**: increasingly, TPU programming involves writing custom kernels in Pallas. This series discusses how to write kernels and many lower level TPU details that aren't mentioned here.
- **How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog**: while GPU and CUDA specific, this is an excellent blog post showing how to optimize a matmul kernel in CUDA. This might be a good deep dive into how TPUs and GPUs are different.

- **Distributed arrays and automatic parallelization**: this is a really nice guide to parallelism APIs in JAX and is a good way to learn how to actually implement some of the ideas we've discussed here.
- **Rafi Witten's High Performance LLMs 2024 Class**: our former colleague Rafi gave a great course on TPU performance engineering and the slides are all on GitHub. This covers a bunch of things in more depth than we do here.
- **[2211.05102] Efficiently Scaling Transformer Inference**: a detailed paper on the mathematics of Transformer inference. This is the inspiration for a lot of this document.
- **Huggingface Ultra-Scale Playbook**: something of a GPU analog to this book, this talks more at depth about how PyTorch implements parallelism techniques and memory-saving techniques during training.
- **Transformer Inference Arithmetic**: a blog with many of the same ideas as this book and some excellent illustrations.
- **Stanford CS336 Slides and Videos**: a fantastic Stanford course covering many details of LLM training and serving, with some useful exercises. Assignments 1 and 2 are particularly relevant.
- **Stas Bekman's ML Engineering Handbook**: a highly practical guide to ML infrastructure, covering topics not addressed in this book like how to negotiate with cloud providers, cluster management, and empirical measurements of GPU throughput.
- **ezyang's blog**: a PyTorch lead's blog on all things sharding + PyTorch, including a guide to PyTorch internals and a writeup of sharded matrix multiplication. Lots of other good things here.
- **The Anatomy of Collective Communication**: a nice walkthrough of GPU and TPU collectives in the spirit of this book. Has a better writeup of N-D and GPU collectives than this book.

There remains a lot of room for comprehensive writing in this area, so we hope this manuscript encourages more of it! We also believe that this is a fruitful area to study and research. In many cases, it can be done even without having many hardware accelerators on hand.

Feedback

Please leave comments or questions so that we can improve this further. You can reach our corresponding author, Jacob Austin, at jacobaustin123 [at] gmail [dot] com, or suggest edits by posting issues, pull requests, or discussions on GitHub.

Chapter 12

How to Think About GPUs

What Is a GPU?

A modern ML GPU (e.g. H100, B200) is basically a bunch of compute cores that specialize in matrix multiplication (called **Streaming Multiprocessors** or **SMs**) connected to a stick of fast memory (called **HBM**). Here's a diagram:

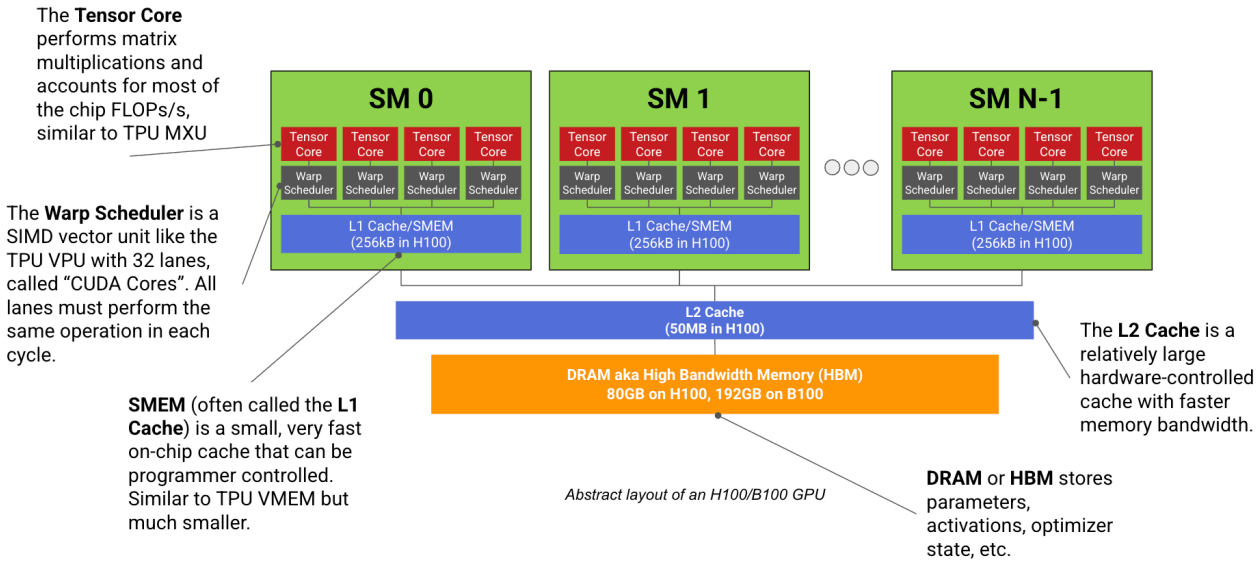


Figure 24: Figure: a diagram showing the abstract layout of an H100 or B200 GPU. An H100 has 132 SMs while a B200 has 148. We use the term 'Warp Scheduler' somewhat broadly to describe a set of 32 CUDA SIMD cores and the scheduler that dispatches work to them. Note how much this looks like a TPU!

Each SM, like a TPU's Tensor Core, has a dedicated matrix multiplication core (unfortunately also called a **Tensor Core**[58]), a vector arithmetic unit (called a **Warp Scheduler**[59]), and a fast on-chip cache (called **SMEM**). Unlike a TPU, which has at most 2 independent "Tensor Cores", a modern GPU has more than 100 SMs (132 on an H100). Each of these SMs is much less powerful than a TPU Tensor Core but the system overall is more flexible. Each SM is more or less totally independent, so a GPU can do hundreds of separate tasks at once.[60]

Let's take a more detailed view of an H100 SM:

Each SM is broken up into 4 identical quadrants, which NVIDIA calls **SM subpartitions**, each containing a

[58]The GPU Tensor Core is the matrix multiplication sub-unit of the SM, while the TPU TensorCore is the umbrella unit that contains the MXU, VPU, and other components.

[59]NVIDIA doesn't have a good name for this, so we use it only as the best of several bad options. The Warp Scheduler is primarily the unit that dispatches work to a set of CUDA cores, but we use it here to describe the control unit and the set of cores it controls.

[60]Although SMs are independent, they are often forced to coordinate for peak performance because they all share a capacity-limited L2 cache.



Figure 25: Figure: a diagram of an H100 SM (source) showing the 4 subpartitions, each containing a Tensor Core, Warp Scheduler, Register File, and sets of CUDA Cores of different precisions. The 'L1 Data Cache' near the bottom is the 256kB SMEM unit. A B200 looks similar, but adds a substantial amount of Tensor Memory (TMEM) for feeding the bulky Tensor Cores.

Tensor Core, 16k 32-bit registers, and a SIMD/SIMT vector arithmetic unit called a Warp Scheduler, whose lanes (ALUs) NVIDIA calls **CUDA Cores**. The core component of each partition is arguably the Tensor Core, which performs matrix multiplications and makes up the vast majority of its FLOPs/s, but it's not the only component worth noting.

- **CUDA Cores:** each subpartition contains a set of ALUs called CUDA Cores that do SIMD/SIMT vector arithmetic. Each ALU can generally do 1 arithmetic op each cycle, e.g. f32.add.[61] Each subpartition contains 32 fp32 cores (and a smaller number of int32 and fp64 cores) that all execute the same instruction in each cycle. Like the TPU's VPU, CUDA cores are responsible for ReLUs, pointwise vector operations, and reductions (sums).[62]
- **Tensor Core (TC):** each subpartition has its own Tensor Core, which is a dedicated matrix multiplication unit like a TPU MXU. The Tensor Core represents the vast majority of the GPU's FLOPs/s (e.g. on an H100, we have 990 bf16 TC TFLOP/s compared to just 66 TFLOPs/s from the CUDA cores).
  - 990 bf16 TFLOPs/s with 132 SM running at 1.76GHz means each H100 TC can do `7.5e12 / 1.76e9 / 4 ~ 1024` bf16 FLOPs/cycle, roughly an 8x8x8 matmul.[63]
  - Like TPUs, GPUs can do lower precision matmuls at higher throughput (e.g. H100 has 2x fp8 FLOPs/s vs. fp16). Low-precision training or serving can be significantly faster.
  - Each GPU generation since Volta has increased the TC size over the previous generation (good article on this). With B200 the TC has gotten so large it can no longer fit its inputs in SMEM, so B200s introduce a new memory space called TMEM.[64]

CUDA cores are more flexible than a TPU's VPU: GPU CUDA cores (since V100) use what is called a SIMT (*Single Instruction Multiple Threads*) programming model, compared to the TPU's SIMD (*Single Instruction Multiple Data*) model. Like ALUs in a TPU's VPU, CUDA cores within a subpartition must execute the same operation in each cycle (e.g. if one core is adding two floats, then every other CUDA core in the subpartition must also do so). Unlike the VPU, however, each CUDA core (or "thread" in the CUDA programming model) has its own instruction pointer and can be *programmed* independently. When two threads in the same warp are instructed to perform different operations, you effectively do *both* operations, masking out the cores that don't need to perform the divergent operation.

This enables flexible programming at the thread level, but at the cost of silently degrading performance if warps diverge too often. Threads can also be more flexible in what memory they can access; while the VPU can only operate on contiguous blocks of memory, CUDA cores can access individual floats in shared registers and maintain per-thread state.

CUDA core scheduling is also more flexible: SMs run a bit like multi-threaded CPUs, in the sense that they can "schedule" many programs (**warps**) concurrently (up to 64 per SM) but each *Warp Scheduler* only ever executes a single program in each clock cycle.[65] The Warp Scheduler automatically switches between active warps to hide I/O operations like memory loads. TPUs are generally single threaded by comparison.

Memory

Beyond the compute units, GPUs have a hierarchy of memories, the largest being HBM (the main GPU memory), and then a series of smaller caches (L2, L1/SMEM, TMEM, register memory).

[61] Newer GPUs support FMA (Fused-Multiply Add) instructions which technically do two FLOPs each cycle, a fact NVIDIA uses ruthlessly to double their reported specs.

[62] Historically, before the introduction of the Tensor Core, the CUDA cores were the main component of the GPU and were used for rendering, including ray-triangle intersections and shading. On today's gaming GPUs, they still do a bulk of the rendering work, while TensorCores are used for up-sampling (DLSS), which allows the GPU to render at a lower resolution (fewer pixels = less work) and upsample using ML.

[63] NVIDIA doesn't share many TC hardware details, so this is more a guess than definite fact – certainly, it doesn't speak to how the TC is implemented. We know that a V100 can perform 256 FLOPs/TC/cycle. An A100 can do 512, H100 can do 1024, and while the B200 details aren't published, it seems likely it's about 2048 FLOPs/TC/cycle, since `2250e12 / (148 * 4 * 1.86e9)` is about 2048. Some more details are confirmed here.

[64] In Ampere, the Tensor Core could be fed from a single warp, while in Hopper it requires a full SM (warpgroup) and in Blackwell it's fed from 2 SMs. The matmuls have also become so large in Blackwell that the arguments (specifically, the accumulator) no longer fit into register memory/SMEM, so Blackwell adds TMEM to account for this.

[65] Warps scheduled on a given SM are called "resident".

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```

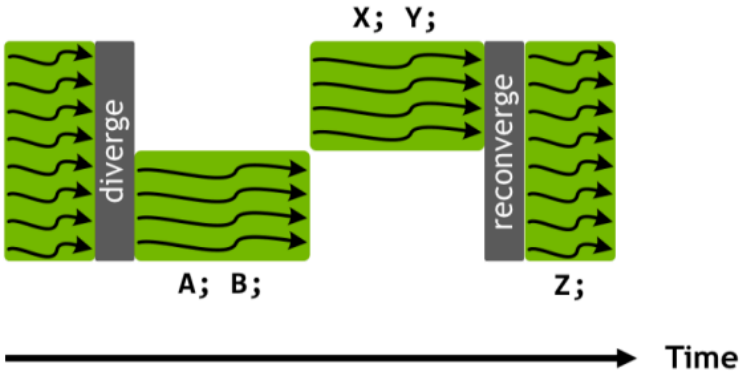


Figure 26: Figure: an example of warp divergence within a set of threads (source). White spaces indicate stalls of at least some fraction of the physical CUDA cores

- **Registers:** Each subpartition has its own register file containing 16,384 32-bit words on H100/B200 (`4 * 16384 * 4 = 256kiB` per SM) accessible by the CUDA cores.
  - Each CUDA core can only access up to 256 registers at a time, so although we can schedule up to 64 "resident warps" per SM, you can only fit 8 (`256 * 1024 / (4 * 32 * 256)`) at a time if each thread uses 256 registers.
- **SMEM (L1 Cache):** each SM has its own 256kB on-chip cache called SMEM, which can either be programmer controlled as "shared memory" or used by the hardware as an on-chip cache. SMEM is used for storing activations and inputs to TC matmuls.
- **L2 Cache:** all SMs share[66] a relatively large ~50MB L2 cache used to reduce main memory accesses.
  - This is similar in size to a TPU's VMEM but it's **much** slower and isn't programmer controlled. This leads to a bit of "spooky action at a distance" where the programmer needs to modify memory access patterns to ensure the L2 cache is well used.[67]
  - NVIDIA does not publish the L2 bandwidth for their chips, but it's been measured to be about 5.5TB/s. This is roughly 1.6x the HBM bandwidth but it's full-duplex, so the effective bidirectional bandwidth is closer to 3x. By comparison, a TPU's VMEM is 2x larger *and* has much more bandwidth (around 40TB/s).
- **HBM:** the main GPU memory, used for storing model weights, gradients, activations, etc.
  - The HBM size has increased a lot from 32GB in Volta to 192GB in Blackwell (B200).
  - The bandwidth from HBM to the CUDA Tensor Core is called HBM bandwidth or memory bandwidth, and is about 3.35TB/s on H100 and 9TB/s on B200.

Summary of GPU specs

Here is a summary of GPU specs for recent models. The number of SMs, clock speed, and FLOPs differ somewhat between variants of a given GPU. Here are memory capacity numbers:

GPU	Generation	Clock Speed	SMs/chip	SMEM capacity/SM	L2 capacity/chip	HBM capacity/chip
V100	Volta	1.25GHz/1.38GHz	80	96kB	6MB	32GB
A100	Ampere	1.10GHz/1.41GHz	108	192kB	40MB	80GB
H100	Hopper	1.59GHz/1.98GHz	132	256kB	50MB	80GB

[66]Technically, the L2 cache is split in two, so half the SMs can access 25MB a piece on an H100. There is a link connecting the two halves, but at lower bandwidth.

[67]The fact that the L2 cache is shared across all SMs effectively forces the programmer to run the SMs in a fairly coordinated way anyway, despite the fact that, in principle, they are independent units.

GPU	Generation	Clock Speed	SMs/chip	SMEM capacity/SM	L2 capacity/chip	HBM capacity/chip
H200	Hopper	1.59GHz/1.98GHz	132	256kB	50MB	141GB
B200	Blackwell	?	148	256kB	126MB	192GB

All generations have 256kB of register memory per SM. Blackwell adds 256kB of TMEM per SM as well. Here are the FLOPs and bandwidth numbers for each chip:

GPU	Generation	HBM BW/chip	FLOPs/s/chip (bf16/fp16)	FLOPs/s/chip (fp8/int8)	FLOPs/s/chip (fp4)
V100	Volta	9.0e11	—	—	—
A100	Ampere	2.0e12	3.1e14	6.2e14	—
H100	Hopper	3.4e12	9.9e14	2.0e15	—
H200	Hopper	4.8e12	9.9e14	2.0e15	—
B200	Blackwell	8.0e12	2.3e15	4.5e15	9.0e15

We exclude B100 since it wasn't mass-produced.[68] Some specs depend slightly on the precise version of the GPU, since NVIDIA GPUs aren't as standard as TPUs.

Here's a helpful cheat sheet comparing GPU and TPU components:

GPU	TPU	What is it?
Streaming Multiprocessor (SM)	Tensor Core	Core "cell" that contains other units
Warp Scheduler	VPU	SIMD vector arithmetic unit
CUDA Core	VPU ALU	SIMD ALU
SMEM (L1 Cache)	VMEM	Fast on-chip cache memory
Tensor Core	MXU	Matrix multiplication unit
HBM (aka GMEM)	HBM	High bandwidth high capacity memory

GPUs vs. TPUs at the chip level

GPUs started out rendering video games, but since deep learning took off in the 2010s, they've started acting more and more like dedicated matrix multiplication machines – in other words, more like TPUs.[69] To an extent, this history explains why modern GPUs look the way they do. They weren't designed purely for LLMs or ML models but as general-purpose accelerators, and the hardware aims for a level of "generality" that can be both a blessing and a curse. GPUs much more often "just work" when applied to new tasks and lean far less on a good compiler than TPUs do. But this also makes them much harder to reason about or get roofline performance out of, since so many compiler features can cause bottlenecks.

GPUs are more modular. TPUs have 1-2 big Tensor Cores, while GPUs have hundreds of small SMs. Likewise, each TC has a single big VPU composed of 4 independently programmable 8x128 units (for a total of 4096 ALUs); by comparison, an H100 has 132 * 4 = 528 independent SIMD units, each 32-wide (16k ALUs total). Here is a 1:1 comparison of GPUs to TPU that highlights this point:

[68] While NVIDIA made a B100 generation, they were only briefly sold and produced, allegedly due to design flaws that prevented them from running close to their claimed specifications. They struggled to achieve peak FLOPs without throttling due to heat and power concerns.

[69] Before the deep learning boom, GPUs ("Graphics Processing Units") did, well, graphics – mostly for video games. Video games represent objects with millions of little triangles, and the game renders (or "rasterizes") these triangles into a 2D image that gets displayed on a screen 30-60 times a second (this frequency is called the framerate). Rasterization involves projecting these triangles into the coordinate frame of the camera and calculating which triangles overlap which pixels, billions of times a second. As you can imagine, this is very expensive, and it's just the beginning. You then have to color each pixel by combining the colors of possibly several semi-opaque triangles that intersect the ray. GPUs were designed to do these operations extremely fast, with an eye towards versatility; you need to run many different GPU workloads (called "shaders") at the same time, with no single operation dominating. As a result, consumer graphics-focused GPUs can do matrix multiplication, but it's not their primary function.

GPU	TPU	H100 #	TPU v5p #
SM (streaming multiprocessor)	Tensor Core	132	2
Warp Scheduler	VPU slots	528	8
SMEM (L1 cache)	VMEM	32MB	128MB
Registers	Vector Registers (VRegs)	32MB	256kB
Tensor Core	MXU	528	8

This difference in modularity on the one hand makes TPUs much cheaper to build and simpler to understand, but it also puts more burden on the compiler to do the right thing. Because TPUs have a single thread of control and only support vectorized VPU-wide instructions, the compiler needs to manually pipeline all memory loads and MXU/VPU work to avoid stalls. A GPU programmer can just launch dozens of different kernels, each running on a totally independent SM. On the other hand, those kernels might get horrible performance because they are thrashing the L2 cache or failing to coalesce memory loads; because the hardware controls so much of the runtime, it becomes hard to reason about what's going on behind the scenes. As a result, TPUs can often get closer to peak roofline performance with less work.

Historically, individual GPUs are more powerful (and more expensive) than a comparable TPU: A single H200 has close to 2x the FLOPs/s of a TPU v5p and 1.5x the HBM. At the same time, the sticker price on Google Cloud is around \$10/hour for an H200 compared to \$4/hour for a TPU v5p. TPUs generally rely more on networking multiple chips together than GPUs.

TPUs have a lot more fast cache memory. TPUs also have a lot more VMEM than GPUs have SMEM (+TMEM), and this memory can be used for storing weights and activations in a way that lets them be loaded and used extremely fast. This can make them faster for LLM inference if you can consistently store or prefetch model weights into VMEM.

Quiz 1: GPU hardware

Here are some problems to work through that test some of the content above. Answers are provided, but it's probably a good idea to try to answer the questions before looking, pen and paper in hand.

Question 1 [CUDA cores]: How many fp32 CUDA cores (ALUs) does an H100 have? B200? How does this compare to the number of independent ALUs in a TPU v5p?

Click here for the answer.

Answer: An H100 has 132 SMs with 4 subpartitions each containing 32 fp32 CUDA cores, so we `132 * 4 * 32 = 16896` CUDA cores. A B200 has `148` SMs, so a total of `18944`. A TPU v5p has 2 TensorCores (usually connected via Megacore), each with a VPU with (8, 128) lanes and 4 independent ALUs per lane, so `2 * 4 * 8 * 128 = 8192` ALUs. This is roughly half the number of vector lanes of an H100, running at roughly the same frequency.

Question 2 [Vector FLOPs calculation]: A single H100 has 132 SMs and runs at a clock speed of 1.59GHz (up to 1.98GHz boost). Assume it can do one vector op per cycle per ALU. How many vector fp32 FLOPs can be done per second? With boost? How does this compare to matmul FLOPs?

Click here for the answer.

Answer: `132 * 4 * 32 * 1.59e9 = 26.9TFLOPs/s`. With boost its 33.5 TFLOPs/s. This is half what's reported in the spec sheet because technically we can do an FMA (fused-multiply-add) in one cycle which counts as two FLOPs, but this isn't useful in most cases. We can do 990 bfloat16 matmul TFLOPs/s, so ignoring FMAs, Tensor Cores do around 30x more FLOPs/s.

Question 3 [GPU matmul intensity]: What is the peak fp16 matmul intensity on an H100? A B200? What about fp8? *By intensity we mean the ratio of matmul FLOPs/s to memory bandwidth.*

Click here for the answer.

Answer: For an H100, we have a peak 990e12 fp16 FLOPs and 3.35e12 bytes / s of bandwidth. So the critical intensity is `990e12 / 3.35e12 = 295`, fairly similar to the 240 in a TPU. For B200 its `2250e12 / 8e12 = 281`,

very similar. This means, similar to TPUs, that we need a batch size of around 280 to be compute-bound in a matmul.

For both H100 and B200 we have exactly 2x fp8 FLOPs, so the peak intensity also doubles to 590 and 562 respectively, although in some sense it stays constant if we take into account the fact that our weights will likely be loaded in fp8 as well.

Question 4 [Matmul runtime]: Using the answer to Question 3, how long would you expect an `fp16[64, 4096] * fp16[4096, 8192]` matmul to take on a single B200? How about `fp16[512, 4096] * fp16[4096, 8192]`?

Click here for the answer.

From the above, we know we'll be communication-bound below a batch size of 281 tokens. Thus the first is purely bandwidth bound. We read or write $2BD + 2DF + 2BF$ bytes (`2*64*4096 + 2*4096*8192 + 2*64*8192=69e6`) with `8e12` bytes/s of bandwidth, so it will take about `69e6 / 8e12 = 8.6us`. In practice we likely get a fraction of the total bandwidth, so it may take closer to 10-12us. When we increase the batch size, we're fully compute-bound, so we expect `T=2*512*4096*8192/2.3e15=15us`. We again only expect a fraction of the total FLOPs, so we may see closer to 20us.

Question 5 [L1 cache capacity]: What is the total L1/SMEM capacity for an H100? What about register memory? How does this compare to TPU VMEM capacity?

Click here for the answer.

Answer: We have 256kB SMEM and 256kB of register memory per SM, so about 33MB (`132 * 256kB`) of each. Together, this gives us a total of about 66MB. This is about half the 120MB of a modern TPU's VMEM, although a TPU only has 256kB of register memory total! TPU VMEM latency is lower than SMEM latency, which is one reason why register memory on TPUs is not that crucial (spills and fills to VMEM are cheap).

Question 6 [Calculating B200 clock frequency]: NVIDIA reports here that a B200 can perform 80TFLOPs/s of vector fp32 compute. Given that each CUDA core can perform 2 FLOPs/cycle in a FMA (fused multiply add) op, estimate the peak clock cycle.

Click here for the answer.

Answer: We know we have 148 * 4 * 32 = 18944 CUDA cores, so we can do `18944 * 2 = 37888 FLOPs / cycle`. Therefore `80e12 / 37888 = 2.1GHz`, a high but reasonable peak clock speed. B200s are generally liquid cooled, so the higher clock cycle is more reasonable.

Question 7 [Estimating H100 add runtime]: Using the figures above, calculate how long it ought to take to add two `fp32[N]` vectors together on a single H100. Calculate both T_{math} and T_{comms}. What is the arithmetic intensity of this operation? If you can get access, try running this operation in PyTorch or JAX as well for `N = 1024` and `N=1024 * 1024 * 1024`. How does this compare?

Click here for the answer.

Answer: Firstly, adding two `fp32[N]` vectors performs N FLOPs and requires `4 * N * 2` bytes to be loaded and 4 * N bytes to be written back, for a total of `3 * 4 * N = 12N`. Computing their ratio, we have `total FLOPs / total bytes = N / 12N = 1 / 12`, which is pretty abysmal.

As we calculated above, we can do roughly 33.5 TFLOPs/s boost, ignoring FMA. This is only if all CUDA cores are used. For `N = 1024`, we can only use *at most* 1024 CUDA cores or 8 SMs, which will take longer (roughly 16x longer assuming we're compute-bound). We also have a memory bandwidth of 3.35e12 bytes/s. Thus our peak hardware intensity is `33.5e12 / 3.35e12 = 10`.[70] So we're going to be horribly comms bound. Thus our runtime is just

$$T = \max(T_{\text{comms}}, T_{\text{math}}) = \frac{12 \cdot N}{3.35e12} = \frac{N}{2.8e11}$$

For `N = 65,536`, this is about 0.23us. In practice we see a runtime of about 1.5us in JAX, which is fine because we expect to be super latency bound here. For `N = 1024 * 1024 * 1024`, we have a roofline of about 3.84ms,

[70] It's notable that this intensity stays constant across recent GPU generations. For H100s it's 33.5 / 3.5 and for B200 it's 80 / 8. Why this is isn't clear, but it's an interesting observation.

and we see 4.1ms, which is good!

Networking

Networking is one of the areas where GPUs and TPUs differ the most. As we've seen, TPUs are connected in 2D or 3D tori, where each TPU is only connected to its neighbors. This means sending a message between two TPUs must pass through every intervening TPU, and forces us to use only uniform communication patterns over the mesh. While inconvenient in some respects, this also means the number of links per TPU is constant and we can scale to arbitrarily large TPU "pods" without loss of bandwidth.

GPUs on the other hand use a more traditional hierarchical tree-based switching network. Sets of 8 GPUs called **nodes** (up to 72 for GB200[71]) are connected within 1 hop of each other using high-bandwidth interconnects called NVLinks, and these nodes are connected into larger units (called **SUs** or Scalable Units) with a lower bandwidth InfiniBand (IB) or Ethernet network using NICs attached to each GPU. These in turn can be connected into arbitrarily large units with higher level switches.

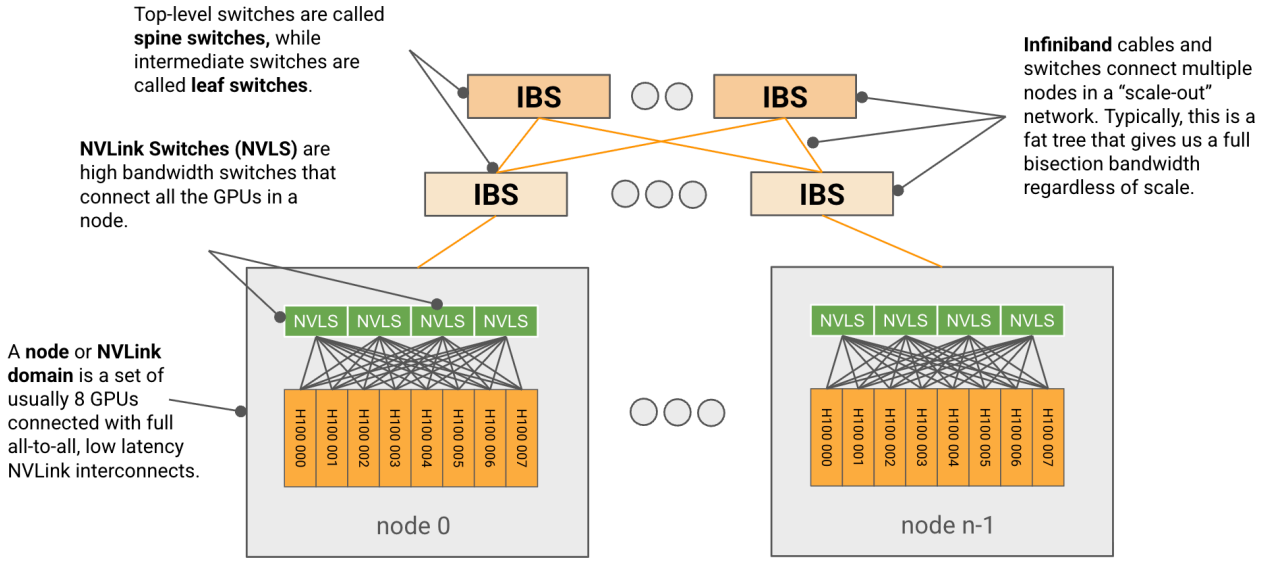


Figure 27: Figure: a diagram showing a typical H100 network. A set of 8 GPUs is connected into a node or NVLink domain with NVSwitches (also called NVLink switches), and these nodes are connected to each other with a switched InfiniBand fabric. H100s have about 450GB/s of egress bandwidth each in the NVLink domain, and each node has 400GB/s of egress bandwidth into the IB network.

At the node level

A GPU node is a small unit, typically of 8 GPUs (up to 72 for GB200), connected with all-to-all, full-bandwidth, low latency NVLink interconnects.[72] Each node contains several high-bandwidth NVSwitches which switch packets between all the local GPUs. The actual node-level topology has changed quite a bit over time, including the number of switches per node, but for H100, we have 4 NVSwitches per node with GPUs connected to them in a `5 + 4 + 4 + 5` link pattern, as shown:

For the Hopper generation (NVLink 4.0), each NVLink link has 25GB/s of full-duplex[73] bandwidth (50GB/s

[71]The term node is overloaded and can mean two things: the NVLink domain, aka the set of GPUs fully connected over NVLink interconnects, or the set of GPUs connected to a single CPU host. Before B200, these were usually the same, but in GB200 NVL72, we have an NVLink domain with 72 GPUs but still only 8 GPUs connected to each host. We use the term node here to refer to the NVLink domain, but this is controversial.

[72]NVLink has been described to me as something like a souped-up PCIe connection, with low latency and protocol overhead but not designed for scalability/fault tolerance, while InfiniBand is more like Ethernet, designed for larger lossy networks.

[73]Full-duplex here means 25GB/s each way, with both directions independent of each other. You can send a total of 50GB/s over the link, but at most 25GB/s in each direction.

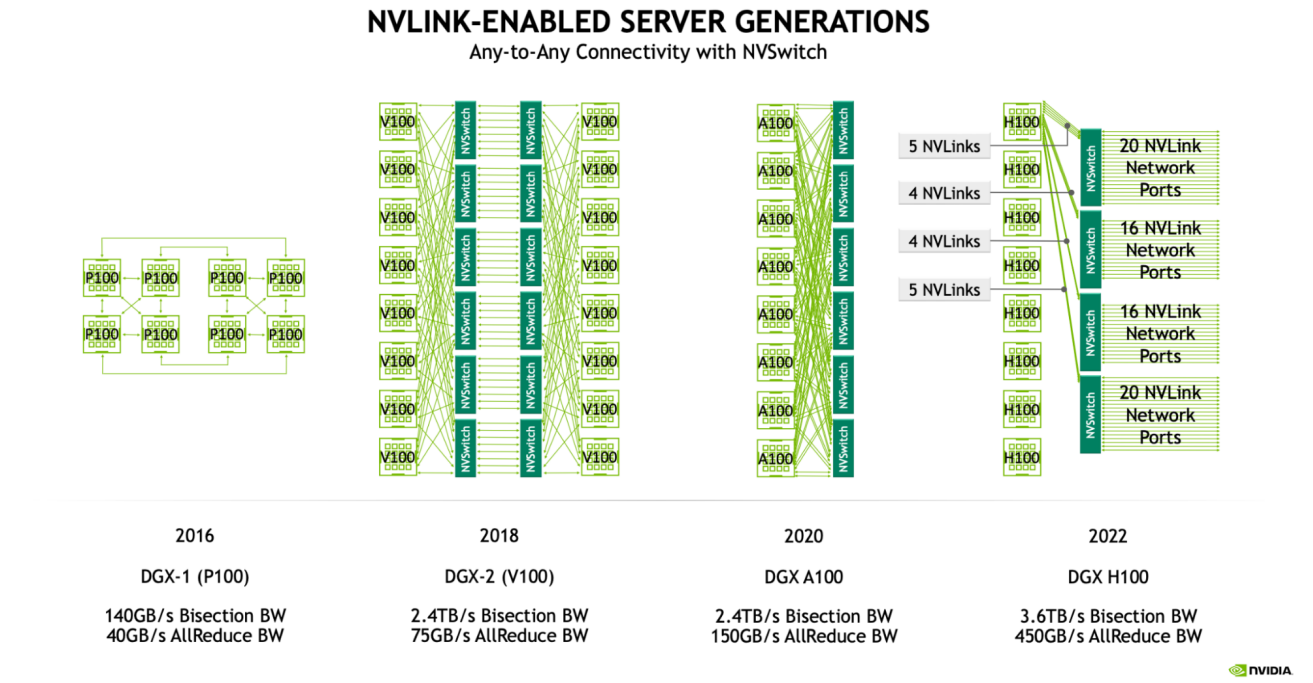


Figure 28: Figure: node aka NVLink domain diagrams from Pascal (P100) onward. Since Volta (V100), we have had all-to-all connectivity within a node using a set of switches. The H100 node has 4 NVSwitches connected to all 8 GPUs with 25GB/s links.

for B200), giving us `18 * 25=450GB/s` of full-duplex bandwidth from each GPU into the network. The massive NVSwitches have up to 64 NVLink ports, meaning an 8xH100 node with 4 switches can handle up to `64 * 25e9 * 4=6.4TB/s` of bandwidth. Here's an overview of how these numbers have changed with GPU generation:

NVLink Gen	NVSwitch Gen	GPU Generation	NVLink Bandwidth (GB/s, full-duplex)	NVLink Ports / GPU	Node GPU to GPU bandwidth (GB/s full-duplex)	Node size (NVLink domain)	NVSwitches per node
3.0	**2.0**	Ampere	25	12	300	8	6
4.0	**3.0**	Hopper	25	18	450	8	4
5.0	**4.0**	Blackwell	50	18	900	8/72	2/18

Blackwell (B200) has nodes of 8 GPUs. GB200NVL72 support larger NVLink domains of 72 GPUs. We show details for both the 8 and 72 GPUs systems.

Quiz 2: GPU nodes

Here are some more Q/A problems on networking. I find these particularly useful to do out, since they make you work through the actual communication patterns.

Question 1 [Total bandwidth for H100 node]: How much total bandwidth do we have per node in an 8xH100 node with 4 switches? *Hint:* consider both the NVLink and NVSwitch bandwidth.

Click here for the answer.

Answer: We have Gen4 4xNVSwitches, each with `64 * 25e9=1.6TB/s` of unidirectional bandwidth. That would give us `4 * 1.6e12=6.4e12` bandwidth at the switch level. However, note that each GPU can only handle 450GB/s of unidirectional bandwidth, so that means we have at most `450e9 * 8 = 3.6TB/s` bandwidth. Since this is smaller, the peak bandwidth is 3.6TB/s.

Question 2 [Bisection bandwidth]: Bisection bandwidth is defined as the smallest bandwidth available between any even partition of a network. In other words, if we split a network into two equal halves, how much bandwidth crosses between the two halves? Can you calculate the bisection bandwidth of an 8x H100 node? *Hint:* bisection bandwidth typically includes flow in both directions.

Click here for the answer.

Answer: Any even partition will have 4 GPUs in each half, each of which can egress `4 * 450GB/s` to the other half. Taking flow in both directions, this gives us `8 * 450GB/s` of bytes cross the partition, or 3.6TB/s of bisection bandwidth. This is what NVIDIA reports e.g. here.

Question 3 [AllGather cost]: Given an array of B bytes, how long would a (throughput-bound) AllGather take on an 8xH100 node? Do the math for bf16[DX, F] where `D=4096`, `F=65,536`. *It's worth reading the TPU collectives section before answering this. Think this through here but we'll talk much more about collectives next.*

Click here for the answer.

Answer: Each GPU can egress 450GB/s, and each GPU has B/N bytes (where `N=8`, the node size). We can imagine each node sending its bytes to each of the other $N-1$ nodes one after the other, leading to a total of (N - 1) turns each with $T_{\text{comms}} = (B/(N * W_{\text{unidirectional}}))$, or $T_{\text{comms}} = (N-1) * B/(N * W_{\text{unidirectional}})$. This is approximately B/W_{uni} or B/450e9.

For the given array, we have `B = 4096 * 65536 * 2 = 536e6` bytes, so the total time is `536e6 * (8 - 1) / (8 * 450e9) = 1.04ms` (or `536e6 / 450e9 = 1.19ms` with the approximation). This could be latency-bound, so it may take longer than this in practice (in practice it takes about 1.5ms).

Beyond the node level

Beyond the node level, the topology of a GPU network is less standardized. NVIDIA publishes a reference DGX SuperPod architecture that connects a larger set of GPUs than a single node using InfiniBand, but customers and datacenter providers are free to customize this to their needs.[74]

Here is a diagram for a reference 1024 GPU H100 system, where each box in the bottom row is a single 8xH100 node with 8 GPUs, 8 400Gbps CX7 NICs (one per GPU), and 4 NVSwitches.

Scalable Units: Each set of 32 nodes is called a "Scalable Unit" (or SU), under a single set of 8 leaf InfiniBand switches. This SU has 256 GPUs with 4 NVSwitches per node and 8 Infiniband leaf switches. All the cabling shown is InfiniBand NDR (50GB/s full-duplex) with 64-port NDR IB switches (also 50GB/s per port). *Note that the IB switches have 2x the bandwidth of the NVSwitches (64 ports with 400 Gbps links).*

SuperPod: The overall SuperPod then connects 4 of these SUs with 16 top level "spine" IB switches, giving us 1024 GPUs with 512 node-level NVSwitches, 32 leaf IB switches, and 16 spine IB switches, for a total of 512 + 32 + 16 = 560 switches. Leaf switches are connected to nodes in sets of 32 nodes, so each set of 256 GPUs has 8 leaf switches. All leaf switches are connected to all spine switches.

How much bandwidth do we have? The overall topology of the InfiniBand network (called the "scale out network") is that of a **fat tree**, with the cables and switches guaranteeing full bisection bandwidth above the node level (here, 400GB/s). That means if we split the nodes in half, each node can egress 400GB/s to a node in the other partition at the same time. More to the point, this means we should have a roughly constant AllReduce bandwidth in the scale out network! While it may not be implemented this way, you can imagine doing a ring reduction over arbitrarily many nodes in the scale-out network, since you can construct a ring including every one.

[74] For instance, Meta trained LLaMA-3 on a datacenter network that differs significantly from this description, using Ethernet, a 3 layer switched fabric, and an oversubscribed switch at the top level.

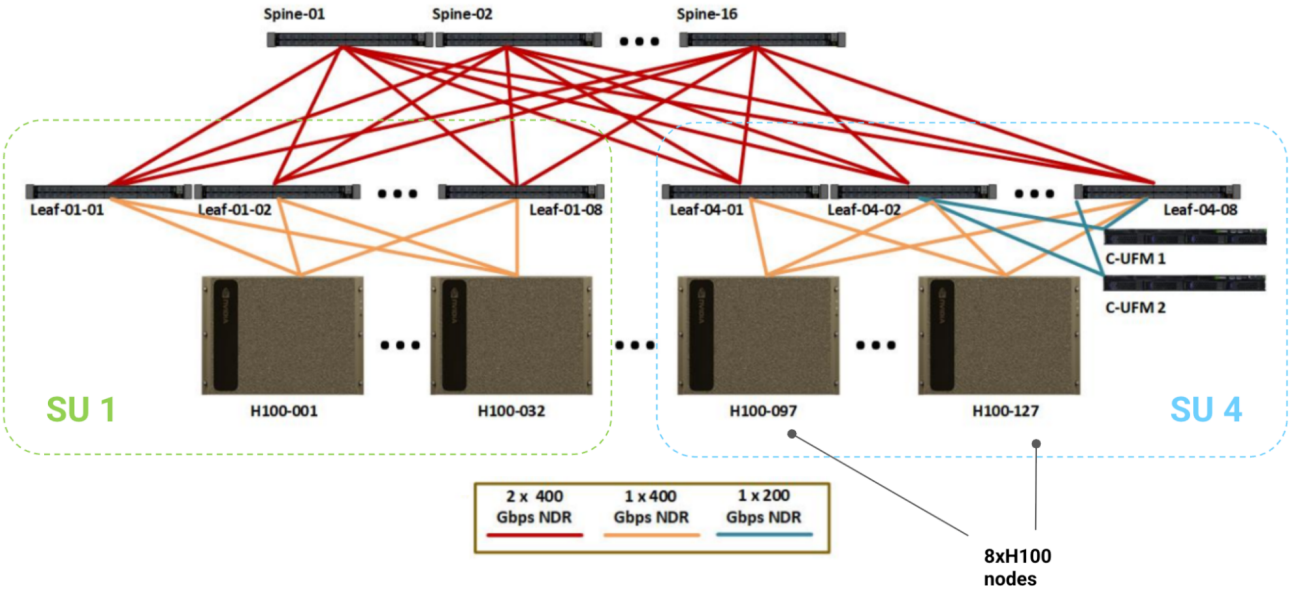


Figure 29: Figure: diagram of the reference 1024 H100 DGX SuperPod with 128 nodes (sometimes 127), each with 8 H100 GPUs, connected to an InfiniBand scale-out network. Sets of 32 nodes (256 GPUs) are called 'Scalable Units' or SUs. The leaf and spine IB switches provide enough bandwidth for full bisection bandwidth between nodes.

Level	GPUs	Switches per Unit	Switch Type	Bandwidth per Unit (TB/s, full-duplex)	GPU-to-GPU Bandwidth (GB/s, full-duplex)	Fat Tree Bandwidth (GB/s, full-duplex)
Node	8	4	NVL	3.6	450	450
Leaf	256	8	IB	12.8	50	400
Spine	1024	16	IB	51.2	50	400

By comparison, a TPU v5p has about 90GB/s egress bandwidth per link, or 540GB/s egress along all axes of the 3D torus. This is not point-to-point so it can only be used for restricted, uniform communication patterns, but it still gives us a much higher TPU to TPU bandwidth that can scale to arbitrarily large topologies (at least up to 8960 TPUs).

The GPU switching fabric can in theory be extended to arbitrary sizes by adding additional switches or layers of indirection, at the cost of additional latency and costly network switches.

Takeaway: Within an H100 node, we have a full fat tree bandwidth of 450GB/s from each GPU, while beyond the node, this drops to 400GB/s node-to-node. This will turn out to be critical for communication primitives.

GB200 NVL72s: NVIDIA has recently begun producing new GB200 NVL72 GPU clusters that combine 72 GPUs in a single NVLink domain with full 900GB/s of GPU to GPU bandwidth. These domains can then be linked into larger SuperPods with proportionally higher (9x) IB fat tree bandwidth. Here is a diagram of that topology:

Counting the egress bandwidth from a single node (the orange lines above), we have `4 * 18 * 400 / 8 = 3.6TB/s` of bandwidth to the leaf level, which is 9x more than an H100 (just as the node contains 9x more GPUs). That means the critical node egress bandwidth is much, *much* higher and our cross-node collective bandwidth can actually be *lower* than within the node. See Appendix A for more discussion.

Node Type	GPUs per node	GPU egress bandwidth	Node egress bandwidth
H100	8	450e9	400e9
B200	8	900e9	400e9
GB200 NVL72	72	900e9	3600e9

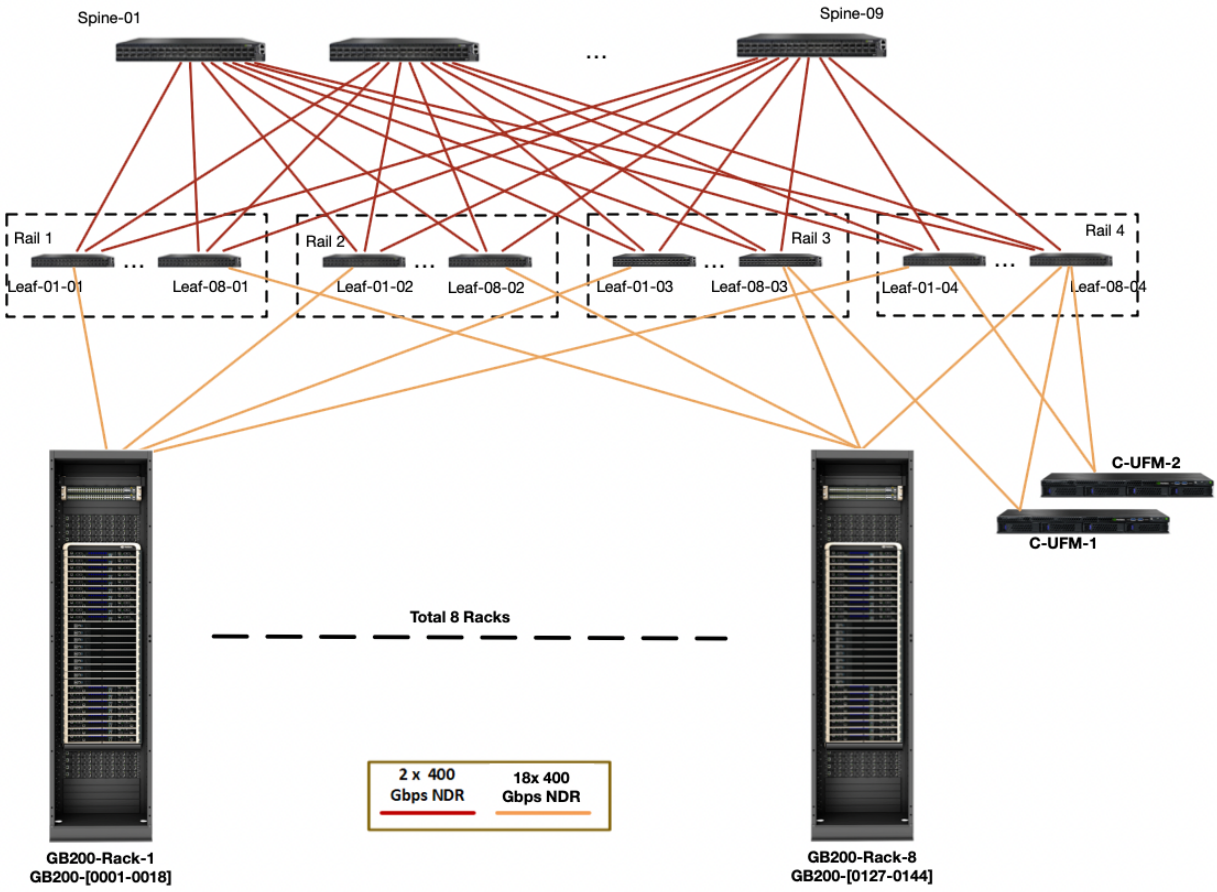


Figure 30: Figure: a diagram showing a GB200 DGX SuperPod of 576 GPUs. Each rack at the bottom layer contains 72 GB200 GPUs.

node. We have 16 spine switches, each with 3.2TB/s, giving us `16 * 3.2 = 51.2 TB/s`, which with 128 nodes is again 400GB/s.

Thus if we bisect our nodes in any way, we will have 400GB/s per node between them. Every component has exactly the requisite bandwidth to ensure the fat tree.

Question 2 [Scaling to a larger DGX pod]: Say we wanted to train on 2048 GPUs instead of 1024. What would be the simplest/best way to modify the above DGX topology to handle this? What about 4096? *Hint: there's no single correct answer, but try to keep costs down. Keep link capacity in mind. This documentation may be helpful.*

Click here for the answer.

Answer: One option would be to keep the SU structure intact (32 nodes under 8 switches) and just add more of them with more top-level switches. We'd need 2x more spine switches, so we'd have 8 SUs with 32 spine switches giving us enough bandwidth.

One issue with this is that we only have 64 ports per leaf switch, and we're already using all of them in the above diagram. But instead it's easy to do 1x 400 Gbps NDR cable per spine instead of 2x, which gives the same total bandwidth but saves us some ports.

For 4096 GPUs, we actually run out of ports, so we need to add another level of indirection, that is to say, another level in the hierarchy. NVIDIA calls these "core switches", and builds a 4096 GPU cluster with 128 spine switches and 64 core switches. You can do the math to show that this gives enough bandwidth.

How Do Collectives Work on GPUs?

GPUs can perform all the same collectives as TPUs: ReduceScatters, AllGathers, AllReduces, and AllToAlls. Unlike TPUs, the way these work changes depending on whether they're performed at the node level (over NVLink) or above (over InfiniBand). These collectives are implemented by NVIDIA in the NVSHMEM and NCCL (pronounced "nickel") libraries. NCCL is open-sourced here. While NCCL uses a variety of implementations depending on latency requirements/topology (details), from here on, we'll discuss a theoretically optimal model over a switched tree fabric.

Intra-node collectives

AllGather or ReduceScatter: For an AllGather or ReduceScatter at the node level, you can perform them around a ring just like a TPU, using the full GPU-to-GPU bandwidth at each hop. Order the GPUs arbitrarily and send a portion of the array around the ring using the full GPU-to-GPU bandwidth.[75] The cost of each hop is $T_{\text{hop}} = \text{bytes}/(N * \text{GPU egress bandwidth})$, so the overall cost is

$$T_{\text{AG or RS comms}} = \frac{\text{bytes} \cdot (N-1)}{N \cdot \text{GPU egress bandwidth}} \to \frac{\text{bytes}}{\text{GPU egress bandwidth}}$$

You'll note this is exactly the same as on a TPU. For an AllReduce, you can combine an RS + AG as usual for twice the cost.

If you're concerned about latency (e.g. if your array is very small), you can do a tree reduction, where you AllReduce within pairs of 2, then 4, then 8 for a total of $\log(N)$ hops instead of $N - 1$, although the total cost is still the same.

Takeaway: the cost to AllGather or ReduceScatter an array of B bytes within a single node is about $T_{\text{comms}} = B * (8-1)/(8 * W_{\text{GPU egress}}) \approx B/W_{\text{GPU egress}}$. This is theoretically around B/450e9 on an H100 and B/900e9 on a B200. An AllReduce has 2x this cost unless in-network reductions are enabled.

Pop Quiz 1 [AllGather time]: Using an 8xH100 node with 450 GB/s full-duplex bandwidth, how long does AllGather(bf16[BX, F]) take? Let $B = 1024$, $F = 16,384$.

Click here for the answer.

[75] You can also think of each GPU sending its chunk of size bytes/N to each of the other $N-1$ GPUs, for a total of $(N-1)*N*bytes/N$ bytes communicated, which gives us the same answer.

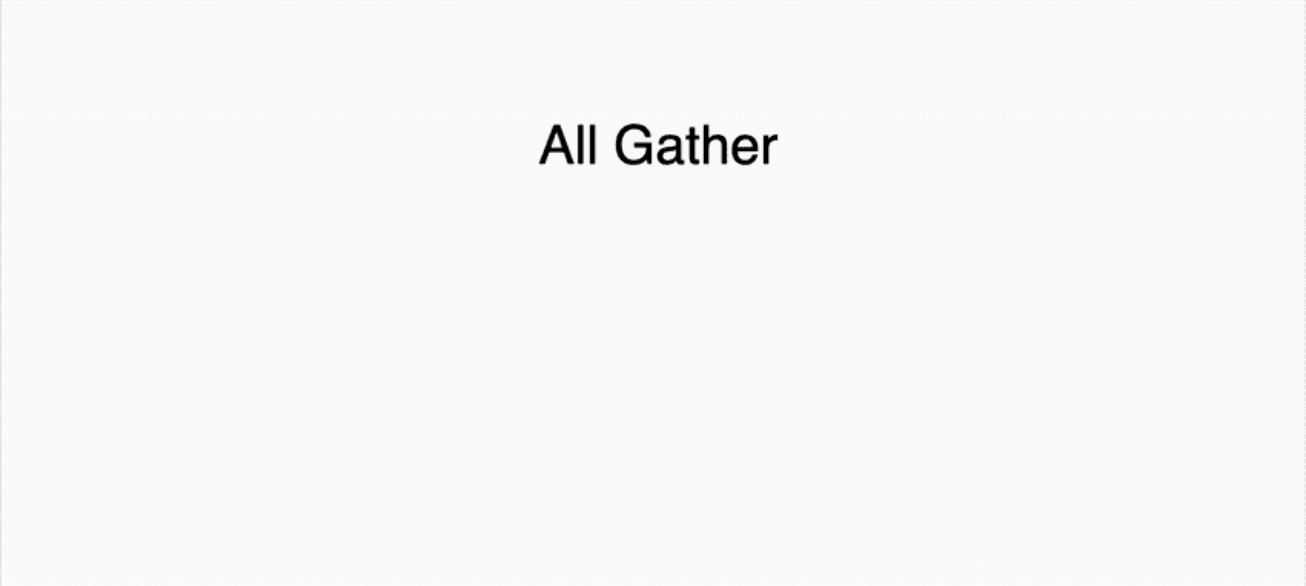


Figure 31: Figure: bandwidth-optimal 1D ring AllGather algorithm. For B bytes, this sends B / X bytes over the top-level switches X - 1 times.

Answer: We have a total of $2 \cdot B \cdot F$ bytes, with 450e9 unidirectional bandwidth. This would take roughly $T_{\text{comms}} = (2 \cdot B \cdot F)/450\text{e}9$, or more precisely $(2 \cdot B \cdot F \cdot (8-1))/(8 \cdot 450\text{e}9)$. Using the provided values, this gives us roughly $(2 \cdot 1024 \cdot 16384)/450\text{e}9 = 75\text{us}$, or more precisely, 65us.

AllToAlls: GPUs within a node have all-to-all connectivity, which makes AllToAlls, well, quite easy. Each GPU just sends directly to the destination node. Within a node, for B bytes, each GPU has B/N bytes and sends (B/N^2) bytes to $N-1$ target nodes for a total of

$$T_{\text{AllToAll comms}} = \frac{B \cdot (N-1)}{W \cdot N^2} \approx \frac{B}{W \cdot N}$$

Compare this to a TPU, where the cost is $B/(4W)$. Thus, within a single node, we get a 2X theoretical speedup in runtime ($B/4W$ vs. $B/8W$).

For Mixture of Expert (MoE) models, we frequently want to do a *sparse or ragged AllToAll,* where we guarantee at most k of N shards on the output dimension are non-zero, that is to say $T_{\text{AllToAll}} \to K[B, N]$ where at most k of N entries on each axis are non-zero. The cost of this is reduced by k/N, for a total of about $\min(k/N, 1) \cdot B/(W \cdot N)$. For an MoE, we often pick the non-zero values independently at random, so there's some chance of having fewer than k non-zero, giving us approximately $(N-1)/N \cdot \min(k/N, 1) \cdot B/(W \cdot N)$.[76]

Pop Quiz 2 [AllToAll time]: Using an 8xH100 node with 450 GB/s unidirectional bandwidth, how long does AllToAllX->N(bf16[BX, N]) take? What if we know only 4 of 8 entries will be non-zero?

Click here for the answer.

Answer: Note that here B is the batch dimension of the array, so the total array size is $V = 2 \cdot B \cdot N$ bytes. From the above, we know that in the dense case, the cost is $V \cdot (N-1)/(W \cdot N^2)$, or roughly $V/(W \cdot N)$. If we know only $\frac{1}{2}$ the entries will be non-padding, we can send $V \cdot k/N/(W \cdot N) = V/(2 \cdot W \cdot N)$, roughly half the overall cost.

Takeaway: The cost of an AllToAll on an array of B bytes on GPU within a single node is about $T_{\text{comms}} = (B \cdot (8-1))/(8^2 \cdot W_{\text{GPU egress}}) \approx B/(8 \cdot W_{\text{GPU egress}})$. For a ragged (top-$k$) AllToAll, this is decreased further to $(B \cdot k)/(64 \cdot W_{\text{GPU egress}})$.

[76] The true cost is actually

$$(1 - \left(\frac{Z-1}{Z}\right)^K) \cdot \frac{Z-1}{Z}$$

the expected number of distinct outcomes in K dice rolls, but it is very close to the approximation given. See the Appendix for more details.

Empirical measurements: here is an empirical measurement of AllReduce bandwidth over an 8xH100 node. The Algo BW is the measured bandwidth (bytes / runtime) and the Bus BW is calculated as $2 \cdot W \cdot (8-1)/8$, theoretically a measure of the actual link bandwidth. You'll notice that we do achieve close to 370GB/s, less than 450GB/s but reasonably close, although only around 10GB/device. This means although these estimates are theoretically correct, it takes a large message to realize it.

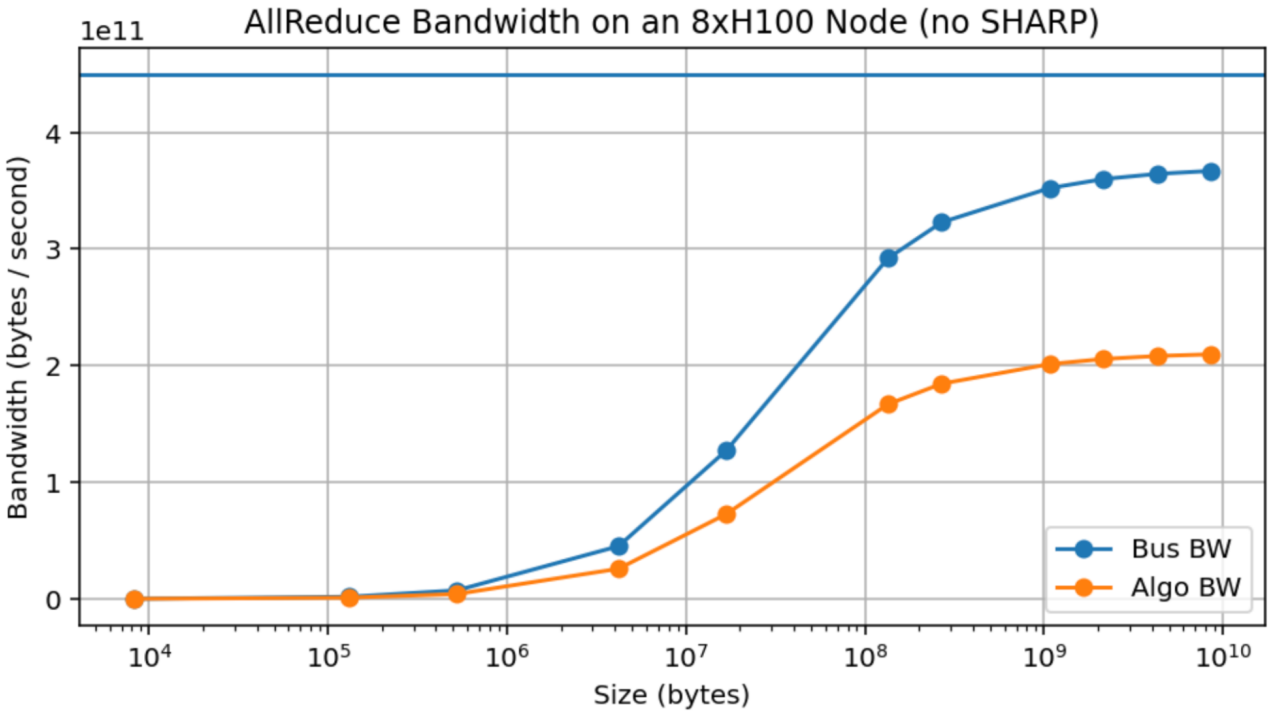


Figure 32: Figure: AllReduce throughput for an 8xH100 node with SHARP disabled. The blue curve is the empirical link bandwidth, calculated as $2 * \text{bytes} * (N-1)/(N * \text{runtime})$ from the empirical measurements. Note that we do not get particularly close to the claimed bandwidth of 450GB/s, even with massive 10GB arrays.

This is a real problem, since it meaningfully complicates any theoretical claims we can make, since e.g. even an AllReduce over a reasonable sized array, like LLaMA-3 70B's MLPs (of size `bf16[8192, 28672]`, or with 8-way model sharding, `bf16[8192, 3584] = 58MB`) can achieve only around 150GB/s compared to the peak 450GB/s. By comparison, TPUs achieve peak bandwidth at much lower message sizes (see Appendix B).

Takeaway: although NVIDIA claims bandwidths of about 450GB/s over an H100 NVLink, it is difficult in practice to exceed 370 GB/s, so adjust the above estimates accordingly.

In network reductions: Since the Hopper generation, NVIDIA switches have supported "SHARP" (Scalable Hierarchical Aggregation and Reduction Protocol) which allows for "in-network reductions". This means *the network switches themselves* can do reduction operations and multiplex or "MultiCast" the result to multiple target GPUs:

Theoretically, this close to halves the cost of an AllReduce, since it means each GPU can send its data to a top-level switch which itself performs the reduction and broadcasts the result to each GPU without having to egress each GPU twice, while also reducing network latency.

$$T_{\text{SHARP AR comms}} = \frac{\text{bytes}}{\text{GPU egress bandwidth}}$$

Note that this is exact and not off by a factor of $1/N$, since each GPU egresses $B \cdot (N-1)/N$ first, then receives the partially reduced version of its local shard (ingress of B/N), finishes the reductions, then egresses B/N again, then ingresses the fully reduced result (ingress of $B \cdot (N-1)/N$), resulting in exactly B bytes ingressed.

However, in practice we see about a 30% increase in bandwidth with SHARP enabled, compared to the predicted

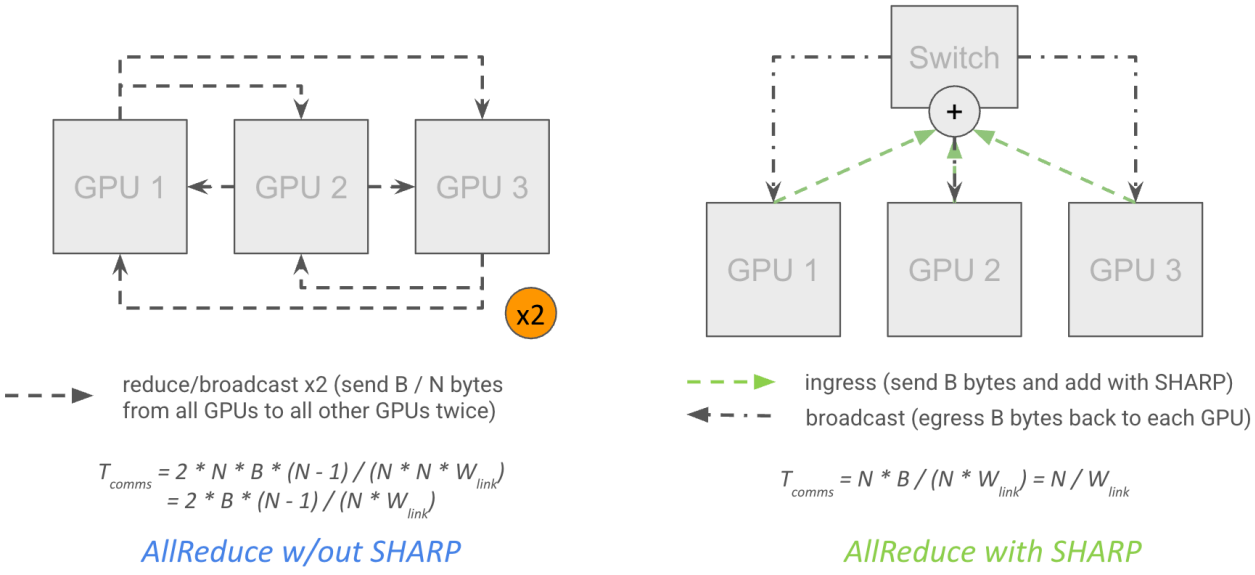


Figure 33: Figure: an AllReduce without SHARP has 2x the theoretical cost because it has to pass through each GPU twice. In practice, speedups are only about 30% (from NCCL 2.27.5).

75%. This gets us up merely to about 480GB/s effective collective bandwidth, not nearly 2x.

Takeaway: in theory, NVIDIA SHARP (available on most NVIDIA switches) should reduce the cost of an AllReduce on B bytes from about $2*B/W$ to B/W. However, in practice we only see a roughly 30% improvement in bandwidth. Since pure AllReduces are fairly rare in LLMs, this is not especially useful.

Cross-node collectives

When we go beyond the node-level, the cost is a bit more subtle. When doing a reduction over a tree, you can think of reducing from the bottom up, first within a node, then at the leaf level, and then at the spine level, using the normal algorithm at each level. For an AllReduce especially, you can see that this allows us to communicate less data overall, since after we AllReduce at the node level, we only have to egress B bytes up to the leaf instead of $B * N$.

How costly is this? To a first approximation, because we have full bisection bandwidth, the cost of an AllGather or ReduceScatter is roughly the buffer size in bytes divided by the node egress bandwidth (400GB/s on H100) *regardless of any of the details of the tree reduction.*

$$T_{\text{AG or RS comms}} = \frac{\text{bytes}}{W_{\text{node egress}}} \underset{H100}{=} \frac{\text{bytes}}{400\text{e}9}$$

where W_{node} egress is generally 400GB/s for the above H100 network (8x400Gbps IB links egressing each node). The cleanest way to picture this is to imagine doing a ring reduction over *every node in the cluster*. Because of the fat tree topology, we can always construct a ring with W_{node} egress between any two nodes and do a normal reduction. The node-level reduction will (almost) never be the bottleneck because it has a higher overall bandwidth and better latency, although in general the cost is

$$T_{\text{total}} = \max(T_{\text{comms at node}}, T_{\text{comms in scale-out network}}) = \max\left[\frac{\text{bytes}}{W_{\text{GPU egress}}}, \frac{\text{bytes}}{W_{\text{node egress}}}\right]$$

You can see a more precise derivation here.

We can be more precise in noting that we are effectively doing a ring reduction at each layer in the network, which we can mostly overlap, so we have:

$$T_{\text{AG or RS comms}} = \text{bytes} \cdot max_{\text{depth i}}\left[\frac{D_i - 1}{D_i \cdot W_{\text{link i}}}\right]$$

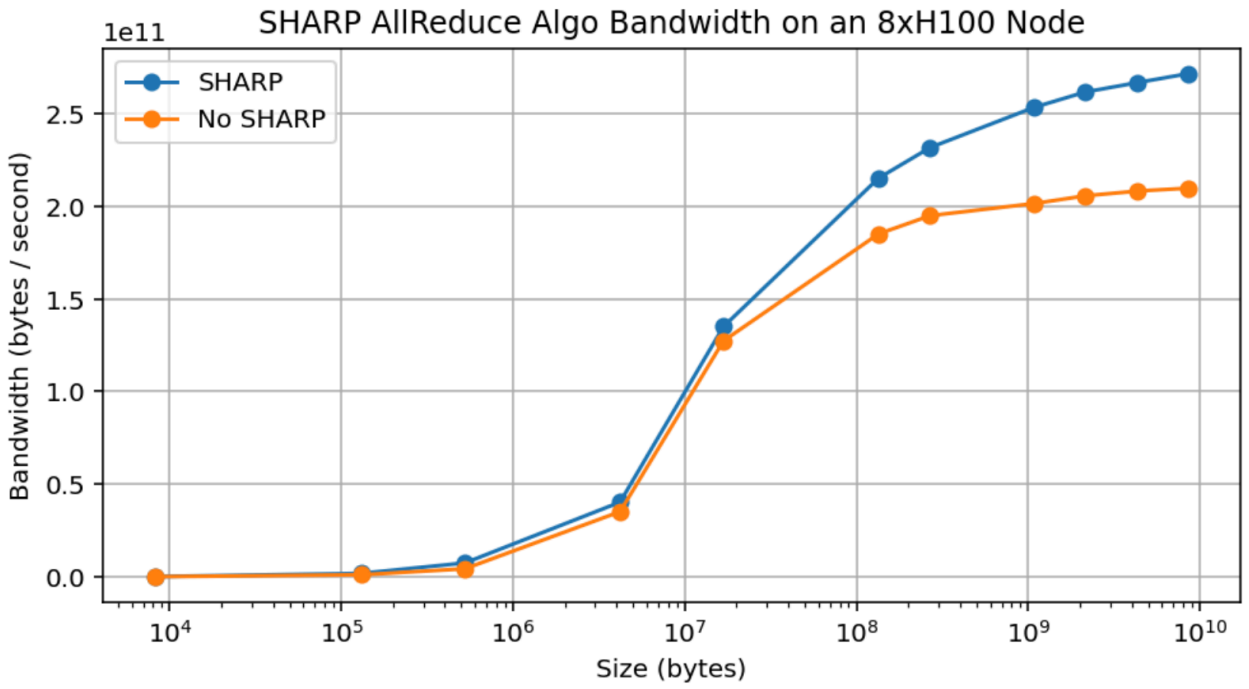


Figure 34: Figure: empirical measurements of AllReduce algo bandwidth with and without NVIDIA SHARP enabled within a node. The gains amount to about 30% throughput improvement at peak, even though algorithmically it ought to be able to achieve closer to a 75% gain.

where D_i is the degree at depth i (the number of children at depth i), $W_{\text{link i}}$ is the bandwidth of the link connecting each child to node i.

Using this, we can calculate the available AllGather/AllReduce bandwidth as $min_{\text{depth i}}(D_i * W_{\text{link i}}/(D_i - 1))$ for a given topology. In the case above, we have:

- **Node:** $D_{\text{node}} = 8$ since we have 8 GPUs in a node with Wlink i = 450GB/s. Thus we have an AG bandwidth of `450e9 * 8 / (8 - 1) = 514GB/s`.
- **Leaf:** $D_{\text{leaf}} = 32$ since we have 32 nodes in an SU with Wlink i = 400GB/s (8x400Gbps IB links). Thus our bandwidth is `400e9 * 32 / (32 - 1) = 413GB/s`.
- **Spine:** $D_{\text{spine}} = 4$ since we have 4 SUs with $W_{\text{link i}} = 12.8$TB/s (from `8 * 16 * 2 * 400Gbps` links above). Our bandwidth is `12.8e12 * 4 / (4 - 1) = 17.1TB/s`.

Hence our overall AG or RS bandwidth is `min(514GB/s, 413GB/s, 17.1TB/s) = 413GB/s` at the leaf level, so in practice $T_{\text{AG or RS comms}} = B/413\text{GB/s}$, i.e. we have about 413GB/s of AllReduce bandwidth even at the highest level. For an AllReduce with SHARP, it will be slightly lower than this (around 400GB/s) because we don't have the $(N-1)/N$ factor. Still, 450GB/s and 400GB/s are close enough to use as approximations.

Other collectives: AllReduces are still 2x the above cost unless SHARP is enabled. NVIDIA sells SHARP-enabled IB switches as well, although not all providers have them. AllToAlls do change quite a bit cross-node, since they aren't "hierarchical" in the way AllReduces are. If we want to send data from every GPU to every other GPU, we can't use take advantage of the full bisection bandwidth at the node level. That means if we have an N-way AllToAll that spans $M = N/8$ nodes, the cost is

$$T_{\text{AllToAll comms}} = \frac{B \cdot (M-1)}{M^2 \cdot W_{\text{node egress}}} \approx \frac{B}{M \cdot W_{\text{node egress}}}$$

which effectively has 50GB/s rather than 400GB/s of bandwidth. We go from $B/(8 * 450\text{e}9)$ within a single H100 node to $B/(2 \cdot 400\text{e}9)$ when spanning 2 nodes, a more than 4x degradation.

Here is a summary of the 1024-GPU DGX H100 SuperPod architecture:

Level	Number of GPUs	Degree (# Children)	Switch Bandwidth (full-duplex, TB/s)	Cable Bandwidth (full-duplex, TB/s)	Collective Bandwidth (GB/s)
Node	8	8	6.4	3.6	450
Leaf (SU)	256	32	25.6	12.8	400
Spine	1024	4	51.2	51.2	400

We use the term "Collective Bandwidth" to describe the effective bandwidth at which we can egress either the GPU or the node. It's also the bisection bandwidth $* 2/N$.

Takeaway: beyond the node level, the cost of an AllGather or ReduceScatter on B bytes is roughly $B/W_{\text{node egress}}$, which is $B/400e9$ on an H100 DGX SuperPod, while AllReduces cost twice as much unless SHARP is enabled. The overall topology is a fat tree designed to give constant bandwidth between any two pairs of nodes.

Reductions when array is sharded over a separate axis: Consider the cost of a reduction like

$\text{AllReduce}_X(A[I_Y, J]\ \{U_X\})$

where we are AllReducing over an array that is itself sharded along another axis Y. On TPUs, the overall cost of this operation is reduced by a factor of $1/Y$ compared to the unsharded version since we're sending $1/Y$ as much data per axis. On GPUs, the cost depends on which axis is the "inner" one (intra-node vs. inter-node) and whether each shard spans more than a single node. Assuming Y is the inner axis, and the array has bytes total bytes, the overall cost is reduced effectively by Y, but only if Y spans multiple nodes:

$T_{\text{comms at node}} = \frac{\text{bytes}}{W_{\text{GPU egress}}} \cdot \frac{1}{\min(Y, D_{\text{node}})}$

$T_{\text{comms in scale-out network}} = \frac{\text{bytes}}{W_{\text{node egress}}} \cdot \frac{D_{\text{node}}}{\max(D_{\text{node}}, Y)}$

$T_{\text{total}} = \max(T_{\text{comms at node}}, T_{\text{comms in scale-out network}})$

where N is the number of GPUs and again D_{node} is the number of GPUs in a node (the degree of the node). As you can see, if $Y < D_{\text{node}}$, we get a win at the node level but generally don't see a reduction in overall runtime, while if $Y > D_{\text{node}}$, we get a speedup proportional to the number of nodes spanned.

If we want to be precise about the ring reduction, the general rule for a tree AllGatherX(AY { UX }) (assuming Y is the inner axis) is

$T_{\text{AR or RS comms}} = \text{bytes} \cdot \max_{\text{depth } i} \left[\frac{D_i - 1}{D_i \cdot \max(Y, S_{i-1}) \cdot W_{\text{link } i}}\right]$

where S_i is M * N * …, the size of the subnodes below level i in the tree. This is roughly saying that the more GPUs or nodes we span, the greater our available bandwidth is, but only within that node.

Pop Quiz 3 [Sharding along 2 axes]: Say we want to perform $\text{AllGather}_X(\text{bf16}[D_X, F_Y])$ where Y is the inner axis over a single SU (256 chips). How long will this take as a function of D, F, and Y?

Click here for the answer.

Answer: We can break this into two cases, where Y <= 8 and when Y > 8. When $Y <= 8$, we remain bounded by the leaf switch, so the answer is, as usual, $T_{\text{comms}} = 2 * D * F * (32 - 1)/(32 * 400e9)$. When Y > 8, we have from above, roughly

$T_{\text{comms}} = \frac{2 \cdot D \cdot F \cdot 256}{Y \cdot 12.8\text{e}12} = \frac{2DF}{Y \cdot 50\text{GB/s}}$

For `D = 8192`, `F = 32,768`, we have:

Note how, if we do exactly 8-way model parallelism, we do in fact reduce the cost of the node-level reduction by 8 but leave the overall cost the same, so it's free but not helpful in improving overall bandwidth.

Takeaway: when we have multiple axes of sharding, the cost of the outer reduction is reduced by a factor of the number of nodes spanned by the inner axis.

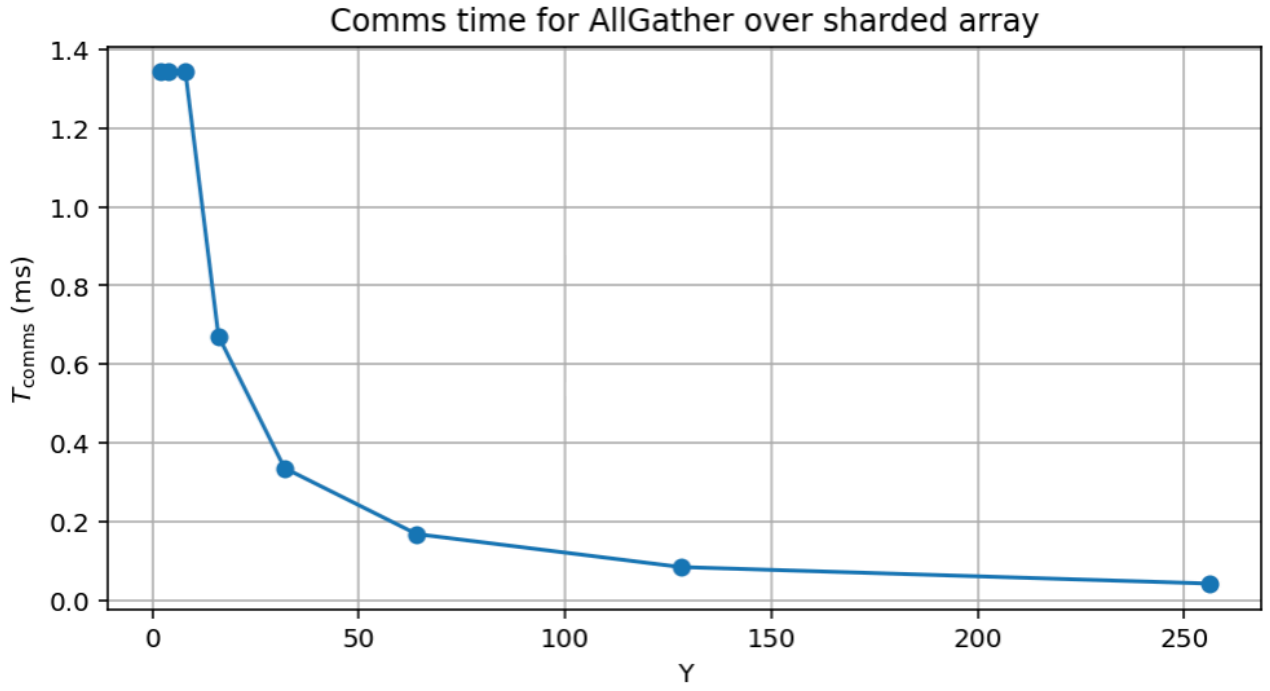


Figure 35: Figure: theoretical cost of a sharded AllGather as the inner axis spans more nodes.

Quiz 4: Collectives

Question 1 [SU AllGather]: Consider only a single SU with M nodes and N GPUs per node. Precisely how many bytes are ingressed and egressed by the node level switch during an AllGather? What about the top-level switch?

Click here for the answer.

Answer: Let's do this step-by-step, working through the components of the reduction:

1. Each GPU sends B/MN bytes to the switch, for a total ingress of $NB/MN = B/M$ bytes ingress.
2. We egress the full B/M bytes up to the spine switch.
3. We ingress $B * (M-1)/M$ bytes from the spine switch
4. We egress $B - B/MN$ bytes N times, for a total of $N * (B - B/MN) = NB - B/M$.

The total is B ingress and BN egress, so we should be bottlenecked by egress, and the total time would be $T_{\text{AllGather}} = BN/W_{\text{node}} = B/450\text{e}9$.

For the spine switch, the math is actually simpler. We must have B/M bytes ingressed M times (for a total of B bytes), and then $B(M-1)/M$ egressed M times, for a total of $B * (M-1)$ out. Since this is significantly larger, the cost is $T_{\text{AllGather}} = B \cdot (M-1)/(M \cdot W_{\text{node}}) = B \cdot (M-1)/(M \cdot 400\text{e}9)$.

Question 2 [Single-node SHARP AR]: Consider a single node with N GPUs per node. Precisely how many bytes are ingressed and egressed by the switch during an AllReduce using SHARP (in-network reductions)?

Click here for the answer.

Answer: As before, let's do this step-by-step.

1. Each GPU sends $B * (N-1)/N$ bytes, so we have $N * B * (N-1)/N = B * (N-1)$ ingressed.
2. We accumulate the partial sums, and we send back B/N bytes to each GPU, so $N * B/N = B$ bytes egressed.
3. We do a partial sum on the residuals locally, then send this back to the switch. This is a total of $N * B/N = B$ bytes ingressed.

4. We capture all the shards and multicast them, sending $B*(N-1)/N$ to N destinations, for a total of $B*(N-1)/N*N = B*(N-1)$ egressed.

Therefore the total is $B*(N-1)+B = BN$ bytes ingressed and egressed. This supports the overall throughput being exactly B/W_{egress}.

Question 3 [Cross-node SHARP AR]: Consider an array bf16[DX, FY] sharded over a single node of N GPUs. How long does AllReduce(bf16[D, FY] { UX }) take? You can assume we do in-network reductions. Explain how this differs if we have more than a single node?

Click here for the answer.

Answer: We can try to modify the answer to the previous question above. Basically, we first egress $B*(X-1)/XY$ bytes from each GPU, then send back B/XY to each GPU, then send that same amount back to the switch, then send $B*(X-1)/XY$ back to each GPU. The total is NB/Y ingress and egress, so the total time is $T_{\text{comms}} = NB/(Y*N*W_{\text{link}}) = N*2DF/(Y*N*W_{\text{link}}) = 2*D*F/(Y*W_{\text{link}})$, so the total time does decrease with Y.

If we go beyond a single node, we can do roughly the same reduction as above, but when we egress the node-level switch, we need to send all B bytes, not just B/Y. This is because we need to keep each shard separate.

Question 4 [Spine level AR cost]: Consider the same setting as above, but with $Y = 256$ (so the AR happens at the spine level). How long does the AllReduce take? Again, feel free to assume in-network reductions.

Click here for the answer.

Answer: This lets us take advantage of the rather ludicrous amount of bandwidth at the spine level. We have 51.2TB/s of bandwidth at the spine over 4 SUs, or 12.8TB/s per SU. Using SHARP, this could take as little as `2 * D * F / 12.8e12` seconds.

Question 5 [2-way AllGather cost]: Calculate the precise cost of an AllGather of B bytes over exactly 2 nodes. *Make sure to calculate the precise cost and not the approximation, and consider both the intra-node and cross-node cost.*

Click here for the answer.

Answer: At the node level, we have $T_{\text{comms}} = B*7/(8*450\text{e}9) = B/514\text{e}9$ while beyond we actually have $T_{\text{comms}} = B*(2-1)/(2*400\text{e}9) = B/800\text{e}9$. Thus, we're actually bounded by the node level reduction and not the leaf level! This motivates e.g. DeepSeek v3 which does 2-way Data Parallelism.

Rooflines for LLM Scaling on GPUs

Now let's look at what this has all been building towards: understanding rooflines for LLM scaling on GPU. This is to complement the TPU training chapter here. As we did there, the goal here is to look at the total T_{math} and T_{comms} for different parallelism strategies and understand at what point $T_{\text{comms}} > T_{\text{math}}$. As before, we consider only the MLP block with operations

$\text{MLP}(x) \equiv x[B,D] *_D W_{\text{in}}[D,F] \cdot_F W_{\text{out}}[F,D]$

where B is the global batch size **in tokens** (i.e. B = batch size $\cdot$ sequence length).

Here we'll reproduce the table above showing effective bandwidths at both the GPU and node level:

Node Type	GPUs per node	GPU egress bandwidth	Node egress bandwidth
H100	8	450e9	400e9
B200	8	900e9	400e9
GB200 NVL72	72	900e9	3600e9

Note: Both the GPU and node egress bandwidths determine rooflines for our LLMs. We'll use the term $W_{\text{collective}}$ to describe either the GPU or node bandwidths depending on whether we are operating within or above the node level.

Let's look at the compute communication rooflines as we did for TPUs for **data parallelism, tensor parallelism, pipeline parallelism, expert parallelism,** and combinations thereof. For the rest of this section we'll focus on H100 rooflines for specific calculations. GB200-NVL72 has the same general rooflines but because we have a larger node egress bandwidth, we can sometimes be bottlenecked at the node level instead.

Data Parallelism

As noted before, DP and ZeRO sharding involve either a weight AllReduce or a ReduceScatter + AllGather in the backward pass. Since these both have the same cost, to be compute-bound for pure data parallelism or FSDP *without in-network reductions*, we have, per layer, in the backward pass, with an axis of size X:

$T_{\text{math}} = \frac{2 \cdot 2 \cdot 2 \cdot BDF}{X \cdot C}$

$T_{\text{comms}} = \frac{2 \cdot 2 \cdot 2 \cdot DF}{W_{\text{collective}}}$

Therefore, for $T_{\text{math}} > T_{\text{comms}}$, we need $B/(XC) > 1/W_{\text{collective}}$ or

$\frac{B}{X} > \frac{C}{W_{\text{collective}}}$

where $W_{\text{collective}}$ is either the GPU or node level egress bandwidth depending on whether we're sharding within a node or across nodes. Thus:

- **Within a node**, we just need the per-GPU **token** batch size > 990e12/450e9 = 2200.
- **Within an SU or at the spine level**, BS > 990e12/400e9 = 2475.

This is quite a bit higher than on a TPU, where the number is 850 with all three axes. For instance, LLaMA-3, which trained on 16000 H100s would need a batch size of at least 40M tokens (for reference, they used 16M). DeepSeek v3 trained on 2048 H800 GPUs with lower 300GB/s of bandwidth (instead of 450GB/s on H100) would need 990e12/300e9 = 3300 tokens per GPU, or about 6.7M (in practice, they used 4M).

With in-network reductions enabled and using pure data parallelism, theoretically we have 2x the AllReduce bandwidth, which would halve both of these numbers. However, in practice the benefit is closer to 30%, which only really makes up for the fact that we typically struggle to reach the reported numbers. Furthermore, because pure data parallelism is rarely useful, this basically doesn't matter in practice.

MoE models: For a Mixture of Experts (MoE) model, where we have E experts and k experts per token, this increases to

$T_{\text{math}} = \frac{2 \cdot 2 \cdot 2 \cdot k \cdot BDF}{X \cdot C}$

$T_{\text{comms}} = \frac{2 \cdot 2 \cdot 2 \cdot EDF}{W_{\text{collective}}}$

which inflates the per-GPU token batch size by a factor of E/k, i.e.

$\frac{B}{X} > \frac{E}{k} \frac{C}{W_{\text{collective}}}$

For example, the new OpenAI OSS model with $k = 4$ and $E = 128$, this increases to `32 * 2475 = 79,200` across nodes, a kind of ridiculously high number.

What happens when X is small? When we do only e.g. 2-node data parallelism, we benefit from the $(X-1)/X$ scaling, which gives us

$T_{\text{math}} = \frac{2 \cdot 2 \cdot 2 \cdot BDF}{N * C}$

$T_{\text{comms}} = \frac{2 \cdot 2 \cdot 2 \cdot DF \cdot (X-1)}{X \cdot W_{\text{collective}}}$

where X is the number of nodes and $N = 8 \cdot X$. Then for a dense model we have $B/N > \alpha \cdot (X-1)/X$, or e.g. $B/N > 1237$, half the above value. You'll notice 2-way data parallelism fairly often for this reason.

Takeaway: Data parallelism and ZeRO sharding require a per-GPU batch size of about 2500 tokens to be compute-bound on an H100 or B200, assuming perfect overlap and FLOPs utilization. For MoE models, this increases by a factor of E/k, the ratio of total to activated parameters. When doing a small amount of data parallelism, the critical batch size decreases.

Tensor Parallelism

Tensor parallelism requires an AllGather and ReduceScatter over the activations, which we need to overlap with the MLP FLOPs. In other words, in the forward pass, we have

$T_{\text{math}} = \frac{2 \cdot 2 \cdot BDF}{Y \cdot C}$

$T_{\text{comms}} = \frac{2 \cdot 2 \cdot BD}{W_{\text{collective}}}$

which to be compute-bound gives us the rule

$Y < \frac{F \cdot W_{\text{collective}}}{C}$

Within a node, this gives us about $F/2200$ or $F/2475$ beyond a node. For $F = 28000$ like LLaMA-3, this is about 11-way TP (or rounding down, about 8-way, which is how large a node is). As with above, we get an extra 2X bandwidth when we span exactly 2 nodes, so we can generally do 16-way tensor parallelism ($F > 2475 \cdot (Y - 8)$), which gives us up to 19-way model parallelism in theory.

Takeaway: Tensor parallelism over an axis of size Y with feed-forward dimension F becomes communication-bound when the $Y > F/2475$, which generally constrains us to only intra-node TP or at most 2-node TP.

Expert Parallelism

As we've already noted above, Mixture of Expert (MoE) models come with E times more model weights with only k times more FLOPs, making data parallelism significantly harder. We can mitigate this somewhat by sharding our weights along the expert dimension, i.e. Win[EZ, D, F]. To do the MLP block, we need to introduce 2x AllToAll to send our activations to the corresponding experts.

As noted above, the cost of this AllToAllZ->k([B, D, k]) if it spans multiple nodes is roughly $T_{\text{AllToAll}} = 2 \cdot B \cdot D \cdot (Z - 8)/Z \min(8 * k/Z, 1)$, so for pure expert parallelism we need

$T_{\text{math}} = \frac{4 \cdot B \cdot k \cdot D \cdot F}{Z \cdot C}$

$T_{\text{comms}} = \frac{4 \cdot B \cdot D \cdot (Z-8)}{W \cdot Z} \cdot \min\left(\frac{8 \cdot k}{Z}, 1\right)$

We either need $K > Z/8$ with $F > \alpha \cdot (Z - 8)/k$ or $Z \gg K$ and $F > 8 \cdot \alpha$, where $\alpha = C/W$. This gives you two domains in which expert parallelism is possible, one with a small amount of expert parallelism (roughly 2-node) and small F, or one with large F and Z arbitrarily large (up to E-way expert parallelism).

You'll see both cases in practice, either a small amount of expert-parallelism (like DeepSeek v3 which has very small F and relatively small, restricted cross-node expert parallelism), or models with large F, in which case we can do significant cross-node EP alongside TP.

Takeaway: if $F < 8 * C/W_{\text{node}}$, expert parallelism can span 1-2 nodes with similar (slightly lower) cost to TP, or if $F > 8 * C/W_{\text{node}}$, we can do a significant amount of expert parallelism (up to E nodes) with relatively low cost.

Pipeline Parallelism

Pipeline parallelism splits layers across nodes with an extremely low communication cost, since we are just sending small microbatches of activations every couple layers. Historically pipelining has suffered from "pipeline bubbles", but with new zero-bubble pipelining approaches, it is typically possible to do without.

The overall communication cost of pipelining is tiny: with N_{MB} microbatches and N_{stages}, we have $T_{\text{comms per hop}} = 2 \cdot B \cdot D/(W \cdot N_{\text{MB}})$ and $N_{\text{MB}} + N_{\text{stages}} - 2$ hops, so roughly

$T_{\text{total PP comms}} = \frac{2BD}{W \cdot N_{\text{MB}}} \cdot (N_{\text{MB}} + N_{\text{stages}} - 2)$

$T_{\text{per-layer comms}} \approx 1.5 \cdot \frac{2BD}{W \cdot N_{\text{layers}}}$

Since we are dividing by N_{layers}, this is vastly smaller than any of the other costs. In other words, from a communication standpoint, pipelining is basically free. So why don't we just do pipelining? There are a few reasons:

(1) **Code complexity:** pipelining doesn't fit as nicely into automatic parallelism frameworks (like XLA's GSPMD) as other approaches. Because it introduces microbatching to hide pipeline bubbles, it changes the structure of the program, and custom zero-bubble pipeline schedules exacerbate this problem by requiring complicated interleaving of the forward and backward pass.

(2) **Pipelining makes data parallelism and FSDP hard:** probably the biggest reason not to do pipelining is that it plays badly with FSDP and data parallelism. ZeRO-3 sharding in particular works badly, since it requires us to AllGather the weights on every microbatch which doesn't work when we have only $B/N_{\text{microbatches}}$ tokens to amortize the AllGather cost. Furthermore, during the backward pass, *we can't AllReduce or ReduceScatter the gradients until the last microbatch has passed a given stage, which means we have significant non-overlapped communication time.*

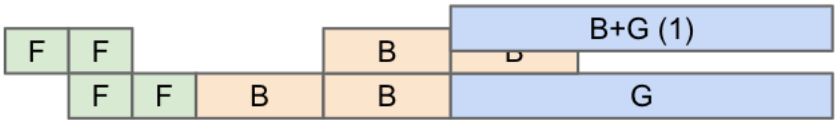


Figure 36: Figure: an example 2 stage, 2 microbatch pipeline. F denotes a stage forward pass and B is a stage backward pass (2x the cost). G denotes the data-parallel AllReduces, which can be significantly longer than the time of a single microbatch.

(3) **Pipeline bubbles and step imbalance:** As you can see in the (bad) pipeline schedule above, it is easy to have significant bubbles (meaning wasted compute) during a naive pipeline schedule. Above, the second stage is idle on step 0, the first stage is idle from step 2 to 3, and the second stage is again idle on the last step. While we can avoid these somewhat with careful scheduling, we still often have some bubbles. We also have to pass activations from one stage to the next on the critical path, which can add overhead:

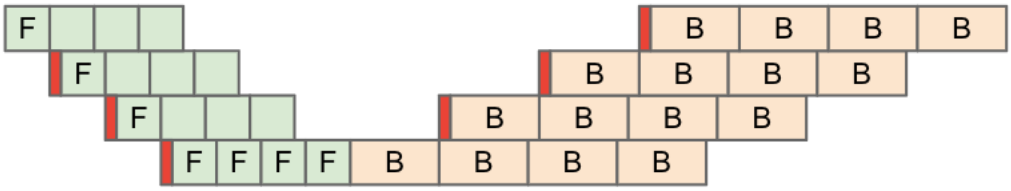


Figure 37: Figure: an example pipeline showing transfer cost in red. This shifts stages relative to each other and increases the pipeline bubble overhead.

There are workarounds for each of these issues, but they tend to be complicated to implement and difficult to maintain; pipelining remains a technique with low communication cost relative to other methods.

Caveat about latency: As noted before, GPUs struggle to achieve full AllReduce bandwidth even with fairly large messages. This means even if we in theory can scale e.g. expert-parallel AllToAlls across multiple nodes, we may struggle to achieve even 50% of the total bandwidth. This means we do try to keep TP or EP within a smaller number of nodes to minimize latency overhead.

Examples

What does DeepSeek do? For reference, DeepSeek V3 is trained with 2048 H800 GPUs with:

- 64-way Expert Parallelism (EP) spanning 8 nodes
- 16-way Pipeline Parallelism (PP)
- 2-way ZeRO-1 Data Parallelism (DP)

They had a steady state batch size of `4096 * 15360 = 62,914,560` tokens, or 30k tokens per GPU. You can see that this is already quite large, but their model is also very sparse (k=8, E=256) so you need a fairly large batch size. You can see that with 64-way EP and 16-way PP, we end up with 1024-way model parallelism in total, which

means the AllReduce is done at the spine level, and because it's only 2-way, we end up with $2/(2-1) = 2$ times more bandwidth in practice. This also helps reduce the cost of the final data-parallel AllReduce overlapping with the final pipeline stages.

What does LLaMA-3 do? LLaMA-3 trains with a BS of 16M tokens on 16k GPUs, or about 1k tokens per GPU. They do:

- 8-way Tensor Parallelism within a node (TP)
- 16-way Pipeline Parallelism (PP)
- 128-way ZeRO-1 Data Parallelism

This is also a dense model so in general these things are pretty trivial. The 16-way PP reduces the cost of the data parallel AllReduce by 16x, which helps us reduce the critical batch size.

TLDR of LLM Scaling on GPUs

Let's step back and come up with a general summary of what we've learned so far:

- **Data parallelism or FSDP (ZeRO-1/3) requires a local batch size of about 2500 tokens per GPU**, although in theory in-network reductions + pure DP can reduce this somewhat.
- **Tensor parallelism is compute-bound up to about 8-ways** but we lack the bandwidth to scale much beyond this before becoming comms-bound. This mostly limits us to a single NVLink domain (i.e. single-node or need to use GB200NVL72 with up to 72 GPUs).
- **Any form of model parallelism that spans multiple nodes can further reduce the cost of FSDP**, so we often want to mix PP + EP + TP to cross many nodes and reduce the FSDP cost.
- **Pipeline parallelism works well if you can handle the code complexity of zero-bubble pipelining and keep batch sizes fairly large to avoid data-parallel bottlenecks.** Pipelining usually makes ZeRO-3 impossible (since you would need to AllGather on each pipeline stage), but you can do ZeRO-1 instead.

At a high level, this gives us a recipe for sharding large models on GPUs:

- For relatively small dense models, aggressive FSDP works great if you have the batch size, possibly with some amount of pipelining or tensor parallelism if needed.
- For larger dense models, some combination of 1-2 node TP + many node PP + pure DP works well.
- For MoEs, the above rule applies but we can also do expert parallelism, which we prefer to TP generally. If $F > 8 * C/W_{\text{node}}$, we can do a ton of multi-node expert parallelism, but otherwise we're limited to roughly 2-node EP.

Quiz 5: LLM rooflines

Question 1 [B200 rooflines]: A B200 DGX SuperPod (**not GB200 NVL72**) has 2x the bandwidth within a node (900GB/s egress) but the same amount of bandwidth in the scale-out network (400GB/s) (source). The total FLOPs are reported above. How does this change the model and data parallel rooflines?

Click here for the answer.

Answer: Our FLOPs/s in bfloat16 increases from 990 to 2250 TFLOPs, a 2.25x increase. With 2x the bandwidth, within a node, our rooflines stay roughly the same. For TP, for example, the critical intensity goes up to `2250e12 / 900e9 = 2500`, so we have a limit of $Y < F/2500$, only slightly higher (and this doesn't help us unless the node size increases).

Beyond a node, however, the lack of additional bandwidth actually makes it even harder for us to be compute-bound! For instance, for data parallelism, our critical batch size increases to `2250e12 / 400e9 = 5625`, because our GPU can do significantly more FLOPs with the same bandwidth.

GB200 SuperPods with 72-GPU nodes change this by adding more egress bandwidth (source).

Question 2 [How to shard LLaMA-3 70B]: Consider LLaMA-3 70B, training in bfloat16 with fp32 optimizer state with Adam.

1. At a minimum, how many H100s would we need simply to store the weights and optimizer?

2. Say we want to train on 4096 H100 GPUs for 15T tokens. Say we achieved 45% MFU (Model FLOPs Utilization). How long would it take to train?
3. LLaMA-3 70B has `F = 28,672` and was trained with a batch size of about 4M tokens. What is the most model parallelism we could do without being comms-bound? With this plus pure DP, could we train LLaMA-3 while staying compute-bound on 4k chips? What about ZeRO-3? What about with 8-way pipelining? *Note: consider both the communication cost and GPU memory usage.*

Click here for the answer.

1. We need 2 bytes for the weights and 8 for the optimizer state, so at least 700GB. With 80GB of DRAM, we'll need at least 9 GPUs at a minimum, or (rounding up) at least 2 8xH100 nodes. This would take forever to train and wouldn't hold the gradient checkpoints, but it's a lower bound.
2. This will require a total of `6 * 70e9 * 15e12 = 6.3e24 bf16 FLOPs`. Each GPU can do `990e12` FLOPs, so at 45% MFU we can do 1.8e18 FLOPs/s. Thus the whole thing will take 3.5e6 seconds, or 40 days.
3. Within a node, we have 450GB/s of bandwidth, so the limit is roughly `F / 1995 = 28672 / 1995 = 14.372`. Since this doesn't span 2 nodes, it realistically means we'd go up to 8-way model parallelism.
    1. This would then require us to do 512 way DP. Firstly, we need to see if we have enough memory. Since our model is only sharded 8-ways, this would mean `700GB / 8 = 87.5GB / GPU`, which won't fit, so no!
    2. With ZeRO-3 and 8-way TP, we'll be doing 512-way ZeRO-3. This won't have any issue with memory because we're sharding everything aggressively. We'll have a per-GPU batch size of `4e6 / 4096 = 976`. This is quite low, even below our pure DP limit, and this is twice that limit because we have to move our weights. So no.
    3. With 8-way pipelining, each model parallel shard now spans 8 nodes. As we've seen, this reduced the cost of our leaf-level AllGathers by 8, so the overall AllReduce/AllGather bandwidth there goes from 400GB/s to `8 * 400GB/s = 3200GB/s`. The roofline then is `990e12 / 3200e9 = 309`, so we should be good! We just need to implement pipelining efficiently.

Question 3 [Megatron-LM hyperparams]: Consider this figure from the Megatron-LM repository highlighting their high MFU numbers.

Model size	Attention heads	Hidden size	Number of layers	Tensor MP size	Pipeline MP size	Data-parallel size	Number of GPUs	Batch size	Per-GPU teraFLOP/s	MFU	Aggregate petaFLOP/s
1.7B	16	2048	24	1	1	48	48	192	408.8	41%	19.6
7.1B	32	4096	30	2	1	48	96	192	465.9	47%	44.7
16B	48	6144	32	4	1	48	192	192	489.1	49%	93.9
32B	56	7168	48	8	1	48	384	192	459.6	46%	176.5
70B	64	8192	84	8	2	48	768	384	419.7	42%	322.3
119B	80	10240	92	8	4	48	1536	768	420.5	43%	645.9
177B	96	12288	96	8	6	48	2304	1152	432.8	44%	997.2
314B	128	16384	96	8	8	48	3072	1536	474.4	48%	1457.4
462B	144	18432	112	8	16	48	6144	3072	459.9	47%	2825.6

Note that their sequence length is 4096 everywhere. For the 16B, 70B, and 314B models, what is the per-GPU token batch size? Assuming data parallelism is the outermost axis and assuming bfloat16 reductions, determine whether each of these is theoretically compute-bound or communication-bound, and whether there is a more optimal configuration available?

Click here for the answer.

Answer: Let's start with batch sizes per GPU.

- **16B**: `192 * 4096 / 192 = 4096` tokens per GPU
- **70B**: `384 * 4096 / 768 = 2048` tokens per GPU
- **314B**: `1536 * 4096 / 3072 = 2048` tokens per GPU

This means with the exception of the first, these all hover around 2k tokens per batch, which is notably around the critical threshold we calculated for FSDP. We had calculated that bound to be 2,472 tokens / GPU based on the spine level reduction, which should roughly come into play here. For both the 70B and 314B though, because

we have 16 and 64-way model (PP + TP) sharding respectively, we get 2x and 8x better throughput at the spine level, which means we should be compute-bound at roughly 1k and 300 tokens / step respectively.

Acknowledgements and Further Reading

This chapter relied heavily on help from many knowledgeable GPU experts, including:

- Adam Paszke, who helped explain the realities of kernel programming on GPUs.
- Swapnil Patil, who first explained how GPU networking works.
- Stas Bekman, who pointed out that the empirical realities of GPUs are often different from the purported specs.
- Reiner Pope, who helped clarify how GPUs and TPUs compare at a hardware level.
- Frédéric Bastien, who gave detailed feedback on the chip-level story.
- Nouamane Tazi, whose experience with LLM training on GPUs helped improve the roofline section.
- Sanford Miller, who helped me understand how GPUs are networked and how NVIDIA's specifications compare to what's often deployed in the field.

There's a great deal of good reading on GPUs, but some of my favorites include:

- SemiAnalysis' History of the NVIDIA Tensor Core: a fantastic article describing how GPUs transformed from video game engines to ML accelerators.
- SemiAnalysis' Analysis of Blackwell Performance: worth reading to understand the next generation of NVIDIA GPUs.
- H100 DGX SuperPod Reference: dry but useful reading on how larger GPU clusters are networked. Here is a similar document about the GB200 systems.
- Hot Chips Talk about the NVLink Switch: fun reading about NVLink and NCCL collectives, especially including in-network reductions.
- DeepSeek-V3 Technical Report: a good example of a large semi-open LLM training report, describing how they picked their sharding setup.
- How to Optimize a CUDA Matmul: a great blog describing how to implement an efficient matmul using CUDA Cores, with an eye towards cache coherence on GPU.
- HuggingFace Ultra-Scale Playbook: a guide to LLM parallelism on GPUs, which partly inspired this chapter.
- Making Deep Learning Go Brrrr From First Principles: a more GPU and PyTorch-focused tutorial on LLM rooflines and performance engineering.
- Cornell Understanding GPU Architecture site: a similar guide to this book, comparing GPU and CPU internals more specifically.

Appendix A: How does this change with GB200?

Blackwell introduces a bunch of major networking changes, including NVLink 5 with twice the overall NVLink bandwidth (900GB/s). B200 still has 8-GPU nodes, just like H100s, but GB200 systems (which combine B200 GPUs with Grace CPUs) introduce much larger NVLink domain (72 GPUs in NVL72 and in theory up to 576). This bigger NVLink domain also effectively increases the node egress bandwidth, which reduces collective costs above the node level.

Within a node, this increased bandwidth (from 450GB/s to 900GB/s) doesn't make much of a difference because we also double the total FLOPs/s of each GPU. Our rooflines mostly stay the same, although because NVLink has much better bandwidth, Expert Parallelism becomes easier.

Beyond a node, things change more. Here's a SuperPod diagram from here.

As you can see, the per-node egress bandwidth increases to `4 * 18 * 400 / 8 = 3.6TB/s`, up from 400GB/s in H100. This improves the effective cross-node rooflines by about 4x since our FLOPs/chip also double. Now we may start to worry about whether we're bottlenecked at the node level rather than the scale-out level.

Grace Hopper: NVIDIA also sells GH200 and GB200 systems which pair some number of GPUs with a Grace CPU. For instance, a GH200 has 1 H200 and 1 Grace CPU, while a GB200 system has 2 B200s and 1 Grace CPU. An advantage of this system is that the CPU is connected to the GPUs using a full bandwidth NVLink connection

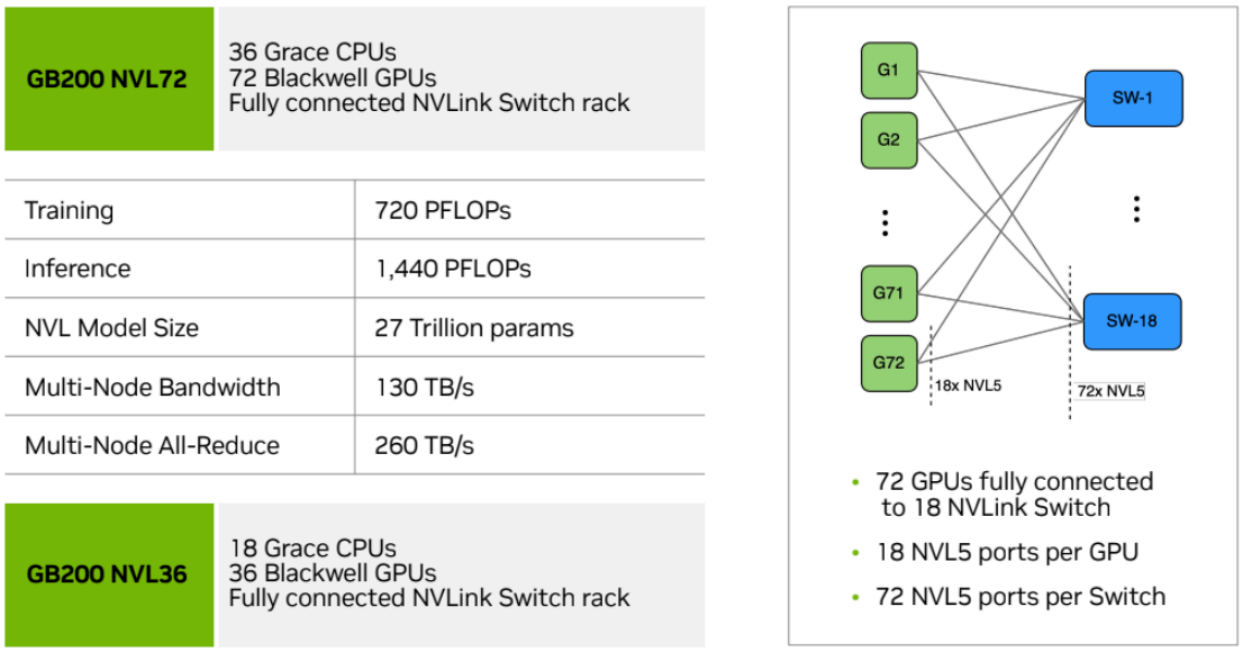


Figure 38: Figure: a diagram showing how a GB200 NVL72 unit is constructed, with 18 switches and 72 GPUs.

(called NVLink C2C), so you have very high CPU to GPU bandwidth, useful for offloading parameters to host RAM. In other words, for any given GPU, the bandwidth to reach host memory is identical to reaching another GPU's HBM.

Appendix B: More networking details

Here's a diagram of an NVLink 4 switch. There are 64 overall NVLink4 ports (each uses 2 physical lanes), and a large crossbar that handles inter-lane switching. TPUs by contrast use optical switches with mirrors that can be dynamically reconfigured.

At each level we can be bottlenecked by the available link bandwidth or the total switch bandwidth.

- **Node level:** at the node level, we have 4 * 1.6TB/s = 6.4TB/s of NVSwitch bandwidth, but each of our 8 GPUs can only egress 450GB/s into the switch, meaning we actually have a peak bandwidth of 450e9 * 8 = 3.6TB/s (full-duplex) within the node.
- **SU/leaf level:** at the SU level, we have 8 switches connecting 32 nodes in an all-to-all fashion with 1x400 Gbps Infiniband. This gives us 8 * 32 * 400 / 8 = 12.8TB/s of egress bandwidth from the nodes, and we have 8 * 1.6TB/s = 12.8TB/s at the switch level, so both agree precisely.
- **Spine level:** at the spine level, we have 16 switches connecting 32 leaf switches with 2x400 Gbps links, so we have 32 * 16 * 400 * 2 / 8 = 51.2TB/s of egress bandwidth. Unlike a leaf switch, all 64 ports of a spine switch face downward, so each switch can move 64 * 400 / 8 = 3.2TB/s of traffic, giving us 16 * 3.2TB/s = 51.2TB/s at the switch level, again agreeing precisely.

Per GPU, this gives us 450GB/s of GPU to GPU bandwidth at the node level, and 50GB/s at both the SU and spine levels.

GPU empirical AR bandwidth:

TPU v5p bandwidth (1 axis):

Here's AllGather bandwidth as well:

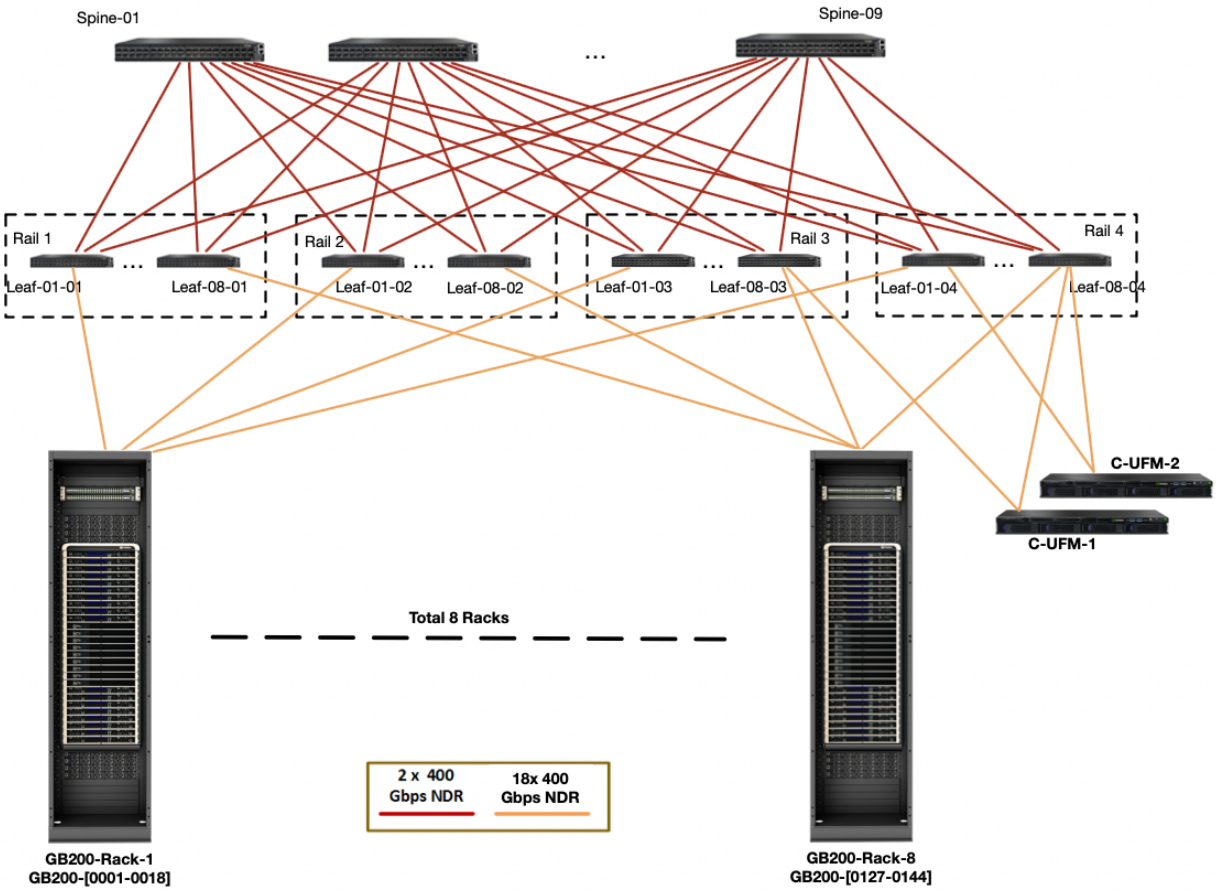


Figure 39: Figure: a diagram showing a GB200 DGX SuperPod of 576 GPUs.

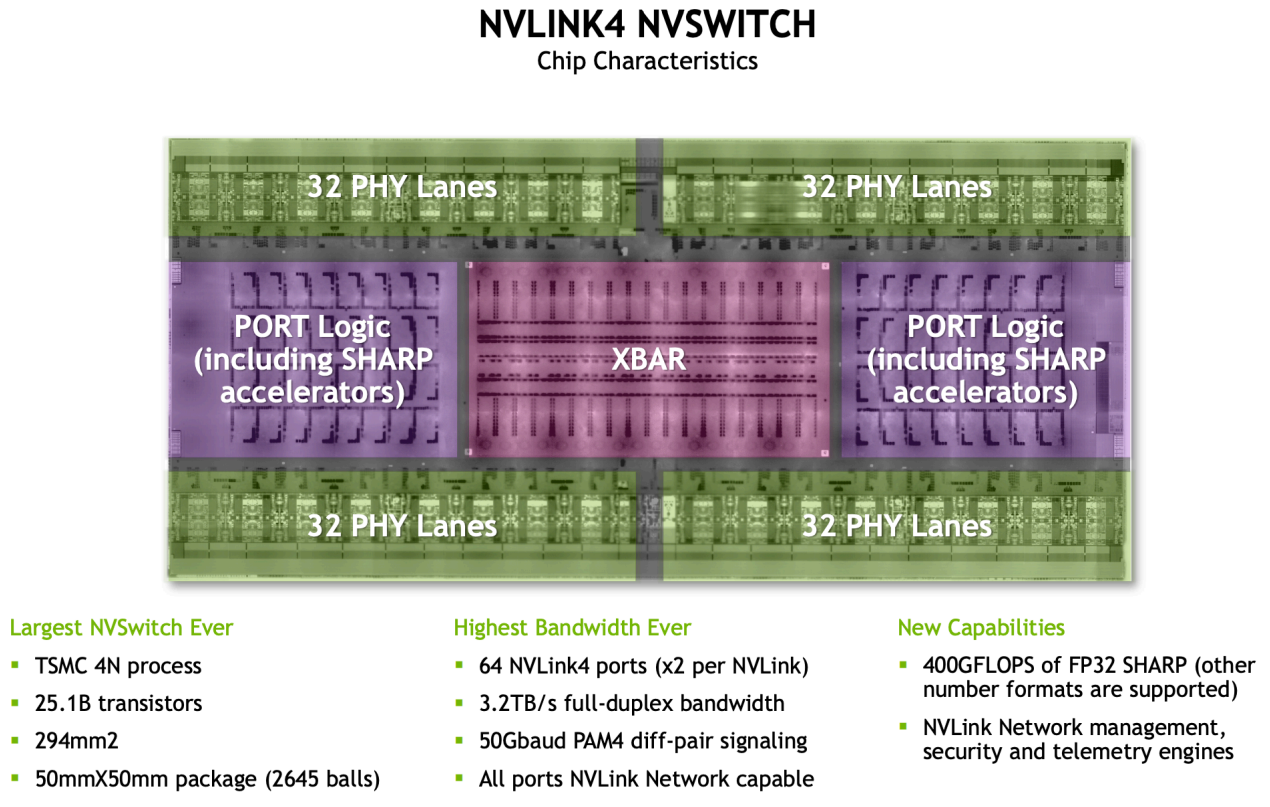


Figure 40: Figure: a lower level view of a single NVLink4 Switch.

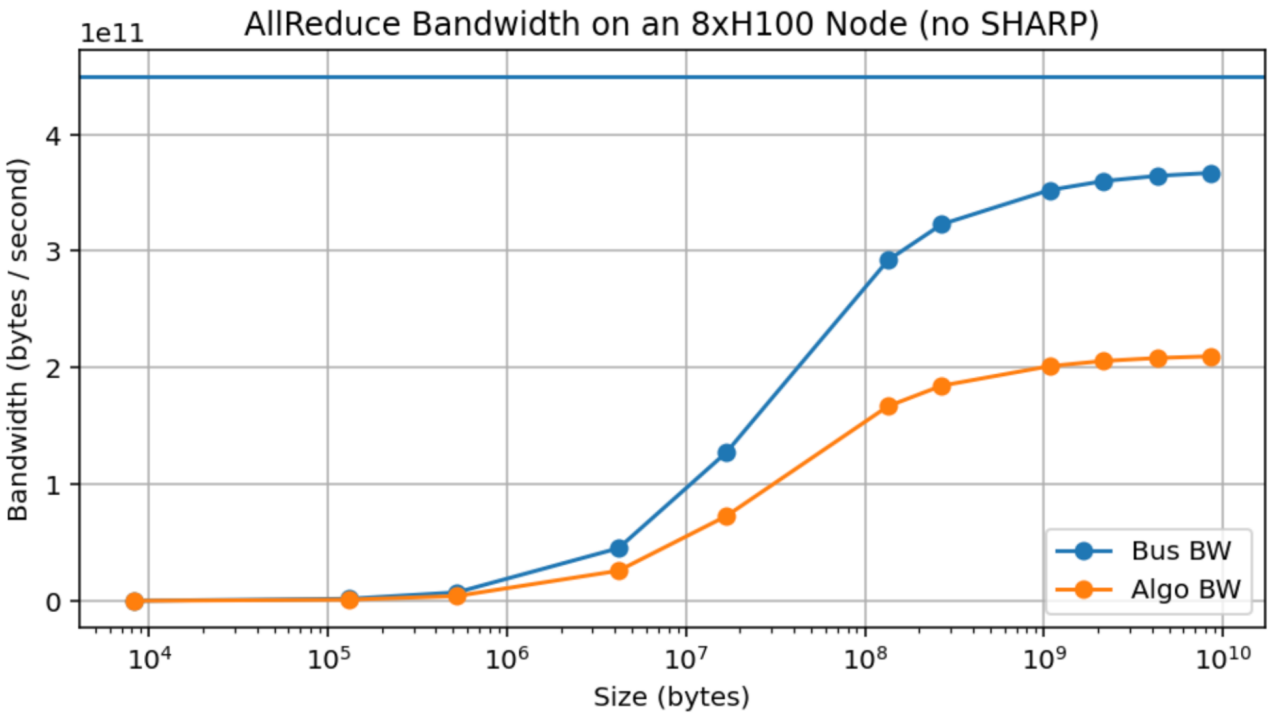


Figure 41: Figure: AllReduce bandwidth on an 8xH100 cluster (intra-node, SHARP disabled).

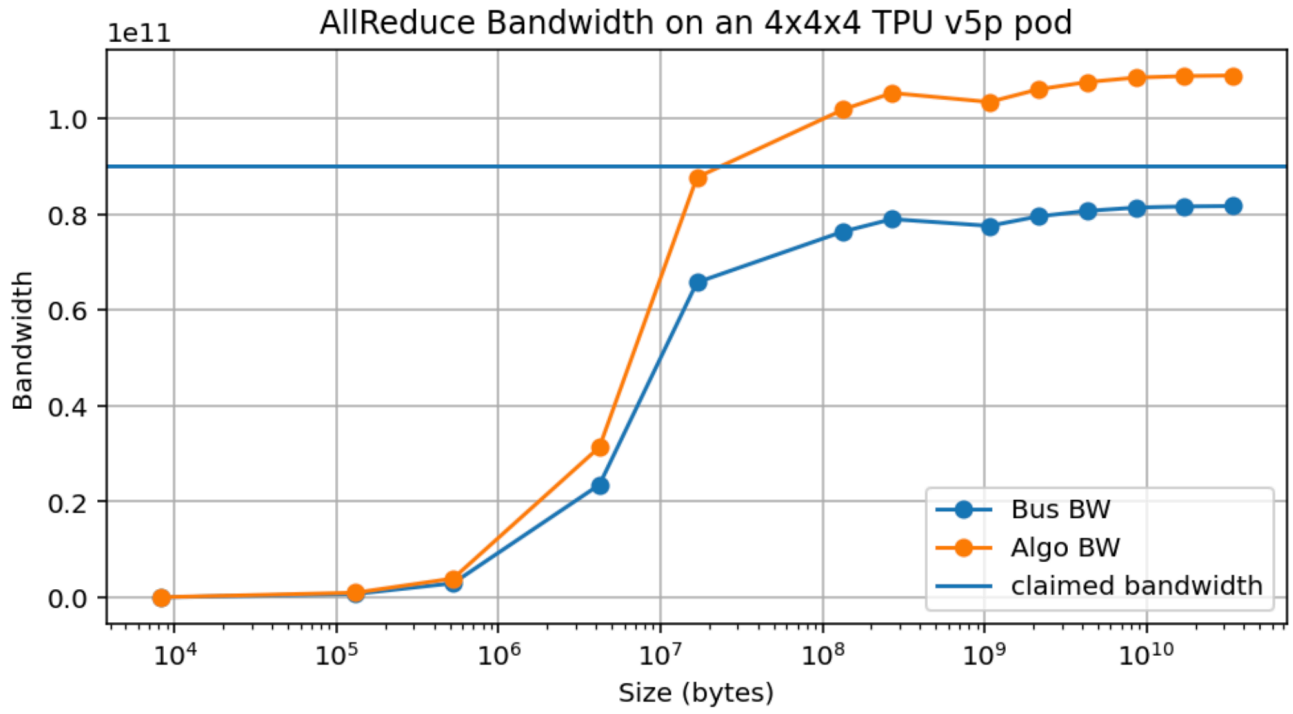


Figure 42: Figure: AllReduce bandwidth on a TPU v5p 4x4x4 cluster (along one axis).

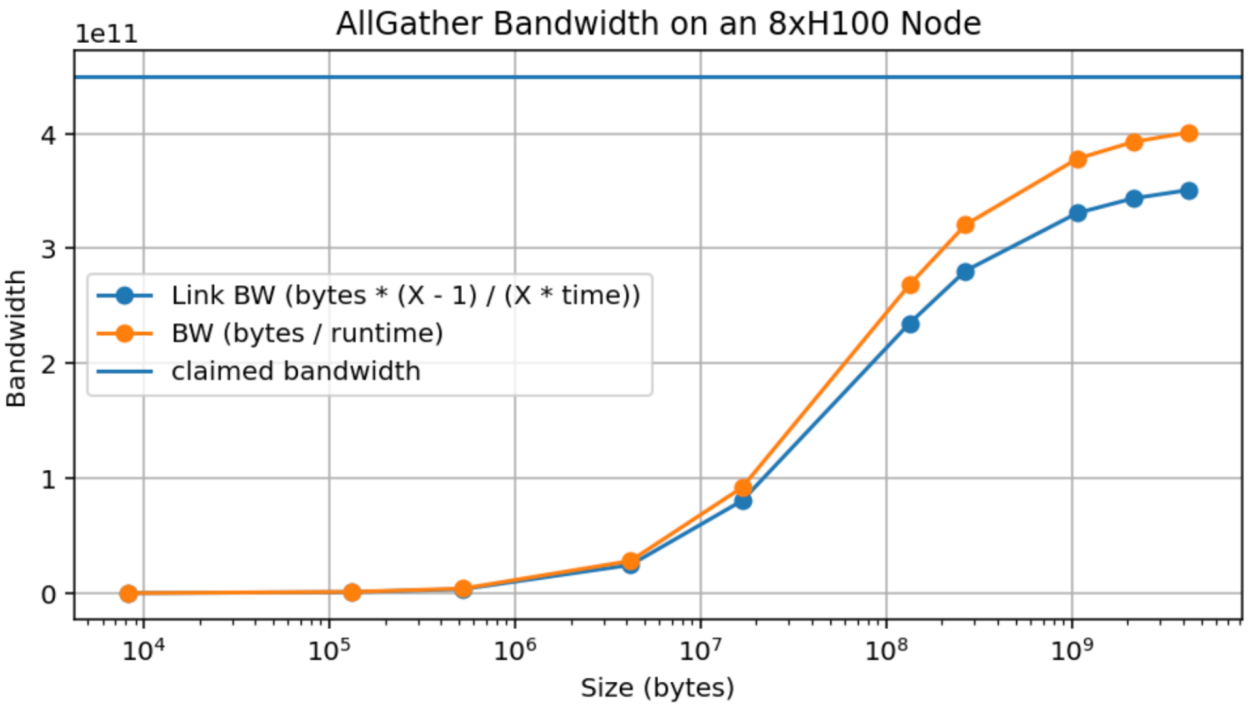


Figure 43: Figure: AllGather bandwidth on an 8xH100 cluster (intra-node).

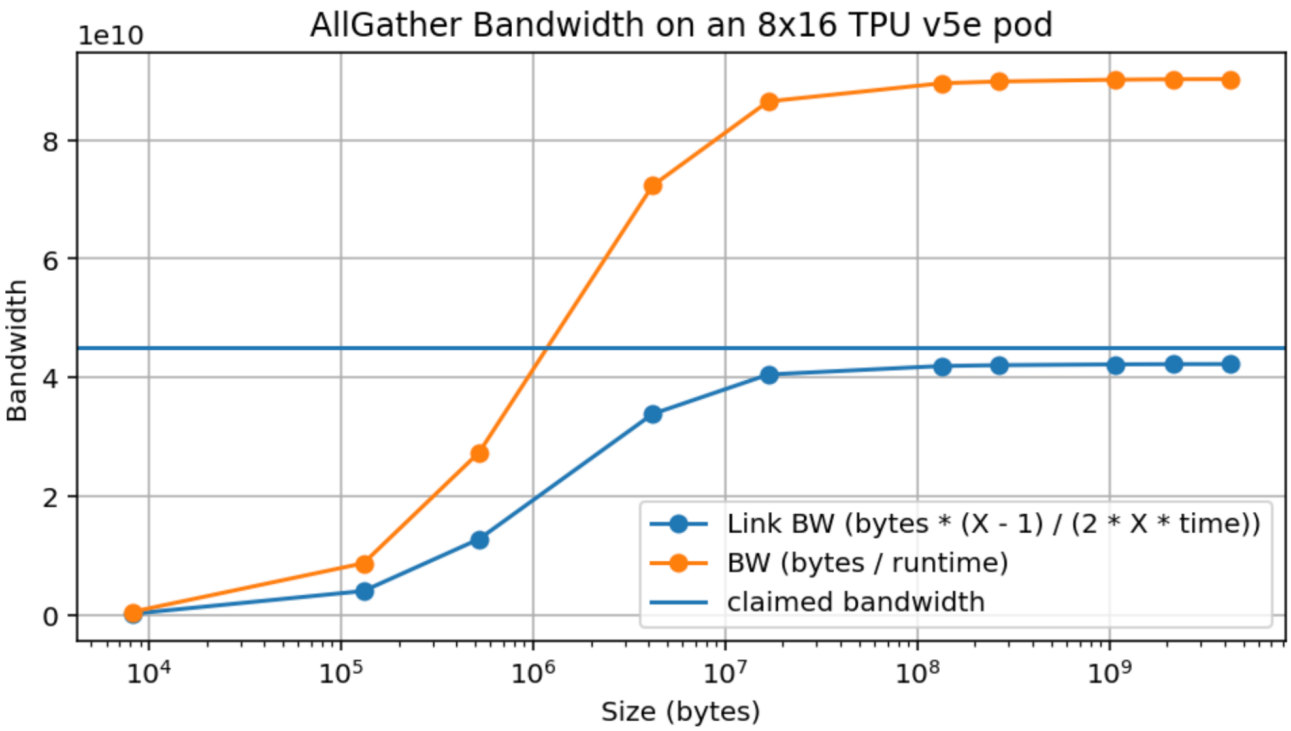


Figure 44: Figure: AllGather bandwidth on a TPU v5e 8x16 cluster (along one axis).

More on AllToAll costs:

Here we can compare the approximation $\min(K/Z)*(Z-1)/Z$ to the true value of $(1-((Z-1)/Z)**K)*(Z-1)/Z$. They're similar except for small values of Z.

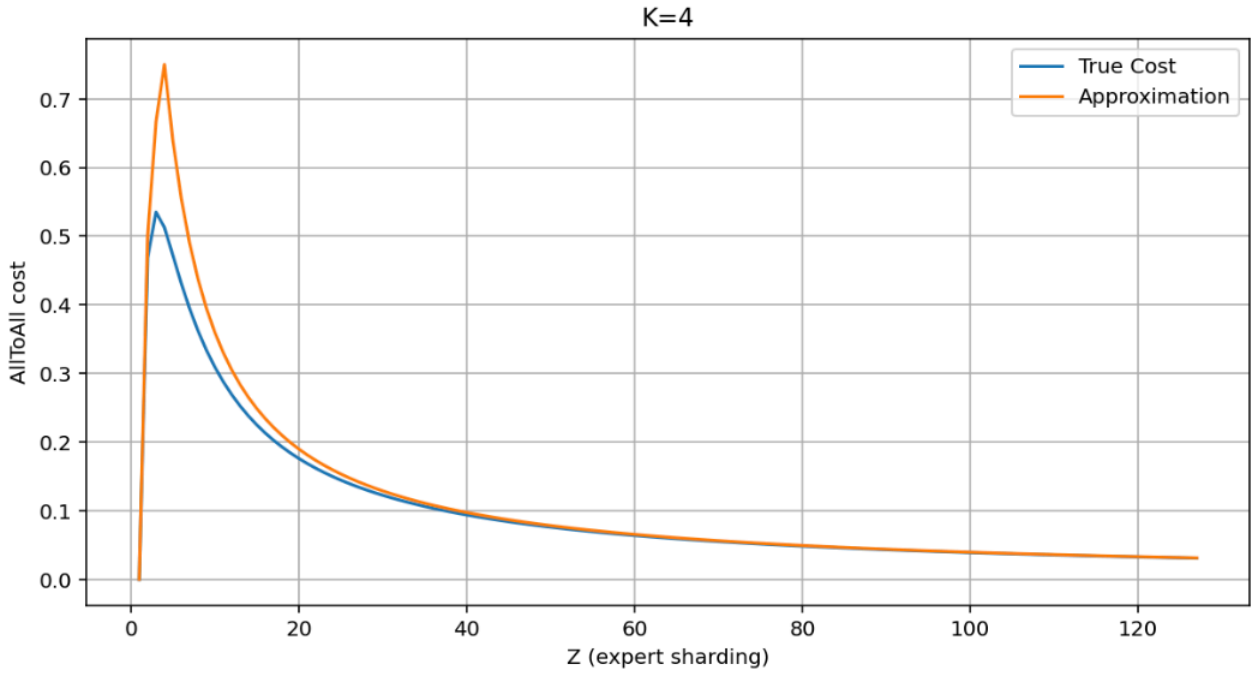


Figure 45: Figure: a comparison of the approximate and true cost of a ragged AllToAll as the number of shards increases.